ФУНДАМЕНТАЛЬНЫЙ УЧЕБНИК ПО РЕВЕРС-ИНЖИНИРИНГУ

Windows x64, PE/COFF, исключения, unwind и полная анатомия EH/Unwind Analyzer

Системный курс от двоичного представления, C и x86-64 до практического статического и динамического анализа. Отдельный том пошагово разбирает фактический исходный код проекта, отчёты, эвристики, тестирование и интеграцию.

Версия проекта: 1.0.0
Составлено по фактическому исходному коду проекта
26 июня 2026 года

Содержание

Windows x64 SEH/Unwind . . . 22
Как устроено чтение . . . 22
Нулевой фундамент: память, адреса, стек и куча . . . 23
Глава 1. База: что вообще происходит при исключении . . . 24
1. Что такое исключение в Windows вообще . . . 24
2. Что такое вызов функции, стек и кадр стека . . . 24
3. Как Windows ищет обработчик по кадрам . . . 25
4. Чем x64-модель отличается от x86-модели . . . 26
Глава 2. База PE: в каком файле все это живет . . . 27
5. Что такое PE-файл и как он устроен в общем виде . . . 27
6. Что такое PE32+ и почему для x64 это важно . . . 27
7. Что такое RVA, VA, Image Base и секции . . . 28
8. Что такое Data Directory и где там Exception Directory . . . 28
Глава 3. Табличная модель x64: откуда Windows берет данные для unwind . . . 30
9. Что такое .pdata и зачем она нужна . . . 30
10. Что такое RUNTIME_FUNCTION и как читать ее поля . . . 30
11. Что такое .xdata и что там лежит . . . 31
12. Что такое UNWIND_INFO и его общая идея . . . 31
Глава 4. Как функция описывается для unwind . . . 33
13. Что такое пролог функции и почему он описывается отдельно . . . 33
14. Что такое эпилог функции и почему он тоже важен . . . 33
15. Что такое UNWIND_CODE и как он кодирует действия пролога . . . 34
16. Все основные UWOP_* операции по одной и очень подробно . . . 34
17. Что такое frame register и frame pointer на x64 . . . 35
18. Что такое volatile и nonvolatile регистры . . . 36
19. Что такое x64 calling convention и как она связана с unwind . . . 36
Глава 5. Как Windows реально поднимается по стеку . . . 37
20. Как система по RIP находит запись функции . . . 37
21. Что делает RtlLookupFunctionEntry . . . 37
22. Что делает RtlVirtualUnwind . . . 37
23. Что такое EstablisherFrame . . . 38
24. Что такое DispatcherContext . . . 38
Глава 6. Фазы обработки исключения . . . 40
25. Что такое search phase и unwind phase . . . 40
26. Что такое language-specific handler . . . 40
27. Что такое __C_specific_handler и его роль . . . 40
28. Что такое SEH на x64 в реальном смысле . . . 41
29. Что такое VEH и чем она отличается от frame-based обработки . . . 41
30. Что такое top-level / unhandled exception handling . . . 42
Глава 7. Полные сценарии . . . 43
31. Что происходит при RaiseException пошагово . . . 43
32. Что происходит при access violation пошагово . . . 43

Глава 8. Особые случаи и тонкости . . . 44
33. Как leaf function отличается от non-leaf function . . . 44
34. Почему не у каждой функции есть полноценная unwind-метаинформация . . . 44
35. Что такое chained unwind info . . . 45
36. Что такое dynamic function tables . . . 45
37. Что делают RtlAddFunctionTable и RtlInstallFunctionTableCallback . . . 45
Глава 9. Почему это важно не только для исключений . . . 47
38. Как unwind связан с отладчиками и stack trace . . . 47
39. Как unwind связан с crash dump analysis . . . 47
Глава 10. Практика ручного разбора . . . 48
40. Как вручную читать .pdata и .xdata в PE . . . 48
41. Как вручную разбирать конкретный UNWIND_INFO по байтам . . . 48
42. Как связывать дизассемблированный пролог с UNWIND_CODE . . . 49
Глава 11. Reverse engineering и твой проект . . . 50
43. Какие бывают аномалии и несоответствия в unwind-метаданных . . . 50
44. Как это все смотреть глазами reverse engineer . . . 51
45. Как это все переносится на твой Inspector/Protector . . . 51
Часть II. Углубленные лекции: широкий фундамент вокруг основной линии . . . 53
Расширенная лекция 1. От исходного кода к выполняющемуся процессу . . . 53
Часть I. От исходного текста до выполняющегося процесса . . . 53
Исходная точка . . . 53
Программа как описание вычисления . . . 53
Translation unit и preprocessing . . . 54
Compilation . . . 54
Object file . . . 55
Linking . . . 55
Исполняемый файл как структурированный результат . . . 56
Файл как последовательность байтов . . . 56
Безопасное чтение диапазона . . . 57
Порядок байтов . . . 58
От файла к процессу . . . 58
Почему входной EXE не запускается в статическом режиме . . . 59
Три состояния одной программы . . . 59
Место reverse engineering . . . 60
Вывод первой части . . . 60
Проверка связности понимания . . . 60
Расширенная лекция 2. Двоичные данные, память и системы адресов . . . 61
Часть II. Двоичные данные, память и системы адресов . . . 61
Почему следующий этап посвящён представлению данных . . . 61
Разрядность и множество представимых значений . . . 61
Знаковая интерпретация . . . 62
Шестнадцатеричная запись . . . 62
Побитовые поля . . . 62
Переполнение . . . 63
Порядок байтов . . . 63

Объект памяти процесса . . . 64
Физическая и виртуальная память . . . 64
Image . . . 65
ImageBase . . . 65
RVA . . . 65
Section и две системы координат . . . 65
File offset, RVA и VA как разные типы . . . 66
ASLR . . . 66
Адреса модулей в разных процессах . . . 67
Размер и диапазон . . . 67
Что проект проверяет на уровне адресов . . . 67
Итоговая связная модель . . . 67
Контрольные задачи . . . 68
Расширенная лекция 3. Процессор, функции, стек и Windows x64 ABI . . . 68
Часть III. Процессор, функции, стек и Windows x64 ABI . . . 68
Исполнение инструкции . . . 68
Регистры . . . 68
Операнды и память . . . 69
Арифметика адреса . . . 69
Управление потоком . . . 69
Function на машинном уровне . . . 70
Caller и callee . . . 70
Return value . . . 71
Volatile и nonvolatile registers . . . 71
Stack . . . 71
call и адрес возврата . . . 71
Stack frame . . . 72
Shadow space . . . 72
Alignment stack . . . 72
Prolog . . . 72
Epilog . . . 73
Frame pointer . . . 73
Leaf и nonleaf function . . . 73
Машинное исключение и обычный возврат . . . 73
Почему metadata отделена от машинного кода . . . 74
Пример связи с benign_richer.exe . . . 74
Context потока . . . 74
Почему проект сохраняет register masks . . . 74
Итоговая причинная цепочка . . . 75
Контрольные вопросы . . . 75
Расширенная лекция 4. Процесс, поток, контекст и исключение Windows . . . 75
Лекция 4. Процесс, поток, контекст и исключение в Windows . . . 75
Зачем эта лекция нужна до изучения проекта . . . 75
Процесс как объект операционной системы . . . 76
Модуль и загруженный образ . . . 76

Поток как носитель исполнения . . . 77
Контекст потока . . . 77
Нормальное управление и исключительное управление . . . 77
Structured Exception Handling . . . 78
Frame-based handlers, VEH, VCH и UEF . . . 78
First chance и second chance . . . 79
Debug object и поток событий . . . 79
Software breakpoint как исключение . . . 80
Trap flag и single-step . . . 80
Динамические таблицы функций . . . 81
Что динамический collector устанавливает, а чего не устанавливает . . . 81
Связь с последующими лекциями . . . 82
Нормативные источники . . . 82
Расширенная лекция 5. PE/COFF как контракт loader-а и анализатора . . . 82
Лекция 5. PE/COFF как контракт компоновщика, загрузчика и анализатора . . . 82
Формат файла и смысл структуры . . . 82
Object file, image file и этап link . . . 83
File offset, RVA и VA как три координатные системы . . . 83
DOS header и поле e_lfanew . . . 83
PE signature . . . 84
COFF file header . . . 84
Почему optional header называется optional . . . 84
ImageBase . . . 85
SectionAlignment и FileAlignment . . . 85
SizeOfImage . . . 85
SizeOfHeaders . . . 86
Data directories . . . 86
Section header . . . 86
Почему parser использует max(VirtualSize, RawSize) . . . 87
Structural validation секций . . . 87
Exception directory . . . 87
Связь .pdata и .xdata . . . 88
Loader contract и parser contract . . . 88
Что должен уметь доказать студент . . . 89
Нормативный источник . . . 89
Расширенная лекция 6. Диспетчеризация исключений и теория unwind . . . 89
Лекция 6. Диспетчеризация исключений и теория раскрутки Windows x64 . . . 89
Центральная проблема . . . 89
Leaf и nonleaf function . . . 89
RUNTIME_FUNCTION как индекс диапазона кода . . . 90
UNWIND_INFO как описание обратного действия . . . 90
Version и Flags . . . 90
SizeOfProlog и CodeOffset . . . 91
FrameRegister и FrameOffset . . . 91
UWOP_PUSH_NONVOL . . . 91

UWOP_ALLOC_SMALL . . . 92
UWOP_ALLOC_LARGE . . . 92
UWOP_SET_FPREG . . . 92
UWOP_SAVE_NONVOL и UWOP_SAVE_NONVOL_FAR . . . 92
XMM operations . . . 93
UWOP_PUSH_MACHFRAME . . . 93
Padding и выравнивание хвоста . . . 93
Handler tail . . . 93
Chained unwind info . . . 94
Virtual unwind . . . 94
Search phase и unwind phase . . . 94
Почему metadata интересна reverse engineer . . . 95
Инварианты, которые проверяет проект . . . 95
Разбор фактической записи sample . . . 96
Граница доказательства . . . 96
Нормативные источники . . . 96
Расширенная лекция 7. Метод reverse engineering: Ghidra, debugger, отчет . . . 97
Лекция 7. Метод reverse engineering: от наблюдения к проверяемому выводу . . . 97
Reverse engineering как дисциплина вывода . . . 97
Статический и динамический анализ . . . 97
Нормативный факт, implementation fact и аналитический вывод . . . 97
Chain of custody для бинарного артефакта . . . 98
Первый проход: определить класс объекта . . . 98
Ghidra как статическая платформа . . . 98
Адресная синхронизация . . . 99
Импорт CSV из проекта . . . 99
Анализ функции по слоям . . . 99
Сопоставление prolog и UNWIND_INFO . . . 100
Control flow graph . . . 100
Data flow . . . 101
Cross-reference analysis . . . 101
Debugger breakpoint strategy . . . 101
Software breakpoint limitations . . . 102
First-chance workflow . . . 102
Handler investigation . . . 102
Static-dynamic correlation . . . 103
Evidence strength и falsifiability . . . 103
Отчёт reverse engineer . . . 104
Практический протокол для EH/Unwind Analyzer . . . 104
Что означает компетентность после лекции . . . 104
Расширенная лекция 8. Системная модель проекта . . . 105
Том II. EH/Unwind Analyzer: полная инженерная модель проекта . . . 105
Что именно должно быть понято во втором томе . . . 105
Предметная область проекта . . . 105
Граница продукта . . . 106

Архитектурный тип . . . 106
Модули и их строгие контракты . . . 107
util.c . . . 107
pe.c . . . 107
unwind.c . . . 107
heuristics.c . . . 108
dynamic_win.c . . . 108
report.c . . . 108
app_info.c . . . 108
main.c . . . 108
Почему CLI и ядро разделены . . . 108
Главные структуры как состояния анализа . . . 109
Buf . . . 109
PeImage . . . 109
RuntimeFunction[] . . . 109
UnwindSummary[] и us_ok[] . . . 109
HeuristicReport . . . 109
DynamicReport . . . 109
ReportMetadata . . . 110
Почему используются параллельные массивы . . . 110
Ownership graph . . . 110
Error model . . . 111
CLI error . . . 111
Input or structural fatal error . . . 111
Per-record parse failure . . . 111
Analytical finding . . . 111
Output error . . . 111
Dynamic session failure . . . 111
Почему analysis продолжается при локальной ошибке . . . 111
Доказательная модель . . . 111
Сквозной инвариант неизменности входа . . . 112
Связь модулей с теоретическим фундаментом . . . 113
Как читать последующие главы . . . 113
Расширенная лекция 9. CLI lifecycle и границы ответственности . . . 113
Том II. main.c: полный жизненный цикл одного анализа . . . 113
Роль main.c . . . 113
Входной интерфейс процесса . . . 114
Внутренняя структура Args . . . 114
Почему parsing arguments отделён . . . 115
Позиционный input . . . 115
Разделитель -- . . . 115
Проверка чисел . . . 116
Диапазоны options . . . 116
Exit code CLI parsing . . . 116
Чтение input . . . 117

Error buffers . . . 117
PE parsing . . . 117
Runtime function table . . . 117
Allocations для per-function state . . . 118
Options unwind parser . . . 118
Основной цикл unwind . . . 118
Почему операции могут разбираться повторно . . . 119
Heuristic stage . . . 119
Dynamic stage . . . 119
Outputs CSV . . . 120
Подготовка provenance . . . 120
Redaction arguments target . . . 120
SHA-256 . . . 120
JSON и HTML . . . 120
Финальный cleanup . . . 121
Полный автомат состояний main . . . 121
Tests, относящиеся к main . . . 121
Что можно улучшить . . . 122
Проверка полного понимания main . . . 122
Расширенная лекция 10. Безопасное чтение байтов и util-слой . . . 122
Том II. util.c: основание безопасного parser и генерации отчётов . . . 122
Почему util.c является частью доказательства корректности . . . 122
Buf как read-only view . . . 123
buf_init_from_file . . . 123
Определение размера . . . 123
Пустой файл . . . 124
Полнота fread . . . 124
buf_free_owned . . . 124
buf_read . . . 124
Typed read functions . . . 125
le16, le32, le64 . . . 125
StrBuf . . . 125
Почему error state sticky . . . 126
Рост capacity . . . 126
sb_appendn . . . 126
sb_appendf . . . 126
Атомарная запись . . . 127
SHA-256 . . . 127
Связь util.c с отчётом и tests . . . 127
Полная роль модуля . . . 128
Расширенная лекция 11. PE parser проекта . . . 128
Том II. pe.c: построение модели PE из недоверенных байтов . . . 128
Почему PE parser выполняется раньше unwind decoder . . . 128
Почему parser не использует Windows headers . . . 128
PeSection . . . 129

PeImage . . . 129
Инициализация результата . . . 129
DOS header . . . 130
Проверка минимального PE header range . . . 130
PE signature . . . 130
COFF file header . . . 130
Optional header . . . 131
Magic PE32+ . . . 131
Ключевые поля optional header . . . 131
ImageBase . . . 131
SectionAlignment . . . 131
FileAlignment . . . 132
SizeOfImage . . . 132
SizeOfHeaders . . . 132
NumberOfRvaAndSizes . . . 132
Фатальные проверки полей . . . 132
Data directory . . . 132
Section table . . . 132
Чтение section fields . . . 133
Virtual span policy . . . 133
Raw OOB . . . 133
Virtual OOB . . . 134
Alignment . . . 134
Overlap . . . 134
Exception directory boundary . . . 134
Завершение модели . . . 135
pe_free . . . 135
pe_find_section_for_rva . . . 135
pe_rva_to_fileoff . . . 135
Сквозной пример benign_richer.exe . . . 136
Переход от parser counter к finding . . . 136
Tests . . . 136
Ограничения pe.c . . . 137
Что должно быть понятно перед unwind.c . . . 137
Расширенная лекция 12. Unwind decoder проекта . . . 137
Том II. unwind.c: от таблицы функций к модели восстановления стека . . . 137
Почему Windows x64 хранит таблицу функций . . . 137
RuntimeFunction . . . 138
Чтение exception directory . . . 138
Почему reader не проверяет begin/end сразу . . . 138
UnwindSummary . . . 138
Register encoding . . . 139
Flags . . . 139
Вход unwind_parse . . . 139
Инициализация и локализация ошибки . . . 140

Options . . . 140
Alignment UNWIND_INFO . . . 140
Mapping header . . . 140
Header layout . . . 140
Version policy . . . 141
Frame register checks . . . 141
CountOfCodes . . . 141
Structure slot . . . 141
Определение длины operation . . . 142
UWOP_PUSH_NONVOL . . . 142
UWOP_ALLOC_SMALL . . . 142
UWOP_ALLOC_LARGE . . . 143
UWOP_SET_FPREG . . . 143
UWOP_SAVE_NONVOL . . . 143
UWOP_SAVE_NONVOL_FAR . . . 143
XMM saves . . . 144
V1 opcodes 6 и 7 . . . 144
V2 epilog descriptors . . . 144
UWOP_PUSH_MACHFRAME . . . 144
Защита суммы fixed stack . . . 144
Tail после slots . . . 144
Handler tail . . . 145
Chain tail . . . 145
Следование по chain . . . 145
valid и anomalies . . . 145
parse_error . . . 146
Сквозной sample . . . 146
Связь с heuristics . . . 146
Tests . . . 147
Ограничения decoder . . . 147
Расширенная лекция 13. Heuristics evidence engine . . . 148
Том II. Глава 6. heuristics.c: от структурных фактов к оценке свидетельств . . . 148
Место модуля в системе . . . 148
Что означает слово heuristic . . . 148
Выходная модель . . . 148
Валидация arguments и initialization . . . 149
Evidence families . . . 149
Frequency map . . . 150
Первый проход по RUNTIME_FUNCTION . . . 150
Invalid range . . . 150
Function range out of image . . . 150
Unwind RVA out of image . . . 151
Unsorted input table . . . 151
Reuse counts . . . 151
Placement unwind metadata . . . 151

Placement handler . . . 152
Parse failure и anomaly . . . 152
Overlap и duplicate begin . . . 152
Ratios . . . 153
Score saturation . . . 153
PE structural findings и веса . . . 153
Empty exception table . . . 154
Процентные integrity thresholds . . . 154
Unwind anomaly threshold . . . 154
Table order findings . . . 154
Strong unwind reuse . . . 155
Handler dominance с benign-context suppression . . . 155
Moderate unwind reuse требует corroboration . . . 155
Placement weights . . . 156
Combination findings . . . 156
Executable unwind metadata и non-executable handler . . . 156
Placement и integrity . . . 156
Reuse и dominance . . . 157
Writable metadata и writable handler . . . 157
Finding storage limit . . . 157
Severity counts и evidence level . . . 157
Confidence calculation . . . 158
Weakening rules . . . 158
single_signal_only . . . 158
small_sample_size . . . 158
placement_only_signal . . . 159
single_family_only . . . 159
Manual review flag . . . 159
Verdict . . . 159
Фактический benign sample . . . 159
Фактический suspicious sample suspicious_unwind_oob.exe . . . 160
Фактический suspicious sample suspicious_unwind_writable.exe . . . 160
Тесты модуля . . . 160
Что следует улучшать в будущей версии . . . 161
Итоговая модель главы . . . 161
Расширенная лекция 14. Windows dynamic evidence collector . . . 161
Том II. Глава 7. dynamic_win.c: Windows dynamic evidence collector . . . 161
Назначение и граница платформы . . . 161
Defensive intent . . . 162
Модель результата . . . 162
Module map . . . 162
Разрешение forwarded API . . . 163
Наблюдаемые API . . . 163
Breakpoint model . . . 164
Запись bytes в remote process . . . 164

Установка breakpoint . . . 164
Self-check . . . 165
Per-thread pending re-arm . . . 165
Командная строка target . . . 165
Создание процесса . . . 166
Static handler addresses как input dynamics . . . 166
Debug loop и timeout . . . 166
Continuation status . . . 167
CREATE_PROCESS_DEBUG_EVENT . . . 167
LOAD_DLL_DEBUG_EVENT . . . 167
Ordinary exception event . . . 167
Initial system breakpoint . . . 168
Hit собственного INT3 . . . 168
Capture регистрации VEH . . . 169
Capture VCH . . . 169
Capture UEF . . . 169
Capture RtlAddFunctionTable . . . 169
Capture callback table . . . 170
Handler hits . . . 170
EXCEPTION_SINGLE_STEP . . . 171
Thread exit . . . 171
Process exit . . . 171
Session finalization . . . 171
Semantics dynamic signal count . . . 172
Runtime evidence level . . . 172
Structured event limitations . . . 172
Граница безопасности и доверия . . . 173
Bundled dynamic smoke targets . . . 173
Фактическая Windows validation 8 июня 2026 года . . . 173
Что сделано правильно . . . 174
Что остаётся улучшить . . . 174
Итоговая state machine . . . 175
Расширенная лекция 15. Reporting, assessment и интеграция . . . 175
Том II. Глава 8. report.c: отчёты, итоговая assessment и интеграционный контракт . . . 175
Отчёт не является форматированным printf . . . 175
Brief console report . . . 176
CSV report . . . 176
Ghidra CSV . . . 176
Anomaly text . . . 177
Report metadata . . . 177
Metadata validation . . . 178
Model validation . . . 178
JSON escaping . . . 178
JSON numeric representation . . . 179
Top-level JSON model . . . 179

pe_validation . . . 179
sections . . . 179
heuristic_analysis . . . 180
dynamic_validation . . . 180
Functions . . . 181
Итоговая static/dynamic correlation . . . 181
Dynamic signal count . . . 182
Runtime evidence level . . . 182
Case strength . . . 182
Final confidence . . . 182
Final verdict . . . 182
Manual validation plan . . . 183
HTML security model . . . 183
HTML sections . . . 183
HTML status nuance . . . 184
Atomic file write . . . 184
Memory failure behavior . . . 184
JSON schema . . . 185
Как backend должен вызывать core . . . 185
Рекомендуемая database model . . . 185
Сравнение analyses . . . 186
API adapter не должен менять core . . . 186
Что сделано правильно . . . 186
Что улучшить . . . 187
Итог . . . 187
Расширенная лекция 16. Build, tests и release readiness . . . 188
Том II. Глава 9. Сборка, тестирование, воспроизводимость и степень готовности . . . 188
Что значит «проект работает» . . . 188
CMake project model . . . 188
Core sources . . . 188
Platform checks . . . 189
Warning policy . . . 189
Instrumentation . . . 189
Presets . . . 189
Unit test executables . . . 190
unit_unwind . . . 190
unit_heuristics . . . 190
unit_heuristics_mitigations . . . 190
unit_heuristics_context . . . 190
unit_heuristics_combos . . . 190
unit_core_edges . . . 190
unit_report_json . . . 191
unit_report_html . . . 191
unit_matrix_report . . . 191
Assertions в release unit tests . . . 191

Python integration tests . . . 191
regression_samples . . . 191
dynamic_validator . . . 192
report_semantics . . . 192
release_metadata . . . 192
release_hygiene . . . 192
cli_contract . . . 192
mutation_fuzz . . . 192
Общее число CTest cases . . . 192
Regression corpus semantics . . . 192
Реальный benign corpus . . . 193
Fuzzing levels . . . 193
Sanitizer limitations . . . 194
Dynamic smoke validation . . . 194
CLI exit codes . . . 195
Report schema validation . . . 195
Release metadata . . . 195
Installation . . . 195
CPack . . . 196
Windows x64 package . . . 196
macOS arm64 package . . . 196
Source package . . . 196
Evidence package . . . 196
Readiness dimensions . . . 197
Static core . . . 197
Dynamic core . . . 197
Product platform . . . 197
Можно ли защищать как ВКР . . . 198
Release gate перед передачей . . . 198
Фактический паспорт качества final revision . . . 199
Итог . . . 199
Расширенная лекция 17. Разбор нормального benign-файла . . . 200
Том II. Глава 10. Полный разбор benign_richer.exe: от bytes до assessment . . . 200
Цель сквозного разбора . . . 200
Первые bytes . . . 200
PE identity . . . 200
Alignment . . . 201
Sections . . . 201
.text . . . 201
.rdata . . . 201
.pdata . . . 201
.xdata . . . 201
.bss . . . 202
.idata . . . 202
Structural checks . . . 202

Exception directory . . . 202
Raw .pdata bytes . . . 202
Entry 0 . . . 202
Entry 1 . . . 203
Entry 2 . . . 203
Table invariants . . . 203
Conversion unwind RVA . . . 203
Raw .xdata bytes . . . 203
Record 0 at 0xA00 . . . 203
Record 1 at 0xA04 . . . 204
Record 2 at 0xA0C . . . 204
Parallel arrays . . . 204
Heuristic aggregation . . . 204
Static assessment . . . 205
Final assessment . . . 205
Что именно доказал этот run . . . 205
Как выполнить ручную correspondence check . . . 206
Почему этот sample важен . . . 206
Расширенная лекция 18. Разбор подозрительных samples . . . 206
Том II. Глава 11. Расследование четырёх подозрительных regression samples . . . 206
Назначение samples . . . 206
suspicious_unwind_oob.exe . . . 207
Фактический report . . . 207
Почему 0x9000 invalid . . . 207
Score derivation . . . 207
Confidence derivation . . . 207
Что проверять вручную . . . 208
Ограниченный вывод . . . 208
suspicious_unwind_writable.exe . . . 208
Фактический report . . . 208
Structural fact . . . 208
Score . . . 208
Confidence . . . 208
Почему score low, evidence medium . . . 209
Возможные benign explanations . . . 209
Manual verification . . . 209
Ограниченный вывод . . . 209
suspicious_section_raw_oob.exe . . . 209
Фактический report . . . 209
Score . . . 210
Confidence . . . 210
Почему пять raw overlaps . . . 210
Manual verification . . . 210
Ограниченный вывод . . . 210
suspicious_eh_unwind_tamper.exe . . . 210

Фактический report . . . 210
Interpretation . . . 211
Score . . . 211
Confidence . . . 211
Why combination affected count is zero . . . 211
Manual investigation . . . 211
Ограниченный вывод . . . 212
Почему все cases unconfirmed . . . 212
Как превратить case в correlated . . . 212
Regression test и реальное расследование . . . 212
Расширенная лекция 19. Типовые проблемы и решения . . . 213
Том II. Глава 12. Задачи с полными решениями . . . 213
Как работать с задачами . . . 213
Задача 1. Little-endian field . . . 213
Решение . . . 213
Задача 2. RVA в file offset . . . 213
Решение . . . 213
Задача 3. Virtual-only address . . . 214
Решение . . . 214
Задача 4. Число runtime functions . . . 214
Решение . . . 214
Задача 5. Range invariants . . . 214
Решение . . . 214
Задача 6. Decode header . . . 215
Решение . . . 215
Задача 7. UWOP_ALLOC_SMALL . . . 215
Решение . . . 215
Задача 8. UWOP_ALLOC_LARGE . . . 216
Решение . . . 216
Задача 9. Code ordering . . . 216
Решение . . . 216
Задача 10. Alignment tail . . . 216
Решение . . . 216
Задача 11. Score calculation . . . 217
Решение . . . 217
Задача 12. Confidence calculation . . . 217
Решение . . . 218
Задача 13. Breakpoint state . . . 218
Решение . . . 218
Задача 14. First-chance continuation . . . 218
Решение . . . 218
Задача 15. Dynamic function table event . . . 219
Решение . . . 219
Задача 16. Correlation . . . 219
Решение . . . 219

Задача 17. Backend result . . . 220
Решение . . . 220
Задача 18. Сравнение двух runs . . . 220
Решение . . . 220
Задача 19. Критика verdict . . . 220
Решение . . . 220
Задача 20. Проектирование нового finding . . . 221
Решение . . . 221
Итог контроля . . . 221
Расширенная лекция 20. Термины, источники и карта защиты . . . 222
Приложение A. Связный терминологический указатель . . . 222
Как пользоваться указателем . . . 222
Программа, файл и исполнение . . . 222
Данные и адреса . . . 222
CPU и ABI . . . 223
PE/COFF . . . 224
Исключения и unwind . . . 225
Debugging . . . 226
Анализ и доказательства . . . 227
Отчёты и интеграция . . . 228
Приложение B. Нормативные и технические источники . . . 229
Microsoft . . . 229
Стандарты и документация tools . . . 229
Приложение C. Карта ответов на защите . . . 230
Что делает проект . . . 230
Почему это нужно . . . 230
В чём научно-техническая задача . . . 230
Почему архитектура не microservices . . . 230
Почему static работает на macOS/Linux . . . 230
Почему score не probability . . . 230
Что означает successful analysis . . . 230
Что является главным ограничением . . . 230
Можно ли подключить frontend и database без изменения core . . . 230
Почему проект готов к ВКР . . . 231
Что развивать после ВКР . . . 231
Расширенная лекция 21. Evidence, coverage и проверяемость . . . 231
Том III. Техническая теория обнаружения аномалий и границ доказательства . . . 231
Зачем нужна отдельная теория над parser и эвристиками . . . 231
Наблюдение . . . 231
Инвариант . . . 232
Инварианты безопасного чтения . . . 232
Инварианты PE layout . . . 232
Инварианты RUNTIME_FUNCTION . . . 232
Инварианты UNWIND_INFO . . . 233
Нарушение и аномалия . . . 233

Finding как публичная единица результата . . . 233
Почему finding не равен verdict . . . 234
Evidence family . . . 234
Severity . . . 235
Score . . . 235
Suspicion . . . 235
Confidence . . . 235
Weakening rule . . . 236
Manual review . . . 236
Модель классов вмешательств . . . 237
Класс A. Повреждение файлового layout . . . 237
Класс B. Повреждение function table . . . 237
Класс C. Повреждение UNWIND_INFO . . . 237
Класс D. Перемещение metadata . . . 237
Класс E. Перенаправление handler . . . 237
Класс F. Централизация . . . 237
Класс G. Runtime-only изменение . . . 237
Класс H. Семантически согласованная подмена . . . 238
Покрытие не является полнотой . . . 238
Почему число признаков не доказывает покрытие . . . 238
Двенадцать внутренних anomaly reasons . . . 239
Traceability . . . 239
False positive и false negative . . . 239
Технически корректная формулировка результата . . . 240
Контрольные вопросы . . . 240
Расширенная лекция 22. Подробная логика findings . . . 241
Том III. Полная система 27 признаков: основания, связи и ручная проверка . . . 241
Что именно означает число 27 . . . 241
Критерии включения признака . . . 241
Группа I. Шесть признаков PE layout . . . 241
1. pe_section_raw_oob . . . 241
2. pe_section_virtual_oob . . . 242
3. pe_section_raw_overlap . . . 242
4. pe_section_virtual_overlap . . . 243
5. pe_section_misaligned . . . 243
6. pe_exception_directory_oob . . . 244
Группа II. Семь признаков RUNTIME_FUNCTION . . . 244
7. no_runtime_functions . . . 244
8. invalid_ranges . . . 245
9. rf_oob . . . 245
10. unwind_rva_oob . . . 245
11. rf_unsorted . . . 246
12. rf_overlaps . . . 246
13. duplicate_begin . . . 247
Группа III. Два признака UNWIND_INFO integrity . . . 247

14. unwind_parse_fail . . . 247
15. unwind_anomalies . . . 247
Группа IV. Три статистических признака . . . 248
16. unwind_reuse_moderate . . . 248
17. unwind_reuse_strong . . . 248
18. handler_dominance . . . 249
Группа V. Пять placement-признаков . . . 249
19. unwind_in_exec_section . . . 249
20. unwind_unusual_section . . . 250
21. unwind_writable_section . . . 250
22. handler_nonexec_section . . . 250
23. handler_writable_section . . . 251
Группа VI. Четыре combined findings . . . 251
24. placement_exec_nonexec_handler_combo . . . 251
25. placement_integrity_combo . . . 251
26. reuse_dominance_combo . . . 252
27. writable_metadata_handler_combo . . . 252
Почему отдельных findings не больше . . . 252
Почему отдельных findings не меньше . . . 253
Матрица покрытия классов вмешательств . . . 253
Какие вмешательства 27 признаков не гарантируют обнаружить . . . 253
Как добавить новый признак правильно . . . 254
Итоговое обоснование числа 27 . . . 254
Контрольные вопросы . . . 254
Расширенная лекция 23. Accuracy validation и дизайн эксперимента . . . 254
Том III. Точность, валидация и проектирование строгого эксперимента . . . 254
Почему вопрос «какова точность» неполон . . . 254
Единица оценки . . . 255
File-level . . . 255
Record-level . . . 255
Finding-level . . . 255
Event-level . . . 255
Ground truth . . . 256
Confusion matrix . . . 256
True Positive, TP . . . 256
False Positive, FP . . . 256
True Negative, TN . . . 256
False Negative, FN . . . 256
Sensitivity и recall . . . 257
Specificity . . . 257
Precision . . . 257
Negative Predictive Value . . . 258
F1 . . . 258
False Positive Rate . . . 258
False Negative Rate . . . 258

Threshold . . . 258
ROC и PR curves . . . 259
Calibration . . . 259
Confidence interval . . . 259
Independence assumption . . . 259
Что доказывает corpus 106/106 . . . 260
Что доказывают 6 regression samples . . . 260
Что доказывают 1000 mutations . . . 261
Что доказывают 124/124 Windows reports . . . 261
Что доказывают 4/4 focused smoke scenarios . . . 261
Parser accuracy и detector accuracy . . . 262
Parser structural accuracy . . . 262
Heuristic detection accuracy . . . 262
Runtime observation accuracy . . . 262
Строгий benchmark plan . . . 262
Шаг 1. Зафиксировать taxonomy . . . 262
Шаг 2. Собрать corpus . . . 263
Шаг 3. Разделить development и test . . . 263
Шаг 4. Выполнить независимую разметку . . . 263
Шаг 5. Измерить per-class metrics . . . 263
Шаг 6. Проверить thresholds . . . 263
Шаг 7. Провести error analysis . . . 264
Шаг 8. Опубликовать limitations . . . 264
Какая метрика главная для этого проекта . . . 264
Технически корректный ответ о текущей точности . . . 264
Технические источники . . . 264
Контрольные вопросы . . . 265
Расширенная лекция 24. UNWIND_INFO v3 compatibility . . . 265
Том III. UNWIND_INFO V3: новая модель, совместимость и план поддержки . . . 265
Почему V3 требует отдельной главы . . . 265
Статус спецификации . . . 266
Назначение V3 . . . 266
Что осталось неизменным . . . 266
Header V3 . . . 266
PayloadWords . . . 267
Tail offset . . . 267
Flags V3 . . . 267
Large prolog . . . 268
Prolog смещения IP . . . 268
Epilog descriptors . . . 268
Epilog offset . . . 269
Parent fragment transfer . . . 269
Large epilog . . . 269
WOD . . . 269
Почему variable-length decoding сложнее . . . 269

Порядок dispatch WOD . . . 270
Основные WOD . . . 270
Integer register encoding . . . 270
WOD_PUSH . . . 271
WOD_PUSH2 . . . 271
WOD_PUSH_CONSECUTIVE_2 . . . 271
Allocation WOD . . . 271
Save WOD . . . 271
WOD_SET_FPREG . . . 272
WOD_PUSH_CANONICAL_FRAME . . . 272
Fragment и chaining . . . 272
Почему текущий проект правильно отказывает V3 . . . 272
Почему отказ всё равно является продуктовым ограничением . . . 273
Как report должен различать malformed и unsupported . . . 273
План безопасной реализации V3 . . . 273
Этап 1. Version dispatch . . . 273
Этап 2. Новая модель данных . . . 273
Этап 3. Bounded cursor . . . 274
Этап 4. Descriptor validation . . . 274
Этап 5. WOD decoding . . . 274
Этап 6. Summary . . . 275
Этап 7. Reports и schema . . . 275
Этап 8. Tests . . . 275
Этап 9. Corpus gate . . . 276
Как V3 влияет на 27 findings . . . 276
Технически корректный ответ о V3 . . . 276
Нормативный источник . . . 276
Контрольные вопросы . . . 276
Приложение A. Таблица основных UWOP . . . 278
Приложение B. Словарь терминов без тумана . . . 279
Приложение C. Нормативные источники и ориентиры . . . 280
Приложение D. Чек-лист ручной проверки одного finding . . . 281
Приложение E. Как говорить о проекте на защите . . . 282

Windows x64 SEH/Unwind

Фундаментальный учебник для понимания темы ВКР: PE32+ AMD64, Exception Directory, `.pdata`, `.xdata`, `RUNTIME_FUNCTION`, `UNWIND_INFO`, x64 calling convention, фазы обработки исключений Windows и логика проекта Inspector/Protector.

Эта книга написана как связный курс, а не как словарь и не как набор карточек. Главная линия простая: сначала мы разбираем, как процессор, память, стек и вызов функции создают реальное состояние программы; потом смотрим, как Windows описывает это состояние в PE-файле; затем читаем таблицы unwind по байтам; после этого связываем все с обработкой исключений, отладчиками, crash dump analysis и твоим анализатором.

В книге остаются английские имена из документации Microsoft, потому что именно такие имена ты увидишь в WinAPI, PE-спецификации, дизассемблере, Ghidra, IDA, WinDbg и исходном коде проекта. Но термин не бросается в текст без расшифровки. Каждый важный термин объясняется через роль: что это, где лежит, кто этим пользуется, какая ошибка возникает при неправильном понимании и как это проверить руками.

Нормативная база курса - официальная документация Microsoft по PE Format, x64 exception handling, x64 calling convention, `RtlLookupFunctionEntry`, `RtlVirtualUnwind`, dynamic function tables, `RaiseException`, vectored exception handling и top-level exception filters. Книги уровня Eldad Eilam, Chris Eagle, Bruce Dang, Alexandre Gazet, Elias Bachaalany, Bryant/O'Hallaron и Tanenbaum используются здесь как ориентир по инженерному мышлению: читать систему слоями, доказывать выводы адресами и байтами, не путать удобную гипотезу с фактом.

Как устроено чтение

Если идти с середины, материал будет казаться лесом из сокращений. Поэтому курс построен лестницей:

```
байт -> адрес -> виртуальная память -> стек -> вызов функции ->
кадр стека -> x64 calling convention -> PE32+ -> RVA/VA ->
Exception Directory -> .pdata -> RUNTIME_FUNCTION -> .xdata ->
UNWIND_INFO -> UNWIND_CODE -> RtlLookupFunctionEntry ->
RtlVirtualUnwind -> обработчики -> отчет анализатора
```

Каждый следующий слой использует предыдущий. Например, `RUNTIME_FUNCTION` нельзя нормально понять без RVA. `UNWIND_CODE` нельзя понять без пролога функции. `RtlVirtualUnwind` нельзя понять без идеи "восстановить состояние вызывающей функции". А аномалии проекта нельзя объяснить на защите, если не отделять нарушение формата от подозрительного, но возможного решения компилятора или защитного рантайма.

Нулевой фундамент: память, адреса, стек и куча

Перед исключениями нужен базовый язык разговора. Программа в Windows не "просто выполняется". У процесса есть виртуальное адресное пространство: большой набор адресов, которые процесс видит как свои. Часть этих адресов сопоставлена с кодом PE-образа, часть с данными, часть со стеком потоков, часть с кучей, часть может быть вообще не сопоставлена. Если инструкция обращается к адресу, для которого нет допустимой страницы или нет нужного права доступа, процессор и ОС получают событие ошибки доступа. В Windows на уровне приложения это часто проявляется как exception code `0xC0000005`, то есть access violation.

Стек - область памяти, которая используется для короткоживущего состояния выполнения: адресов возврата, сохраненных регистров, локальных переменных, временных spill-значений и иногда аргументов сверх первых регистровых аргументов. На x64 Windows стек растет вниз: когда функция резервирует место, `RSP` обычно уменьшается. `RSP` - stack pointer, указатель на текущую вершину стека. Слово "вершина" здесь немного обманчиво: численно вершина часто имеет меньший адрес, чем старые данные выше по стеку.

Куча - область для объектов, срок жизни которых не обязан совпадать с одним вызовом функции. `malloc`, `new`, `HeapAlloc` и внутренние аллокаторы берут память именно оттуда или из похожих runtime-структур. Ошибка на куче может испортить указатель, объект или метаданные аллокатора. Ошибка на стеке может испортить локальные переменные, сохраненный регистр или адрес возврата. Для reverse engineering это различие важно: стек помогает восстановить "как мы сюда пришли", а куча часто помогает понять "какие объекты живут и кто ими владеет".

Адрес - это не магическое имя объекта, а число в выбранной адресной модели. В PE-файле ты часто видишь не абсолютные виртуальные адреса, а относительные виртуальные адреса, `RVA`. В памяти после загрузки тот же участок будет иметь `VA`, virtual address. В файле на диске у него будет file offset. Эти три числа могут описывать одни и те же байты, но отсчитываются от разных начал. Если перепутать их, анализатор начнет читать правильные структуры из неправильного места, и вся дальнейшая логика станет мусором.

Offset - смещение. Самое честное определение: это расстояние от заранее выбранной точки отсчета до нужного места. File offset отсчитывается от начала файла. RVA отсчитывается от image base, то есть от базы образа в памяти. Смещение внутри структуры отсчитывается от начала структуры. Поэтому фраза "offset равен 8" сама по себе неполная. Всегда надо спрашивать: смещение от чего? В `RUNTIME_FUNCTION` поле `BeginAddress` - это не file offset, а RVA начала функции. В `UNWIND_INFO` второй байт `SizeOfProlog` - это не адрес, а размер пролога в байтах от начала функции. Одно слово "offset" может встречаться в разных слоях, но смысл задает точка отсчета.

Регистр - маленькая быстрая ячейка состояния процессора. `RIP` хранит адрес текущей или следующей исполняемой инструкции в архитектурной модели. `RSP` связан со стеком. `RAX`, `RCX`, `RDX`, `R8` и другие регистры несут аргументы, результаты и временные значения. Для unwind особенно важны nonvolatile-регистры: если функция их меняет, она должна сохранить старое значение и восстановить его перед возвратом. Именно поэтому unwind metadata описывает push/save этих регистров: без этого Windows не сможет корректно восстановить контекст вызывающей функции.

Глава 1. База: что вообще происходит при исключении

В этой главе мы строим модель исключения не с WinAPI, а с обычного исполнения программы. Исключение становится понятным только тогда, когда видно, какое состояние было у процессора, где лежал стек и почему системе нужно подняться по кадрам, а не просто вызвать случайную функцию-обработчик.

1. Что такое исключение в Windows вообще

Exception в Windows - это формализованный способ сказать: при выполнении потока произошло событие, которое нельзя обработать как обычный возврат из функции. Слово "exception" переводят как исключение, но лучше держать в голове не бытовое "что-то редкое", а техническое "особый путь управления". Обычный путь управления выглядит так: инструкция выполнилась, `RIP` перешел к следующей инструкции, функция когда-нибудь сделала `ret`. Исключение ломает эту прямую дорогу: управление передается диспетчеру исключений, а он решает, есть ли обработчик, который согласен принять событие.

Источник исключения бывает разным. Иногда программа сама вызывает `RaiseException`, то есть явно просит ОС сгенерировать исключение с заданным кодом и аргументами. Иногда событие приходит снизу: доступ к недопустимой памяти, деление на ноль, illegal instruction, breakpoint, single step. Для исследователя важно не смешивать эти случаи. Вызов `RaiseException` - сознательная часть логики программы или runtime. Access violation - часто следствие обращения по адресу, который не разрешен текущей карте памяти. Но для диспетчера Windows оба случая превращаются в объект исключения и контекст потока.

Контекст потока - снимок регистров в момент события. Там есть `RIP`, который показывает точку, где произошло исключение, `RSP`, который показывает текущий стек, регистры аргументов, флаги и другое состояние. Windows не может обработать исключение без этого снимка, потому что ей нужно понять: в какой функции мы были, где находится ее кадр, как выйти на вызывающую функцию и какой обработчик связан с этим уровнем стека.

На x64 ключевая идея такая: обработчик ищется не по скрытой цепочке записей, вписанной в стек каждой функцией, а по таблицам, которые компилятор и linker положили в PE-образ. Поэтому исключения Windows x64 тесно связаны с `.pdata`, `.xdata`, `RUNTIME_FUNCTION` и `UNWIND_INFO`. Исключение - это не только "ошибка"; это момент, когда ОС проверяет, насколько честно код описал свои stack frames.

2. Что такое вызов функции, стек и кадр стека

Вызов функции на x64 - это не абстрактная стрелка в псевдокоде. Инструкция `call target` делает две вещи: кладет адрес возврата на стек и передает управление по адресу `target`. Адрес возврата - это адрес инструкции, которая должна выполниться после завершения функции. Когда функция выполнит `ret`, процессор достанет этот адрес со стека и продолжит выполнение у caller, то есть у вызывающей функции.

Перед `call` по Microsoft x64 calling convention первые четыре целочисленных или pointer-аргумента обычно идут в `RCX`, `RDX`, `R8`, `R9`, а floating-point аргументы - в `XMM0`-`XMM3`. Кроме того, caller резервирует 32 байта shadow space, или home space. Это место на стеке, куда callee при необходимости может "доместить" регистровые аргументы. Shadow space - частая причина, почему в простом прологе ты видишь `sub rsp, 28h`, а не только место под локальные

переменные: стек должен оставаться выровненным, и calling convention задает обязательную зону.

Кадр стека, stack frame, - это участок стека, который функция считает своим рабочим пространством во время выполнения. В нем может лежать сохраненный `RBP`, сохраненный `RBX`, локальный буфер, временные значения после register spilling, копии аргументов и служебные данные runtime. Границы кадра не всегда явно подписаны. Если функция использует frame pointer, например `RBP`, человеку проще видеть кадр. Если компилятор оптимизировал код и обошелся без `RBP`, кадр все равно существует логически, но его приходится восстанавливать через `RSP` и unwind data.

Для исключений важна не красота кадра, а возможность откатить действия пролога. Если функция в начале сделала `push rbx` и `sub rsp, 40h`, то при виртуальном подъеме к caller нужно сделать обратную картину: восстановить `RBX` из стека, увеличить `RSP`, достать return address и получить контекст вызывающей функции. `UNWIND_INFO` как раз и говорит системе: вот какие действия пролог сделал со стеком и nonvolatile- регистрами, поэтому вот как их надо обратить.

Миниатюрная схема:

```
caller:
    sub  rsp, 20h        ; shadow space для вызова
    mov  rcx, arg1
    call callee          ; на стек попал return address
    add  rsp, 20h

callee:
    push rbx             ; сохранить nonvolatile RBX
    sub  rsp, 40h        ; место под локальные данные
    ...
    add  rsp, 40h
    pop  rbx
    ret
```

Если исключение случилось внутри `callee`, Windows должна понять эту маленькую историю задом наперед. Без unwind metadata это было бы гадание по стеку.

3. Как Windows ищет обработчик по кадрам

Когда исключение возникло, Windows начинает не с того, что сразу завершает процесс. Она пытается найти обработчик. Обработчик - это код, который говорит диспетчеру: "я понимаю это исключение и могу продолжить или корректно размотать выполнение". На x64 поиск связан с кадрами стека: сначала рассматривается функция, где произошло исключение, потом ее caller, потом caller caller-a и так далее.

Чтобы перейти от текущей функции к вызывающей, системе нужен unwind. Unwind буквально означает "размотать", но технически это не уничтожение стека, а вычисление предыдущего контекста. `RtlVirtualUnwind` берет текущий `RIP`, текущий context record и запись `RUNTIME_FUNCTION`, читает `UNWIND_INFO`, затем вычисляет, каким был бы контекст caller. Это называется virtual unwind, потому что он может вычислять состояние кадра без немедленного реального выполнения всех destructors и handlers.

Поиск обработчика обычно идет в search phase. В этой фазе система спрашивает у frame-based handlers: "вы берете это исключение?" Если обработчик возвращает, что он не занимается событием, диспетчер поднимается выше. Если подходящий обработчик найден, начинается unwind phase: теперь Windows реально проходит обратно по кадрам, вызывает нужные termination/unwind handlers, освобождает уровни стека и передает управление найденному обработчику или продолжению.

Важно не путать "нашли запись функции" и "нашли обработчик". Запись `RUNTIME_FUNCTION` нужна почти для каждого non-leaf кадра, потому что без нее нельзя подняться по стеку. Но наличие записи не означает, что функция ловит исключения. За обработчик отвечают флаги в `UNWIND_INFO`: `UNW_FLAG_EHANDLER` и `UNW_FLAG_UHANDLER` указывают, что после unwind codes есть handler RVA и language-specific data. Если флагов нет, функция может быть важна для stack trace, но не обязана перехватывать исключение.

Отсюда рождается центральная связь ВКР: если `.pdata/.xdata` повреждены, искусственно изменены или ведут себя странно, это влияет не только на "табличку в файле". Это может влиять на поиск обработчиков, построение stack trace, crash dump analysis, работу отладчика и точность вывода reverse engineer-а.

4. Чем x64-модель отличается от x86-модели

На 32-битной Windows классическая SEH-модель часто объясняется через linked list записей на стеке. Упрощенно: функция, которая хочет иметь SEH-обработчик, создает registration record, связывает его с предыдущей записью, а указатель на вершину цепочки доступен через TEB. В старых материалах это часто связывают с `FS:[0]`. Для реверса x86 это важная тема: можно видеть установку обработчика прямо в инструкциях функции.

На x64 Windows основная модель другая: table-based exception handling. Информация о диапазонах функций и способе unwind хранится в таблицах образа и dynamic function tables. Вместо того чтобы каждая функция строила на стеке явную linked list для SEH, компилятор описывает функцию в `.pdata/.xdata`. Поэтому исследователь x64 не должен искать x86-шаблоны и делать вывод "SEH нет", если не видит старую цепочку. Надо читать Exception Directory.

Различие не косметическое. В x86 часто можно восстановить обработчик через код установки записи. В x64 нужно уметь прочитать `RUNTIME_FUNCTION`, найти `UNWIND_INFO`, посмотреть flags, handler RVA и language-specific data. Плюс x64-компилятор накладывает строгие правила на prolog/epilog, потому что unwind должен быть возможен по компактному описанию, а не по полноценному исполнению функции.

Для твоего проекта это принципиально: Inspector/Protector анализирует именно x64 PE32+ AMD64 слой. Если на защите спросить "почему вы не читаете цепочку SEH на стеке?", ответ должен быть спокойным: потому что в x64 Windows frame-based exception handling опирается на таблицы runtime functions и unwind metadata; стековые x86-приемы здесь не являются основной моделью.

Глава 2. База PE: в каком файле все это живет

Теперь переносим разговор из абстрактного стека в PE-файл. Windows должна загрузить образ, сопоставить секции с памятью, найти каталоги данных и понять, где лежит Exception Directory. Ошибка в этом слое делает невозможным весь последующий разбор.

5. Что такое PE-файл и как он устроен в общем виде

PE, Portable Executable, - основной формат исполняемых файлов, DLL и некоторых системных образов Windows. Слово portable здесь историческое: формат описывает образ так, чтобы loader мог разместить его в памяти, настроить импорты, релокации, права секций и служебные каталоги. PE - это не просто "файл с кодом". Это договор между linker-ом, loader-ом, отладчиком, ОС и инструментами анализа.

В начале PE обычно есть DOS header с сигнатурой `MZ`. В поле `e_lfanew` хранится file offset до NT headers, где находится сигнатура `PE\0\0`, COFF File Header и Optional Header. Optional Header называется "optional" исторически, но для executable image он фактически обязателен: там лежат `ImageBase`, `SectionAlignment`, `FileAlignment`, `SizeOfImage`, entry point, таблица Data Directory и другие поля.

После заголовков идут section headers. Секция - это описание области образа: имя, virtual address, virtual size, file offset сырого содержимого, raw size и характеристики вроде executable, readable, writable. `.text` обычно содержит код, `.rdata` - read-only данные, `.pdata` - таблицу runtime functions для x64, `.xdata` или `.rdata` - unwind structures. Но имена секций не являются абсолютной гарантией. Инструмент должен смотреть не только на имя, но и на адреса, размеры и flags.

PE-файл живет в двух представлениях. На диске секции лежат с выравниванием `FileAlignment`; в памяти loader размещает их с выравниванием `SectionAlignment`. Поэтому нельзя читать RVA как file offset напрямую. Нужно найти секцию, в чей virtual диапазон попадает RVA, и пересчитать:

```
file_offset = PointerToRawData + (RVA - VirtualAddress)
```

Именно эту работу делает слой `pe.c` в проекте. Он строит карту секций, проверяет выходы за пределы файла/образа, учитывает Exception Directory и дает остальным модулям безопасную функцию "RVA -> file offset".

6. Что такое PE32+ и почему для x64 это важно

PE32+ - вариант PE Optional Header для 64-битных образов. Плюс в названии не означает "лучше", он означает другой layout заголовка. Для AMD64 `Machine` обычно равен `0x8664`, а `Magic` Optional Header равен `0x20b`. В PE32+ поле `ImageBase` 64-битное, некоторые размеры и адресные поля устроены иначе, а `BaseOfData`, привычное по PE32, отсутствует.

Почему это важно для unwind? Потому что Windows x64 exception handling описан для 64-битной модели выполнения: регистры `RAX-R15`, 64-битный `RIP`, 64-битный `RSP`, Microsoft x64 calling convention, table-based unwind. Если инструмент случайно примет PE32 за PE32+ или наоборот, он будет читать заголовки со сдвигом, неверно найдет Data Directory и неправильно интерпретирует Exception Directory.

В `RUNTIME_FUNCTION` поля остаются 32-битными RVA, даже в 64-битном образе. Это не противоречие. PE-образ имеет base address, а большинство внутренних ссылок хранятся как image-relative 32-bit values. VA получается добавлением `ImageBase`. Такой формат компактнее и удобнее для relocation. Но сам контекст выполнения, который unwinder восстанавливает, содержит 64-битные регистры.

Для проекта правильная проверка PE32+ - фильтр применимости. Если файл не AMD64 PE32+, анализ x64 unwind metadata не должен притворяться универсальным. Он может сообщить "неподдерживаемая архитектура", но не должен выдавать псевдовыводы. Это часть инженерной честности инструмента.

7. Что такое RVA, VA, Image Base и секции

`ImageBase` - предпочтительный базовый адрес, по которому loader пытается разместить образ в виртуальной памяти процесса. `VA`, virtual address, - реальный адрес в виртуальном адресном пространстве процесса. `RVA`, relative virtual address, - смещение от ImageBase до объекта внутри образа. Формула простая:

```
VA = ImageBase + RVA
RVA = VA - ImageBase
```

Но простая формула часто ломает новичка, потому что рядом есть file offset. File offset - позиция байта в файле на диске. RVA - позиция объекта в загруженном образе. Между ними стоит section table. Например, секция `.text` может иметь `VirtualAddress = 0x1000`, `PointerToRawData = 0x400`. Тогда RVA `0x1234` находится на `0x234` байта от начала секции в памяти, а file offset будет `0x400 + 0x234 = 0x634`. Если прочитать файл по offset `0x1234`, ты попадешь в другое место.

Секции задают не только перевод адресов, но и контекст. Если `unwind_rva` указывает в `.rdata`, это обычно спокойно: metadata read-only. Если unwind metadata лежит в executable section, это не автоматическое доказательство вредоносности, но вопрос: почему служебная таблица попала туда, где должен исполняться код? Если handler RVA указывает в non-executable section, это сильный структурный сигнал: обработчик как код должен быть исполнимым, иначе нормальный вызов невозможен.

В проекте `pe_find_section_for_rva` и `pe_rva_to_fileoff` - не вспомогательная мелочь, а фундамент. Все дальнейшие проверки `rf_oob`, `unwind_rva_oob`, handler placement и writable/executable metadata зависят от того, правильно ли построена карта секций.

8. Что такое Data Directory и где там Exception Directory

Data Directory - таблица указателей на специальные области PE-образа. В Optional Header есть массив из 16 entries, где каждая entry обычно содержит RVA и size. Там находятся export table, import table, resource table, relocation table, debug directory и другие каталоги. Для нас ключевой entry - `IMAGE_DIRECTORY_ENTRY_EXCEPTION`, индекс 3.

В PE32+ AMD64 Exception Directory указывает на массив записей `RUNTIME_FUNCTION`. На практике он часто лежит в секции `.pdata`. Размер directory должен быть кратен 12 байтам, потому что одна запись `RUNTIME_FUNCTION` состоит из трех 32-битных полей: BeginAddress, EndAddress, UnwindData. Если размер не кратен 12, это уже структурная проблема: таблицу нельзя полностью разложить на записи без остатка.

Важно понимать: Data Directory дает вход в таблицу, но не объясняет каждую функцию. Он говорит "здесь находится exception table". Дальше нужно читать записи, проверять диапазоны, сортировку, пересечения, выходы за `SizeOfImage`, и только потом переходить к `.xdata`. Нельзя прыгать сразу к `UNWIND_INFO`, пока не доказано, что сам путь через PE корректен.

Если Exception Directory отсутствует, это не всегда катастрофа. Образ может состоять из leaf functions или иметь специфическую структуру. Но для обычного x64 executable с нетривиальными функциями отсутствие `.pdata` ограничивает stack walking и exception handling. Поэтому проект не должен кричать "вредоносно" только из-за отсутствия таблицы, но должен объяснять, что именно

невозможно проверить и почему.

Глава 3. Табличная модель x64: откуда Windows берет данные для unwind

Здесь начинается ядро темы. `.pdata` и `.xdata` - это не "дополнительные секции для компилятора", а карта, по которой Windows восстанавливает стек.

9. Что такое .pdata и зачем она нужна

`.pdata` обычно содержит массив `RUNTIME_FUNCTION` для x64. Каждая запись говорит: для диапазона кода от `BeginAddress` до `EndAddress` unwind information находится по адресу `UnwindData`. Диапазон считается half-open: начало включено, конец не включен. То есть `RIP` принадлежит функции, если `BeginAddress <= RIP_RVA < EndAddress`.

Назначение `.pdata` - быстро ответить на вопрос "какой функции принадлежит текущий instruction pointer?". Когда исключение случилось по адресу `RIP`, `RtlLookupFunctionEntry` ищет в function table запись, чей диапазон покрывает этот адрес. Если запись найдена, можно читать `.xdata` и делать unwind. Если запись не найдена, Windows может рассматривать функцию как leaf function и попытаться подняться через return address, но это работает только при ограниченных условиях.

Для reverse engineer-а `.pdata` ценна еще и как независимый источник границ функций. Дизассемблер может ошибиться в function discovery, особенно в обфусцированном коде. Таблица runtime functions показывает, какие диапазоны важны для Windows. Это не абсолютная истина о "логических функциях" исходного кода, но сильный технический факт: loader/runtime будут использовать именно эти диапазоны для unwind.

В твоем анализаторе `.pdata` превращается в массив `RuntimeFunction`. Если записи несортированы, пересекаются, имеют нулевой или отрицательный размер, указывают за пределы образа или дублируют начало, это уже не "странный стиль". Это нарушение табличной модели, которое влияет на lookup и может ломать или маскировать stack walking.

10. Что такое RUNTIME_FUNCTION и как читать ее поля

`RUNTIME_FUNCTION` в x64 PE - структура из трех 32-битных image-relative полей:

```
typedef struct _RUNTIME_FUNCTION {
    DWORD BeginAddress;
    DWORD EndAddress;
    DWORD UnwindData;
} RUNTIME_FUNCTION;
```

`BeginAddress` - RVA начала диапазона функции. `EndAddress` - RVA конца диапазона, причем конец не включается. `UnwindData` - RVA структуры `UNWIND_INFO`, если запись не использует специальную форму chained information. Все три поля в файле little-endian, то есть байты младшей части числа идут первыми.

Ручное чтение одной записи:

```
Байты:
00 10 00 00   50 10 00 00   00 30 00 00

BeginAddress = 0x00001000
EndAddress   = 0x00001050
UnwindData   = 0x00003000
```

Это значит: если `RIP_RVA` находится от `0x1000` включительно до `0x1050` не включительно, нужно читать `UNWIND_INFO` по RVA `0x3000`. Чтобы физически прочитать байты из файла, `0x3000` надо перевести через section table в file offset. Если `0x3000` не попадает ни в одну секцию или выходит

за raw data, `unwind_rva_oob` - не придирка, а невозможность безопасно прочитать структуру.

Нормальная запись должна иметь `BeginAddress < EndAddress`. Если начало равно концу, диапазон пустой. Если начало больше конца, lookup становится логически бессмысленным. Если диапазоны пересекаются, возникает неоднозначность: один `RIP` может подходить сразу под две записи. Windows ожидает корректную function table; reverse engineer обязан отмечать такие места как структурный риск.

11. Что такое .xdata и что там лежит

`.xdata` - условное имя области, где лежат структуры unwind information. В реальных файлах эти структуры могут находиться и в `.rdata`; важно не имя секции, а то, что `UnwindData` из `RUNTIME_FUNCTION` указывает на корректную `UNWIND_INFO`. `.pdata` отвечает на вопрос "какой диапазон кода?", `.xdata` отвечает на вопрос "как размотать кадр этой функции?".

В `.xdata` лежит не машинный код, а компактное описание действий пролога: сколько байтов занимает prolog, какие nonvolatile-регистры были сохранены, сколько места выделено на стеке, используется ли frame register, есть ли exception/unwind handler или chained information. Это описание не равно дизассемблированному прологу один в один, но должно быть с ним согласовано по смыслу.

Почему нельзя просто дизассемблировать пролог вместо `.xdata`? Потому что unwinder должен работать надежно и быстро в сложных ситуациях: исключение может произойти в середине функции, отладчик может строить stack trace, crash dump может анализироваться без выполнения кода. Табличное описание дает системе формальную грамматику действий, которую можно читать без догадок. Дизассемблер нужен человеку, а `.xdata` - runtime-у.

Для анализа защищенного или обфусцированного PE `.xdata` часто дает интересные несоответствия: таблица может указывать в необычную секцию, `UNWIND_INFO` может обрываться, один и тот же блок может переиспользоваться слишком многими функциями, handler может доминировать. Такие признаки надо объяснять аккуратно: это не мгновенный verdict, а evidence, то есть инженерное свидетельство для дальнейшей проверки.

12. Что такое UNWIND_INFO и его общая идея

`UNWIND_INFO` - структура, которая описывает, как отменить действия пролога функции. Ее первый байт содержит version и flags. Затем идут `SizeOfProlog`, `CountOfCodes`, packed-поле `FrameRegister/FrameOffset`, массив `UNWIND_CODE`, возможное выравнивание до четного количества slots, а затем handler RVA или chained `RUNTIME_FUNCTION`, если этого требуют flags.

Смысл структуры можно сформулировать так: "когда выполнение находится внутри этой функции, вот что нужно знать, чтобы восстановить состояние caller-a". Если функция сделала `push rbx`, unwind code скажет, что `RBX` лежит на стеке и при unwind его надо вернуть. Если функция сделала `sub rsp, 40h`, unwind code скажет, что при подъеме нужно прибавить к `RSP` эти 0x40 байт. Если функция установила frame pointer, структура объяснит, какой регистр играет эту роль и с каким scaled offset.

Упрощенный пример:

```
01 04 01 00   04 42   00 00
```

Первый байт `0x01`: version 1, flags 0. Второй байт `0x04`: prolog size 4 байта. Третий `0x01`: один unwind code slot. Четвертый `0x00`: frame register отсутствует. Slot `04 42`: code offset 4, opcode 2 (`UWOP_ALLOC_SMALL`), opinfo 4. Размер allocation считается как `opinfo * 8 + 8`, то есть `4 * 8 + 8`

= 40 байт. Иначе говоря, пролог сделал маленькое выделение стека на 40 байт, например sub rsp, 28h.

Вот почему UNWIND_INFO надо читать медленно. Один байт может быть разделен на bit fields, один slot может требовать дополнительных slots, а после массива могут идти данные разного смысла в зависимости от flags. Ошибка на один slot сдвигает весь разбор и превращает handler RVA в случайные байты.

Глава 4. Как функция описывается для unwind

Теперь связываем машинный код функции с таблицей. Пролог, эпилог, calling convention и классы регистров - это не отдельные темы, а одна механика.

13. Что такое пролог функции и почему он описывается отдельно

Пролог функции - начальный участок, где функция подготавливает свой кадр: сохраняет nonvolatile-регистры, резервирует стек, иногда задает frame pointer. Пролог важен для unwind потому, что именно он меняет состояние так, что простой `ret` уже не объясняет, где находится caller. Если функция ничего не меняет и сразу делает вычисления в volatile- регистрах, unwind почти не нужен. Если функция двигает `RSP` и сохраняет регистры, системе нужен план обратного восстановления.

Microsoft x64 накладывает ограничения на prolog/epilog: действия, влияющие на unwinding, должны быть описуемы через набор unwind opcodes. Это не прихоть документации. Представь, что исключение произошло после `push rbx`, но до `sub rsp, 80h`. Состояние стека отличается от состояния после полного пролога. `SizeOfProlog` и code offsets помогают unwinder-у понять, какие действия уже успели выполниться, если control PC находится внутри prolog.

Типичный пролог:

```
push rbx
push rbp
sub  rsp, 80h
lea  rbp, [rsp+20h]
```

Здесь `RBX` и `RBP` сохраняются, `RSP` уменьшается на `0x80`, а `RBP` становится frame register с некоторым offset. `UNWIND_INFO` не хранит строку assembly. Он хранит компактные операции: `PUSH_NONVOL RBX`, `PUSH_NONVOL RBP`, `ALLOC_LARGE` или `ALLOC_SMALL`, `SET_FPREG`. Человек видит инструкции, Windows видит обратимые действия над контекстом.

Когда проект связывает disassembly prolog с `UNWIND_CODE`, он проверяет не "красоту" компиляции, а соответствие договора: metadata должна объяснять реальные изменения кадра. Если metadata говорит, что сохранен `RBX`, а пролог этого не делает, stack walking может восстановить ложное значение.

14. Что такое эпилог функции и почему он тоже важен

Эпилог - участок, где функция возвращает кадр в исходное состояние: освобождает stack allocation, восстанавливает nonvolatile-регистры и выполняет `ret` или tail jump. На первый взгляд эпилог можно не описывать: ведь `UNWIND_INFO` хранит действия пролога, а обратные действия вроде бы очевидны. Но для unwinder-а важно распознать, что control PC находится в эпилоге, где часть обратных действий уже могла выполниться.

Пример:

```
add rsp, 80h
pop rbp
pop rbx
ret
```

Если исключение или отладочное событие попало между `add rsp, 80h` и `pop rbp`, текущий `RSP` уже не соответствует "тело функции после пролога". Unwinder должен учитывать, что мы находимся на пути выхода. Поэтому x64 ABI ограничивает форму эпилогов: они должны быть распознаваемыми, без произвольных инструкций между восстановлением `RSP`, pops и transfer of control.

В классической UNWIND_INFO v1 эпилоги не перечисляются так же явно, как пролог, но runtime может распознавать допустимые эпилогические последовательности. В новых и расширенных случаях встречаются дополнительные сведения, например epilog descriptors в связанных форматах. Для ВКР достаточно понять базу: эпилог важен, потому что unwind должен быть корректным не только в середине тела функции, но и на участке выхода.

Для reverse engineering эпилог помогает проверять размер кадра. Если prolog сделал `sub rsp, 80h`, нормальный epilog должен вернуть этот размер. Если дизассемблер показывает странную асимметрию, надо спросить: это оптимизация, multiple exits, tail call, obfuscation или ошибка анализа границ функции?

15. Что такое UNWIND_CODE и как он кодирует действия пролога

UNWIND_CODE - элемент массива внутри UNWIND_INFO. В документации он выглядит маленьким, но смысл у него плотный. Один slot обычно занимает два байта: CodeOffset и байт, разделенный на UnwindOp и OpInfo. CodeOffset говорит, на каком смещении от начала prolog-а произошло действие. UnwindOp говорит тип действия. OpInfo уточняет регистр, размер или режим.

Важно: массив unwind codes хранится в порядке, удобном для unwind, а не обязательно в том виде, как новичок ожидает по листингу. При ручном разборе нужно не "угадывать", а применять правила формата: прочитать count, затем идти по slots, учитывая, что некоторые opcodes занимают дополнительные slots. Например UWOP_ALLOC_LARGE может занимать 2 или 3 slots в зависимости от режима, а UWOP_SAVE_NONVOL хранит offset в следующем slot.

Пример slot `04 42`:

```
CodeOffset = 0x04
второй байт = 0x42
low nibble  = 0x2 -> UWOP_ALLOC_SMALL
high nibble = 0x4 -> OpInfo
size = 4 * 8 + 8 = 40 bytes
```

Здесь low nibble - младшие 4 бита, high nibble - старшие 4 бита. Это хороший пример, почему "байт" не всегда читается как одно число. В бинарных форматах один байт часто упаковывает два маленьких поля. Если перепутать nibbles, ALLOC_SMALL превратится в другой opcode, и весь вывод станет ложным.

В проекте decoder обязан быть bounds-safe: перед чтением дополнительных slots он проверяет, что они существуют. Если CountOfCodes говорит, что slots мало, а opcode требует operand, это UA_TRUNCATED, а не повод читать за пределами буфера.

16. Все основные UWOP_* операции по одной и очень подробно

UWOP_PUSH_NONVOL описывает push nonvolatile-регистра. Если prolog сделал `push rbx`, то при unwind нужно прочитать старый RBX из стека и увеличить RSP на 8. OpInfo хранит номер регистра. Операция важна не потому, что push часто встречается, а потому, что nonvolatile-регистр принадлежит caller-у по договору ABI: callee может временно использовать его, но обязан вернуть значение.

UWOP_ALLOC_SMALL описывает небольшое выделение места на стеке. Размер вычисляется как `OpInfo * 8 + 8`. Если `OpInfo = 4`, размер равен 40 байт. При unwind это действие отменяется прибавлением того же размера к RSP. UWOP_ALLOC_LARGE нужен для больших allocations. У него есть mode: в одном режиме следующий slot хранит размер, деленный на 8, в другом используются два slots для полного значения. Поэтому decoder не может читать ALLOC_LARGE как один slot.

UWOP_SET_FPREG говорит, что функция установила frame register. Сам регистр и scaled offset лежат не в этом slot, а в header UNWIND_INFO. Frame pointer нужен, когда доступ к локальным данным удобнее вести от стабильной базы, а не от постоянно меняющегося RSP. На x64 RBP не обязан всегда быть frame pointer, но если metadata заявляет frame register, должен быть понятен момент его установки.

UWOP_SAVE_NONVOL и UWOP_SAVE_NONVOL_FAR описывают сохранение nonvolatile-регистра через store, а не push. Например компилятор может сделать `mov [rsp+30h], rsi`. Обычная форма хранит offset, масштабируемый на 8, far-форма хранит более широкий offset. При unwind нужно восстановить регистр из соответствующего stack slot. Ошибка в offset опасна: регистр будет восстановлен из чужой локальной переменной или мусора.

UWOP_SAVE_XMM128 и UWOP_SAVE_XMM128_FAR аналогичны, но относятся к nonvolatile XMM-регистрам, например XMM6-XMM15. Размер единицы - 16 байт, потому что XMM-регистр 128-битный. Многие учебники для новичков пропускают XMM, но для корректного ABI это не второстепенная тема: floating-point/vector state тоже может быть частью контракта caller/callee.

UWOP_PUSH_MACHFRAME описывает machine frame, связанный с исключением или interrupt-подобным механизмом. Это более специальный случай: на стеке лежит системно сформированный frame, иногда с error code. Для обычных пользовательских функций чаще встречаются push/save/alloc, но decoder должен знать и эту операцию, иначе он будет слеп на часть допустимого формата.

Хороший decoder не просто печатает имена UWOP. Он связывает каждую операцию с эффектом на RSP, сохраненными регистрами, handler flags, parse errors и возможными ABI warnings. Именно так `unwind.c` в проекте формирует `fixed_stack_bytes`, masks сохраненных регистров и bitmask anomalies.

17. Что такое frame register и frame pointer на x64

Frame pointer - регистр, который функция использует как стабильную базу для доступа к своему кадру. Исторически многие привыкли к RBP: локальные переменные лежат по отрицательным смещениям от RBP, аргументы - по положительным. На оптимизированном x64 это не обязательно. Компилятор может использовать RSP-relative addressing и освободить RBP под обычные вычисления.

Frame register в UNWIND_INFO - формальная запись о том, что такой стабильный регистр есть. Поле frame register хранит номер регистра, а frame offset хранится в масштабе 16 байт. Если frame register равен нулю, функция не заявляет frame pointer. Если не ноль, в unwind codes должна быть операция UWOP_SET_FPREG, иначе metadata говорит о frame register, который фактически не установлен.

Зачем offset масштабируется? Формат экономит место. Вместо хранения произвольного байтового значения он хранит небольшое число, умножаемое на 16. Это соответствует выравниванию и типичным layout-ам кадров. Но для ручного разбора это ловушка: `FrameOffset = 2` не означает "2 байта", а означает 32 байта.

Для reverse engineer-а отсутствие frame pointer - не аномалия. Это нормальная оптимизация. Аномалией становится противоречие: metadata заявляет frame register, но unwind code не устанавливает его; или decoder видит зарезервированный register class; или disassembly явно расходится с описанием. Правильный вывод всегда строится на связи "что заявлено" -> "что возможно по ABI" -> "что видно в коде".

18. Что такое volatile и nonvolatile регистры

Volatile-регистр - регистр, значение которого caller не имеет права считать сохраненным после вызова функции. Callee может свободно его испортить. В Microsoft x64 к volatile general-purpose относятся `RAX`, `RCX`, `RDX`, `R8`, `R9`, `R10`, `R11`; среди XMM - обычно `XMM0`- `XMM5`. Это не "плохие" регистры, а рабочие временные регистры по ABI.

Nonvolatile-регистр - регистр, который callee обязан вернуть в прежнем состоянии, если он его менял. Сюда относятся `RBX`, `RBP`, `RDI`, `RSI`, `R12`-`R15`, а также `XMM6`-`XMM15`. `RSP` имеет особый статус: стек должен быть восстановлен к ожидаемому состоянию при возврате. Именно nonvolatile-регистры появляются в unwind metadata, потому что unwind должен восстановить состояние caller-а.

Пример: caller хранит важный указатель в `RBX` и вызывает функцию. Если callee хочет использовать `RBX`, она делает `push rbx` или store в свой stack frame. Потом перед возвратом восстанавливает `RBX`. Если в середине функции произошло исключение, обычного возврата не будет, но caller все равно ожидает корректный `RBX` при обработке выше по стеку. Поэтому `UNWIND_INFO` обязан объяснить, где лежит старое значение.

В проекте `saved_nonvol_mask` и `saved_xmm_mask` - это не декоративные поля. Они показывают, какие части calling convention задействованы функцией. A strict ABI warnings помогают отделять формально странные записи от реально опасных parse failures.

19. Что такое x64 calling convention и как она связана с unwind

Calling convention - соглашение о том, как функции передают аргументы, возвращают результат, кто сохраняет какие регистры и как должен выглядеть стек в точках вызова. На Windows x64 первые четыре аргумента идут через `RCX`, `RDX`, `R8`, `R9` для integer/pointer и через `XMM0`-`XMM3` для floating-point. Caller резервирует shadow space 32 байта. Стек должен быть выровнен на 16 байт в большинстве точек вне prolog/epilog.

Unwind связан с calling convention напрямую. ABI говорит, какие регистры callee обязан сохранить; unwind metadata говорит, как именно конкретная функция это сделала. ABI говорит, что стек должен вернуться к корректному состоянию; unwind metadata описывает, сколько байт функция выделила и как это отменить. ABI ограничивает форму prolog/epilog; unwind metadata ссылается на эти ограничения.

Если не знать calling convention, легко сделать ложный вывод. Например, увидеть `sub rsp, 28h` и решить, что функция имеет 40 байт локальных переменных. На самом деле часть этого размера связана с выравниванием и shadow space. Или увидеть, что `RCX` "испорчен" после вызова, и считать это ошибкой, хотя `RCX` volatile. Или удивиться отсутствию `RBP`, хотя frame pointer omission нормален.

Для ВКРсильная формулировка такая: проект анализирует unwind metadata, а unwind metadata является машинно-читаемым отражением соглашения ABI для конкретных функций PE32+ AMD64. Поэтому анализ `.pdata/.xdata` - это не отдельная игрушка, а способ проверить, как бинарь описывает свои кадры стека с точки зрения Windows runtime.

Глава 5. Как Windows реально поднимается по стеку

Эта глава описывает центральный runtime-процесс: по текущему RIP найти запись функции, прочитать metadata и получить контекст caller-а.

20. Как система по RIP находит запись функции

Когда у системы есть RIP, ей сначала нужно перевести его в ControlPc внутри некоторого image. Если адрес принадлежит модулю, у модуля есть base address. Тогда RIP_RVA = RIP - ImageBase. Именно этот RVA можно сравнивать с BeginAddress и EndAddress в RUNTIME_FUNCTION.

Поиск работает по function table. Для статического PE эта таблица пришла из Exception Directory. Для JIT или runtime-generated code таблица может быть зарегистрирована динамически. Запись считается найденной, если control PC попадает в half-open диапазон. Если несколько записей перекрываются, модель становится неоднозначной. Поэтому пересечения в .pdata важны не как "неаккуратность", а как потенциальная поломка lookup.

Если запись не найдена, x64 unwinder может трактовать функцию как leaf function: она не меняла nonvolatile-регистры и не выделяла сложный stack frame, поэтому caller можно получить через return address на стеке. Но это предположение безопасно не всегда. Если функция реально меняла стек, а записи нет, stack trace может сломаться.

Для ручного анализа полезно задавать три вопроса: какой module base? какой RIP_RVA? какая запись .pdata покрывает этот RVA? Пока эти три ответа не получены, разговор о UNWIND_INFO преждевременен.

21. Что делает RtlLookupFunctionEntry

RtlLookupFunctionEntry - функция Windows, которая по ControlPc ищет соответствующую function table entry и возвращает RUNTIME_FUNCTION вместе с image base. На уровне идеи она отвечает на вопрос: "какая запись описывает функцию, где сейчас находится выполнение?".

Для обычного загруженного модуля поиск опирается на exception directory образа. Для dynamic function tables учитываются таблицы, зарегистрированные через RtlAddFunctionTable или callback-механизм. Поэтому статический анализ PE не всегда видит всю runtime-картину: JIT, protectors и runtime code generation могут добавить function entries после загрузки.

Результат RtlLookupFunctionEntry - еще не unwind. Это адрес записи. Следующий шаг - RtlVirtualUnwind, который использует эту запись и контекст. Если lookup вернул null, обработчик stack walking должен перейти к leaf-логике или сообщить ограничение. Ошибка новичка - считать, что отсутствие записи означает отсутствие функции. На самом деле это означает отсутствие табличного описания для данного control PC.

В проекте статический аналог этой функции - поиск по массиву RuntimeFunction. Он не вызывает WinAPI в static mode, но проверяет те же предпосылки: диапазон функции, границы образа, сортировка, отсутствие перекрытий и корректность unwind_rva.

22. Что делает RtlVirtualUnwind

RtlVirtualUnwind - сердце механики. Она берет HandlerType, ImageBase, ControlPc, RUNTIME_FUNCTION, context record и возвращает обновленный context, который соответствует caller frame. Кроме того, она может вернуть handler data и establisher frame, если текущая функция имеет обработчик.

"Virtual" означает, что функция вычисляет результат unwind, а не просто выполняет машинные инструкции эпилога. Она читает `UNWIND_INFO`, применяет обратные эффекты unwind operations, восстанавливает nonvolatile-регистры, корректирует `RSP`, извлекает return address и обновляет `RIP`. На выходе context выглядит так, как будто текущая функция уже вернулась к caller-у.

Пример логики:

```
Было внутри callee:
RSP -> локальный кадр callee
RIP -> адрес внутри callee

UNWIND_INFO говорит:
сохранен RBX, выделено 0x40 байт стека

После virtual unwind:
RBX восстановлен из stack slot
RSP указывает на caller frame
RIP равен return address
```

Для exception handling это позволяет пройти кадр за кадром. Для отладчика это позволяет построить call stack. Для crash dump analysis это позволяет понять путь к падению. Для твоего проекта понимание `RtlVirtualUnwind` объясняет, почему битая `UNWIND_INFO` не мелкая ошибка парсинга, а нарушение механизма восстановления контекста.

23. Что такое EstablisherFrame

`EstablisherFrame` - значение, связанное с кадром функции, чей обработчик рассматривается. Грубо говоря, это frame base, через который language- specific handler может обращаться к данным своего кадра и scope table. Точное вычисление зависит от frame register и формы unwind, но смысл один: обработчику нужен якорь в стеке текущей функции.

Почему это важно? Обработчик вроде `__C_specific_handler` не живет в пустоте. Он должен понять, какие `__try/__except` или `__finally` области относятся к текущему кадру, где лежит scope data и какие условия применимы. `EstablisherFrame` помогает связать абстрактную запись обработчика с конкретным stack frame.

Если frame metadata повреждена, establisher frame может быть вычислен неправильно. Тогда обработчик получит неверную базу и начнет читать неправильные данные. В обычном корректном бинаре это не тема для волнения, но в reverse engineering защищенных образцов такие связи становятся интересными: неправильный frame может маскировать реальные control-flow paths или ломать анализ.

Для ВКРне надо притворяться, что проект полностью эмулирует Windows dispatcher. Достаточно правильно объяснить: `EstablisherFrame` - часть runtime-контекста обработчика, а корректность его вычисления зависит от тех же unwind structures, которые анализирует проект.

24. Что такое DispatcherContext

`DispatcherContext` - структура контекста, которую Windows передает language-specific handler-у во время обработки исключения. Она содержит сведения о текущей function entry, image base, control PC, handler data, establisher frame и другую информацию, нужную обработчику для решения.

Если говорить человечески, dispatcher context - это папка с материалами по текущему кадру. Handler не должен заново искать `.pdata`, считать base address и угадывать scope table. Диспетчер уже собрал нужные данные и передает их в стандартизованном виде.

Для `__C_specific_handler` dispatcher context особенно важен: handler должен пройти language-specific scope information и решить, есть ли подходящий filter, finally или unwind action.

Поэтому flags в `UNWIND_INFO` ведут не просто к адресу функции-обработчика, а к связке handler + handler data + frame context.

В статическом анализаторе ты не создаешь настоящий `DispatcherContext`, но можешь восстановить его источники: `RUNTIME_FUNCTION`, `UNWIND_INFO`, handler RVA, chained info, placement секций. Это дает объяснимый мост между байтами PE и поведением Windows dispatcher.

Глава 6. Фазы обработки исключения

Теперь разбираем путь исключения как процесс: поиск, размотка, language-specific обработчики, VEH и финальная необработанная ошибка.

25. Что такое search phase и unwind phase

Search phase - фаза поиска обработчика. Система идет по кадрам стека и спрашивает связанные обработчики, готовы ли они обработать исключение. В этой фазе стек еще не обязан реально разрушаться. Главная задача - найти место, которое принимает ответственность за событие.

Unwind phase - фаза реальной размотки после того, как цель найдена или принято решение завершать выполнение. Теперь система проходит кадры, которые надо покинуть, и вызывает unwind/termination logic. В языках и runtime-ах это связано с `finally`, destructors, cleanup actions. Если перепутать фазы, можно неправильно понять, почему один handler "видит" исключение, но выполнение потом идет через другие cleanup paths.

В Windows x64 обе фазы опираются на возможность подняться по кадрам. Search требует понять, какие функции имеют handlers. Unwind требует корректно восстановить состояние и вызвать cleanup. Поэтому `.pdata` и `.xdata` нужны не только для красивого stack trace, а для самой логики exception dispatch.

Для reverse engineering полезный вопрос: наблюдаемая функция участвует в поиске, в cleanup или вообще не имеет handler-а? Ответ часто скрыт не в псевдокоде, а в flags `UNWIND_INFO` и language-specific data.

26. Что такое language-specific handler

Language-specific handler - обработчик, который знает правила конкретного языка или runtime-модели. Windows dispatcher знает общую механику кадров и unwind, но он не обязан понимать все детали `__try/__except`, C++ exceptions, destructors, filters и scope tables. Поэтому `UNWIND_INFO` может указывать на handler-функцию, которая интерпретирует дополнительные данные.

Слово "language-specific" не значит "только C++". Это значит: после базового x64 unwind есть слой, зависящий от компилятора и runtime. Например, один handler может понимать C SEH scope table, другой - C++ exception metadata, третий - runtime своего языка. Windows дает handler-у dispatcher context, exception record и frame context, а handler решает, применим ли он.

В PE это обычно выглядит так: flags в `UNWIND_INFO` говорят, что handler есть; после массива unwind codes лежит `ExceptionHandler` RVA; после него могут идти language-specific data. Статический analyzer может прочитать handler RVA и placement, но полная интерпретация scope data требует знания конкретного handler-а.

Поэтому проект должен быть точным в формулировках: "у функции есть handler RVA" - доказуемый факт; "это именно такой-то фильтр и он перехватывает именно это исключение" - более сильное утверждение, для которого нужны дополнительные данные.

27. Что такое __C_specific_handler и его роль

`__C_specific_handler` - один из важных handlers в Windows x64 для C structured exception handling. Он получает dispatcher context и language-specific scope table, затем решает, какие `__try`, `__except` или `__finally` области относятся к текущему исключению и текущему кадру.

В 64-битном коде ты обычно не увидишь классическую x86-цепочку SEH records на стеке. Вместо этого функция имеет `UNWIND_INFO` с флагом handler, а handler RVA может указывать на `__C_specific_handler`. Дополнительные данные после handler RVA помогают связать диапазоны кода с filters/finally blocks.

Для reverse engineering это полезный маркер. Если много функций указывают на один и тот же `__C_specific_handler`, это само по себе нормально: runtime handler общий, а различие сидит в scope data. Но если один необычный handler доминирует в огромном числе функций, лежит в странной секции или связан с поврежденной metadata, это уже повод для ручной проверки.

В твоем проекте handler dominance нельзя трактовать грубо. Нормальные системные handlers могут переиспользоваться широко. Поэтому evidence matrix учитывает family, severity, mitigations и контекст: dominance становится сильнее в сочетании с placement anomalies, parse failures или runtime handler hits.

28. Что такое SEH на x64 в реальном смысле

SEH, Structured Exception Handling, на x64 - это не просто ключевые слова `__try/__except` и не старая x86 linked list. В реальном смысле это система: exception record, context record, dispatcher, function tables, unwind info, language-specific handlers и правила ABI.

Если кто-то говорит "SEH на x64", надо уточнять слой. На уровне исходного кода это конструкция языка/компилятора. На уровне PE это `.pdata/.xdata` и handler RVA. На уровне runtime это search/unwind phases. На уровне reverse engineering это набор признаков, которые помогают понять control flow, cleanup и защитные техники.

На x64 SEH тесно связан с table-based unwind. Даже если функция не ловит исключение, ее unwind metadata может понадобиться, чтобы пройти выше к функции, которая ловит. Поэтому "SEH-слой" шире, чем список функций с `EHANDLER`. Он включает всю карту кадров, через которую dispatcher поднимается.

Для ВКРможно формулировать так: проект анализирует не весь SEH в смысле всех языковых semantics, а фундаментальный EH/Unwind слой PE32+ AMD64, без которого frame-based exception handling, stack walking и crash analysis не работают корректно.

29. Что такое VEH и чем она отличается от frame-based обработки

VEH, Vectored Exception Handling, - механизм обработчиков, регистрируемых на уровне процесса через `AddVectoredExceptionHandler`. Такой handler не привязан к конкретному кадру стека через `UNWIND_INFO`. Он получает шанс обработать exception как процесс-wide callback.

Главное различие: frame-based handler связан с функцией и ее stack frame. Чтобы дойти до него, Windows использует unwind и function tables. VEH стоит сбоку от этой иерархии: он может быть вызван до обычного поиска frame-based handlers и может наблюдать исключения независимо от того, в какой функции они произошли. Поэтому VEH часто используют отладчики, instrumentation, защитные runtime-ы и программы, которым нужен глобальный перехват событий.

Для reverse engineering VEH важна потому, что control flow может не объясняться только `.pdata/.xdata`. Программа может поставить vectored handler, намеренно вызвать breakpoint/access violation, изменить context record и продолжить выполнение в другом месте. Статический unwind-анализ не обязан полностью раскрывать такую схему, но dynamic collector может увидеть registrations и hits.

В проекте это отражено в `dynamic_win.c`: он наблюдает VEH/VCH/UEF activity, exception events и handler hits. Это не замена статике, а второй слой evidence. Если static placement anomalies сочетаются с runtime VEH activity, вывод становится сильнее.

30. Что такое top-level / unhandled exception handling

Top-level или unhandled exception handling - последний уровень, когда обычный поиск не нашел обработчика, который взял exception. В Windows можно установить top-level filter через `SetUnhandledExceptionFilter`. Такой filter не является frame-based handler конкретной функции. Он вступает в игру, когда исключение иначе осталось бы необработанным.

Для пользователя это часто выглядит как crash: программа падает, ОС показывает диалог, создается crash dump или запускается crash reporter. Но технически перед финальным завершением могут быть уведомлены debugger, vectored handlers, frame-based handlers и top-level filter. Порядок и детали зависят от ситуации, first-chance/second-chance и наличия отладчика.

Для анализа важно отличать UEF от `UNWIND_INFO` handlers. Если проект видит `SetUnhandledExceptionFilter` в dynamic session, это runtime evidence. Но оно не означает, что в каждой функции есть EHANDLER. И наоборот: наличие EHANDLER в `.xdata` не означает установленный unhandled filter.

На защите это можно объяснить как разные уровни перехвата: VEH - глобальный ранний слой, frame-based SEH - слой кадров стека, top-level filter - финальный слой для необработанного случая.

Глава 7. Полные сценарии

Теперь прогоняем два сценария целиком: искусственный `RaiseException` и реальный access violation. Здесь важно видеть последовательность, а не отдельные термины.

31. Что происходит при RaiseException пошагово

`RaiseException` - WinAPI-функция, с помощью которой программа явно создает исключение. Она принимает exception code, flags, количество аргументов и массив arguments. Это полезно для runtime-ов, тестов, сигнализации и иногда защитных схем. В отличие от access violation, здесь нет обязательной аппаратной ошибки доступа: программа сама просит dispatcher начать exception flow.

Сценарий можно разложить так. Поток вызывает `RaiseException`. Windows формирует exception record с кодом и параметрами. Сохраняется context record текущего потока. Если подключен debugger, он может получить first-chance notification. Затем могут отработать vectored handlers. Если они не обработали событие, начинается frame-based search: по текущему `RIP` ищется `RUNTIME_FUNCTION`, читается `UNWIND_INFO`, проверяется handler, затем через virtual unwind система переходит к caller frame и повторяет процесс.

Если найден подходящий handler, dispatcher запускает unwind phase для кадров, которые покидаются. Если handler говорит продолжить выполнение, context может быть изменен, и поток продолжит работу. Если обработчик не найден, exception становится unhandled: возможен second-chance для debugger, top-level filter, crash handling и завершение процесса.

Для ручного reverse engineering важно смотреть, зачем программа вызывает `RaiseException`. Это может быть обычная ошибка/сигнал, C++/runtime механизм, anti-debug trick или часть virtualized control flow. Но вывод надо доказывать: exception code, место вызова, handlers, изменения context, повторяемость и связь с `.pdata/.xdata`.

32. Что происходит при access violation пошагово

Access violation возникает, когда код пытается прочитать, записать или выполнить память без соответствующего права или без сопоставленной страницы. В user-mode Windows это обычно exception code `0xC0000005`. Параметры exception record помогают понять тип доступа и адрес: read, write или execute, faulting address.

На нижнем уровне процессор обнаруживает нарушение при обращении к памяти. Ядро и memory manager переводят это в исключение для user-mode потока. Дальше логика похожа на другие exceptions: context record, возможное уведомление debugger, vectored handlers, frame-based search, virtual unwind по кадрам, language-specific handlers, second-chance и top-level path, если никто не обработал.

Отличие от `RaiseException` в причине. При `RaiseException` событие создано программно. При access violation есть конкретный faulting memory access. Для reverse engineering это огромная подсказка: надо смотреть регистр или адресный operand, который дал bad address. Например `mov rax, [rcx]` при `RCX = 0` почти сразу объясняет null dereference. `call qword ptr [rax]` при поврежденном `RAX` может говорить о corrupt vtable или intentional control-flow trap.

Для unwind access violation важна потому, что падение может произойти где угодно: в prolog, в epilog, в середине тела, внутри handler-а. Stack trace после такого падения будет достоверным только настолько, насколько корректны `.pdata/.xdata` и динамические function tables. Поэтому crash dump analysis и твоя тема напрямую связаны.

Глава 8. Особые случаи и тонкости

Здесь разбираем случаи, из-за которых нельзя строить примитивное правило "нет записи - ошибка" или "есть reuse - защита".

33. Как leaf function отличается от non-leaf function

Leaf function - функция, которая не вызывает другие функции и не требует сложного unwind-состояния. В документации Microsoft leaf function может быть unwound простым моделированием return, потому что она не меняет nonvolatile-регистры и не нуждается в полноценной function table entry.

Non-leaf function вызывает другие функции или создает stack frame, который нужно описывать. Она может сохранять `RBX`, выделять стек, устанавливать frame pointer, иметь handlers. Для нее отсутствие корректной unwind info уже серьезнее, потому что простой `ret` не восстановит контекст правильно.

Ошибка новичка - думать, что каждая функция обязана иметь запись в `.pdata`. В реальных бинарях часть маленьких leaf routines может не иметь полноценной metadata. Поэтому анализатор не должен считать отсутствие записи универсальным признаком проблемы. Нужно смотреть контекст образа, executable sections, наличие calls, stack adjustment и общую структуру.

Для reverse engineering leaf/non-leaf помогает объяснить пробелы function table. Если дизассемблер показывает tiny stub без stack frame, отсутствие записи нормально. Если большая функция с `sub rsp, 200h`, saves и calls не имеет записи, это уже вопрос.

34. Почему не у каждой функции есть полноценная unwind-метаинформация

Причины бывают нормальные и подозрительные. Нормальная причина - leaf function. Еще одна - код, который не является обычной функцией в смысле ABI: padding, thunks, jump stubs, import-related glue, hotpatch fragments, compiler/runtime helpers. Границы дизассемблера и границы `.pdata` не всегда совпадают один к одному.

Подозрительные причины - поврежденная таблица, искусственно удаленная metadata, runtime-generated code без статического описания, protector, который переносит управление в области, плохо видимые статическому анализу. Но слово "подозрительно" не равно "доказано". Нужны признаки: исполняемый диапазон без описания, фактические stack modifications, dynamic function table registrations, handler activity, crash behavior.

Полноценная unwind info также может быть переиспользована. Несколько похожих маленьких функций могут ссылаться на один `UNWIND_INFO`, если их прологи одинаковы. Это допустимо. А вот массовое доминирование одного блока metadata при разных функциях, особенно в сочетании с необычным handler placement, требует проверки.

В проекте это отражается через mitigations. Небольшой sample size, single signal only, placement-only signal - все это ослабляет уверенность. Такой подход защищает ВКРот грубых утверждений.

35. Что такое chained unwind info

Chained unwind info включается флагом `UNW_FLAG_CHAININFO`. В этом случае после unwind codes лежит не handler RVA, а еще одна `RUNTIME_FUNCTION`, которая связывает текущую запись с другой. Идея нужна для описания сложных случаев, например разделенных или оптимизированных функций, где часть информации переиспользуется.

Важное правило: `CHAININFO` несовместим с `EHANDLER` и `UHANDLER` в той же записи. Если flags одновременно заявляют chain и handler, структура становится противоречивой. Decoder должен отметить bad operand или chain invalid, а не пытаться трактовать одни и те же байты сразу как handler RVA и chained runtime function.

Chained info опасна для парсера тем, что создает переходы. Если цепочка указывает сама на себя или образует цикл, наивный parser зависнет или уйдет в бесконечную рекурсию. Поэтому в проекте есть max chain depth, detection loop и anomalies вроде `UA_CHAIN_LOOP` и `UA_CHAIN_DEPTH_EXCEEDED`.

Для ручного анализа chain надо читать как ссылку в графе metadata. Нормальная chain объясняет переиспользование. А странная chain, выход за image, цикл или глубина без смысла - сильный повод проверить файл внимательнее.

36. Что такое dynamic function tables

Dynamic function tables - function tables, которые появляются во время выполнения, а не лежат статически в `.pdata` PE-файла. Они нужны JIT- компиляторам, runtime-generated code, некоторым защитным системам и программам, которые создают executable memory на лету. Windows дает API для регистрации таких таблиц, чтобы unwind и exception handling работали для динамического кода.

Это закрывает важную дыру статического анализа. Если ты читаешь только PE на диске, ты видишь только статическую Exception Directory. Но во время выполнения процесс может добавить новые диапазоны функций и metadata. Stack trace, debugger и dispatcher будут учитывать runtime-registered tables, а твой static report без dynamic layer может их не знать.

Dynamic function tables не являются признаком вредоносности сами по себе. JIT - нормальный сценарий. Но в reverse engineering protector-ов это интересный сигнал: код и его unwind model могут создаваться после загрузки, скрываясь от простого PE parser-а. Поэтому проект собирает evidence о `RtlAddFunctionTable` и `RtlInstallFunctionTableCallback` в dynamic mode.

Правильная формулировка: static analyzer дает картину файла; dynamic collector проверяет, не появляется ли во время исполнения дополнительный EH/Unwind слой.

37. Что делают RtlAddFunctionTable и RtlInstallFunctionTableCallback

`RtlAddFunctionTable` регистрирует массив `RUNTIME_FUNCTION` для заданного диапазона кода. Caller передает таблицу, количество entries и base address. После этого Windows может использовать эти entries при lookup и unwind для динамически созданного кода. Когда таблица больше не нужна, ее lifetime должен быть корректно завершен, иначе останутся висячие ссылки на память.

`RtlInstallFunctionTableCallback` регистрирует callback, который может возвращать function entries по запросу для заданного диапазона. Это полезно, когда таблицу не хотят или не могут держать целиком в готовом массиве. Для анализа это сложнее: статически таблицы может не быть, а runtime будет получать ее лениво через callback.

Для reverse engineering эти API важны как мост между PE и runtime. Если программа вызывает `RtlAddFunctionTable`, значит часть unwind картины существует не только в `.pdata`. Если callback-механизм активен, статический parser тем более не видит всей модели. Dynamic collector не обязан восстанавливать каждую entry полностью, но сам факт регистрации - evidence.

В проекте эти события попадают в dynamic report. Они не должны сразу увеличивать подозрительность до максимума, но в сочетании с placement anomalies, handler dominance или runtime exceptions могут усиливать вывод о защитном или нестандартном control-flow механизме.

Глава 9. Почему это важно не только для исключений

Unwind metadata часто воспринимают слишком узко. На самом деле она нужна отладчикам, профайлерам, crash dumps, трассировке и доказательному реверсу.

38. Как unwind связан с отладчиками и stack trace

Stack trace - список кадров вызовов: где мы сейчас, кто вызвал текущую функцию, кто вызвал caller-а и так далее. На x64 нормальный stack trace строится через unwind. Отладчик берет текущий context, ищет `RUNTIME_FUNCTION`, вызывает логику virtual unwind, получает caller context и повторяет.

Если unwind info корректна, отладчик может показать цепочку даже без frame pointer. Это причина, почему modern optimized builds все еще отлаживаемы. Если metadata повреждена или отсутствует, stack trace может оборваться, показать неверный caller, пропустить кадры или уйти в мусор.

Для reverse engineer-а stack trace - не просто удобная панель в debugger. Это доказательство пути выполнения. Когда ты видишь, что exception прошел через handler, dispatcher, runtime stub и функцию проекта, ты строишь гипотезу о control flow. Если stack trace опирается на ложные unwind data, гипотеза может быть красивой, но неверной.

Поэтому анализ `.pdata/.xdata` помогает оценивать качество отладочной картины. Твой проект не заменяет WinDbg, но может заранее подсказать: в этом образе stack walking надежен, а здесь есть structural findings, которые требуют ручной проверки.

39. Как unwind связан с crash dump analysis

Crash dump - снимок процесса или его части после падения. Аналитик открывает dump и пытается понять, почему процесс упал. Главные данные: exception record, context record, memory around fault, modules, threads и stack traces. На x64 stack trace снова зависит от unwind metadata.

Если dump содержит только состояние после падения, ты не можешь "заново выполнить" программу назад. Ты можешь только интерпретировать сохраненную память и metadata. `UNWIND_INFO` становится инструкцией, как подняться от faulting frame к caller frames. Поэтому корректность `.pdata/.xdata` влияет на качество postmortem analysis.

В защищенных программах или поврежденных бинарях crash dump может специально выглядеть непонятно: stack trace обрывается, handler-и указывают в необычные места, dynamic code не описан в статическом образе. Не каждое такое явление злонамеренно, но каждый такой случай надо описывать адресами и ограничениями.

Для ВКРсильная связь такая: анализатор EH/Unwind metadata помогает заранее классифицировать, насколько надежно по данному PE можно будет строить stack traces и интерпретировать исключения. Это полезно не только для malware/protector анализа, но и для диагностики сбоев.

Глава 10. Практика ручного разбора

Практика здесь не ради кнопок. Она нужна, чтобы теория стала проверяемой: байт -> поле -> смысл -> связь с инструкциями.

40. Как вручную читать .pdata и .xdata в PE

Ручной разбор начинается с PE headers. Найди `e_lfanew`, перейди к NT headers, проверь `Machine = AMD64` и `OptionalHeader.Magic = PE32+`. Затем открой Data Directory index 3: это Exception Directory. Его RVA и size дают область runtime function table. Найди секцию, в которую попадает RVA, и переведи RVA в file offset.

Допустим Exception Directory RVA `0x5000`, size `0x24`. Размер 0x24 = 36 байт, одна запись 12 байт, значит записей три. Читай каждые 12 байт как little-endian triple: Begin, End, Unwind. Для каждой записи проверь: Begin < End, End <= SizeOfImage, Unwind mappable, диапазон не пересекается с предыдущими, записи примерно отсортированы.

Потом переходи к `UnwindData`. Переведи `unwind_rva` в file offset, прочитай первые 4 байта `UNWIND_INFO`, вычисли version/flags, prolog size, count, frame register/offset. Затем читай `CountOfCodes` slots, учитывая дополнительные operands. После slots выровняйся до четного количества slots. Если flags имеют EHANDLER/UHANDLER, следующие 4 байта

- handler RVA. Если CHAININFO, следующие 12 байт - chained

`RUNTIME_FUNCTION`.

Ручной разбор должен каждый раз фиксировать точку отсчета. "0x5000" как RVA, "0x2400" как file offset, "slot 3" как индекс в массиве unwind codes, "CodeOffset 0x0A" как смещение от начала prolog-a. Когда эти точки смешиваются, ошибка становится почти неизбежной.

41. Как вручную разбирать конкретный UNWIND_INFO по байтам

Возьмем простой набор байтов:

```
01 06 02 00   06 30   02 52
```

Первый байт `0x01`: младшие 3 бита дают version 1, старшие bits дают flags 0. Второй байт `0x06`: prolog size 6. Третий `0x02`: два unwind code slots. Четвертый `0x00`: frame register отсутствует.

Первый slot `06 30`: `CodeOffset = 6`, второй байт `0x30`. Low nibble `0` - `UWOP_PUSH_NONVOL`, high nibble `3` - номер регистра `RBX`. Значит в prolog на смещении 6 завершилось действие push/save старого `RBX` через push. При unwind надо восстановить `RBX` и поднять `RSP` на 8.

Второй slot `02 52`: `CodeOffset = 2`, low nibble `2` - `UWOP_ALLOC_SMALL`, high nibble `5`. Размер allocation: `5 * 8 + 8 = 48` байт. При unwind надо прибавить 48 к `RSP`. Обрати внимание на порядок code offsets: slots могут быть отсортированы не так, как ты ожидаешь по чтению assembly. Важно применять правила документации.

Возможный пролог:

```
sub  rsp, 30h
push rbx
```

Но такой порядок assembly странен для обычного push-before-alloc; другой листинг может быть более реалистичным в зависимости от offsets. Поэтому ручная проверка всегда заканчивается сравнением с дизассемблированным prolog-ом. Байты metadata дают формальную модель, disassembly проверяет ее соответствие коду.

42. Как связывать дизассемблированный пролог с UNWIND_CODE

Связь строится по действиям, а не по буквальному тексту. Сначала выпиши инструкции prolog-а, которые меняют unwind-состояние: pushes nonvolatile, `sub rsp, imm`, stores nonvolatile registers, установка frame register. Затем выпиши unwind operations и их offsets. Потом сопоставь: каждая операция metadata должна объяснять реальное действие, а каждое действие, важное для unwind, должно быть описано.

Пример:

```
00: push rbx
01: push rbp
02: sub  rsp, 40h
06: lea  rbp, [rsp+20h]
```

Ожидаемые идеи в `UNWIND_CODE`: `PUSH_NONVOL RBX`, `PUSH_NONVOL RBP`, `ALLOC_SMALL` или `ALLOC_LARGE` на 0x40, `SET_FPREG` с frame register `RBP` и offset 0x20. Если metadata не содержит `SET_FPREG`, но header заявляет frame register, это противоречие. Если metadata содержит save `RDI`, а prolog его не трогает, это тоже вопрос.

Но осторожно: disassembly может ошибаться, function boundary может быть неверной, compiler может иметь несколько prolog-like участков, hot/cold splitting и chained entries. Поэтому вывод должен быть не "я глазами не увидел", а "для диапазона Begin-End metadata заявляет такие действия; в байтах prolog-а по этим offsets видны/не видны соответствующие инструкции; требуется такая-то проверка".

В Ghidra/IDA полезно комментировать не только функцию, но и сами ranges: begin/end RVA, unwind RVA, flags, fixed stack bytes, saved registers, handler RVA. Тогда reverse engineering становится воспроизводимым: другой человек может открыть те же адреса и проверить вывод.

Глава 11. Reverse engineering и твой проект

Финальная глава переводит фундамент в язык проекта. Здесь особенно важно не делать голословных выводов: anomaly - это наблюдаемое несоответствие модели, а не автоматическое обвинение файла.

43. Какие бывают аномалии и несоответствия в unwind-метаданных

Аномалия в этом проекте - не ругательство и не доказательство вредоносности. Это проверяемое несоответствие ожидаемой инженерной модели. Нормальная модель взята из PE Format, x64 exception handling и x64 ABI: таблица функций должна состоять из корректных диапазонов; `UNWIND_INFO` должна быть читаемой по правилам формата; handler должен указывать туда, где вообще может быть исполнимый код; reuse metadata должен иметь разумное объяснение. Когда одно из этих ожиданий нарушается, появляется finding.

Первая семья - `table_integrity`. Она относится к самой `.pdata` и PE- карте. `invalid_ranges` означает, что у `RUNTIME_FUNCTION` `BeginAddress >= EndAddress`. Это ломает идею half-open диапазона: функция либо пустая, либо имеет отрицательный размер. `rf_oob` означает, что begin/end выходят за `SizeOfImage`; Windows не должна искать функцию за пределами образа. `unwind_rva_oob` означает, что `UnwindData` нельзя безопасно преобразовать в file offset или он не попадает в допустимый image range. `rf_unsorted`, overlaps и duplicate begins говорят о проблемах lookup: поиск по `RIP` может стать неоднозначным или нестабильным.

Вторая семья - `unwind_integrity`. Здесь таблица может указывать в существующее место, но сама `UNWIND_INFO` ломается. Например, header говорит `CountOfCodes = 5`, а в файле хватает только на 3 slots. Это `UA_TRUNCATED`: читать дальше опасно. Или opcode неизвестен для поддерживаемой версии. Или `UWOP_ALLOC_LARGE` заявляет operand, которого нет. Или `CodeOffset` больше `SizeOfProlog`, то есть metadata говорит, что действие пролога произошло за пределами пролога. Или flags содержат одновременно `CHAININFO` и `EHANDLER`: одни и те же байты нельзя одновременно трактовать как chained runtime function и handler data.

Третья семья - `metadata_placement`. Здесь структура может парситься, но ее расположение странное. Unwind metadata обычно ожидается в read-only data вроде `.pdata/.xdata/.rdata`. Если она лежит в writable section, ее можно изменить во время выполнения, и это усиливает риск. Если она лежит в executable section, служебные данные смешаны с кодом. Если handler RVA указывает в non-executable section, обычный вызов handler-а невозможен без дополнительных трюков. Если handler указывает в writable section, это не доказательство атаки, но хороший вопрос о self-modifying/runtime behavior.

Четвертая семья - reuse и dominance. Один `UNWIND_INFO` может легально описывать много одинаковых функций, а один runtime handler вроде `__C_specific_handler` может встречаться часто. Поэтому reuse сам по себе слабее, чем parse failure. Но если 80-90% функций ссылаются на один необычный `UnwindInfoRVA`, а handler dominance совпадает с writable placement или dynamic exception activity, это уже связанная картина. Проект правильно хранит это как heuristic evidence, а не как бинарный вердикт "защита/не защита".

Пятая группа - strict ABI diagnostics. Например, reserved opinfo, странный порядок push, подозрительные save offsets, register class, chain restrictions. Такие предупреждения полезны эксперту, но не должны автоматически повышать suspicion score до небес. Производственные toolchains, оптимизации и edge cases иногда дают формально неожиданные записи. Поэтому `--strict-abi` показывает диагностику отдельно.

Правильная форма вывода по аномалии всегда одна: норма -> нарушение -> адрес -> последствия -> ограничение вывода. Например: "Запись `RUNTIME_FUNCTION` at index 42 имеет `BeginAddress=0x2100`, `EndAddress=0x2000`; это нарушает half-open range model, поэтому lookup по этому диапазону невозможен; finding относится к `table_integrity`; сам по себе факт не доказывает вредоносность, но требует ручной проверки PE layout и происхождения файла".

44. Как это все смотреть глазами reverse engineer

Reverse engineer смотрит на unwind metadata как на второй слой карты программы. Первый слой - disassembly: инструкции, переходы, calls. Второй слой - runtime metadata: какие диапазоны функций признает Windows, как восстанавливаются кадры, где handlers, какие registers сохраняются. Когда оба слоя совпадают, уверенность растет. Когда они расходятся, появляется исследовательский вопрос.

Нормальный порядок анализа такой. Сначала получить PE summary: machine, PE32+, image base, sections, exception directory. Потом прочитать `.pdata`: количество записей, диапазоны, сортировка, overlaps. Потом выбрать несколько representative функций: простую без handler-a, функцию с allocation, функцию с saved nonvolatiles, функцию с handler flags, подозрительную запись. Для каждой вручную или инструментально разобрать `UNWIND_INFO` и сравнить с prolog.

После этого смотреть findings. Плохой отчет говорит "файл подозрительный". Хороший отчет говорит: какие family сработали, сколько entries affected, где representative RVA, какая severity, какие mitigations ослабили вывод, что проверять руками. В reverse engineering ценится не громкость вывода, а воспроизводимость. Адрес, байты, поле, правило, следствие - вот нормальная цепочка.

Если finding связан с placement, открой section headers. Посмотри flags: executable, readable, writable. Если finding связан с handler dominance, проверь, что это за handler: импорт из runtime, локальная функция, thunk, dispatcher protector-a. Если finding связан с parse failure, открой bytes around unwind RVA и проверь, где decoder остановился. Если finding связан с dynamic events, сопоставь runtime address с module base и static RVA.

Глаз reverse engineer-a не должен верить ни одному инструменту полностью. Инструмент дает организованное evidence. Человек связывает evidence с контекстом: compiler, linker, protector, packed/unpacked state, debug symbols, imports, strings, behavior. Поэтому лучший результат проекта - не "магический verdict", а карта мест, где ручной анализ станет быстрее и точнее.

45. Как это все переносится на твой Inspector/Protector

Твой проект можно объяснять как Inspector EH/Unwind слоя PE32+ AMD64. Он берет файл как недоверенные байты, строит PE model, извлекает Exception Directory, читает массив `RUNTIME_FUNCTION`, декодирует `UNWIND_INFO`, оценивает structural findings и при необходимости добавляет dynamic evidence. Protector-oriented часть не означает, что проект "ломает защиту"; она означает, что проект умеет замечать признаки, которыми protectors и нестандартные runtime-ы могут пользоваться.

Архитектурная цепочка проекта:

```
input bytes
  -> util.c: bounds-safe чтение, буферы, строки
  -> pe.c: PE32+ AMD64, секции, RVA -> file offset, Exception Directory
  -> unwind.c: RUNTIME_FUNCTION, UNWIND_INFO, UWOP, handlers, chains
  -> heuristics.c: families, findings, score, confidence, mitigations
  -> dynamic_win.c: Debug API, exceptions, handlers, VEH/VCH/UEF, dynamic tables
  -> report.c: console, CSV, JSON, HTML, Ghidra CSV
```

`pe.c` отвечает за карту мира. Если этот слой ошибся, `unwind.c` будет читать не те байты. Поэтому проверки raw/virtual out-of-bounds, overlaps, misalignment и exception directory bounds - не второстепенные. Это защита анализатора от недоверенного input и основа корректного вывода.

`unwind.c` отвечает за грамматику EH/Unwind. Он не должен "верить" полю `CountOfCodes`, пока не проверил границы. Он не должен бесконечно идти по chain. Он должен различать parse failure, decodable anomaly и strict ABI warning. Именно поэтому в `UnwindSummary` есть `valid`, flags, prolog size, count codes, frame register, handler RVA, chain, stack bytes, saved register masks, anomalies и parse error.

`heuristics.c` отвечает за честность интерпретации. Он не делает вид, что score - статистическая вероятность. Score - инженерная шкала suspiciousness, confidence - уверенность в интерпретации, findings - структурированные сигналы, mitigations - причины ослабить вывод. Это очень сильная позиция для ВКР: проект не подменяет науку громким словом, а показывает прозрачную evidence matrix.

`dynamic_win.c` нужен там, где static PE не вся история. Runtime может регистрировать function tables, ставить VEH/VCH/UEF, генерировать exceptions, попадать в handlers. Dynamic mode опаснее, потому что он запускает target, поэтому он должен быть явно включаемым, иметь timeout, лимиты событий и аккуратный report. Но без него нельзя уверенно говорить о runtime-only EH/Unwind behavior.

На защите проект можно описать одной сильной фразой: "Я сделал анализатор, который превращает скрытый для большинства пользователей EH/Unwind слой Windows x64 из набора байтов в проверяемую инженерную картину: где функции, как восстанавливается стек, где обработчики, какие несоответствия есть в metadata и какие из них подтверждаются во время выполнения".

Часть II. Углубленные лекции: широкий фундамент вокруг основной линии

Первая часть книги ведет по твоему плану от исключения до проекта. Вторая часть расширяет фундамент: связывает исходный код, процесс загрузки, память, стек, PE loader, ABI, decoder, эвристики, dynamic collector, отчетность, validation и защиту ВКР. Эти лекции нужны не для раздувания страниц, а чтобы у читателя после основной линии появилась толстая техническая подложка: почему каждый байт и каждое поле в проекте имеют именно такую роль.

Расширенная лекция 1. От исходного кода к выполняющемуся процессу

Часть I. От исходного текста до выполняющегося процесса

Исходная точка

Работу проекта удобно начать рассматривать с конкретной команды:

```
eh_unwind_analyzer benign_richer.exe --heuristics --json report.json
```

В этой строке участвуют три файла разного назначения. `eh_unwind_analyzer` является собранным исполняемым модулем анализатора. `benign_richer.exe` является входным исполняемым модулем Windows, который анализатор должен исследовать. `report.json` является ещё не существующим выходным файлом, который должен быть создан после анализа.

Одинаковое слово «файл» скрывает принципиально различную роль каждого объекта. Первый файл операционная система загружает и выполняет. Второй в статическом режиме открывается как последовательность данных и не выполняется. Третий создаётся как сериализованное представление результата.

Чтобы понять эту разницу строго, нужно проследить весь путь от исходного текста проекта до процесса анализатора.

Программа как описание вычисления

Программа задаёт вычислительный процесс: допустимые входные данные, последовательность преобразований, условия выбора ветвей, операции с памятью и формирование результата. На языке C этот процесс описан исходным кодом.

Рассмотрим начало функции из `src/main.c`:

```
int main(int argc, char** argv) {
    Args a;
    ParseResult pres = parse_args(argc, argv, &a);
```

Здесь исходный текст содержит синтаксические конструкции языка C. Конструкция:

```
int main(int argc, char** argv)
```

объявляет функцию с именем `main`. Функция получает два параметра и возвращает значение типа `int`.

Термин **function**, или функция, в программировании обозначает именованную часть программы с определённым интерфейсом входных параметров и результата. На уровне C функция является языковой конструкцией. После компиляции она обычно соответствует одному или нескольким диапазонам машинных инструкций, но такое соответствие не обязано быть взаимно однозначным: компилятор может встроить функцию, разделить её на участки, создать вспомогательные блоки

или удалить неиспользуемый код.

Параметр `argc` имеет тип `int` и содержит количество элементов командной строки, переданных процессу. Параметр `argv` имеет тип `char**`, то есть является указателем на указатели на `char`. На практическом уровне `argv` предоставляет доступ к массиву C-строк. `argv[0]` обычно содержит путь или имя запущенной программы, `argv[1]` в рассматриваемой команде содержит `benign_richer.exe`.

Сейчас важно не запоминать синтаксис указателей, а зафиксировать связь уровней: строка, введённая пользователем в оболочке, превращается операционной системой и C runtime в данные, доступные функции `main`.

Исходный код сам по себе не является последовательностью инструкций AMD64. Он является текстом в формальном языке C. Чтобы процессор мог выполнять описанное вычисление, исходный текст должен пройти трансляцию.

Translation unit и preprocessing

Компилятор C обрабатывает не изолированный файл в его исходном виде, а единицу трансляции - `translation unit`. Она получается после работы препроцессора.

В `main.c` присутствуют строки:

```
### include "util.h"
### include "pe.h"
### include "unwind.h"
```

Директива `#include` сообщает препроцессору, что содержимое указанного заголовочного файла должно участвовать в формировании единицы трансляции. Заголовок содержит объявления типов и функций, которыми пользуется `main.c`.

Например, в `include/pe.h` объявлена функция:

```
bool pe_parse(PeImage* out, const Buf* b, StrBuf* err);
```

Это объявление сообщает компилятору тип возвращаемого значения и типы параметров. Тело функции находится в `src/pe.c`. На стадии компиляции `main.c` достаточно знать интерфейс. Фактическая связь с реализацией будет установлена линкером.

Разделение на заголовок и исходный файл позволяет нескольким единицам трансляции согласованно использовать один интерфейс. Если объявление и определение расходятся, программа нарушает контракт языка или ABI и может не собраться либо работать неверно.

Препроцессор также обрабатывает условную компиляцию. В `src/dynamic_win.c` используется:

```
### ifdef _WIN32
```

Если макрос `_WIN32` определён, в сборку входит реализация через Windows Debug API. Если нет, компилируется заглушка, сообщающая о недоступности динамического режима. Поэтому один набор исходников создаёт разные машинные варианты в зависимости от целевой платформы.

Здесь возникает первое важное различие: переносимость исходного кода не означает идентичность возможностей всех собранных файлов. Статическая часть не зависит от Windows API и работает на нескольких ОС. Динамическая часть включается только в Windows-сборку.

Compilation

После preprocessing компилятор анализирует C.

Синтаксический анализ устанавливает структуру выражений и операторов. Проверка типов определяет допустимость операций. Затем компилятор строит промежуточное представление, выполняет оптимизации, выбирает инструкции целевой архитектуры и формирует объектный файл.

Целевая архитектура проекта для исследуемого формата - AMD64, также обозначаемая x86-64. Архитектура набора команд определяет регистры, инструкции, правила адресации и модель выполнения, наблюдаемую машинным кодом.

Фрагмент:

```
if (!buf_init_from_file(&b, a.input, &owned)) {
    fprintf(stderr, "Failed to read file: %s\n", a.input);
    return 1;
}
```

не переводится механически «одна строка - одна инструкция». Компилятор должен:

- подготовить аргументы вызова по соглашению о вызовах;
- выполнить вызов `buf_init_from_file`;
- проверить возвращённое булево значение;
- выбрать адрес следующей инструкции;
- при ошибке подготовить аргументы `fprintf`;
- вернуть целочисленный код из `main`.

В оптимизированной сборке часть операций может быть преобразована. Однако наблюдаемое поведение должно соответствовать требованиям языка C для корректной программы.

Object file

Результатом отдельной компиляции обычно является объектный файл. Он уже содержит машинный код, но ещё не является полностью готовым процессом или окончательным исполняемым image.

Допустим, машинный код `main.c` вызывает `pe_parse`. Адрес `pe_parse` на стадии отдельной компиляции может быть неизвестен, потому что функция определена в другом объектном файле. Компилятор сохраняет символическую ссылку и relocation information.

Символом называется именованная сущность toolchain, с которой связывается функция, объект данных или другое адресуемое значение. Relocation record описывает место, которое необходимо скорректировать после определения окончательного размещения.

Объектный файл, таким образом, содержит не только инструкции, но и информацию, необходимую следующей стадии.

Linking

Линкер получает объектные файлы:

```
main.obj
util.obj
pe.obj
unwind.obj
heuristics.obj
report.obj
dynamic_win.obj
app_info.obj
```

и используемые библиотеки. Он разрешает ссылки между символами и создаёт итоговый модуль.

В `CMakeLists.txt` ядро объявлено:

```
add_library(ehua_core STATIC ${EHUA_CORE_SOURCES})
```

Это означает, что перечисленные объектные компоненты объединяются в статическую библиотеку `ehua_core`. Далее:

```
add_executable(eh_unwind_analyzer src/main.c)
target_link_libraries(eh_unwind_analyzer PRIVATE ehua_core)
```

Линкер связывает CLI с ядром и создаёт исполняемый файл.

На этом этапе определяется размещение секций, адреса функций внутри image, import information и служебные таблицы. Для Windows x64 toolchain также формирует сведения, необходимые для раскрутки стека. Эти сведения впоследствии станут объектом анализа самого проекта.

Это важная причинная связь. `UNWIND_INFO` не является произвольной дополнительной базой, которую вручную создаёт пользователь. Обычно её генерирует toolchain, потому что операционной системе необходимо восстанавливать состояние функций при исключениях и stack walking.

Исполняемый файл как структурированный результат

Итоговый файл Windows имеет формат PE - Portable Executable. Термин `portable` исторически указывает на единый формат для нескольких поддерживаемых архитектур Windows, но не означает, что один и тот же машинный код выполняется на любой архитектуре или ОС.

PE является не только контейнером машинных инструкций. Он содержит:

- заголовки, описывающие image;
- таблицу секций;
- код;
- данные;
- описание импортируемых DLL и функций;
- relocation information;
- exception directory;
- другие служебные каталоги.

Операционная система должна знать, какие байты являются кодом, где их разместить и какие зависимости подключить. Без такой структуры конечная последовательность байтов не определяет полноценный алгоритм загрузки.

EH/Unwind Analyzer использует тот же формат как источник аналитических данных. Но его задача отличается от задачи loader. Loader работает с целью создания выполняемого image. Анализатор работает с недоверенным вводом и должен обнаруживать несогласованности, не выходя за границы файла.

Файл как последовательность байтов

На уровне файловой системы исполняемый модуль хранится как конечная последовательность байтов. Если размер равен `N`, отдельные байты имеют индексы от `0` до `N - 1`.

Файловое смещение, `file offset`, - количество байтов от начала файла до рассматриваемой позиции. Это определение задаёт систему координат файла.

Когда `buf_init_from_file` открывает `benign_richer.exe`, выполняются:

```
FILE* f = fopen(path, "rb");
```

Режим `rb` означает binary read. Затем:

```
fseek(f, 0, SEEK_END);
long sz = ftell(f);
fseek(f, 0, SEEK_SET);
```

Функция определяет длину и возвращает позицию в начало. После выделения памяти:

```
uint8_t* mem = (uint8_t*)malloc(alloc_size);
```

байты читаются:

```
size_t rd = fread(mem, 1, (size_t)sz, f);
```

В результате образуются два связанных объекта:

```
out->data = mem;
out->size = (size_t)sz;
*owned_mem = mem;
```

`mem` является адресом выделенной области памяти процесса анализатора. `Buf` содержит невладеющее представление этой области: адрес начала и размер. `owned_mem` сохраняет указатель, который позднее должен быть передан `free`.

Здесь термин «владение» не относится к правам доступа ОС. В C-коде ownership означает ответственность за освобождение динамически выделенного ресурса. Если несколько указателей указывают на одну область, необходимо однозначно знать, какая часть программы должна её освободить и когда.

`Buf` не освобождает память самостоятельно. В конце `main` вызывается:

```
buf_free_owned(owned);
```

Эта модель предотвращает неопределённость жизненного цикла.

Безопасное чтение диапазона

После загрузки parser многократно читает поля по известным смещениям. Функция:

```
bool buf_read(const Buf* b, size_t off, void* out, size_t len)
```

получает:

- `b` - описание входной последовательности;
- `off` - начальный индекс;
- `out` - адрес области назначения;
- `len` - число требуемых байтов.

Проверки:

```
if (off > b->size) return false;
if (len > b->size - off) return false;
```

доказывают существование диапазона.

Пусть размер файла равен 100. Запрос четырёх байтов по смещению 96 допустим: требуются индексы 96, 97, 98 и 99. Запрос по смещению 97 недопустим: четвёртый байт имел бы индекс 100,

которого нет.

Это не бытовая аналогия, а непосредственная проверка полуоткрытого диапазона:

```
[off, off + len)
```

Диапазон допустим, если его конец не превышает размер буфера.

Почему код не записан как:

```
if (off + len > b->size) return false;
```

Тип `size_t` беззнаковый. При достаточно больших значениях сумма может переполниться по модулю и стать малой. Вычитание `b->size - off` безопасно только после доказательства `off <= b->size`, поэтому проверки расположены именно в таком порядке.

Эта последовательность является одним из центральных инженерных свойств ядра: все внешние смещения считаются недоверенными.

Порядок байтов

Файл содержит отдельные байты, но поля PE часто являются 16-, 32- или 64-битными числами. Для объединения байтов требуется правило порядка.

AMD64 и формат PE используют little-endian representation для рассматриваемых целых полей. Младший байт числа располагается по меньшему адресу или file offset.

Четыре байта:

```
50 45 00 00
```

при little-endian чтении образуют:

```
0x00004550
```

Проект не приводит произвольный адрес внутри файла к указателю на C-структуру. Он читает байты и собирает значение явно:

```
static inline uint32_t le32(const uint8_t* p) {
    return (uint32_t)(
        p[0] |
        ((uint32_t)p[1] << 8) |
        ((uint32_t)p[2] << 16) |
        ((uint32_t)p[3] << 24)
    );
}
```

Каждый байт переводится в `uint32_t` и сдвигается на число бит, соответствующее его позиции. Это устраняет зависимость parser от alignment и layout C-структур.

От файла к процессу

До запуска `eh_unwind_analyzer` является файлом. Чтобы выполнить его инструкции, операционная система создаёт процесс.

Процесс - управляемый ОС объект, включающий:

- виртуальное адресное пространство;
- один или несколько потоков;
- таблицу handles;
- security context;
- отображённые executable modules;

- выделенные области памяти;
- состояние runtime.

Loader читает PE анализатора, резервирует виртуальные диапазоны и отображает sections. Он разрешает imports, применяет relocations при необходимости и передаёт управление entry point.

Entry point - RVA, указанное в PE header, которое после загрузки преобразуется в виртуальный адрес. Оно обычно ведёт в startup code, а не непосредственно в исходную `main`. Runtime подготавливает параметры и вызывает `main(argc, argv)`.

Таким образом, до первой строки `main` уже произошли:

```
чтение PE анализатора
-> создание процесса
-> создание начального потока
-> отображение image
-> разрешение imports
-> инициализация runtime
-> формирование argc/argv
-> вызов main
```

Эта последовательность необходима для понимания динамического режима. Когда анализатор запускает target под debugger, Windows выполняет аналогичный процесс создания, но посылает debug events управляющему процессу.

Почему входной EXE не запускается в статическом режиме

В команде присутствуют два исполняемых файла, но выполняется только анализатор. Путь `benign_richer.exe` передаётся как текстовый аргумент. `buf_init_from_file` использует его для `fopen`, а не для `CreateProcess`.

После чтения байты target находятся в обычной heap memory процесса анализатора. Процессор не выбирает оттуда инструкции для выполнения. Функции `pe_parse` и `unwind_parse` интерпретируют байты как данные.

Это точный смысл static analysis проекта: свойства target извлекаются из его файлового представления без передачи управления его entry point.

При `--dynamic` вызывается другая ветвь:

```
dynamic_collect_windows(...)
```

Windows-реализация использует API создания процесса с флагами отладки. Теперь target становится отдельным процессом и его инструкции выполняются. Поэтому динамический режим имеет другой trust boundary и требует изолированной Windows-среды.

Три состояния одной программы

Для системного анализа необходимо различать как минимум три представления.

Первое - исходный код. Он содержит типы, имена и структуры языка C.

Второе - PE-файл. Он содержит машинный код и служебные бинарные структуры, сформированные compiler/linker toolchain.

Третье - загруженный процесс. Его память содержит отображённый image, разрешённые imports, применённые relocations, stack, heap и runtime allocations.

Некоторые свойства существуют только на одном уровне.

Имя локальной переменной может существовать только в исходнике или debug symbols.

File offset существует в системе координат файла, но не является адресом памяти процесса.

Фактический virtual address зависит от размещения image и существует в процессе.

Heap object создаётся во время выполнения и отсутствует в исходном PE как готовый объект.

Реверс-инженер должен постоянно указывать, к какому представлению относится адрес или структура.

Место reverse engineering

Обычный engineering process движется от требований и исходного кода к executable implementation. Reverse engineering начинает с реализации и строит модель её структуры и поведения.

EH/Unwind Analyzer автоматизирует конкретный участок этой работы. Он не восстанавливает исходный C и не анализирует всю семантику машинного кода. Он:

- проверяет PE layout;
- находит exception directory;
- читает runtime-function entries;
- декодирует x64 unwind descriptions;
- агрегирует аномалии;
- при необходимости наблюдает Windows exception-related runtime events.

Для полноценной работы реверс-инженер дополняет результат disassembler, decompiler и debugger.

Вывод первой части

Теперь можно точно описать начальный участок работы.

Исходные файлы C компилируются в объектный код. Линкер связывает модули и создаёт исполняемый файл анализатора. Loader операционной системы создаёт процесс и передаёт управление через startup code в `main`. Командная строка становится `argc/argv`. Путь к target используется для открытия файла. Файл целиком читается в динамически выделенную память процесса анализатора. `Buf` представляет байты и размер, а безопасные функции чтения гарантируют, что parser не обращается за пределы входа.

Следующая часть должна определить, как процессор адресует память, что такое virtual address и почему файловое смещение target нельзя напрямую использовать как адрес загруженного image. Без этого невозможно корректно перейти к PE sections и RVA.

Проверка связности понимания

Ответьте развёрнуто, без однословных определений.

1. Почему `src/pe.c`, собранный analyzer и процесс analyzer являются тремя разными объектами?
2. На какой стадии ссылка из `main.c` на `pe_parse` получает окончательное разрешение?
3. Почему object file может содержать машинный код, но не быть готовым executable image?
4. Как loader использует PE иначе, чем EH/Unwind Analyzer?
5. Почему `benign_richer.exe` не выполняется без `--dynamic`?

6. Как связаны `owned_mem`, `Buf.data` и освобождение памяти?
7. Почему порядок двух bounds-check в `buf_read` имеет значение?
8. Какие свойства target относятся к файлу, а какие появляются только в процессе?

Расширенная лекция 2. Двоичные данные, память и системы адресов

Часть II. Двоичные данные, память и системы адресов

Почему следующий этап посвящён представлению данных

После `buf_init_from_file` анализатор располагает двумя значениями: адресом первого байта target в своей памяти и размером последовательности. Однако поля PE не представлены как готовые C-объекты. Проект должен извлечь из байтов числа, а затем интерпретировать эти числа как размеры, смещения и адреса.

Чтобы понимать этот процесс, необходимо последовательно различить:

- математическое значение;
- двоичное представление;
- тип C;
- физический набор байтов;
- адрес области памяти;
- file offset;
- virtual address;
- RVA.

Ошибочное смешение этих категорий является одной из основных причин ошибок бинарных parsers.

Разрядность и множество представимых значений

Последовательность из `N` битов имеет `2^N` возможных конфигураций. Если конфигурации интерпретируются как беззнаковые целые, диапазон равен:

```
0 <= x <= 2^N - 1
```

Для 8 бит верхняя граница равна 255, для 16 - 65 535, для 32 - 4 294 967 295.

Типы `uint8_t`, `uint16_t`, `uint32_t` и `uint64_t` в C предназначены для беззнаковых целых точной ширины. Префикс `u` означает `unsigned`, число указывает ширину, суффикс `_t` является частью имени typedef.

В PE многие поля имеют фиксированную ширину независимо от платформы, на которой работает parser. Поэтому проект использует `uint32_t`, а не `unsigned long`, размер которого зависит от data model компилятора.

Например:

```
uint32_t exception_rva;
uint32_t exception_size;
```

Оба поля PE имеют размер 4 байта даже тогда, когда анализатор собран как 64-битная программа.

Знаковая интерпретация

Те же `N` битов могут интерпретироваться как знаковое число в two's complement representation. Диапазон:

```
-2^(N-1) <= x <= 2^(N-1) - 1
```

Байты не содержат отдельного признака «это signed». Интерпретацию задаёт тип или инструкция.

Конфигурация из 32 единичных битов:

```
FF FF FF FF
```

как `uint32_t` равна `4 294 967 295`, а как `int32_t` равна `-1`.

В parser полей размера и RVA используется беззнаковая интерпретация, потому что отрицательные адресные смещения не входят в модель этих полей.

Шестнадцатеричная запись

Двоичная запись неудобна для длинных значений. Одна шестнадцатеричная цифра представляет четыре бита:

```
0 = 0000
1 = 0001
...
9 = 1001
A = 1010
B = 1011
C = 1100
D = 1101
E = 1110
F = 1111
```

Две hex-цифры представляют один байт. Поэтому последовательность:

```
4D 5A
```

однозначно показывает два байта.

Префикс `0x` в C и технической документации обозначает шестнадцатеричную систему:

```
0x10 = 16
0x100 = 256
0x1000 = 4096
```

Hex удобен не только краткостью. Границы степеней двойки визуально выражены нулями. В PE типичные alignments `0x200` и `0x1000` непосредственно связаны с двоичным выравниванием.

Побитовые поля

Несколько логических значений могут храниться в одном целом. Каждый бит или группа битов имеет отдельную семантику.

Первый байт `UNWIND_INFO` содержит:

```
bits 0..2: Version
bits 3..7: Flags
```

Чтобы извлечь младшие три бита, проект применяет mask:

```
version = vflags & 0x07u;
```

`0x07` в двоичном виде:

```
00000111
```

Операция bitwise AND оставляет только младшие три позиции.

Чтобы получить старшие пять битов:

```
flags = vflags >> 3;
```

Логический сдвиг вправо перемещает их в младшие позиции.

Здесь понятие `flags` обозначает набор независимых или комбинируемых признаков, закодированных битами. В проекте flags unwind определяют наличие exception handler, termination handler или chained info.

Переполнение

Операция над целым фиксированной ширины может дать математический результат, не представимый этим типом.

Для беззнакового 32-битного значения арифметика выполняется по модулю `2^32`:

```
0xFFFFFFFF + 1 = 0x00000000
```

В `src/pe.c` сложение двух `uint32_t` проверяется через расширение:

```
static bool safe_add_u32(uint32_t a, uint32_t b, uint32_t* out) {
    uint64_t t = (uint64_t)a + (uint64_t)b;
    if (t > 0xffffffffu) return false;
    *out = (uint32_t)t;
    return true;
}
```

Операнды сначала переводятся в 64-битный тип. Сумма всех возможных двух `uint32_t` помещается в `uint64_t`, поэтому переполнение на промежуточной стадии исключено. Затем проверяется возможность вернуть результат в 32 бита.

Функция используется при вычислении file offset:

```
off = section.raw_ptr + delta
```

Если сумма не помещается в поле, mapping отклоняется.

Порядок байтов

Целое число шире одного байта должно быть разложено по последовательным адресам.

Endianness определяет порядок расположения байтов многобайтного значения.

В little-endian representation младший по значимости байт расположен по меньшему адресу.

Число:

```
0x12345678
```

представлено:

```
offset +0: 78
offset +1: 56
offset +2: 34
offset +3: 12
```

Это не означает, что число «записано наоборот» на всех уровнях. Hex viewer показывает байты по возрастающим смещениям. Register view показывает уже собранное числовое значение.

Функция `le32` проекта определяет интерпретацию явно, поэтому результат одинаков на поддерживаемых host-платформах.

Объект памяти процесса

Когда `malloc` возвращает указатель, процесс получает область своего виртуального адресного пространства.

Address - адрес. Числовой идентификатор позиции в определённом адресном пространстве.

Определение требует указания пространства. Число `0x1000` само по себе не сообщает, является ли оно:

- file offset;
- RVA;
- virtual address;
- физическим адресом;
- смещением внутри структуры.

Virtual address, VA - виртуальный адрес. Адрес в виртуальном адресном пространстве процесса.

Указатель `mem` из `buf_init_from_file` содержит VA первого байта копии target внутри процесса анализатора. Этот адрес не обязан совпадать ни с ImageBase target, ни с адресом, по которому target был бы загружен.

Выражение:

```
b->data + off
```

выполняет pointer arithmetic. Для указателя на `uint8_t` прибавление `off` перемещает адрес на `off` байтов. Поэтому file offset после чтения файла может использоваться как индекс внутри копии:

```
host VA байта = Buf.data + file_offset
```

Это адрес байта в процессе анализатора, а не runtime VA target.

Физическая и виртуальная память

Процессор выполняет инструкции с виртуальными адресами. Операционная система совместно с аппаратным механизмом трансляции отображает виртуальные страницы на физические frames либо фиксирует отсутствие mapping.

Для user-mode анализа важны свойства виртуальной памяти:

- каждый процесс имеет собственное адресное пространство;
- один virtual address в разных процессах может указывать на разные данные;
- region может быть reserved, committed или unmapped;
- страницы имеют permissions;
- mapping может связывать память с файлом;
- адрес не гарантирует допустимость чтения или выполнения.

Page - страница памяти. Блок фиксированного размера, являющийся базовой единицей управления virtual memory. На x64 Windows обычный размер страницы составляет 4096 байт, то есть `0x1000`, хотя существуют large pages.

Memory protection - защита памяти. Атрибуты страницы, определяющие разрешённость чтения, записи и выполнения.

PE section characteristics используются loader для выбора начальных protections. Поэтому проект рассматривает writable или executable placement unwind metadata как аналитический признак.

Image

В Windows PE-контексте **image** - модуль, подготовленный форматом для отображения в virtual memory с определённой структурой sections и адресов.

Следует различать:

- file representation - байты на диске;
- image layout - логическое размещение согласно PE;
- mapped image - фактическое отображение в процессе.

Размер файла не равен `SizeOfImage`. В памяти sections выравниваются иначе, чем на диске. Некоторые виртуальные области заполняются нулями и не имеют соответствующих raw bytes.

ImageBase

ImageBase - базовый виртуальный адрес image. Предпочтительный адрес начала image, указанный в optional header PE.

Для `benign_richer.exe`:

```
ImageBase = 0x140000000
```

Это preferred base. Из-за ASLR loader может выбрать другой actual base. Адреса PE, представленные как RVA, сохраняют относительную позицию и могут быть прибавлены к фактической базе.

RVA

Relative Virtual Address, RVA - относительный виртуальный адрес. Неотрицательное смещение от начала image в его виртуальной системе координат.

Связь:

```
VA = actual_image_base + RVA
```

Если:

```
actual_image_base = 0x140000000
RVA = 0x3000
```

то:

```
VA = 0x140003000
```

RVA не является file offset. В памяти sections могут начинаться на границах `0x1000`, а в файле raw data - на границах `0x200`. Поэтому одна и та же логическая позиция имеет разные координаты.

Section и две системы координат

PE section header связывает virtual layout и file layout.

Поля:

```
VirtualAddress
VirtualSize
PointerToRawData
SizeOfRawData
```

задают два диапазона.

Виртуальный диапазон начинается с `VirtualAddress`, являющегося RVA.

Файловый диапазон начинается с `PointerToRawData`, являющегося file offset.

Если требуемый RVA принадлежит section:

```
delta = RVA - section.VirtualAddress
```

Тот же относительный сдвиг внутри raw data:

```
file_offset = section.PointerToRawData + delta
```

Но преобразование допустимо только при:

```
delta < section.SizeOfRawData
```

Если `VirtualSize` больше `SizeOfRawData`, часть section существует в mapped image как zero-filled memory, но не имеет исходных байтов в файле.

Именно поэтому `pe_rva_to_fileoff` сначала ищет section, затем проверяет `delta`:

```
uint32_t delta = rva - s->virtual_address;
if (delta >= s->raw_size) {
    return false;
}
```

File offset, RVA и VA как разные типы

Хотя все три значения представлены целыми числами, логически они являются разными типами.

File offset отвечает на вопрос: где байт находится в файле?

RVA отвечает на вопрос: где объект находится относительно начала image?

VA отвечает на вопрос: где объект находится в virtual address space конкретного процесса?

Формулы перехода требуют контекста:

```
RVA -> VA:
actual base + RVA

RVA -> file offset:
найти section, вычислить delta, проверить raw backing
```

Переход:

```
file offset -> VA
```

невозможен без знания section и actual image base.

ASLR

Address Space Layout Randomization, ASLR - рандомизация размещения адресного пространства. Механизм, при котором loader изменяет фактические базы modules и другие placements, чтобы абсолютные адреса были менее предсказуемыми.

ASLR объясняет, почему статический отчёт преимущественно использует RVA. RVA устойчиво относительно image, тогда как VA конкретного запуска зависит от base.

Dynamic collector после события создания процесса получает loaded image base и вычисляет:

```
handler VA = loaded base + handler RVA
```

Если бы он использовал preferred `ImageBase` без учёта фактической базы, breakpoint мог бы быть установлен по неверному адресу.

Адреса модулей в разных процессах

Системная DLL загружена и в analyzer, и в target, но из-за ASLR bases могут различаться. Поэтому адрес `RtlAddFunctionTable` в analyzer process нельзя напрямую считать адресом функции в target.

Collector определяет offset функции внутри локального module:

```
symbol_offset = local_symbol_va - local_module_base
```

Затем:

```
remote_symbol_va = remote_module_base + symbol_offset
```

Эта операция является ещё одним применением относительной адресации.

Размер и диапазон

В бинарном анализе почти каждый объект задаётся началом и размером либо началом и концом.

Проект использует полуоткрытые диапазоны:

```
[begin, end)
```

Начало включено, конец исключён. Размер:

```
size = end - begin
```

Соседние диапазоны:

```
[0x1000, 0x1010)
[0x1010, 0x1020)
```

не пересекаются. Адрес `0x1010` принадлежит только второму.

`RUNTIME_FUNCTION` использует `BeginAddress` и `EndAddress` именно как границы диапазона кода.

Что проект проверяет на уровне адресов

`pe_parse` проверяет:

- raw section range относительно file size;
- virtual section range относительно `SizeOfImage`;
- raw overlaps;
- virtual overlaps;
- alignment;
- exception directory относительно image.

`unwind_read_runtime_functions` преобразует RVA каталога в file offset.

`heuristics_analyze` проверяет begin/end/unwind RVA отдельных записей.

`dynamic_collect_windows` преобразует статические RVA в runtime VA.

`report.c` сериализует оба вида в соответствующих полях, не смешивая их.

Итоговая связная модель

Входной PE существует как файл с file offsets. После чтения его байты существуют также в heap анализатора по host virtual addresses, но семантика полей остаётся файловой. PE section table связывает file offsets с RVA image. Loader в отдельном target process связывает RVA с фактическими VA. ASLR может изменить base, но не относительную структуру image.

Следовательно, любое адресное значение в анализе должно сопровождаться указанием системы координат.

Контрольные задачи

Для section:

```
VirtualAddress = 0x3000
VirtualSize = 0x180
PointerToRawData = 0x800
SizeOfRawData = 0x200
```

и RVA `0x3050`:

```
delta = 0x50
file offset = 0x850
```

Для RVA `0x31F0` raw backing существует, потому что `0x1F0 < 0x200`.

Для RVA `0x3200` рассматриваемый полуоткрытый raw диапазон уже завершён.

Ответьте:

1. Чем адрес `Buf.data + 0x850` отличается от target VA соответствующего байта?
2. Почему `SizeOfImage` не должен сравниваться с file size как равное значение?
3. Почему RVA устойчивее VA при сравнении двух запусков?
4. Как virtual overlap делает RVA mapping неоднозначным?
5. Почему zero-filled tail существует в процессе, но отсутствует в файле?
6. Где проект использует `uint64_t` для проверки 32-битных диапазонов?

Расширенная лекция 3. Процессор, функции, стек и Windows x64 ABI

Часть III. Процессор, функции, стек и Windows x64 ABI

Исполнение инструкции

После загрузки процесса поток имеет register context, включающий instruction pointer и другие архитектурные регистры. На AMD64 регистр `RIP` содержит virtual address следующей инструкции.

Упрощённый архитектурный цикл:

```
получить байты по RIP
-> декодировать инструкцию
-> получить операнды
-> выполнить операцию
-> изменить регистры, flags или память
-> определить следующий RIP
```

Современный процессор реализует этот цикл значительно сложнее: конвейер, out-of-order execution, speculation, caches. Для reverse engineering пользовательского кода сначала изучается архитектурно наблюдаемое поведение ISA, а не микроархитектурная реализация.

Регистры

Register - регистр. Архитектурно видимое хранилище фиксированной ширины внутри модели процессора.

Основные 64-битные general-purpose registers AMD64:

```
RAX RBX RCX RDX
RSI RDI RBP RSP
R8  R9  R10 R11
```

```
R12 R13 R14 R15
```

RIP содержит instruction pointer. RFLAGS содержит condition and control flags.

Названия частей одного регистра отражают ширину:

```
RAX: 64 бита
EAX: младшие 32 бита
AX:  младшие 16 бит
AL:  младшие 8 бит
```

Запись в EAX обнуляет старшие 32 бита RAX. Это свойство часто используется компилятором.

Операнды и память

Инструкция работает с операндами. Операнд может находиться:

- в регистре;
- в памяти;
- непосредственно в кодировке инструкции;
- быть неявно задан ISA.

Пример Intel syntax:

```
mov eax, dword ptr [rcx]
```

Инструкция читает 4 байта памяти по адресу из RCX и помещает значение в EAX.

Квадратные скобки означают обращение к памяти. Без скобок RCX обозначал бы само числовое содержимое регистра.

Арифметика адреса

AMD64 поддерживает эффективный адрес:

```
base + index * scale + displacement
```

где scale принимает 1, 2, 4 или 8.

Пример:

```
mov eax, dword ptr [rcx + rdx*4 + 8]
```

может соответствовать чтению 32-битного элемента массива, если RCX является base, RDX - индексом, а 8 - смещением поля.

Инструкция lea вычисляет адресное выражение, но не читает память:

```
lea rax, [rcx + rdx*4]
```

Компилятор использует lea и для обычной арифметики.

Управление потоком

По умолчанию RIP переходит к следующей инструкции. Instructions branch изменяют эту последовательность.

jmp выполняет безусловный переход.

Условные переходы je, jne, ja, jb, jg, jl используют flags, установленные предыдущими арифметическими или сравнительными операциями.

Различие signed и unsigned comparison существенно. Например:

- `ja` рассматривает operands как unsigned;
- `jg` рассматривает их как signed.

Decompiler восстанавливает `if`, `while` и другие конструкции на основе control-flow graph, но реальный machine code состоит из сравнений и переходов.

Function на машинном уровне

На машинном уровне функция не является встроенным объектом ISA с полным описанием типов. Обычно это диапазон кода с соглашением:

- где находятся входные аргументы;
- где вернуть результат;
- какие регистры можно изменять;
- какие регистры нужно восстановить;
- как организован stack;
- как передаётся управление.

Такое соглашение называется ABI.

Application Binary Interface, ABI - двоичный интерфейс приложений. Набор правил совместимости скомпилированных компонентов на уровне машинного кода и бинарных объектов.

ABI включает calling convention, layout некоторых объектов, правила stack alignment, symbol naming и другие аспекты.

Если caller и callee используют разные правила, они неверно интерпретируют аргументы или повреждают состояние.

Caller и callee

Caller - вызывающая функция.

Callee - вызываемая функция.

При вызове caller подготавливает аргументы и передаёт управление. Callee выполняет работу и возвращает результат.

В `main.c`:

```
bool ok = unwind_parse(&b, &img, rf[i].unwind_rva,
                       &uopts, &us[i], NULL, &perr);
```

`main` является caller, `unwind_parse` - callee.

На Windows x64 первые четыре integer/pointer arguments передаются:

```
RCX, RDX, R8, R9
```

Оставшиеся передаются через stack.

Для вызова `unwind_parse`:

```
RCX = &b
RDX = &img
R8  = unwind_rva
R9  = &uopts
остальные параметры = stack
```

Компилятор может использовать промежуточные инструкции, но состояние в точке вызова должно соответствовать convention.

Return value

Integer and pointer return value обычно помещается в `RAX`. Для `bool` значим младший участок.

После `unwind_parse` caller проверяет результат и записывает его в `us_ok[i]`.

Volatile и nonvolatile registers

Windows x64 делит registers на volatile и nonvolatile.

Volatile register callee может изменить без восстановления. Caller должен сохранить значение самостоятельно, если оно требуется после вызова.

Nonvolatile register callee обязан восстановить перед возвратом, если использовал.

К nonvolatile general-purpose относятся:

```
RBX RBP RDI RSI RSP R12 R13 R14 R15
```

RCX, RDX, R8, R9 и часть других являются volatile.

Это правило напрямую связано с unwind. Если функция сохранила nonvolatile register на stack, Windows должна знать, где находится сохранённое значение, чтобы восстановить context при исключении.

Stack

Stack - стек. Область virtual memory потока, используемая calling convention и программой для хранения адресов возврата, сохранённых регистров, локальных объектов и временных данных.

На AMD64 stack обычно растёт в сторону меньших адресов. Регистр `RSP` содержит текущий stack pointer.

Инструкция:

```
sub rsp, 40
```

уменьшает `RSP` на 40 и резервирует диапазон stack memory.

Инструкция:

```
add rsp, 40
```

освобождает тот же фиксированный диапазон перед возвратом.

Слово «освобождает» здесь означает изменение границы текущего stack frame. Память не обязательно немедленно удаляется или очищается.

`call` и адрес возврата

Инструкция `call target` выполняет два архитектурных действия:

1. помещает адрес следующей инструкции на stack;
2. устанавливает `RIP` в адрес target.

Сохранённый адрес называется return address.

Инструкция `ret` извлекает адрес со stack и передаёт ему управление.

Следовательно, корректность `RSP` критична. Если stack pointer не восстановлен, `ret` прочитает неверное значение.

Stack frame

Stack frame - стековый кадр функции. Часть stack и связанное register state, относящиеся к конкретному активному вызову функции.

Frame может включать:

- return address;
- caller-provided home space;
- сохранённые nonvolatile registers;
- локальные переменные;
- временные значения;
- stack arguments для других вызовов.

Границы frame не хранятся в одном универсальном hardware field. Они следуют из ABI, выполняемых инструкций и unwind metadata.

Shadow space

Windows x64 требует, чтобы caller перед вызовом резервировал 32 байта home space для первых четырёх register parameters.

Этот диапазон также называют `shadow space`. Callee может использовать соответствующие slots для сохранения register arguments.

Даже если callee их не использует, caller обязан обеспечить пространство согласно ABI.

Это объясняет распространённый размер:

```
sub rsp, 40
```

32 байта составляют shadow space, ещё 8 участвуют в поддержании требуемого alignment с учётом return address и текущей структуры функции.

Нельзя автоматически утверждать, что любое выделение 40 имеет только эту причину, но для вызывающей функции это типичный pattern.

Alignment stack

Вне специально разрешённых участков prolog/epilog stack должен быть выровнен на 16 байт. Это необходимо для ABI и некоторых инструкций.

Caller учитывает, что `call` помещает 8-байтовый return address. Компилятор выбирает размер frame так, чтобы в точках вызовов соблюдалось правило.

Prolog

Prolog - пролог функции. Начальная последовательность инструкций, создающая frame и сохраняющая необходимое nonvolatile state.

Типичный вариант:

```
push rbx
sub rsp, 32
```

Семантика:

- `push rbx` уменьшает RSP на 8 и записывает прежний RBX;
- `sub rsp, 32` резервирует дополнительный stack space.

После этого fixed stack change относительно состояния при входе составляет 40 байт.

Epilog

Epilog - эпилог функции. Допустимая заключительная последовательность, восстанавливающая frame и возвращающая управление caller.

Для примера:

```
add rsp, 32
pop rbx
ret
```

Эпилог должен быть распознаваемым Windows unwind mechanism. Microsoft ограничивает допустимые формы, потому что система должна уметь определить, находится ли RIP внутри epilog, и корректно восстановить context.

Frame pointer

Некоторые функции используют отдельный nonvolatile register как frame pointer. На x64 это часто RBP, но ABI допускает и другие разрешённые registers в metadata.

Frame pointer предоставляет устойчивую базу для доступа к frame, если RSP временно изменяется или используется dynamic allocation.

Оптимизированный код часто обходится без отдельного frame pointer. Отсутствие RBP как frame pointer не является аномалией.

Leaf и nonleaf function

В контексте Windows x64 unwind leaf function обычно не изменяет nonvolatile state и не требует сложной раскрутки. Она может не иметь записи в exception table.

Nonleaf function выполняет действия, требующие описания восстановления: изменяет RSP, сохраняет nonvolatile registers, вызывает другие функции или имеет handler.

Поэтому отсутствие `.pdata` нельзя автоматически считать повреждением для небольшого leaf-only image. Проект учитывает это и выдаёт умеренный, а не максимальный сигнал при executable code без runtime functions.

Машинное исключение и обычный возврат

При нормальном выполнении функция достигает epilog и `ret`. При исключении управление может быть передано системе из произвольной точки функции. Обычный epilog ещё не выполнен, поэтому RSP и nonvolatile registers находятся в состоянии текущей точки.

Чтобы перейти к предыдущему frame, Windows должна определить:

- какие изменения prolog уже были выполнены;
- сколько stack space выделено;
- где сохранены registers;
- есть ли frame pointer;

• связан ли с функцией handler.

На Windows x64 эти данные кодируются в unwind metadata.

Почему metadata отделена от машинного кода

Теоретически система могла бы дизассемблировать пролог во время каждого исключения. Однако variable-length x86-64 encoding, необходимость точного и эффективного восстановления, оптимизации и ограничения безопасного распознавания делают table-driven model предпочтительной.

Compiler генерирует описание prolog operations. Linker размещает runtime function table и unwind structures в PE. OS runtime использует их для stack walking и exception dispatch.

EH/Unwind Analyzer читает ту же metadata статически и проверяет её структуру.

Пример связи с `benign_richer.exe`

Одна runtime function sample имеет:

```
Begin RVA: 0x1010
End RVA: 0x1029
Unwind RVA: 0x4004
SizeOfProlog: 4
fixed_stack_bytes: 40
```

Декодированный `UWOP_ALLOC_SMALL` сообщает о фиксированном выделении 40 байт.

Чтобы проверить metadata, реверс-инженер должен:

1. перейти к VA `ImageBase + 0x1010`;
2. дизассемблировать первые инструкции;
3. определить байтовую длину prolog;
4. найти изменение `RSP`;
5. сравнить размер с 40;
6. проверить отсутствие необъяснённых сохранений.

Текущая версия анализатора выполняет пункты разбора metadata, но не автоматизирует сопоставление с disassembly.

Context потока

Thread context - контекст потока. Набор register values и другого архитектурного состояния, необходимого для продолжения выполнения.

Windows structure `CONTEXT` хранит, среди прочего, `RIP`, `RSP` и general-purpose registers.

При debug event collector получает или изменяет context для обработки software breakpoint. При unwind Windows вычисляет context caller.

Почему проект сохраняет register masks

`UnwindSummary` содержит:

```
uint16_t saved_nonvol_mask;
uint16_t saved_xmm_mask;
uint32_t pushed_nonvol_count;
uint32_t fixed_stack_bytes;
```

Mask позволяет обозначить множество registers битами. Если установлен бит с номером регистра, metadata сообщает о его сохранении.

Эти поля являются сводной моделью. Они не заменяют полный порядок операций, но позволяют отчёту показать, какое state заявлено как сохранённое.

Итоговая причинная цепочка

ABI требует, чтобы caller и callee согласованно передавали arguments и сохраняли nonvolatile state. Callee создаёт stack frame в prolog и удаляет его в epilog. Исключение может произойти до epilog. Поэтому Windows требуется формальное описание уже выполненных изменений. Toolchain создаёт unwind metadata. Проект декодирует это описание и оценивает его целостность.

Контрольные вопросы

1. Почему функция на уровне C и диапазон machine code не всегда взаимно однозначны?
2. Что определяет ABI?
3. Почему caller резервирует shadow space?
4. Чем volatile register отличается от nonvolatile?
5. Почему `RSP` должен быть восстановлен перед `ret`?
6. Как исключение нарушает обычную последовательность prolog-body-epilog?
7. Почему Windows использует metadata, а не только disassembly?
8. Что означает fixed stack size и чего он не учитывает?
9. Как вручную проверить `UWOP_ALLOC_SMALL`?
10. Где register context используется dynamic collector?

Расширенная лекция 4. Процесс, поток, контекст и исключение Windows

Лекция 4. Процесс, поток, контекст и исключение в Windows

Зачем эта лекция нужна до изучения проекта

Статическая часть EH/Unwind Analyzer читает сведения, которые Windows использует во время исполнения программы. Динамическая часть запускает исследуемый файл как отдельный процесс, получает от операционной системы события отладки, читает и изменяет контекст его потоков, временно заменяет байты инструкций и наблюдает прохождение исключений. Поэтому слова «процесс», «поток», «контекст», «исключение», «обработчик» и «отладочное событие» нельзя понимать приблизительно. Это разные системные сущности с разными областями состояния и разными правилами времени жизни.

Ни статический файл на диске, ни загруженный образ, ни исполняющийся процесс не являются одним и тем же объектом. PE-файл хранит байты и описания будущего образа. Загрузчик формирует из него отображение в виртуальном адресном пространстве. Процесс предоставляет адресное пространство и набор системных ресурсов. Потоки внутри процесса исполняют инструкции. Исключение возникает в контексте конкретного потока, однако его обработка может обращаться к данным уровня всего процесса: таблицам функций, зарегистрированным векторным обработчикам, загруженным модулям и глобальному фильтру необработанных исключений.

Процесс как объект операционной системы

Process, далее процесс, - объект Windows, представляющий выполняемый экземпляр программы. Процесс обладает собственным виртуальным адресным пространством, таблицей дескрипторов, параметрами безопасности, набором загруженных модулей и как минимум одним потоком после успешного создания.

Формулировка «процесс выполняет инструкции» допустима только как сокращение. Непосредственное архитектурное состояние исполнения принадлежит потоку. Процесс задаёт среду, в которой потоки видят память и системные ресурсы.

Виртуальное адресное пространство процесса содержит диапазоны страниц. Каждая страница имеет состояние и защиту. Она может быть зарезервирована, зафиксирована, доступна для чтения, записи или исполнения. PE-загрузчик отображает заголовки и секции образа в это пространство, применяет перемещения, разрешает импорты и передаёт управление стартовому коду. Один и тот же PE-файл при разных запусках может получить разные фактические виртуальные адреса из-за Address Space Layout Randomization, сокращённо ASLR. Поэтому проект постоянно различает относительный виртуальный адрес RVA и фактический виртуальный адрес VA.

В динамическом анализаторе дескриптор процесса хранится в `pi.hProcess`. Через него вызываются `ReadProcessMemory`, `WriteProcessMemory`, `VirtualProtectEx`, `FlushInstructionCache` и `TerminateProcess`. Дескриптор не является адресом процесса и не содержит его память. Это значение в таблице дескрипторов анализатора, через которое ядро Windows проверяет права и находит целевой объект.

Модуль и загруженный образ

Module, далее модуль, - загруженный в процесс исполняемый образ: основной EXE или DLL. У модуля есть фактический базовый адрес и размер отображённого образа. В проекте эта информация хранится в `ModInfo`:

```
typedef struct ModInfo {
    char name[MAX_PATH];
    uint64_t base;
    uint64_t size;
} ModInfo;
```

Поле `base` - фактический VA начала образа в исследуемом процессе. Поле `size` задаёт диапазон адресов модуля. Имя приводится к нижнему регистру, чтобы сопоставление не зависело от регистра символов.

Функция `mods_find_by_addr` проверяет принадлежность адреса полуинтервалу:

```
[base, base + size)
```

Полуинтервал включает начало и исключает конец. Такое представление совпадает с тем, как проект описывает функции `[BeginRVA, EndRVA)` и предотвращает двойную принадлежность адреса соседним диапазонам.

Список модулей обновляется через Tool Help API. Анализатор создаёт snapshot, перечисляет модули и записывает их фактические базы. Это необходимо потому, что адрес функции API в процессе анализатора нельзя механически использовать в исследуемом процессе. ASLR может выбрать другую базу. Проект сначала вычисляет смещение функции внутри локального модуля, затем прибавляет то же смещение к базе соответствующего модуля в target process.

Поток как носитель исполнения

Thread, далее поток, - планируемая Windows единица исполнения внутри процесса. Потоки одного процесса разделяют виртуальную память и загруженные модули, но имеют собственные:

- архитектурные регистры;
- instruction pointer;
- stack pointer;
- стек;
- thread-local storage;
- состояние планирования;
- системный идентификатор thread ID.

Поскольку каждый поток имеет собственный `RIP`, `RSP` и flags register, программная точка останова должна обрабатываться применительно к потоку, который её достиг. Именно поэтому исправленная версия динамического коллектора хранит pending re-arm не в одной глобальной переменной, а в таблице, индексируемой `tid`.

Если поток A остановился на `INT3`, анализатор временно снял breakpoint и включил single-step, а затем поток B достиг другой точки останова, единый глобальный указатель потерял бы связь между событием single-step и конкретной точкой, которую надо восстановить. `PendingRearmList` устраняет эту ошибку модели.

Контекст потока

Thread context, далее контекст потока, - структура, содержащая выбранную часть архитектурного состояния потока. На x64 к нему относятся регистры общего назначения, `RIP`, `RSP`, `EFlags`, SIMD-состояние и другие поля. Windows API не всегда читает все части контекста. Поле `ContextFlags` указывает, какие группы состояния требуются.

Проект запрашивает:

```
ctx.ContextFlags = CONTEXT_CONTROL | CONTEXT_INTEGER;
```

`CONTEXT_CONTROL` включает управляющее состояние, необходимое для `RIP`, `RSP` и flags. `CONTEXT_INTEGER` включает целочисленные регистры, в том числе `RCX`, `RDX`, `R8` и `R9`, где Windows x64 ABI передаёт первые четыре целочисленных или указательных аргумента функции.

Контекст читается через `GetThreadContext` и изменяется через `SetThreadContext`. Эти операции выполняются над остановленным потоком. В debug event loop Windows приостанавливает потоки отлаживаемого процесса на время обработки события, поэтому анализатор получает согласованную точку для чтения и модификации.

Контекст не является копией всей памяти потока. Он не включает содержимое стека. Если нужны данные по адресу `RSP`, их надо отдельно читать через `ReadProcessMemory`.

Нормальное управление и исключительное управление

При нормальном исполнении следующая инструкция определяется текущим `RIP`, длиной выполненной инструкции и явными инструкциями передачи управления: `call`, `ret`, условными и безусловными переходами.

Exception, далее исключение, - архитектурное или программно инициированное событие, которое прерывает обычную последовательность исполнения и передаёт управление механизму

диспетчеризации Windows. Причинами могут быть:

- доступ к недопустимой памяти;
- деление на ноль;
- выполнение `INT3`;
- включённый trap flag после одной инструкции;
- явный вызов `RaiseException`;
- другие аппаратные или программные условия.

Исключение описывается кодом, адресом возникновения и дополнительными параметрами. Для анализа принципиально различать причину исключения и способ его обработки. Один код исключения может быть намеренно создан программой, возникнуть из-за ошибки или использоваться защитным механизмом управления потоком.

Structured Exception Handling

Structured Exception Handling, сокращённо SEH, - системный механизм Windows для обработки исключений. Слово structured указывает, что обработка следует определённой модели поиска и раскрутки, а не произвольному переходу на неизвестный адрес.

Для x64 Windows использует table-based exception handling. Сведения о функциях и действиях, необходимых для восстановления состояния вызывающей функции, находятся в таблицах `RUNTIME_FUNCTION` и структурах `UNWIND_INFO`. Это ключевая причина существования проекта.

Обработка исключения имеет две логически разные задачи.

Первая задача - **dispatch**, то есть поиск обработчика, который согласится обработать исключение. Система рассматривает текущую функцию, затем вызывающую, затем следующую по стеку. Чтобы перейти от текущего контекста к контексту вызывающей функции, она выполняет виртуальную раскрутку на основе unwind metadata.

Вторая задача - **unwind**, то есть фактическое сворачивание кадров до выбранной точки обработки с вызовом необходимых termination handlers и восстановлением состояния.

Поиск и раскрутка используют сходные метаданные, но не являются одним действием. Поиск может исследовать несколько кадров, не разрушая реальное состояние исполнения. Фактическая раскрутка применяется после выбора пути обработки.

Frame-based handlers, VEH, VCH и UEF

В проекте наблюдаются несколько классов обработчиков, которые нельзя смешивать.

Frame-based handler связан с конкретной функцией через её `UNWIND_INFO`. Флаги `UNW_FLAG_EHANDLER` и `UNW_FLAG_UHANDLER` указывают наличие соответствующего хвоста с `HandlerRVA`. Такой обработчик рассматривается в связи с кадром стека и языковой моделью функции.

Vectored Exception Handler, сокращённо VEH, регистрируется функцией `AddVectoredExceptionHandler`. Он принадлежит процессу, а не одному стековому кадру. Параметр `First` влияет на место в цепочке векторных обработчиков. Параметр `Handler` является адресом функции.

Vectored Continue Handler, сокращённо VCH, регистрируется `AddVectoredContinueHandler` и относится к фазе продолжения после обработки исключения. В проекте регистрация VEH и VCH учитывается раздельно, потому что их семантические роли различны.

Unhandled Exception Filter, сокращённо UEF, задаётся через `SetUnhandledExceptionFilter`. Это top-level filter процесса, которому управление может быть передано, когда исключение не было обработано ранее. Его наличие не доказывает вредоносность: crash reporter, runtime и диагностическая подсистема также могут устанавливать такой фильтр.

Эти механизмы образуют разные слои наблюдения. Динамический collector ставит breakpoint на API регистрации, извлекает адрес переданного обработчика из аргументного регистра, затем ставит дополнительный breakpoint уже на сам пользовательский handler. Регистрация и фактическое срабатывание поэтому являются разными событиями и разными по силе свидетельствами.

First chance и second chance

Когда отлаживаемый процесс создаёт исключение, Windows сначала уведомляет debugger до обычной окончательной обработки. Это уведомление называют **first-chance exception notification**. Оно не означает, что процесс завершится, и не означает, что исключение является ошибкой.

Если debugger не объявил исключение обработанным, система продолжает стандартный поиск обработчиков. Если ни приложение, ни системный механизм не обработали исключение, debugger может получить **second-chance notification**. Второй шанс указывает, что нормальный поиск не дал обработчика; продолжение как необработанного обычно ведёт к завершению процесса.

В `dynamic_win.c` для исключений, не являющихся внутренними breakpoint или single-step анализатора, устанавливается:

```
cont = DBG_EXCEPTION_NOT_HANDLED;
```

Так collector не поглощает исключение и сохраняет обычный путь его диспетчеризации. Если бы он возвращал `DBG_CONTINUE` для каждого исключения, он мог бы искусственно изменить поведение target и скрыть реальные handlers.

Счётчики `first_chance_seen` и `second_chance_seen` отражают число наблюдений, а не число уникальных причин и не число аварий. Один запуск может породить несколько исключений.

Debug object и поток событий

При создании процесса с флагом `DEBUG_ONLY_THIS_PROCESS` Windows связывает target с debugger. Операционная система посылает debugger события:

- создание процесса;
- создание и завершение потоков;
- загрузка и выгрузка DLL;
- исключения;
- завершение процесса.

Событие представлено структурой `DEBUG_EVENT`. Поле `dwDebugEventCode` выбирает активный вариант объединения `u`. Для исключения используется `u.Exception`; для создания процесса - `u.CreateProcessInfo`; для загрузки DLL - `u.LoadDll`.

Главный цикл имеет обязательную дисциплину:

```
WaitForDebugEvent
-> разобрать ровно одно событие
-> выбрать continuation status
-> ContinueDebugEvent
```

Пока событие не продолжено, target остаётся остановленным. Поэтому потерянный вызов `ContinueDebugEvent` создаёт зависание, а ошибочный status меняет семантику исключения.

`CREATE_PROCESS_DEBUG_EVENT` возникает до начала обычного user-mode исполнения target. Проект использует этот момент для получения `lpBaseOfImage`, обновления module map и установки начальных breakpoints.

`LOAD_DLL_DEBUG_EVENT` важен потому, что функции API могут находиться в DLL, которая ещё не была загружена при предыдущей попытке. Collector обновляет список модулей и повторяет `add_api_bps_if_ready`.

`EXIT_PROCESS_DEBUG_EVENT` содержит код завершения. После `ContinueDebugEvent` отладочная связь завершается; поэтому проект сначала сохраняет `target_exit_code`, добавляет event и только затем продолжает событие.

Software breakpoint как исключение

На x86-64 программная точка останова обычно реализуется однобайтовой инструкцией `INT3` с кодом `0xCC`. Анализатор:

1. читает исходный байт по адресу;
2. сохраняет его в `Breakpoint.orig`;
3. записывает `0xCC`;
4. очищает instruction cache;
5. ждёт `EXCEPTION_BREAKPOINT`.

Когда CPU исполняет `INT3`, Windows сообщает исключение. К этому моменту сохранённый `RIP` указывает после байта `0xCC`. Чтобы исполнить заменённую исходную инструкцию, debugger должен:

1. восстановить исходный байт;
2. установить `RIP` на адрес breakpoint;
3. включить trap flag в `EFlags`;
4. продолжить поток;
5. получить `EXCEPTION_SINGLE_STEP` после одной инструкции;
6. снова записать `0xCC`;
7. снять trap flag, если дальнейшая трассировка не нужна.

Эта последовательность реализована в проекте. Она объясняет поля `armed`, `rearm_pending`, `PendingRearmList`, обработку `EXCEPTION_SINGLE_STEP` и вызов `FlushInstructionCache`.

Trap flag и single-step

Bit 8 регистра flags, называемый **Trap Flag**, сокращённо TF, включает аппаратный режим пошагового выполнения. После завершения следующей инструкции процессор создаёт debug exception. Windows передаёт его debugger как `EXCEPTION_SINGLE_STEP`.

Проект использует TF в двух целях:

- исполнить исходную инструкцию после временного снятия `INT3`;
- собрать ограниченную последовательность `RIP` после попадания в handler.

`trace_steps` ограничивается значением 256, потому что `DynamicReport.trace` имеет фиксированную ёмкость 256. Это не полная instruction trace. Это bounded sample последовательных instruction pointers одного выбранного потока.

Динамические таблицы функций

Обычная `.pdata` находится в загруженном PE. Однако JIT compiler или другой runtime может создавать исполняемый код после загрузки. Для корректной раскрутки такого кода процесс регистрирует dynamic function table через `RtlAddFunctionTable`, `RtlInstallFunctionTableCallback` или связанные API.

Вызов `RtlAddFunctionTable` принимает:

```
RCX = FunctionTable
RDX = EntryCount
R8  = BaseAddress
```

Collector фиксирует эти аргументы на входе функции. Он не копирует и не валидирует всю переданную таблицу. Поэтому событие доказывает факт вызова API с наблюдавшимися аргументами, но ещё не доказывает корректность таблицы и не определяет назначение сгенерированного кода.

`RtlInstallFunctionTableCallback` принимает четыре первых аргумента через `RCX`, `RDX`, `R8`, `R9`; проект записывает table identifier, base, length и callback address. Это важное наблюдение для runtime-generated code, но оно также требует ручного исследования памяти и callback.

Что динамический collector устанавливает, а чего не устанавливает

Collector способен установить наблюдаемые факты:

- target был создан под Windows Debug API;
- конкретный API регистрации был достигнут;
- в аргументных регистрах находились конкретные значения;
- зарегистрированный handler был достигнут;
- исключение было first chance или second chance;
- handler, указанный статической `UNWIND_INFO`, был достигнут;
- процесс завершился или превысил timeout.

Он не устанавливает автоматически:

- что handler является вредоносным;
- что исключение намеренно маскирует control flow;
- что dynamic function table содержит поддельные записи;
- что отсутствие события означает отсутствие соответствующего механизма во всех путях программы;
- что исследованный запуск покрыл полный набор поведения.

Динамический анализ всегда зависит от входных аргументов, окружения, времени ожидания и реально достигнутого execution path. Поэтому отчёт хранит параметры запуска, timeout, число шагов, события и состояние завершения сессии.

Связь с последующими лекциями

Теперь определены системные сущности, которыми оперирует проект. Следующая лекция объясняет, как файл на диске задаёт будущий модуль: какие поля PE являются контрактом загрузки, как RVA превращается в адрес и почему exception directory связывает статический формат с описанным здесь runtime exception dispatch.

Нормативные источники

- Microsoft, Debugging Events.
- Microsoft, Writing the Debugger's Main Loop.
- Microsoft, x64 exception handling.
- Microsoft, AddVectoredExceptionHandler.
- Microsoft, SetUnhandledExceptionFilter.
- Microsoft, RtlAddFunctionTable.

Расширенная лекция 5. PE/COFF как контракт loader-а и анализатора

Лекция 5. PE/COFF как контракт компоновщика, загрузчика и анализатора

Формат файла и смысл структуры

Portable Executable, сокращённо PE, - формат образов Windows. Common Object File Format, сокращённо COFF, - связанная система структур для объектных файлов и частей PE. Название portable не означает, что конкретный EXE запускается на любой операционной системе или процессорной архитектуре. Оно означает, что общий формат способен описывать разные machine types и виды Windows images.

PE - не просто контейнер произвольных байтов. Его заголовки задают проверяемые отношения:

- где находится основной PE header;
- для какой архитектуры создан образ;
- сколько секций описано;
- как секции расположены в файле;
- по каким виртуальным адресам они должны отображаться;
- каков предпочтительный ImageBase;
- каков итоговый SizeOfImage;
- где находятся системные таблицы, включая exception directory.

Эти отношения являются контрактом между генератором файла, загрузчиком Windows и инструментами анализа. EH/Unwind Analyzer реализует ограниченного consumer этого контракта. Он не загружает образ и не исполняет его, но проверяет достаточную часть структуры, чтобы безопасно найти x64 exception metadata.

Object file, image file и этап link

Compiler обычно создаёт object files. В них уже есть machine code, data, symbols и relocation records, но окончательные адреса ещё не назначены. **Linker**, далее компоновщик, объединяет object files и libraries, разрешает ссылки, располагает код и данные, строит import information, relocation information и другие таблицы, после чего создаёт image file.

EXE и DLL являются image files. Они предназначены для отображения загрузчиком в виртуальную память. Различие EXE и DLL выражается флагами и entry semantics, но обе категории используют PE image structure.

В проекте входной файл обязан быть PE32+ AMD64 image. PE32+ - вариант optional header для 64-разрядного адресного пространства. `Machine` должен быть `IMAGE_FILE_MACHINE_AMD64`, значение `0x8664`; optional header magic должен быть `0x20B`.

Проект намеренно не пытается применять x64 unwind decoder к PE32, ARM64 или произвольному COFF object file. Это не недостаток проверки, а граница поддерживаемого формата.

File offset, RVA и VA как три координатные системы

File offset - позиция байта внутри файла на диске, отсчитываемая от нуля. Функции `buf_read`, `buf_u16le`, `buf_u32le` работают с file offsets.

Relative Virtual Address, сокращённо RVA, - виртуальный адрес после загрузки за вычетом base address образа. Поля `VirtualAddress` в data directories и section headers, а также адреса в `RUNTIME_FUNCTION` являются RVA.

Virtual Address, сокращённо VA, - адрес в виртуальном пространстве процесса. Для адреса внутри модуля:

```
VA = ActualModuleBase + RVA
```

На диске такой же объект находится по формуле секции:

```
delta = RVA - Section.VirtualAddress
file_offset = Section.PointerToRawData + delta
```

Формула применима только при выполнении условий принадлежности и наличия raw bytes. Нельзя вычесть `VirtualAddress` произвольной секции и считать результат правильным.

Проект проверяет, что:

```
RVA >= section.VirtualAddress
delta < max(VirtualSize, SizeOfRawData)
delta < SizeOfRawData
PointerToRawData + delta < file_size
```

Первое условие исключает отрицательное смещение. Второе проверяет принадлежность виртуальному диапазону. Третье различает существование виртуального места и наличие соответствующего байта в файле. Четвёрто защищает от выхода за фактический входной buffer.

Секция может иметь `VirtualSize > SizeOfRawData`. Дополнительная область формируется загрузчиком, обычно с нулевым содержимым, и не имеет bytes в файле. Статический parser не должен читать её как будто она записана на диске.

DOS header и поле e_lfanew

PE начинается с DOS-compatible header. Первые два байта содержат signature `MZ`, то есть значения `0x4D 0x5A`.

По file offset `0x3C` находится 32-разрядное поле `e_lfanew`. Оно содержит file offset PE signature. Это значение нельзя считать доверенным. Parser должен проверить, что:

- поле `e_lfanew` целиком присутствует во входе;
- вычисленный offset не выходит за размер файла;
- по этому offset доступны signature и следующие headers.

Проект использует bounded reads и safe addition, чтобы повреждённое `e_lfanew` не создавало integer overflow или out-of-bounds access.

DOS stub между DOS header и PE header для задачи проекта не имеет семантического значения. Parser не исполняет и не дизассемблирует его.

PE signature

По offset `e_lfanew` должны лежать четыре байта:

```
50 45 00 00
 P  E \0 \0
```

Signature отделяет DOS-compatible prefix от COFF file header. Совпадение только `MZ` недостаточно, потому что старые DOS executables и повреждённые файлы также могут начинаться с этих байтов.

COFF file header

Сразу после PE signature находится COFF file header длиной 20 bytes. Для проекта важны:

- `Machine`;
- `NumberOfSections`;
- `SizeOfOptionalHeader`;
- `Characteristics`.

`Machine` определяет target architecture. `NumberOfSections` определяет число последующих section headers. `SizeOfOptionalHeader` задаёт фактический размер optional header и используется как граница доступа. Нельзя предполагать, что все возможные поля optional header существуют только потому, что файл имеет PE signature.

Число секций участвует в вычислении размера section table:

```
section_table_size = NumberOfSections * 40
```

Умножение и сложение должны быть проверены до чтения. В `pe.c` все probes выполняются через безопасные функции и проверки размера `Buf`.

Почему optional header называется optional

Название историческое. Для COFF object file этот header действительно может отсутствовать. Для PE image он обязателен. Поэтому анализатор требует его присутствия.

PE32+ optional header содержит standard fields, Windows-specific fields и data directories. Проект читает только необходимые поля:

- magic;
- `ImageBase`;
- `SectionAlignment`;
- `FileAlignment`;

- `SizeOfImage`;
- `SizeOfHeaders`;
- `NumberOfRvaAndSizes`;
- exception directory RVA и size.

Неиспользуемые поля всё равно остаются частью формата, но отсутствие их разбора означает лишь, что они не входят в аналитическую задачу инструмента.

ImageBase

`ImageBase` - предпочтительный VA начала образа. Для PE32+ это 64-разрядное поле. Если образ загружен по этому адресу, статическая связь имеет вид:

```
VA = ImageBase + RVA
```

При ASLR фактическая база может отличаться. Статический report сохраняет declared ImageBase, потому что анализирует file model. Dynamic collector получает actual base из `CREATE_PROCESS_DEBUG_EVENT`. Смешивать эти значения нельзя.

В `benign_richer.exe` declared ImageBase равен:

```
0x0000000140000000
```

Например, `RVA 0x3000` соответствует preferred VA `0x140003000`, но в конкретном процессе реальный VA может быть `base_main + 0x3000`.

SectionAlignment и FileAlignment

`SectionAlignment` задаёт выравнивание секций в памяти. `FileAlignment` задаёт выравнивание raw data в файле. Это разные пространства, поэтому значения могут различаться.

Для `benign_richer.exe`:

```
SectionAlignment = 0x1000
FileAlignment    = 0x0200
```

Следовательно, section RVA обычно кратен 4096 bytes, а `PointerToRawData` - 512 bytes.

Проект считает section misaligned, если:

```
VirtualAddress % SectionAlignment != 0
```

или для секции с raw content:

```
PointerToRawData % FileAlignment != 0
```

Это structural signal. Само нарушение ещё не классифицирует назначение файла, но указывает несогласованность declared layout.

SizeOfImage

`SizeOfImage` - размер загруженного image с учётом headers и секций, округлённый по правилам alignment. Он задаёт верхнюю границу допустимых image-relative ranges для поддерживаемой модели.

Для RVA внутри образа требуется:

```
RVA < SizeOfImage
```

Для полуинтервала функции:

```
BeginRVA < EndRVA
EndRVA <= SizeOfImage
```

Важно, что `EndRVA == SizeOfImage` допустим для полуинтервала: последний включённый byte находится по адресу `SizeOfImage - 1`.

Проект не использует размер файла как замену `SizeOfImage`. File layout и virtual layout различаются.

SizeOfHeaders

`SizeOfHeaders` задаёт размер заголовочной области image. RVA, попадающий в headers и имеющий file-backed byte, может отображаться непосредственно:

```
file_offset = RVA
```

при условии:

```
RVA < SizeOfHeaders
RVA < file_size
```

Это отдельная ветвь `pe_rva_to_fileoff`. Не все RVA обязаны принадлежать section data.

Data directories

В optional header находится массив `IMAGE_DATA_DIRECTORY`. Каждая запись содержит:

```
DWORD VirtualAddress;
DWORD Size;
```

Несмотря на имя поля `VirtualAddress`, для обычных image data directories это RVA. Exception directory является записью с индексом 3.

Перед чтением directory parser обязан проверить:

- `NumberOfRvaAndSizes` достаточно велико;
- `SizeOfOptionalHeader` включает нужную запись;
- сама запись лежит внутри файла.

Microsoft отдельно указывает, что directory RVA не обязан совпадать с началом секции и что инструменты не должны полагаться только на section name. Поэтому проект находит directory по RVA, а не ищет обязательную секцию с именем `.pdata`.

Section header

Каждый section header имеет размер 40 bytes и описывает одну секцию. Проект сохраняет:

```
typedef struct PeSection {
    char name[9];
    uint32_t virtual_size;
    uint32_t virtual_address;
    uint32_t raw_size;
    uint32_t raw_ptr;
    uint32_t characteristics;
} PeSection;
```

Имя в PE занимает максимум восемь bytes и не обязано иметь завершающий zero. Parser копирует восемь bytes в массив длиной девять и добавляет terminator, чтобы получить безопасную C string.

`VirtualAddress` - RVA начала секции. `VirtualSize` - значимый размер в memory model. `SizeOfRawData` и `PointerToRawData` описывают file-backed bytes. `Characteristics` содержат flags назначения и permissions.

Для эвристик особенно важны:

```
IMAGE_SCN_MEM_EXECUTE = 0x20000000
IMAGE_SCN_MEM_READ    = 0x40000000
```

```
IMAGE_SCN_MEM_WRITE   = 0x80000000
```

Проект не превращает section name в разрешение. Разрешение берётся из `Characteristics`. Имя используется только как дополнительный layout context.

Почему parser использует max(VirtualSize, RawSize)

Разные toolchains и linker layouts могут описывать границу секции неидентично. Для определения того, к какой секции относится RVA, проект использует:

```
span = max(VirtualSize, SizeOfRawData)
```

Это policy parser. Она позволяет распознать RVA в raw tail, если raw size больше declared virtual size. Но для фактического чтения всё равно требуется `delta < RawSize`.

Таким образом разделяются два вопроса:

1. к какой секции относится адрес в принятой модели;
2. существует ли соответствующий byte в файле.

Это разделение необходимо для корректной диагностики virtual-only и malformed ranges.

Structural validation секций

`pe_parse` не прекращает анализ при каждой подозрительной секции. Он собирает counters:

- `section_raw_oob`;
- `section_virtual_oob`;
- `section_raw_overlaps`;
- `section_virtual_overlaps`;
- `section_misaligned`;
- `exception_directory_oob`.

Raw out of bounds означает:

```
PointerToRawData + SizeOfRawData > file_size
```

Virtual out of bounds означает:

```
VirtualAddress + max(VirtualSize, RawSize) > SizeOfImage
```

Raw overlap означает пересечение двух file ranges. **Virtual overlap** означает пересечение двух RVA ranges. Пересечение проверяется симметричной формулой полуинтервалов:

```
a_begin < b_end && b_begin < a_end
```

Parser сохраняет counters в `PeImage`, а `heuristics_analyze` превращает их в structured findings. Это хороший пример разделения уровней: parser устанавливает факт структуры, heuristic engine оценивает его аналитическую значимость.

Exception directory

Для AMD64 exception directory содержит массив `RUNTIME_FUNCTION`. Размер каждой записи равен 12 bytes:

```
BeginAddress  : 4 bytes
EndAddress    : 4 bytes
UnwindData    : 4 bytes
```

Все три значения image-relative. Directory size должен быть совместим с записью фиксированной длины. Проект использует строгую policy: неполный хвост directory считается ошибкой целостности, а не молча игнорируется.

Для `benign_richer.exe`:

```
Exception RVA  = 0x3000
Exception Size = 0x24
```

`0x24 = 36`, следовательно:

```
36 / 12 = 3 RUNTIME_FUNCTION entries
```

Directory RVA `0x3000` попадает в `.pdata`. Если `.pdata` имеет:

```
VirtualAddress  = 0x3000
PointerToRawData = 0x0800
```

то:

```
delta = 0x3000 - 0x3000 = 0
file_offset = 0x0800 + 0 = 0x0800
```

Именно с file offset `0x800` parser начинает читать три записи.

Связь `.pdata` и `.xdata`

`RUNTIME_FUNCTION.UnwindData` указывает RVA `UNWIND_INFO`. В типичном MSVC layout table entries находятся в `.pdata`, а unwind records - в `.xdata`. Однако section names не являются нормативным условием поиска.

В исследуемом sample одна запись указывает:

```
UnwindData RVA = 0x4004
```

Если `.xdata` имеет:

```
VirtualAddress   = 0x4000
PointerToRawData = 0x0A00
```

то:

```
delta = 0x4004 - 0x4000 = 0x4
file_offset = 0x0A00 + 0x4 = 0x0A04
```

По `0xA04` начинается header соответствующего `UNWIND_INFO`. Следующая лекция объясняет его семантику как программу обратного преобразования prolog.

Loader contract и parser contract

Windows loader и EH/Unwind Analyzer имеют разные задачи. Loader должен создать исполнимый image или отклонить его. Analyzer должен безопасно извлечь сведения даже из повреждённого или подозрительного файла и предоставить локализованные diagnostics.

Поэтому parser contract строже в одних местах и уже в других:

- он поддерживает только PE32+ AMD64;
- он не разрешает imports и relocations;
- он не исполняет TLS callbacks;
- он проверяет все reads по file bounds;
- он сохраняет structural anomalies вместо безусловленного продолжения;
- он не считает section name источником истины;

• он не заменяет static file model поведением реального loader.

Успешный `pe_parse` означает, что минимальная поддерживаемая модель PE извлечена безопасно. Это не означает, что Windows обязательно загрузит файл, что все directories корректны или что файл безопасен.

Что должен уметь доказать студент

После этой лекции необходимо уметь без механического заучивания вывести:

1. почему `RVA` нельзя использовать как file offset;
2. почему actual module base отличается от declared ImageBase;
3. почему принадлежность секции не гарантирует наличие raw byte;
4. почему directory ищется по RVA, а не по имени `.pdata`;
5. почему parser counters являются фактами, а score появляется только на следующем уровне;
6. как из exception directory size получить число `RUNTIME_FUNCTION`;
7. как вручную преобразовать `UnwindData RVA` в file offset.

Нормативный источник

Microsoft, PE Format. Документ описывает DOS header, PE signature, COFF header, optional header, data directories, section table, RVA, VA, file pointer и exception table. При работе с проектом следует учитывать оговорку Microsoft: спецификация предназначена для разработки инструментов, но не гарантируется как исчерпывающая во всех деталях.

Расширенная лекция 6. Диспетчеризация исключений и теория unwind

Лекция 6. Диспетчеризация исключений и теория раскрутки Windows x64

Центральная проблема

В момент исключения Windows получает состояние текущего потока: `RIP`, `RSP`, регистры и описание исключения. Чтобы искать обработчик выше по цепочке вызовов, системе необходимо последовательно восстановить состояния вызывающих функций.

На x86-64 это нельзя надёжно сделать только по машинному коду и нельзя считать, что каждая функция использует постоянный frame pointer. Compiler может изменять `RSP`, сохранять nonvolatile registers в разных местах и выполнять оптимизации. Поэтому Windows x64 ABI вводит ограничения на prolog и epilog и требует unwind metadata для nonleaf functions.

Unwind, далее раскрутка, - вычисление состояния caller из состояния callee. Оно включает восстановление stack pointer, instruction pointer и сохранённых nonvolatile registers. Слово «раскрутка» описывает движение от более глубокого stack frame к более раннему.

Leaf и nonleaf function

Leaf function в данном контексте - функция, которая не изменяет nonvolatile state таким способом, который требует unwind description, и не вызывает другие функции. Если для текущего `RIP` отсутствует `RUNTIME_FUNCTION`, Windows может рассматривать frame как leaf: return address находится по `[RSP]`, после чего виртуальный `RSP` увеличивается на восемь.

Nonleaf function может выделять stack space, сохранять nonvolatile registers, устанавливать frame register и вызывать другие функции. Для неё требуется table entry.

Отсутствие `.pdata` не всегда является ошибкой. Image, состоящий только из leaf code, теоретически может не иметь записей. Поэтому эвристика `no_runtime_functions` имеет medium severity и прямо требует проверки сложности code. Если executable sections отсутствуют, проект даже не начисляет score и фиксирует информационное объяснение resource-only или forwarding image.

RUNTIME_FUNCTION как индекс диапазона кода

На AMD64 запись имеет три 32-разрядных RVA:

```
typedef struct RuntimeFunction {
    uint32_t begin_rva;
    uint32_t end_rva;
    uint32_t unwind_rva;
} RuntimeFunction;
```

Она описывает:

```
[BeginAddress, EndAddress) -> UnwindInfoAddress
```

Диапазон является полуинтервалом. Запись применима, если:

```
BeginAddress <= ControlPcRVA < EndAddress
```

Таблица должна быть отсортирована по `BeginAddress`. Сортировка позволяет системе эффективно искать запись по текущему control PC. Overlap и duplicate begin нарушают однозначность отображения code address в metadata.

Все адреса image-relative. Это делает table независимой от actual base при ASLR. Для получения VA:

```
FunctionVA = ModuleBase + BeginRVA
UnwindVA   = ModuleBase + UnwindRVA
```

UNWIND_INFO как описание обратного действия

Prolog изменяет входной context функции. Он может уменьшить `RSP`, сохранить `RBX`, `RBP`, `RSI`, `RDI`, `R12-R15`, сохранить nonvolatile XMM registers и установить frame pointer.

`UNWIND_INFO` кодирует действия, необходимые для отмены этих изменений. Это не копия инструкций и не общий bytecode CPU. Это компактная domain-specific encoding Windows ABI.

Header версии 1 занимает четыре bytes:

```
byte 0: Version, 3 bits; Flags, 5 bits
byte 1: SizeOfProlog
byte 2: CountOfCodes
byte 3: FrameRegister, 4 bits; FrameOffset, 4 bits
```

После header расположен массив `UNWIND_CODE`. Каждая базовая slot занимает два bytes:

```
CodeOffset : 1 byte
UnwindOp   : 4 bits
OpInfo     : 4 bits
```

Некоторые operations потребляют дополнительные slots как operands. Поэтому `CountOfCodes` - число двухбайтовых slots, а не число логических операций.

Version и Flags

Три младших bits первого byte образуют `Version`. Нормативная Microsoft documentation для классической структуры описывает version 1. Проект также поддерживает version 2 epilog descriptors в ограниченной decoder policy, потому что такие данные встречаются в современных toolchains.

Пять старших bits образуют `Flags`. Для version 1 существенны:

```
UNW_FLAG_EHANDLER = 1
UNW_FLAG_UHANDLER = 2
UNW_FLAG_CHAININFO = 4
```

`EHANDLER` означает наличие exception handler, используемого в search phase. `UHANDLER` означает termination handler, используемый в unwind phase. Оба формата хвоста содержат handler RVA после выровненного массива codes.

`CHAININFO` означает, что хвост содержит не handler RVA, а связанную `RUNTIME_FUNCTION`. Chained record переиспользует primary unwind information для фрагмента функции.

Сочетание `CHAININFO` с handler flags противоречит формату. Decoder должен фиксировать invalid chain, а не выбирать произвольную интерпретацию хвоста.

SizeOfProlog и CodeOffset

`SizeOfProlog` - длина prolog в bytes. `CodeOffset` указывает offset после инструкции prolog, эффект которой описывает operation. Codes хранятся в descending order по `CodeOffset`, потому что раскрутка отменяет prolog в обратной последовательности.

Если exception произошёл внутри prolog, нельзя отменять instructions, которые ещё не были выполнены. Windows учитывает control PC относительно prolog. Статический проект не выполняет полный `RtlVirtualUnwind` для конкретного `ControlPc`; он проверяет structural order и агрегирует effects всей записи.

`CodeOffset > SizeOfProlog` является anomaly, потому что operation заявляет эффект за пределами declared prolog. Нарушение descending order также фиксируется.

FrameRegister и FrameOffset

Четыре младших bits четвёртого header byte кодируют frame register. Значение zero означает отсутствие frame register. Остальные значения используют register numbering unwind format.

Четыре старших bits задают scaled offset. Реальный offset:

```
FrameOffsetBytes = FrameOffset * 16
```

Если frame register отсутствует, ненулевой offset не имеет корректного смысла. Если указан volatile register, decoder отмечает operand anomaly, потому что frame register должен соответствовать ABI constraints.

Frame pointer не равен автоматически `RBP`. Encoding может указать другой допустимый nonvolatile register. Поэтому reverse engineer должен читать поле, а не переносить привычку из неиспользуемой compilation model.

UWOP_PUSH_NONVOL

`UWOP_PUSH_NONVOL` описывает сохранение nonvolatile general-purpose register инструкцией push. `OpInfo` кодирует register.

Push уменьшает `RSP` на восемь и записывает register. При unwind операция читает сохранённое значение и увеличивает `RSP`. Для summary проекта фиксированный stack effect увеличивается на 8 bytes, а bit register добавляется в `saved_nonvol_mask`.

Если `OpInfo` указывает volatile register или `RSP`, metadata сомнительна. Decoder отмечает bad operand или operand sanity anomaly.

UWOP_ALLOC_SMALL

`UWOP_ALLOC_SMALL` кодирует небольшое выделение stack:

```
AllocationSize = OpInfo * 8 + 8
```

Operation не требует дополнительного slot. Возможный диапазон при четырёхбитовом `OpInfo`:

```
8, 16, ..., 128 bytes
```

Для sample `benign_richer.exe` `OpInfo = 4`, поэтому:

```
4 * 8 + 8 = 40 bytes
```

Это значение появляется в `fixed_stack_bytes` JSON report.

UWOP_ALLOC_LARGE

`UWOP_ALLOC_LARGE` имеет два encoding modes.

При `OpInfo = 0` следующий slot хранит 16-разрядное значение, масштабированное на восемь:

```
AllocationSize = operand16 * 8
```

При `OpInfo = 1` следующие два slots образуют полное 32-разрядное allocation size без scaling.

Другие `OpInfo` недопустимы. Недостаток operand slots означает truncated record.

Проект вычисляет fixed stack size с saturating behavior. Если сумма выходит за `uint32_t`, она ограничивается максимальным значением и помечается anomaly. Это предпочтительнее wraparound, при котором огромный stack effect ошибочно превратился бы в малое число.

UWOP_SET_FPREG

`UWOP_SET_FPREG` сообщает, что prolog установил declared frame register. Дополнительных operands нет. Смысл операции зависит от header fields `FrameRegister` и `FrameOffset`.

Если header не задаёт frame register, наличие `SET_FPREG` противоречиво. Если declared frame register не является допустимым nonvolatile register, decoder отмечает anomaly.

При анализе disassembly нужно найти instruction, которая формирует relation:

```
FrameRegister = RSP + FrameOffsetBytes
```

или эквивалентную разрешённую форму prolog.

UWOP_SAVE_NONVOL и UWOP_SAVE_NONVOL_FAR

Эти operations описывают сохранение nonvolatile general-purpose register через memory store, а не push.

`UWOP_SAVE_NONVOL` использует один дополнительный slot:

```
StackOffset = operand16 * 8
```

`UWOP_SAVE_NONVOL_FAR` использует два slots:

```
StackOffset = operand32
```

Operation сообщает место сохранения register относительно stack frame. Она не означает дополнительное выделение stack: allocation обычно описывается отдельным `ALLOC_SMALL` или `ALLOC_LARGE`. Поэтому summary добавляет register в mask, но не должен дважды прибавлять offset к `fixed_stack_bytes`.

XMM operations

`UWOP_SAVE_XMM128` и `UWOP_SAVE_XMM128_FAR` описывают сохранение nonvolatile SIMD register.

Обычная форма использует:

```
StackOffset = operand16 * 16
```

Far form использует полный 32-разрядный offset. Масштаб 16 соответствует размеру 128-bit XMM value и alignment constraints.

Проект хранит `saved_xmm_mask`, где bit N означает XMMN. На Windows x64 ABI nonvolatile являются XMM6-XMM15. Сохранение volatile XMM register в unwind metadata вызывает sanity signal.

UWOP_PUSH_MACHFRAME

`UWOP_PUSH_MACHFRAME` описывает machine frame, созданный при переходе через interrupt или exception mechanism. `OpInfo` различает frame с error code и без него.

Эта operation не эквивалентна обычному `push`. Она моделирует системный набор полей, включающий control state. Stack effect зависит от variant. Неподдерживаемое `OpInfo` является bad operand.

В обычном прикладном code такая operation редка, но decoder обязан корректно распознавать её, иначе valid system-oriented metadata будет ошибочно объявлена повреждённой.

Padding и выравнивание хвоста

Поскольку каждый code slot имеет два bytes, конец массива может оказаться не выровнен на четыре bytes. Handler RVA или chained `RUNTIME_FUNCTION` должен начинаться на DWORD boundary.

Если `CountOfCodes` нечётно, после последнего значимого slot присутствует padding slot:

```
tail_offset = 4 + 2 * align_up(CountOfCodes, 2)
```

или эквивалентно количество slots округляется до чётного.

Ошибка здесь смещает чтение handler RVA на два bytes и полностью разрушает последующую интерпретацию. Поэтому alignment является самостоятельным проверяемым invariant.

Handler tail

При `EHANDLER` или `UHANDLER` после aligned code array находится 32-разрядный `HandlerRVA`. За ним может следовать language-specific data, формат которых зависит от конкретного handler.

EH/Unwind Analyzer читает только handler RVA. Он не декодирует C scope tables, C++ exception structures или конкретные compiler runtime records. Поэтому `has_handler = true` означает наличие frame handler pointer по общему формату, но не раскрывает семантику языка.

Heuristic engine проверяет section, в которую попадает handler RVA:

- отсутствующая или non-executable section;
- writable section;
- dominance одного handler среди множества records.

Эти свойства не подменяют disassembly handler.

Chained unwind info

При `CHAININFO` хвост содержит полную тройку:

```
BeginAddress
EndAddress
UnwindInfoAddress
```

Она связывает текущий fragment с другой unwind record. Chaining позволяет описывать discontiguous или разделённые prolog regions без полного дублирования.

Decoder проверяет:

- правильность flag combination;
- валидность range;
- нахождение адресов внутри image;
- отсутствие очевидного self-loop;
- ограничение глубины follow;
- отсутствие цикла по уже посещённым unwind RVA.

Ограничение глубины является защитой parser, а не свойством Windows semantics. Без него специально созданная длинная цепочка могла бы вызвать чрезмерную работу или recursion.

Virtual unwind

Virtual unwind означает вычисление caller context без фактического исполнения epilog. В логической форме система:

1. находит table entry для текущего `RIP`;
2. определяет положение внутри prolog, body или epilog;
3. применяет relevant unwind operations;
4. восстанавливает nonvolatile registers;
5. получает return address;
6. формирует caller `RIP` и `RSP`;
7. повторяет поиск для caller.

Если текущий адрес находится в epilog, Windows может распознать разрешённую форму epilog и симулировать оставшиеся действия. Именно поэтому x64 ABI ограничивает структуру epilog.

EH/Unwind Analyzer не является stack walker. Он декодирует metadata вне конкретного runtime context. Поэтому он способен сказать, что record описывает 40-byte allocation и сохранение register, но не вычисляет caller context для заданных `RIP/RSP`.

Search phase и unwind phase

Во время search phase dispatcher проверяет frame handlers, чтобы определить, может ли какой-либо из них обработать exception. Контекст виртуально перемещается вверх, но реальное выполнение функции ещё не сворачивается.

После выбора handler начинается unwind phase. Система возвращается по frames до target, вызывает termination handlers и восстанавливает состояние согласно правилам.

Различие объясняет флаги `EHANDLER` и `UHANDLER`. Один handler может участвовать в поиске, другой - в cleanup при раскрутке.

Почему metadata интересна reverse engineer

Unwind metadata создана прежде всего для runtime correctness, но содержит структурные сведения:

- границы функций;
- наличие prolog и его размер;
- fixed stack allocation;
- nonvolatile registers;
- frame register;
- handler address;
- chain relations.

Даже если symbols удалены, `.pdata` сохраняет function ranges для многих nonleaf functions. Это делает её источником восстановления function map.

Одновременно защитные системы, packers, virtual machines и JIT runtimes могут:

- централизовать handlers;
- регистрировать dynamic tables;
- размещать metadata необычно;
- использовать exception-driven control flow;
- создавать records, затрудняющие автоматический анализ.

Но необычность не равна вредоносности. Compiler runtime, managed runtime и JIT также законно используют сложные handlers и dynamic tables. Поэтому проект строит evidence, а не бинарный детектор.

Инварианты, которые проверяет проект

Инварианты `RUNTIME_FUNCTION`:

```
EndRVA > BeginRVA
BeginRVA < SizeOfImage
EndRVA <= SizeOfImage
UnwindRVA != 0
UnwindRVA < SizeOfImage
table sorted by BeginRVA
ranges do not overlap
```

Инварианты `UNWIND_INFO`:

```
supported version
header and all operand slots are file-backed
CodeOffset <= SizeOfProlog
CodeOffset order is descending
opcode and OpInfo combination is valid
tail is correctly aligned
handler and chain addresses are plausible
chain traversal is bounded and acyclic
```

Нарушение отдельного invariant локализуется в `UnwindSummary.anomalies` или `parse_error`. Затем heuristic layer агрегирует число нарушений и их распространённость.

Разбор фактической записи sample

Для одной функции `benign_richer.exe`:

```
RUNTIME_FUNCTION:
BeginRVA  = 0x1010
EndRVA    = 0x1029
UnwindRVA = 0x4004
```

Функция занимает:

```
0x1029 - 0x1010 = 0x19 = 25 bytes
```

По `UnwindRVA 0x4004` decoder читает header версии 1. `SizeOfProlog = 4`. Массив codes содержит `UWOP_ALLOC_SMALL` с `OpInfo = 4`.

Следовательно:

```
FixedStackBytes = 4 * 8 + 8 = 40
```

Handler flags отсутствуют, chain отсутствует, anomalies равны zero. Итог summary говорит не «функция безопасна», а:

- record структурно прочитан;
- declared prolog length равен 4;
- metadata описывает fixed allocation 40 bytes;
- дополнительных сохранённых registers и handlers нет;
- поддерживаемые invariants не нарушены.

Граница доказательства

Parser может подтвердить внутреннюю согласованность metadata, но не полное соответствие реальному machine code. Для этого нужно сопоставить каждую unwind operation с prolog instructions в disassembler. Например, `ALLOC_SMALL 40` должен соответствовать изменению `RSP` на 40 bytes разрешённой prolog instruction.

Таким образом имеются три уровня:

```
syntactic validity
-> semantic validity внутри формата
-> correspondence с фактическим кодом
```

Проект автоматизирует первые два уровня частично. Третий остаётся manual reverse-engineering task и должен быть явно назван на защите.

Нормативные источники

- Microsoft, x64 exception handling.
- Microsoft, x64 prolog and epilog.
- Microsoft, x64 calling convention.
- Microsoft, PE Format, `.pdata` section.

Расширенная лекция 7. Метод reverse engineering: Ghidra, debugger, отчет

Лекция 7. Метод reverse engineering: от наблюдения к проверяемому выводу

Reverse engineering как дисциплина вывода

Reverse engineering, далее обратный анализ или реверс-инжиниринг, - восстановление структуры, поведения и назначения программной системы по доступным артефактам без опоры на её исходную design documentation. Для бинарного анализа артефактами служат bytes файла, machine code, metadata, imports, runtime memory, execution events и результаты контролируемых экспериментов.

Обратный анализ не сводится к чтению disassembly. Disassembly является одним представлением. Надёжный вывод строится как цепочка:

```
наблюдаемый факт
-> интерпретация по формату или ABI
-> гипотеза о поведении
-> независимая проверка
-> ограниченный вывод
```

Если аналитик перепрыгивает от одного необычного byte к заключению о назначении программы, он совершает логическую ошибку. EH/Unwind Analyzer специально разделяет raw facts, findings, score, confidence и manual validation steps, чтобы сделать эту дисциплину явной.

Статический и динамический анализ

Static analysis исследует файл и производные представления без исполнения target. Он воспроизводим для одного и того же файла и не зависит от достигнутого runtime path. Статический анализ способен охватить недостигаемые в одном запуске sections и records, но не доказывает, что code был выполнен.

Dynamic analysis наблюдает конкретное исполнение. Он подтверждает достигнутые события и значения состояния, но охватывает только path, определённый аргументами, environment, timing и внешними данными.

Эти методы не конкурируют. Они отвечают на разные вопросы.

Статика может установить:

```
UNWIND_INFO указывает HandlerRVA 0x5120
```

Динамика может установить:

```
VA base+0x5120 был достигнут потоком 1480 после exception
```

Только совместно они подтверждают, что handler, заявленный конкретной metadata, реально участвовал в исследованном execution path. Даже этот вывод не объясняет назначение handler; для этого требуется code analysis.

Нормативный факт, implementation fact и аналитический вывод

Рабочие записи должны маркировать происхождение утверждений.

Нормативный факт следует из specification или ABI. Например: `RUNTIME_FUNCTION` AMD64 содержит три image-relative 32-bit fields.

Implementation fact следует из исходного кода инструмента. Например: проект начисляет 25 score points за exception directory out of bounds.

Наблюдение (observation) следует из конкретного sample или run. Например: в `benign_richer.exe` exception directory имеет RVA `0x3000` и size `0x24`.

Аналитический вывод (analytical conclusion) связывает факты и наблюдения. Например: directory содержит три полных entries и не имеет неполного хвоста.

Вес heuristic не является нормативным фактом Microsoft. Отсутствие такого различия делает объяснение проекта научно некорректным.

Chain of custody для бинарного артефакта

До анализа следует зафиксировать identity input:

- исходный путь;
- размер;
- SHA-256;
- время получения;
- источник;
- условия хранения;
- копию для работы.

SHA-256 - cryptographic hash function, формирующая 256-bit digest. В проекте digest кодируется 64 lowercase hexadecimal symbols. Hash позволяет проверить, что два отчёта относятся к одним bytes. Он не подтверждает добросовестность источника и не является digital signature.

Проект записывает `input_sha256` и `input_size` в report metadata. Это связывает analytical output с exact input.

При динамическом анализе неизвестного binary следует использовать изолированную Windows environment, snapshot и контролируемую сеть. Тот факт, что collector не предназначен для обхода защит, не делает запуск неизвестного файла безопасным.

Первый проход: определить класс объекта

До изучения functions нужно ответить:

- является ли input PE image;
- какая архитектура указана;
- EXE это или DLL;
- каковы section layout и permissions;
- присутствует ли exception directory;
- не повреждены ли ranges;
- совпадает ли hash с ожидаемым sample.

EH/Unwind Analyzer автоматизирует часть этого прохода только для PE32+ AMD64. Он не заменяет универсальный PE viewer, signature verification, import analysis или malware sandbox.

Ghidra как статическая платформа

Ghidra - framework для software reverse engineering. Для проекта важны:

- import PE;

- memory map;
- disassembler;
- function manager;
- decompiler;
- symbol and reference navigation;
- scripting.

После import Ghidra строит собственную program database. Она не равна исходному PE-файлу. В ней хранятся разобранные memory blocks, functions, labels, references, types и пользовательские annotations.

Disassembler переводит machine-code bytes в assembly instructions согласно выбранной architecture и address context.

Decompiler строит high-level intermediate representation и затем C-like text. Его output не является исходным C-кодом. Имена, types и control structures являются реконструкцией, подверженной ошибкам.

Адресная синхронизация

EH/Unwind Analyzer выводит RVA. Ghidra обычно показывает image-based addresses. Для declared base:

```
GhidraAddress = ImageBase + RVA
```

Если Ghidra imported image с base `0x140000000`, то function begin RVA `0x1010` находится по:

```
0x140000000 + 0x1010 = 0x140001010
```

В debugger:

```
RuntimeVA = ActualModuleBase + RVA
```

RVA служит стабильным идентификатором позиции внутри image между static и dynamic representations.

Нужно всегда записывать, какой тип адреса используется. Строка `0x140001010` без обозначения VA или preferred VA недостаточна.

Импорт CSV из проекта

Проект способен сформировать Ghidra-oriented CSV:

```
begin_rva,end_rva,name,comment
```

Импортирующий script может:

- создать или уточнить function boundaries;
- присвоить synthetic names вида `sub_00001010`;
- добавить comment с decoded unwind summary.

Это enrichment, а не доказательство правильности decompilation. Если metadata повреждена или намеренно искажена, автоматически созданные function boundaries также могут быть неверны. Поэтому findings table должна проверяться до массового доверия imported map.

Анализ функции по слоям

Для каждой подозрительной function рекомендуется одна и та же последовательность.

Сначала установить boundary source: определена ли функция по `.pdata`, symbol, call target, prolog pattern или manual creation.

Затем исследовать raw instructions:

- какие bytes образуют prolog;
- как изменяется `RSP`;
- какие nonvolatile registers сохраняются;
- выполняются ли calls;
- где находятся epilogs;
- имеются ли indirect branches.

После этого сравнить metadata:

- совпадает ли `SizeOfProlog`;
- соответствует ли `ALLOC_*` изменению `RSP`;
- совпадают ли `PUSH_NONVOL` и `SAVE_NONVOL`;
- соответствует ли frame register;
- указывает ли handler на executable code.

Лишь затем исследовать decompiler output и data flow.

Сопоставление prolog и UNWIND_INFO

Пусть disassembly начинается:

```
sub rsp, 28h
```

Immediate `0x28` равен 40 decimal bytes. Metadata `UWOP_ALLOC_SMALL` с `OpInfo = 4` также описывает:

```
4 * 8 + 8 = 40
```

Совпадение подтверждает semantic correspondence для allocation.

Если metadata заявляет сохранение `RBX`, нужно найти соответствующий `push rbx` или store в stack slot в пределах prolog. Если instruction находится после declared `SizeOfProlog`, metadata и code расходятся.

Такой анализ выполняется instruction-by-instruction. Недостаточно увидеть, что общий stack size похож.

Control flow graph

Control Flow Graph, сокращённо CFG, - ориентированный граф, vertices которого являются basic blocks, а edges - возможные передачи управления.

Basic block - последовательность instructions с одним входом и завершением на branch, return, call с особой моделью или другом boundary.

Exception-based flow может отсутствовать в обычных explicit branch edges. Handler address приходит из metadata или runtime registration. Поэтому EH/Unwind analysis добавляет к CFG неявные связи:

```
protected function/frame -> frame handler
registration call -> registered VEH/VCH/UEF
```

```
exception event -> reached handler
```

Эти связи нужно хранить отдельно от прямых machine-code jumps, потому что их semantics и условия активации различаются.

Data flow

Data flow analysis исследует происхождение и использование values. Для API registration:

```
RCX/RDX/R8/R9 at function entry
```

являются ABI locations аргументов. Однако регистровое значение само по себе может быть pointer, integer, handle или bit field в зависимости от prototype.

Для `AddVectoredExceptionHandler`:

```
RCX = First
RDX = Handler
```

Для `RtlAddFunctionTable`:

```
RCX = FunctionTable
RDX = EntryCount
R8  = BaseAddress
```

После capture нужно проверить pointer ranges, memory protection, module ownership и pointed data. Collector пока записывает arguments, но не выполняет полный remote table parser.

Cross-reference analysis

Cross-reference, сокращённо xref, - связь между местом использования и target address, symbol или data object.

Для dominant handler следует определить:

- сколько `UNWIND_INFO` records на него ссылаются;
- какие functions покрываются этими records;
- принадлежит ли handler compiler runtime;
- кто вызывает или регистрирует related code;
- есть ли indirect references;
- какие exception codes он проверяет;
- куда передаёт control.

Высокий reuse может быть нормален для language runtime. Решение требует context, а не только ratio.

Debugger breakpoint strategy

Каждый breakpoint должен отвечать на сформулированный вопрос.

Breakpoint на registration API отвечает: была ли регистрация, и какие arguments переданы?

Breakpoint на handler entry отвечает: был ли handler фактически достигнут?

Breakpoint на `RtlAddFunctionTable` отвечает: регистрировалась ли dynamic table?

Breakpoint на конкретный instruction после handler отвечает: какой branch выбран после проверки exception?

Избыточные breakpoints создают data volume, но не обязательно увеличивают evidence.

Software breakpoint limitations

Запись `0xCC` изменяет target memory. Это вмешательство может быть обнаружено anti-debug logic, конфликтовать с self-modifying code или racing threads.

Если другой поток исполняет address между снятием и re-arm breakpoint, hit может быть пропущен. Per-thread pending list исправляет логическую потерю re-arm, но не устраняет все concurrency races inherent software breakpoints.

Hardware breakpoints не изменяют code bytes, но ограничены register resources и имеют собственные semantics. Текущий проект их не использует.

Следовательно, отсутствие hit не доказывает, что code никогда не исполнялся при всех условиях.

First-chance workflow

При first-chance exception аналитик фиксирует:

- exception code;
- exception address;
- thread;
- module and RVA;
- register context;
- stack;
- наличие ожидаемого handler;
- дальнейший continuation status.

Затем debugger должен передать исключение системе как unhandled, если цель - наблюдать обычную application dispatch. Именно так поступает collector.

Если debugger объявит исключение handled, application handlers могут не получить управление. Это пример того, как метод наблюдения способен изменить объект наблюдения.

Handler investigation

При попадании в handler необходимо определить:

- класс handler: frame-based, VEH, VCH или UEF;
- prototype и arguments;
- exception record fields;
- context record usage;
- return disposition;
- writes to context;
- transfers to dispatcher or original path;
- повторное создание exception.

VEH получает `PEXCEPTION_POINTERS`, включающий pointers на exception record и context record. Изменение context может перенаправить выполнение. Но факт записи в context ещё нужно связать с последующим `RIP`.

Текущий collector фиксирует handler hit и ограниченную RIP trace, но не сериализует полный context diff. Это одна из честных границ продукта.

Static-dynamic correlation

Correlation должна иметь явный join key. В проекте им может быть:

- handler RVA;
- module + offset;
- event type;
- function range;
- section property;
- dynamic table base.

Например:

```
static:
HandlerRVA 0x5120 находится в writable section

dynamic:
handler VA ActualBase+0x5120 был достигнут

correlated conclusion:
статически заявленный handler из writable section участвовал в данном run
```

Project-level `has_static_dynamic_correlation` использует более агрегированные rules: suspicious static handler context плюс handler hits, либо placement anomalies плюс runtime function table registration. Это useful summary, но manual investigation должна сохранять exact address relation.

Evidence strength и falsifiability

Хорошая гипотеза допускает проверку, способную её опровергнуть.

Гипотеза:

```
dominant handler является normal compiler runtime helper
```

Проверки:

- symbol or signature identification;
- comparison с binaries того же compiler;
- code semantics;
- exception codes;
- references from normal try/catch functions.

Гипотеза:

```
handler реализует exception-driven dispatcher
```

Проверки:

- изменяет `ContextRecord.Rip`;
- выбирает destinations по state;
- вызывается для множества protected functions;
- dynamic trace уходит в dispatcher loop;
- control-flow pattern воспроизводится при разных inputs.

Вывод должен сохранять альтернативные объяснения, пока они не исключены.

Отчёт reverse engineer

Полноценный analytical report включает:

- input identity;
- environment and tool versions;
- scope;
- observations;
- адреса с указанием coordinate system;
- screenshots или disassembly excerpts;
- static findings;
- runtime events;
- correlations;
- excluded hypotheses;
- remaining uncertainty;
- reproducible commands.

JSON проекта является machine-readable evidence package. HTML является human-review projection той же модели. Ни один из них автоматически не заменяет narrative analyst report о semantics code.

Практический протокол для EH/Unwind Analyzer

1. Вычислить SHA-256 и сохранить sample.
2. Запустить static mode с JSON, HTML и Ghidra CSV.
3. Проверить PE structural counters.
4. Отсортировать findings по severity и family.
5. Для каждого representative RVA выполнить RVA-to-VA и RVA-to-file-offset conversion.
6. Открыть соответствующий record и function в Ghidra.
7. Сопоставить prolog и unwind codes.
8. Исследовать handler xrefs и code semantics.
9. На Windows запустить dynamic mode с контролируемыми arguments.
10. Сопоставить events с static addresses.
11. Проверить manual validation steps из report.
12. Сформулировать conclusion с границами evidence.

Эта последовательность превращает output инструмента в метод исследования, а не в замену профессионального анализа.

Что означает компетентность после лекции

Студент должен уметь:

- отделять disassembly от decompilation;

- объяснять, почему static и dynamic evidence не взаимозаменяемы;
- преобразовывать RVA между Ghidra и runtime debugger;
- проверить `UNWIND_INFO` по prolog instructions;
- исследовать handler не только по адресу, но по data flow и return semantics;
- назвать вмешательство software breakpoint;
- сформулировать вывод так, чтобы было видно, что доказано, а что остаётся гипотезой.

Расширенная лекция 8. Системная модель проекта

Том II. EH/Unwind Analyzer: полная инженерная модель проекта

Что именно должно быть понято во втором томе

Исходный код проекта нельзя полноценно объяснить как независимый список файлов. Функции образуют последовательность преобразований, а структуры данных являются промежуточными состояниями одного анализа.

Полное понимание проекта требует уметь проследить две взаимосвязанные цепочки.

Первая цепочка - цепочка данных:

```
путь к файлу
-> байты файла
-> модель PE image
-> таблица runtime functions
-> summaries unwind metadata
-> статические findings
-> динамические события
-> JSON, HTML и CSV
```

Вторая цепочка - цепочка доверия:

```
недоверенный внешний ввод
-> доказанная доступность диапазона
-> доказанная структурная интерпретация
-> проверенная внутренняя модель
-> эвристическая интерпретация
-> ручная верификация
```

Если читатель понимает только первую цепочку, он знает, куда передаются структуры, но может не понимать надёжность вывода. Если понимает только вторую, но не знает фактический код, он не способен объяснить реализацию. Поэтому каждый модуль будет рассмотрен одновременно как преобразователь данных и как граница доверия.

Предметная область проекта

Проект называется `EH/Unwind Analyzer`.

`EH` - сокращение от `Exception Handling`, обработка исключений.

`Unwind` - процесс восстановления состояния предыдущих вызовов функции, в частности значения stack pointer и сохранённых nonvolatile registers.

`Analyzer` - программный инструмент, который извлекает свойства исследуемого объекта и формирует аналитическое представление.

Полное назначение проекта:

> *Проект статически разбирает Windows PE32+ AMD64 images, извлекает таблицу функций и x64 unwind metadata, проверяет структурную согласованность, формирует объяснимые эвристические*

> *признаки и отчёты, а в Windows-сборке может дополнительно наблюдать exception-related runtime events при запуске target под debugger.*

Каждая часть формулировки ограничивает область продукта.

`PE32+` означает 64-битную разновидность PE optional header. Это не означает поддержку любого 64-битного формата.

`AMD64` означает конкретную архитектуру и её модель unwind. ARM64 использует другой формат.

`Статически разбирает` означает, что основное ядро читает файл как данные и не выполняет target.

`Эвристические признаки` означает, что проект не выдаёт математическое доказательство вредоносности.

`Windows-сборка` ограничивает настоящий dynamic collector, потому что он использует Windows Debug API.

Граница продукта

Проект не выполняет следующие задачи:

- полное дизассемблирование `.text`;
- декомпиляцию программы;
- восстановление исходного C/C++;
- универсальную классификацию malware;
- эмуляцию всех возможных путей;
- kernel-mode tracing;
- поддержку PE32 x86;
- поддержку ARM64 unwind;
- поддержку ELF или Mach-O;
- автоматическое доказательство соответствия каждого `UNWIND_INFO` машинному прологу;
- полное декодирование всех language-specific exception tables.

Это не список недостатков, а определение архитектурной границы. Завершённость продукта оценивается относительно заявленного scope.

Архитектурный тип

Основные алгоритмы собираются в статическую библиотеку:

```
add_library(ehua_core STATIC ...)
```

CLI собирается отдельным executable и связывается с ядром:

```
add_executable(eh_unwind_analyzer src/main.c)
target_link_libraries(eh_unwind_analyzer PRIVATE ehua_core)
```

Такую систему корректно называть модульным монолитом с библиотечным ядром.

`Монолит` означает, что штатный анализ выполняется одним процессом и компоненты взаимодействуют через вызовы функций и C-структуры, а не через сеть.

Модульный означает, что ответственность разделена между исходными модулями с явными интерфейсами.

Библиотечное ядро означает, что основная логика не находится исключительно внутри `main`, а собрана в переиспользуемый target `ehua_core`.

Это решение соответствует характеру задачи. Между чтением PE header и декодированием unwind передаются маленькие структуры и массивы. Выделение этих стадий в отдельные сетевые сервисы не повысило бы точность, но добавило бы сериализацию и failure modes.

Над модульным монолитом существует отдельный слой поставки. Он не участвует в алгоритме анализа и не превращает ядро в микросервисную систему:

```
аналитическое ядро и CLI
-> CMake/CTest/CPack
-> release manifest и quality passport
-> Windows, macOS, source и evidence packages
```

`scripts/create_release_bundles.py` собирает воспроизводимые архивы для разных получателей. Это release orchestration, а не новый analytical module. Поэтому архитектурный тип самого анализатора остаётся прежним: один CLI process, статическая библиотека `ehua_core` и модульные границы внутри процесса.

Модули и их строгие контракты

`util.c`

Предоставляет примитивы, на которых основана безопасность остальных модулей:

- загрузку файла;
- bounds-safe чтение;
- little-endian decoding;
- динамический строковый буфер;
- атомарную запись;
- SHA-256.

`util.c` не знает формат PE. Это уровень общих операций над байтами и выходными файлами.

`pe.c`

Преобразует `Buf` в `PeImage`.

Вход: неизменяемая последовательность байтов.

Выход: проверенные ключевые поля PE и массив секций, а также счётчики структурных нарушений.

`pe.c` определяет систему координат, необходимую всем последующим модулям.

`unwind.c`

Использует `PeImage`, чтобы:

- найти exception directory;
- прочитать массив `RUNTIME_FUNCTION`;
- преобразовать `UnwindInfoAddress` в file offset;

- декодировать `UNWIND_INFO`;
- вычислить сводные свойства;
- зафиксировать аномалии и parse failures.

`heuristics.c`

Не читает исходный файл самостоятельно. Он получает уже построенные модели и агрегирует наблюдения.

Это принципиальная граница: heuristic rule не должна обходить parser и заново интерпретировать недоверенные байты по своим правилам.

`dynamic_win.c`

В Windows-сборке создаёт target process под debugger и собирает bounded runtime evidence. Он использует статические handler RVA для установки breakpoints после определения loaded image base.

В не-Windows сборке экспортирует тот же интерфейс, но возвращает контролируемое сообщение о недоступности режима.

`report.c`

Сериализует внутренние модели. Он не должен менять score, повторно анализировать PE или владеть переданными массивами.

`app_info.c`

Создаёт provenance:

- tool version;
- версию schema;
- compiler;
- platform;
- build mode;
- timestamp.

`main.c`

Является orchestration layer:

- разбирает CLI;
- определяет требуемые режимы;
- вызывает модули в правильном порядке;
- управляет временем жизни объектов;
- агрегирует ошибки в exit code;
- инициирует запись outputs.

Почему CLI и ядро разделены

CLI является одним способом использования ядра. Он решает вопросы пользовательского интерфейса:

- как записываются options;
- какие defaults применяются;
- что печатается в терминал;
- какие файлы создавать;
- какой вернуть exit code.

Ядро решает предметную задачу:

- как разобрать PE;
- как декодировать unwind;
- как вычислить findings;
- как сформировать report model.

Такое разделение позволяет backend запускать CLI как worker. Сетевой API между backend и ядром необязателен. Backend может передать путь процессу и получить JSON.

Если в будущем потребуется прямой library API, `ehua_core` уже отделён, но публичный стабильный ABI потребует дополнительного проектирования opaque handles и versioning structures.

Главные структуры как состояния анализа

`Buf`

```
typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;
```

Состояние: файл загружен, но его формат ещё не доказан.

`PeImage`

Состояние: обязательные PE32+ AMD64 headers разобраны, sections известны, RVA mapping возможен.

`RuntimeFunction[]`

Состояние: exception directory интерпретирован как последовательность троек begin/end/unwind RVA.

`UnwindSummary[]` и `us_ok[]`

Состояние: для каждой runtime function выполнена попытка декодирования. `us_ok[i]` отделяет успешную структурную интерпретацию от failure. `UnwindSummary` сохраняет и результат, и причину ошибки.

`HeuristicReport`

Состояние: низкоуровневые наблюдения агрегированы в findings, score, confidence, verdict и mitigations.

`DynamicReport`

Состояние: один ограниченный runtime experiment завершён либо не выполнен. Содержит counters, events, trace и status.

`ReportMetadata`

Состояние: результат связан с конкретной версией инструмента, временем и SHA-256 входа.

Почему используются параллельные массивы

Для `N` runtime functions проект выделяет:

```
UnwindSummary* us;
bool* us_ok;
```

Индекс связывает:

```
rf[i] <-> us[i] <-> us_ok[i]
```

`rf[i]` описывает диапазон функции и RVA metadata.

`us[i]` содержит декодированную сводку либо заполненную ошибку.

`us_ok[i]` сообщает, допустимо ли использовать основные decoded fields.

Почему success не определяется только `us[i].valid`? Внутренняя структура действительно содержит `valid`, но внешний массив success обеспечивает простой единообразный контракт для heuristics и reports. Нулевая структура после `calloc` не должна ошибочно считаться успешным результатом.

Недостаток модели - необходимость поддерживать одинаковую длину и порядок массивов. `main` создаёт их одновременно на основании `rf_count`, а report validation проверяет наличие pointers при ненулевом количестве.

Ownership graph

Память в C не освобождается автоматически по выходу из всей программы, если она выделена в heap. Для краткоживущего CLI ОС освободила бы адресное пространство после exit, но корректное явное освобождение необходимо для:

- unit tests;
- sanitizer checks;
- библиотечного использования;
- ранних ошибок;
- предотвращения накопления ресурсов.

Граф владения:

```
main
├── owned input bytes
├── PeImage.sections
├── RuntimeFunction[]
├── UnwindSummary[]
├── us_ok[]
├── HeuristicReport.details
├── DynamicReport.details
└── локальные StrBuf
```

Конкретные release functions:

```
buf_free_owned
pe_free
free
heuristics_free
dynamic_report_free
sb_free
```

Report writers получают borrowed pointers. Они не освобождают модели и не хранят их после возврата.

Error model

Проект различает несколько уровней ошибки.

CLI error

Неверная option или значение. Exit code `2`.

Input or structural fatal error

Файл не читается, не является поддерживаемым PE или exception directory невозможно прочитать. Exit code `1`.

Per-record parse failure

Одна `UNWIND_INFO` не декодируется, но анализ остальных records продолжается. Ошибка становится частью отчёта и heuristics.

Analytical finding

Наблюдение подозрительно, но анализ технически успешен. Exit code не обязан становиться ненулевым.

Output error

Не удалось сформировать metadata или записать requested report. Exit code `1`.

Dynamic session failure

Collector не запустился или не завершил требуемую операцию. Exit code `1`, но при возможности status отражается в `DynamicReport`.

Критически важно: score выше нуля не является технической ошибкой выполнения инструмента.

Почему analysis продолжается при локальной ошибке

Если одна unwind structure повреждена, прекращение всего анализа уничтожило бы информацию о тысячах остальных функций. Проект использует устойчивую модель:

```
ошибка конкретной записи
-> us_ok[i] = false
-> parse_error сохраняется
-> цикл продолжается
-> heuristic counter увеличивается
-> отчёт содержит failure
```

Такой подход называется fault containment на уровне записи: локальная ошибка не должна разрушать весь результат, если верхнеуровневая структура всё ещё позволяет безопасное продолжение.

Доказательная модель

Проект не переходит непосредственно от байтов к verdict. Между ними существуют уровни.

```
байты
-> parsed fields
-> validation counters
-> findings
-> evidence families
-> score/confidence
-> static/dynamic assessment
-> manual validation
```

`Parsed field` - значение, извлечённое по формату.

`Validation counter` - число нарушений конкретного условия.

`Finding` - структурированное аналитическое сообщение с code, family, severity и location.

`Score` - сумма инженерных weights.

`Confidence` - оценка силы и разнообразия evidence с weakening rules.

`Assessment` - объединение static и dynamic levels.

`Manual validation` - действия, которые должен выполнить человек.

Чем выше уровень, тем больше интерпретации и тем меньше допустимо выдавать результат за нормативный факт.

Сквозной инвариант неизменности входа

Static core объявляет байты:

```
const uint8_t* data;
```

Functions принимают:

```
const Buf* b;
```

`const` выражает контракт: parser не модифицирует вход. Это позволяет:

- вычислить SHA-256 тех же байтов;
- сравнивать результаты разных стадий;
- безопасно повторять parsing;
- исключить случайное patching target;
- использовать вход как immutable evidence artifact.

Динамический target является отдельным process и может изменять собственную memory. Это не меняет сохранённый input buffer анализатора.

Связь модулей с теоретическим фундаментом

Теоретическое понятие	Реализация проекта
конечная последовательность байтов	`Buf`
файловое смещение	параметр `off`, section raw pointers
little-endian integer	`le16`, `le32`, `le64`
виртуальный образ	`PeImage`
RVA	поля sections/runtime functions
ABI и stack frame	decoded unwind operations
exception dispatch	handlers и dynamic events
множество и частота	hash maps heuristics
сериализация	`report.c`
процесс и поток	`dynamic_win.c`
жизненный цикл ресурса	ownership и cleanup
контракт	headers в `include/`
доказательство корректности	unit/regression/schema tests
воспроизводимая поставка	`scripts/create_release_bundles.py`, `release/`

Как читать последующие главы

Каждый модуль будет разобран в одинаковом порядке:

1. зачем он нужен в общей цепочке;
2. какие предпосылки уже доказаны;
3. публичные структуры и функции;
4. внутренний алгоритм;
5. обработка каждого класса ошибки;
6. ownership;
7. передача результата следующему модулю;
8. соответствующие tests;
9. отражение в report;
10. ограничения и возможные улучшения.

Следующая глава начинает с `main.c`, потому что именно там все модули соединяются в реально выполняемый сценарий.

Расширенная лекция 9. CLI lifecycle и границы ответственности

Том II. `main.c`: полный жизненный цикл одного анализа

Роль `main.c`

`main.c` не реализует формат PE или unwind encoding. Его задача - orchestration: создать правильную последовательность вызовов, сохранить результаты, сформировать requested outputs и освободить ресурсы.

Эта роль часто недооценивается. Даже корректные отдельные модули образуют некорректный продукт, если orchestration:

- вызывает их в неверном порядке;
- игнорирует failure;
- передаёт несогласованные pointers;
- создаёт отчёт до вычисления metadata;
- освобождает объект слишком рано;
- возвращает неверный exit code.

Поэтому `main.c` является формальным описанием полного user workflow.

Входной интерфейс процесса

Сигнатура:

```
int main(int argc, char** argv)
```

`argc` содержит число аргументов.

`argv` указывает на массив pointers на null-terminated byte strings.

Для команды:

```
eh_unwind_analyzer benign_richer.exe --heuristics --json report.json
```

логическая модель:

```
argc = 5
argv[0] = "eh_unwind_analyzer"
argv[1] = "benign_richer.exe"
argv[2] = "--heuristics"
argv[3] = "--json"
argv[4] = "report.json"
```

Операционная система и C runtime формируют эти значения до вызова `main`.

Внутренняя структура `Args`

```
typedef struct Args {
    const char* input;
    bool dump;
    bool show_ops;
    bool heuristics;
    const char* csv_path;
    const char* ghidra_csv_path;
    const char* json_path;
    const char* html_path;
    int dump_limit;
    bool follow_chain;
    int max_chain_depth;
    bool dynamic;
    uint32_t timeout_ms;
    uint32_t trace_steps;
    const char** child_argv;
    int child_argc;
} Args;
```

Эта структура является нормализованным представлением command line.

Поля `const char*` не владеют строками. Они указывают на memory, предоставленную через `argv`, и действительны в течение `main`.

Boolean fields определяют включённые режимы.

Числовые fields уже должны быть проверены по диапазонам.

`child_argv` и `child_argc` описывают arguments target после разделителя `--`.

Почему parsing arguments отделён

Функция:

```
static ParseResult parse_args(int argc, char** argv, Args* a)
```

обнуляет output:

```
memset(a, 0, sizeof(*a));
```

а затем задаёт nonzero defaults:

```
a->dump_limit = -1;
a->follow_chain = true;
a->max_chain_depth = 8;
a->timeout_ms = 15000;
a->trace_steps = 64;
```

Обнуление создаёт доказуемое начальное состояние всех optional pointers и flags. Defaults задаются явно после него.

Возвращаемый `ParseResult` различает:

```
PARSE_OK
PARSE_HELP
PARSE_VERSION
PARSE_ERR
```

Boolean результата было бы недостаточно: help и version являются успешным завершением пользовательского запроса, а malformed CLI - ошибкой.

Позиционный input

Если первый аргумент равен `--help` или `--version`, input file не требуется.

Иначе:

```
a->input = argv[1];
```

Проект использует первый позиционный аргумент как путь target. Остальные аргументы до `--` должны быть известными options.

Это означает, что текущий CLI не поддерживает произвольное расположение input после options. Backend должен следовать документированному contract.

Разделитель `--`

Option:

```
--
```

завершает options анализатора. Все последующие strings передаются target в dynamic mode.

Пример:

```
eh_unwind_analyzer target.exe --dynamic -- --input sample.dat
```

До separator:

```
--dynamic
```

принадлежит анализатору.

После separator:

```
--input sample.dat
```

принадлежит target.

Без separator невозможно однозначно определить, чья option `--input`.

Проверка чисел

`parse_long_range` использует `strtol`.

Она проверяет:

- null/empty input;
- `ERANGE`;
- отсутствие цифр;
- наличие trailing characters;
- minimum и maximum.

Условие:

```
end == text
```

означает, что не была распознана ни одна цифра.

Условие:

```
*end != '\0'
```

означает, что после числа остался непотреблённый текст, например `10abc`.

Для unsigned values `parse_u32_range` дополнительно отклоняет начальный `-`, потому что `strtoul` имеет поведение, которое могло бы преобразовать отрицательную запись в большое unsigned value.

Диапазоны options

`--max-chain-depth` допускает `1..16`.

Нижняя граница исключает бессмысленную нулевую глубину при включённом follow chain.

Верхняя граница ограничивает рекурсивную или итеративную работу на malicious chain.

`--trace-steps` допускает `0..256`. Ноль отключает trace. Верхняя граница соответствует фиксированному массиву:

```
uint64_t trace[256];
```

CLI validation и размер модели согласованы.

`--timeout-ms` требует значение больше нуля и допускает диапазон `uint32_t`.

Exit code CLI parsing

После:

```
ParseResult pres = parse_args(argc, argv, &a);
```

`main` обрабатывает результат:

```
if (pres != PARSE_OK) {
    if (pres == PARSE_VERSION) {
        print_version();
        return 0;
    }
    usage(argv[0]);
    return (pres == PARSE_HELP) ? 0 : 2;
}
```

Help и version возвращают 0. Invalid command line возвращает 2.

Различие важно для automated caller:

```
0: запрос выполнен
1: runtime/input/output failure
2: неправильный способ вызова
```

Чтение input

Объявляются:

```
uint8_t* owned = NULL;
Buf b;
```

После успешного `buf_init_from_file`:

```
owned == b.data
b.size == file size
```

но roles различаются. `owned` используется для освобождения, `b` - для чтения.

Если чтение не удалось, другие объекты ещё не созданы, поэтому cleanup минимален.

Error buffers

Проект использует `StrBuf` для накопления подробных diagnostics:

```
StrBuf err;
sb_init(&err, 1024);
```

В текущем `main` return value `sb_init` в некоторых местах не проверяется немедленно. `StrBuf` сохраняет `failed`, а последующие append возвращают false. Для критических output metadata проверки выполняются составным условием.

Инженерное улучшение могло бы унифицировать проверку каждого `sb_init`, но test/sanitizer coverage снижает риск незамеченного поведения.

PE parsing

```
PeImage img;
if (!pe_parse(&img, &b, &err)) {
    ...
    return 1;
}
```

До вызова `img` не инициализирован вызывающим кодом. Контракт `pe_parse` начинается с:

```
memset(out, 0, sizeof(*out));
```

При failure `main` не вызывает `pe_free`, потому что parser сам освобождает локально выделенные sections на соответствующих failure paths и не возвращает успешное owned state.

При success ownership `img.sections` переходит вызывающему, и позднее требуется `pe_free`.

Runtime function table

```
RuntimeFunction* rf = NULL;
size_t rf_count = 0;
```

Использование null и zero перед вызовом обеспечивает определённое состояние даже при failure.

`unwind_read_runtime_functions` выделяет массив. После success:

```
rf[0..rf_count-1]
```

доступен caller. Освобождение выполняется `free(rf)`.

Если exception directory корректно отсутствует, допустим success с `rf_count == 0`. Код последующих allocations учитывает ноль.

Allocations для per-function state

```
UnwindSummary* us =
    (UnwindSummary*)calloc(rf_count ? rf_count : 1,
                           sizeof(UnwindSummary));
bool* us_ok =
    (bool*)calloc(rf_count ? rf_count : 1,
                  sizeof(bool));
```

Почему при `rf_count == 0` выделяется один элемент?

Стандарт C допускает implementation-defined поведение `malloc(0)`/`calloc(0, ...)`, включая null. Проект хочет различать реальный out-of-memory и пустую таблицу, поэтому запрашивает минимум один элемент.

Логическая длина всё равно равна `rf_count`; фиктивный элемент не обрабатывается.

`calloc` обнуляет память:

- `us_ok[i]` изначально false;
- все fields `UnwindSummary` изначально zero;
- `parse_error` содержит empty string.

Options unwind parser

```
UnwindParseOpts uopts = {
    .max_chain_depth = a.max_chain_depth,
    .follow_chain = a.follow_chain
};
```

Designated initializers явно связывают значения с именами fields.

Эта структура отделяет policy пользователя от обязательных параметров parser. В будущем в неё можно добавить controlled options, но изменение public structure потребует проверки ABI/library use.

Основной цикл unwind

```
for (size_t i = 0; i < rf_count; i++) {
    StrBuf perr;
    sb_init(&perr, 256);
    bool ok = unwind_parse(
        &b,
        &img,
        rf[i].unwind_rva,
        &uopts,
        &us[i],
        a.show_ops ? &ops : NULL,
        &perr
    );
    us_ok[i] = ok;
    sb_free(&perr);
}
```

На каждой итерации:

1. выбирается `rf[i].unwind_rva`;
2. parser использует общие immutable `b` и `img`;
3. результат записывается точно в `us[i]`;
4. success записывается в `us_ok[i]`;
5. локальная diagnostic освобождается;

6. цикл продолжается независимо от failure одной записи.

`perr` не печатается в этом цикле, потому что `unwind_parse` копирует конкретную причину в `us[i].parse_error`. Локальный buffer используется для внутреннего diagnostic contract.

Почему операции могут разбираться повторно

В первом цикле текст operations строится только при `a.show_ops`, но общий `ops` не хранит отдельный текст для всех functions.

В dump loop при необходимости `unwind_parse` вызывается повторно для конкретной записи и строит `ops2`.

Преимущество:

- не хранится большой массив текстов для всех функций;
- detailed output создаётся только для отображаемых entries.

Недостаток:

- выбранные structures разбираются второй раз.

Для CLI dump это разумный обмен memory на computation. Parser детерминирован относительно immutable input и options.

Heuristic stage

`HeuristicReport` объявляется на stack:

```
HeuristicReport hr;
HeuristicReport* hr_ptr = NULL;
```

Pointer остаётся null, пока анализ не завершится успешно.

После success:

```
hr_ptr = &hr;
```

Reports получают null, если heuristics не запрошены или failed. Это позволяет сериализатору различать отсутствие этапа и заполненную модель.

Если `heuristics_analyze` failed, `exit_code` становится 1, но код может продолжить формировать другие requested outputs. Такая стратегия позволяет пользователю получить частичный результат, не скрывая failure через process status.

Dynamic stage

`DynamicReport dr` предварительно обнуляется:

```
memset(&dr, 0, sizeof(dr));
```

Флаг:

```
bool dr_initialized = false;
```

нужен для корректного вызова `dynamic_report_free`. После вызова collector считается, что report прошёл init path, даже если session не ran успешно.

`dr_ptr` передаётся reports, если dynamic mode был запрошен. Тогда JSON способен показать failure/status, а не просто отсутствие блока.

Outputs CSV

CSV writers вызываются независимо:

```
report_write_csv(..., false, ...)
report_write_csv(..., true, ...)
```

Последний boolean выбирает full CSV или Ghidra-oriented variant.

Каждый output имеет собственный error buffer и собственную обработку. Failure одного файла устанавливает `exit_code = 1`, но не запрещает попытку создать остальные.

Подготовка provenance

JSON и HTML требуют metadata. `main` строит:

```
StrBuf invocation;
char timestamp[32];
char input_sha256[65];
ReportMetadata metadata;
```

Составное условие требует success всех операций:

```
sb_init
build_invocation
app_utc_timestamp
sha256_hex
```

Только после этого:

```
metadata_ptr = &metadata;
```

Если metadata неполна, JSON/HTML не записываются как будто полноценные. Это сохраняет report invariant: каждый полный report должен быть привязан к tool build и input hash.

Redaction arguments target

`build_invocation` проходит исходный `argv`. При достижении `--` он не копирует последующие значения, а записывает:

```
-- [N target arguments redacted]
```

Причина не связана с синтаксисом JSON. Target arguments могут содержать:

- credentials;
- tokens;
- персональные данные;
- внутренние пути;
- содержимое запроса.

Для воспроизводимости сохраняется факт и количество, но не значения.

SHA-256

`sha256_hex(b.data, b.size, input_sha256)` вычисляет digest всех входных байтов.

Если файл изменится хотя бы в одном бите, ожидается другое digest. SHA-256 связывает отчёт с конкретным содержимым, но не доказывает происхождение или безопасность файла.

JSON и HTML

Writers получают одинаковый analytical model:

```
path
PeImage
RuntimeFunction[]
UnwindSummary[]
us_ok[]
HeuristicReport optional
DynamicReport optional
ReportMetadata
```

Различается presentation:

- JSON предназначен для machine integration;
- HTML предназначен для interactive human review.

Логические значения должны согласовываться между форматами. Unit tests проверяют оба serializers.

Финальный cleanup

Порядок:

```
heuristics_free
dynamic_report_free
sb_free(&ops)
free(us_ok)
free(us)
free(rf)
pe_free(&img)
sb_free(&err)
sb_free(&err2)
buf_free_owned(owned)
```

Порядок следует зависимостям. Reports уже записаны, поэтому borrowed pointers больше не используются.

`PeImage.sections` освобождается до input bytes, но мог бы освобождаться и после: sections является отдельной allocation и не указывает внутрь input. Главное - не использовать models после release.

Полный автомат состояний `main`

```
START
-> CLI_PARSED
-> INPUT_LOADED
-> PE_PARSED
-> RF_TABLE_READ
-> UNWIND_ATTEMPTED_FOR_ALL
-> OPTIONAL_HEURISTICS
-> OPTIONAL_DYNAMIC
-> OPTIONAL_OUTPUTS
-> CLEANUP
-> EXIT
```

Fatal transitions до построения основной модели немедленно ведут в cleanup соответствующего уровня и exit 1.

Локальные analytical failures могут позволять продолжение, но изменяют final exit code.

Tests, относящиеся к `main`

`test_cli_contract.py` проверяет:

- help/version;
- invalid options;
- numeric ranges;

- exit codes;
- output combinations.

`run_regression_samples.py` проверяет end-to-end pipeline на шести PE.

`test_report_semantics.py` запускает CLI и проверяет отношения полей.

`test_release_metadata.py` проверяет согласованность version/schema declarations.

Mutation test многократно вызывает CLI на изменённых входах и требует controlled result вместо crash.

Что можно улучшить

Текущий `main` является последовательным orchestration layer. Возможные улучшения:

- единый cleanup block для уменьшения повторения ранних failure paths;
- structured machine-readable error output;
- более детальные exit subcodes;
- `--quiet`;
- `--output-dir`;
- сохранение duration каждого этапа;
- явное различие `not_requested`, `failed`, `completed` для каждого stage;
- streaming или batching для очень больших reports.

Эти изменения не требуют менять предметные алгоритмы PE/unwind.

Проверка полного понимания `main`

Читатель должен суметь для любой строки основного pipeline ответить:

1. какие объекты уже существуют;
2. кто ими владеет;
3. какие инварианты доказаны;
4. что произойдёт при failure;
5. продолжится ли анализ;
6. какой exit code сформируется;
7. какие outputs ещё могут быть созданы;
8. где объект освобождается.

Расширенная лекция 10. Безопасное чтение байтов и util-слой

Том II. `util.c`: основание безопасного parser и генерации отчётов

Почему `util.c` является частью доказательства корректности

`pe.c` и `unwind.c` содержат предметные знания о формате. Но все их проверки бессмысленны, если базовая операция чтения способна выйти за границу или неверно собрать integer.

`util.c` формирует минимальный trusted computing base проекта на уровне пользовательского кода. Ошибка здесь распространялась бы на все последующие conclusions.

`Buf` как read-only view

```
typedef struct Buf {
    const uint8_t* data;
    size_t size;
} Buf;
```

`Buf` не содержит capacity, mutable pointer или ownership flag. Он описывает только доступное для чтения содержимое.

`const uint8_t*` запрещает изменение байтов через этот pointer на уровне type system C. Это shallow const: сама структура `Buf` может быть присвоена, но pointed bytes через неё не модифицируются.

`size` задаёт единственную допустимую границу. Parser не использует null terminator, потому что binary input может содержать нулевые байты в любой позиции.

`buf_init_from_file`

Контракт:

```
bool buf_init_from_file(
    Buf* out,
    const char* path,
    uint8_t** owned_mem
);
```

Preconditions:

- все три pointers ненулевые;
- `path` указывает на корректную C string.

Postcondition success:

```
out->data == *owned_mem
out->size == фактическое число прочитанных байтов
*owned_mem != NULL
```

Postcondition failure:

```
*owned_mem == NULL
нет открытого FILE
нет неосвобождённой allocation, созданной функцией
```

Функция сначала выполняет:

```
*owned_mem = NULL;
```

Это важно для failure paths caller. Даже если открытие не удалось, caller не получает неопределённый pointer.

Определение размера

Используются `fseek` и `ftell`. `ftell` возвращает `long`, поэтому текущая реализация имеет практическое ограничение для файлов, размер которых не представим `long` на host.

На современных 64-битных POSIX и Windows toolchains это обычно достаточно для анализируемых PE, но строгая поддержка очень больших файлов потребовала бы platform-specific 64-bit file APIs.

Отрицательный результат `ftell` считается failure:

```
if (sz < 0) ...
```

Пустой файл

Для размера zero:

```
size_t alloc_size = sz > 0 ? (size_t)sz : 1u;
```

Выделяется один байт, хотя логический `Buf.size` равен нулю.

Причина: `malloc(0)` может вернуть null или уникальный pointer в зависимости от реализации. Проект хочет, чтобы failure allocation не смешивался с пустым input.

Ни один байт фиктивной allocation не считается частью input, потому что bounds определяет `size`.

Полнота `fread`

```
size_t rd = fread(mem, 1, (size_t)sz, f);
```

Функция требует:

```
rd == sz
```

Частичное чтение не принимается. Если файл изменился, возник I/O error или чтение прервано, allocation освобождается.

Это сохраняет invariant: SHA-256 и parser работают с полной последовательностью заявленного размера.

`buf_free_owned`

```
void buf_free_owned(uint8_t* owned_mem) {
    free(owned_mem);
}
```

Отдельная wrapper function документирует ownership pair:

```
buf_init_from_file -> buf_free_owned
```

`free(NULL)` допустим стандартом C, поэтому функция безопасна для null.

`buf_read`

Контракт:

```
bool buf_read(
    const Buf* b,
    size_t off,
    void* out,
    size_t len
);
```

`void* out` означает, что destination type не фиксирован интерфейсом. Функция копирует raw bytes.

Проверка:

```
if (!b || !out) return false;
```

Даже при `len == 0` проект требует non-null `out`. Это упрощает contract.

Далее:

```
if (off > b->size) return false;
```

`off == size` допустим только для zero-length read.

Затем:

```
if (len > b->size - off) return false;
```

После доказательства диапазона вызывается:

```
memcpy(out, b->data + off, len);
```

`memcpy` требует, чтобы source и destination ranges были валидны и не перекрывались. Destination предоставляется caller. Source доказан `Buf`.

Typed read functions

`buf_read_u16` не записывает прямо в `uint16_t` из arbitrary file address:

```
uint8_t tmp[2];
if (!buf_read(b, off, tmp, 2)) return false;
*out = le16(tmp);
```

Причины:

- file offset может быть невыровнен;
- host endianness не должен влиять;
- чтение C-object через cast может нарушать aliasing/alignment rules;
- bounds должны быть проверены до interpretation.

Такой подход немного многословнее direct cast, но создаёт переносимый parser.

`le16`, `le32`, `le64`

`le64` строится из двух `le32`:

```
return (uint64_t)le32(p) |
       ((uint64_t)le32(p + 4) << 32);
```

Cast к `uint64_t` выполняется до сдвига на 32. Если сдвигать 32-битное значение на 32 без расширения, операция была бы недопустимой.

Эти helpers являются pure functions: результат зависит только от входных байтов, state не изменяется.

`StrBuf`

```
typedef struct StrBuf {
    char* s;
    size_t len;
    size_t cap;
    bool failed;
} StrBuf;
```

`s` указывает на allocation.

`len` - число содержательных bytes без завершающего null.

`cap` - вместимость allocation в bytes.

`failed` - sticky error state.

Invariant успешного initialized buffer:

```
s != NULL
len < cap
s[len] == '\0'
failed == false
```

Наличие null terminator позволяет передавать `s` функциям C strings, а `len` позволяет не вычислять `strlen` после каждого append.

Почему error state sticky

Если одна операция append не смогла выделить память, дальнейшее продолжение могло бы сформировать усечённый JSON. Поэтому:

```
if (!sb || sb->failed) return false;
```

После failure buffer остаётся failed до `sb_free`/новой initialization.

Caller может проверить итог:

```
sb_ok(&buffer)
```

Это transactional-like property на уровне построения текста: нельзя принять partial output как complete.

Рост capacity

`sb_reserve` увеличивает capacity:

```
increment = newcap / 2 + 64
newcap += increment
```

Рост больше линейного снижает число `realloc` при последовательных append.

Перед сложением:

```
if (increment > SIZE_MAX - newcap)
```

проверяется overflow.

Если geometric growth не может быть вычислен, функция использует точно `need_cap`. Затем `realloc` либо возвращает новую allocation, либо оставляет старую действительной и возвращает null. При null устанавливается `failed`.

`sb_appendn`

Проверка required capacity:

```
if (sb->len == SIZE_MAX ||
    n > SIZE_MAX - sb->len - 1u)
```

+1 нужен для null terminator.

После reserve:

```
memcpy(sb->s + sb->len, s, n);
sb->len += n;
sb->s[sb->len] = 0;
```

Invariant восстанавливается после каждой успешной операции.

`sb_appendf`

Variadic function принимает printf format.

Первый `vsnprintf(NULL, 0, ...)` вычисляет необходимую длину. Для повторного использования аргументов создаётся `va_copy`, потому что проход `vsnprintf` изменяет состояние `va_list`.

После reserve второй `vsnprintf` пишет строку. Проверяется:

```
wrote == need
```

Compiler attribute `format(printf, ...)` для GCC/Clang позволяет статически проверять соответствие format и аргументов.

Атомарная запись

`file_write_atomic` создаёт temporary sibling path:

```
<path>.tmp.<pid>
```

Пишет полное содержимое, выполняет `fflush`, закрывает stream, затем заменяет destination.

До replace существующий report остаётся неизменным. Если write failed, temp удаляется.

На Windows:

```
MoveFileExA(
    temp_path,
    path,
    MOVEFILE_REPLACE_EXISTING |
    MOVEFILE_WRITE_THROUGH
)
```

На POSIX:

```
rename(temp_path, path)
```

Соседнее расположение важно для атомарности rename в пределах файловой системы.

Ограничение: текущая реализация не выполняет POSIX `fsync` для file descriptor и parent directory. `fflush` переносит данные из user-space stdio buffer, но не является полной гарантией durability при system crash. Для CLI report это разумный уровень, но high-assurance storage мог бы усилить protocol.

SHA-256

SHA-256 преобразует сообщение произвольной конечной длины в 256-bit digest.

Проект хранит результат как 64 lowercase hex characters:

```
256 bits / 4 bits per hex digit = 64 digits
```

Плюс null terminator:

```
char out_hex[65];
```

Digest используется для identity input bytes. Он не:

- шифрует файл;
- подтверждает автора;
- является digital signature;
- определяет maliciousness.

Связь `util.c` с отчётом и tests

`Buf` используется всеми parsers.

`StrBuf` используется diagnostics и serializers.

Atomic writer используется CSV/JSON/HTML.

SHA-256 попадает в report metadata.

`unit_core_edges` проверяет boundary conditions.

Mutation fuzz проходит через `Buf` operations на изменённых inputs.

JSON/HTML tests косвенно проверяют `StrBuf` и atomic output.

Sanitizers проверяют memory safety на runtime test paths.

Полная роль модуля

`util.c` обеспечивает четыре свойства всего продукта:

1. input bytes имеют явную длину;
2. каждое чтение доказывает границы;
3. многобайтные значения декодируются независимо от host layout;
4. output либо формируется полностью, либо отмечается failure.

Следующий модуль `pe.c` опирается на эти свойства и добавляет семантику формата.

Расширенная лекция 11. PE parser проекта

Том II. `pe.c`: построение модели PE из недоверенных байтов

Почему PE parser выполняется раньше unwind decoder

`UNWIND_INFO` адресуется через RVA. Пока проект не прочитал section table, он не способен установить, какой file offset соответствует RVA. Поэтому `pe_parse` создаёт систему координат для всей последующей работы.

На входе функции известны только:

```
Buf.data
Buf.size
```

На выходе должны быть известны:

```
архитектура image
ImageBase
SizeOfImage
SizeOfHeaders
alignments
exception directory
sections
структурные counters
```

`PeImage` не является полной моделью всего PE. Проект намеренно извлекает только поля, необходимые для своего scope.

Почему parser не использует Windows headers

`pe.h` объявляет собственные минимальные constants и структуры:

```
### define PE_DOS_MAGIC 0x5A4D
### define PE_NT_MAGIC  0x00004550
### define IMAGE_FILE_MACHINE_AMD64 0x8664
### define IMAGE_NT_OPTIONAL_HDR64_MAGIC 0x20b
```

Преимущества:

- static core компилируется на Linux и macOS;
- parser не зависит от packing C-структур Windows SDK;
- каждое поле читается с явным bounds-check;
- поддерживаемый subset виден в коде;
- анализируется файл, а не host representation.

Недостаток - offsets и constants нужно поддерживать в соответствии со спецификацией. Tests и официальная документация являются обязательной частью такого решения.

PeSection

```
typedef struct PeSection {
    char name[9];
    uint32_t virtual_address;
    uint32_t virtual_size;
    uint32_t raw_ptr;
    uint32_t raw_size;
    uint32_t characteristics;
} PeSection;
```

Это нормализованная модель section header, а не byte-for-byte копия 40-байтового PE structure.

В PE имя занимает ровно 8 байт и не обязано завершаться null. В модели выделено 9:

```
memcpy(secs[i].name, name8, 8);
secs[i].name[8] = 0;
```

Так report code безопасно использует имя как C string.

Поля relocation/line number из section header не сохраняются, потому что не используются scope проекта.

PeImage

`PeImage` объединяет три класса данных.

Первый класс - identity/layout:

```
machine
image_base
size_of_image
size_of_headers
section_alignment
file_alignment
```

Второй - предметный каталог:

```
exception_rva
exception_size
```

Третий - результаты валидации:

```
section_raw_oob
section_virtual_oob
section_raw_overlaps
section_virtual_overlaps
section_misaligned
exception_directory_oob
```

Parser не только извлекает поля, но и сохраняет факты о нарушениях для heuristics.

Инициализация результата

```
if (!out || !b || !err) return false;
memset(out, 0, sizeof(*out));
```

После `memset`:

- `out->sections == NULL`;
- counters равны нулю;
- flags false.

Поле ok устанавливается только в конце success path. Caller не должен использовать модель после false return.

DOS header

PE сохраняет исторически совместимый DOS header. Первые два байта содержат `e_magic`.

При little-endian чтении байты:

```
4D 5A
```

дают:

```
0x5A4D
```

ASCII-представление байтов читается как `MZ`.

Функция:

```
read_dos_lfanew
```

читает magic и 32-битное поле по file offset `0x3C`.

`e_lfanew` - file offset новой PE signature.

Почему offset PE header не зафиксирован? Между DOS header и PE header может находиться DOS stub и padding различной длины. Поэтому parser использует поле, а не предполагает значение `0x80`.

Проверка минимального PE header range

```
if ((size_t)lfanew + 4 + 20 > b->size)
```

`4` - размер PE signature.

`20` - размер COFF file header.

Parser доказывает, что обязательная фиксированная часть помещается в файл до чтения полей.

Теоретически сумма может переполнить `size_t`, если `size_t` 32-битный и `lfanew` близок к максимуму. Поддерживаемые современные 64-битные builds исключают это для `uint32_t lfanew`. Для абсолютно общего parser предпочтительна subtractive range check, аналогичная `buf_read`.

Фактические typed reads всё равно проходят через `buf_read`, поэтому out-of-bounds copy не происходит. Предварительная проверка формирует более точную diagnostic.

PE signature

По `lfanew` читается:

```
uint32_t sig;
buf_read_u32(...);
```

Ожидаемое значение:

```
0x00004550
```

Байты:

```
50 45 00 00
```

первые два соответствуют ASCII `PE`, следующие два равны zero.

Наличие `MZ` без корректной PE signature не делает файл поддерживаемым PE.

COFF file header

Сразу после signature находится 20-byte COFF header.

Проект читает:

```
offset +0: Machine
offset +2: NumberOfSections
offset +16: SizeOfOptionalHeader
```

Почему не читается timestamp? Он не нужен для PE/unwind pipeline. Минимальный parser не обязан копировать каждое поле формата.

`Machine` определяет архитектуру machine code и architecture-specific directories. Проект принимает только:

```
0x8664 = IMAGE_FILE_MACHINE_AMD64
```

Это ранняя проверка предотвращает попытку применить AMD64 unwind decoder к ARM64 или x86.

Optional header

Название `Optional Header` историческое. Для PE image эта структура необходима.

Её начало:

```
size_t opthdr = filehdr + 20;
```

`SizeOfOptionalHeader` берётся из COFF header. Parser сначала доказывает:

```
opthdr + opt_size <= file_size
```

и минимальный размер:

```
opt_size >= 0x70
```

Смещение `0x70` является началом data directories в PE32+.

Magic PE32+

Первые 2 байта optional header:

```
0x10B = PE32
0x20B = PE32+
```

`PE32+` используется для 64-битных images и имеет другое layout некоторых fields. Поэтому проверка magic обязательна даже после AMD64 Machine.

Проект отклоняет PE32, а не пытается читать его по PE32+ offsets.

Ключевые поля optional header

Offsets относительно начала PE32+ optional header:

```
0x18 ImageBase, 8 bytes
0x20 SectionAlignment, 4
0x24 FileAlignment, 4
0x38 SizeOfImage, 4
0x3C SizeOfHeaders, 4
0x6C NumberOfRvaAndSizes, 4
```

Offsets определены PE32+ layout.

`ImageBase`

Preferred base image. Используется для presentation и преобразования RVA в preferred VA. Static structural checks в основном работают с RVA.

`SectionAlignment`

Выравнивание sections в virtual image.

`FileAlignment`

Выравнивание raw section data в файле.

`SizeOfImage`

Размер virtual image с учётом section alignment. Используется как верхняя структурная граница RVA.

`SizeOfHeaders`

Размер области headers, отображаемой в image. Для RVA внутри этой области проект допускает прямое соответствие file offset.

`NumberOfRvaAndSizes`

Число реально объявленных data directory entries. Нельзя обращаться к index 3, если число меньше 4.

Фатальные проверки полей

`SizeOfImage == 0` не позволяет сформировать meaningful RVA boundary.

`SizeOfHeaders == 0` или превышение file size отклоняется. Проект требует физическое наличие всех declared headers.

Нулевой alignment недопустим, потому что modulo operation и модель размещения потеряли бы смысл.

Недостаточное число directories делает exception directory недоступным.

Эти conditions являются fatal, потому что последующий mapping не может быть надёжно выполнен.

Data directory

Каждая standard data directory entry состоит из:

```
VirtualAddress: 4 bytes
Size: 4 bytes
```

Exception directory имеет index 3.

Относительный offset:

```
0x70 + 3 * 8 = 0x88
```

Parser отдельно проверяет, что вся entry помещается в фактический `opt_size`, а не только в теоретический standard layout.

Результат:

```
exc_rva
exc_sz
```

Оба могут быть zero, что означает отсутствие directory.

Если одно zero, а второе nonzero, `pe_parse` не завершается fatal failure, но выставляет `exception_directory_oob`. Позднее strict reader отклоняет malformed pair. Такое двухуровневое поведение сохраняет `PeImage` и validation counters для отчёта, когда это возможно.

Section table

Section headers следуют непосредственно после optional header:

```
secthdr = opthdr + opt_size;
```

Каждый header занимает 40 байт.

Общий конец:

```
needed = secthdr + nsects * 40;
```

должен находиться внутри файла.

Массив:

```
calloc(nsects ? nsects : 1, sizeof(PeSection))
```

при zero sections всё равно возвращает обычную allocation. Логическое `num_sections` остаётся zero.

Чтение section fields

Для каждой section:

```
+0  Name[8]
+8  VirtualSize
+12 VirtualAddress
+16 SizeOfRawData
+20 PointerToRawData
+36 Characteristics
```

Все integer fields little-endian.

Если любое чтение не удалось, весь массив освобождается до return false. Ownership не передаётся caller частично.

Virtual span policy

Проект использует:

```
max(VirtualSize, SizeOfRawData)
```

для поиска candidate section и проверки virtual range.

Причина: реальные PE могут иметь одно из fields меньше другого из-за alignment и producer behavior. Использование maximum позволяет классифицировать RVA в расширенном logical span.

Но raw availability проверяется отдельно:

```
delta < raw_size
```

Следовательно, membership section и наличие bytes в файле не смешиваются.

Это осознанная parser policy. Другой parser может выбирать иной span для некоторых malformed images, поэтому overlaps фиксируются как ambiguity.

Raw OOB

Для section:

```
raw_end = raw_ptr + raw_size
```

вычисляется в `uint64_t`.

Если `raw_size > 0` и:

```
raw_end > file_size
```

section заявляет bytes за EOF.

Parser не прекращает работу автоматически. Он увеличивает `section_raw_oob`. Это позволяет heuristics создать finding и сохранить остальные fields.

Однако конкретное чтение через `buf_read` всё равно будет отклонено.

Virtual OOB

```
virtual_end = virtual_address + virtual_span
```

Если:

```
virtual_end > SizeOfImage
```

section не помещается в declared image.

Это структурное противоречие между section table и optional header.

Alignment

Проверяется:

```
virtual_address % section_alignment == 0
```

и для section с raw data:

```
raw_ptr % file_alignment == 0
```

Нарушение увеличивает `section_misaligned`.

Project rule не реализует все специальные тонкости PE для очень малых alignments как отдельную сложную policy. Поэтому finding является аналитическим сигналом и требует сверки со спецификацией/loader behavior.

Overlap

Два полуоткрытых диапазона пересекаются, если:

```
a_begin < b_end
и
b_begin < a_end
```

Zero-sized ranges не считаются пересекающимися.

Проект проверяет каждую новую section со всеми предыдущими. При небольшом числе sections сложность $O(s^2)$ приемлема и сохраняет простой проверяемый код.

Raw overlap означает, что section headers претендуют на общие file bytes.

Virtual overlap означает, что один RVA может соответствовать нескольким sections согласно выбранной span policy.

Virtual overlap особенно опасен для mapping: `pe_find_section_for_rva` возвращает первую подходящую section, а другой parser или loader может интерпретировать ambiguous image иначе.

Exception directory boundary

Если directory не пуст:

```
exc_end = exc_rva + exc_sz
```

вычисляется в `uint64_t`.

Anomaly выставляется, если:

- RVA zero;

- size zero;
- конец выходит за `SizeOfImage`.

На этом этапе ещё не доказывается наличие raw bytes. Это выполняет `unwind_read_runtime_functions` через `pe_rva_to_fileoff`.

Завершение модели

Только после полного чтения sections и validation:

```
out->ok = true;
out->sections = secs;
...
return true;
```

Ownership `secs` переходит `PeImage`.

`pe_free`

```
free(img->sections);
memset(img, 0, sizeof(*img));
```

Обнуление после free:

- удаляет dangling pointer из структуры;
- предотвращает случайное повторное использование старых fields;
- делает повторный `pe_free` безопасным относительно `sections == NULL`.

`pe_find_section_for_rva`

Функция перебирает sections и проверяет:

```
start <= RVA < end
```

`start` и `end` расширены до `uint64_t`, поэтому section у верхней границы `uint32_t` не оборачивается при сложении.

Test `test_section_range_wrap` использует:

```
VirtualAddress = 0xFFFFFF00
VirtualSize = 0x300
```

и подтверждает корректную работу widened arithmetic.

`pe_rva_to_fileoff`

Алгоритм:

```
найти section
-> delta = RVA - VirtualAddress
-> доказать delta < RawSize
-> safely add RawPtr + delta
-> доказать offset < file_size
-> вернуть offset
```

Если section не найдена, проверяется headers:

```
RVA < SizeOfHeaders
RVA < file_size
```

и используется прямое mapping.

Почему headers проверяются после sections? В корректном image их virtual ranges не должны конфликтовать. При malformed overlap chosen policy должна быть документирована. Текущий код отдаёт приоритет section membership.

Сквозной пример `benign_richer.exe`

Report показывает:

```
file size = 5641
ImageBase = 0x140000000
SizeOfImage = 0x7000
SizeOfHeaders = 0x400
SectionAlignment = 0x1000
FileAlignment = 0x200
Exception RVA = 0x3000
Exception Size = 0x24
```

Section `.pdata`:

```
VirtualAddress = 0x3000
VirtualSize = 0x24
RawPtr = 0x800
RawSize = 0x200
```

Для exception RVA:

```
delta = 0x3000 - 0x3000 = 0
file offset = 0x800 + 0 = 0x800
```

Размер `0x24` равен 36 decimal. Конец directory в файле:

```
0x800 + 0x24 = 0x824
```

он находится внутри raw section и файла.

Section `.xdata`:

```
VirtualAddress = 0x4000
RawPtr = 0xA00
```

Unwind RVA `0x4004`:

```
delta = 4
file offset = 0xA04
```

Именно оттуда `unwind_parse` читает 4-byte header второй структуры.

Переход от parser counter к finding

`pe_parse` не назначает score.

Он сохраняет:

```
section_raw_oob = N
```

`heuristics_analyze` позднее создаёт:

```
code = pe_section_raw_oob
family = table_integrity
severity = high
weight = 25
```

Так format layer остаётся отделён от analytical policy.

Tests

`unit_core_edges` проверяет:

- widened section range;
- header mapping по declared size;
- empty exception directory;
- PE counters, передаваемые heuristics.

Regression sample `suspicious_section_raw_oob.exe` проверяет end-to-end:

- raw OOB;
- raw overlap;
- virtual overlap;
- score 60;
- report fields.

Mutation fuzz изменяет headers и требует контролируемый отказ или valid report.

Ограничения `pe.c`

Модуль не:

- проверяет цифровую подпись;
- разбирает imports/exports;
- анализирует relocations;
- разбирает TLS;
- проверяет checksum;
- эмулирует точное loader behavior для всех malformed cases;
- поддерживает другие architectures.

Это минимальный parser, сфокусированный на exception/unwind analysis.

Что должно быть понятно перед `unwind.c`

`pe_parse` не находит функции по machine code. Он находит exception directory по index 3 и создаёт mapping RVA-to-file-offset. Только после этого следующий модуль может прочитать массив 12-byte `RUNTIME_FUNCTION`.

Расширенная лекция 12. Unwind decoder проекта

Том II. `unwind.c`: от таблицы функций к модели восстановления стека

Почему Windows x64 хранит таблицу функций

Исключение может возникнуть в произвольной инструкции функции. В этот момент обычный epilog не выполнен. Значения `RSP` и nonvolatile registers отражают частично или полностью созданный stack frame.

Windows должна:

- определить, какой function range содержит текущий `RIP`;
- найти описание изменений prolog;
- восстановить caller context;
- определить наличие language-specific handler;
- продолжить поиск обработчика или stack walk.

Для nonleaf functions AMD64 image содержит function table. PE exception directory указывает на массив `RUNTIME_FUNCTION`.

`RuntimeFunction`

```
typedef struct RuntimeFunction {
    uint32_t begin_rva;
    uint32_t end_rva;
    uint32_t unwind_rva;
} RuntimeFunction;
```

Каждое поле 4 bytes. Размер записи 12.

Диапазон code:

```
[BeginAddress, EndAddress)
```

`UnwindInfoAddress` указывает на `UNWIND_INFO` либо связанную encoding.

Fields являются image-relative, поэтому хранятся как RVA.

Чтение exception directory

`unwind_read_runtime_functions` сначала обнуляет outputs:

```
*out_rf = NULL;
*out_count = 0;
```

Если RVA и size оба zero, отсутствие таблицы считается корректным:

```
return true;
```

Если zero только одно field, directory malformed.

Далее exception RVA преобразуется через `pe_rva_to_fileoff`.

Проверяется полный файловый диапазон:

```
off + exception_size <= b->size
```

и structural size:

```
size >= 12
size % 12 == 0
```

Текущая policy strict: неполный хвост отклоняет всю таблицу. Альтернативный forensic parser мог бы прочитать complete entries и сохранить remainder anomaly, но проект предпочитает однозначный contract directory.

Количество:

```
n = exception_size / 12;
```

Для sample:

```
0x24 / 0x0C = 3
```

Почему reader не проверяет begin/end сразу

Функция безопасно извлекает все triples. Semantics ranges проверяются в `heuristics.c`.

Разделение позволяет сохранить malformed values в report model и агрегировать counts. Reader отвечает на вопрос «можно ли физически прочитать таблицу», heuristics - «насколько согласованы значения».

`UnwindSummary`

Структура является нормализованным результатом декодирования:

```
valid
version
flags
```

```
prolog_size
count_codes
frame_reg
frame_off
handler
chain
fixed_stack_bytes
pushed_nonvol_count
saved register masks
anomalies
parse_error
```

Она не сохраняет все raw slots. Подробный текст может строиться отдельно через `ops_text`.

Это design choice: массовый analysis хранит компактную summary, а `--ops` повторно декодирует выбранные entries для presentation.

Register encoding

Unwind format кодирует general-purpose register числом `0..15`.

`unwind_reg_name` отображает:

```
0 RAX
1 RCX
2 RDX
3 RBX
4 RSP
5 RBP
6 RSI
7 RDI
8 R8
...
15 R15
```

Unknown value возвращает `REG?`, хотя 4-bit field физически ограничен 0..15.

Flags

Поддерживаемые:

```
UNW_FLAG_EHANDLER  = 0x01
UNW_FLAG_UHANDLER  = 0x02
UNW_FLAG_CHAININFO = 0x04
```

`EHANDLER` указывает exception handler.

`UHANDLER` указывает termination/unwind handler.

`CHAININFO` означает, что tail содержит chained `RUNTIME_FUNCTION`.

CHAININFO не должен одновременно сочетаться с handler flags в той же record. Проект отмечает такое состояние `UA_BAD_OPERAND`.

Вход `unwind_parse`

```
bool unwind_parse(
    const Buf* b,
    const PeImage* img,
    uint32_t unwind_rva,
    const UnwindParseOpts* opts,
    UnwindSummary* out,
    StrBuf* ops_text,
    StrBuf* err
);
```

`b` предоставляет bytes.

`img` предоставляет mapping и `SizeOfImage`.

`unwind_rva` выбирает structure.

`opts` задаёт chain policy.

`out` получает summary.

`ops_text` optional.

`err` получает diagnostic.

Инициализация и локализация ошибки

```
memset(out, 0, sizeof(*out));
size_t err_start = err->len;
```

`err` может уже содержать старый текст. `err_start` позволяет `unwind_parse_fail` скопировать только diagnostic текущего вызова в `out->parse_error`.

Trailing newline удаляется, потому что JSON field должен содержать compact string.

Options

Defaults:

```
max_depth = 8
follow_chain = true
```

Caller может передать options. Значение depth выше 16 clamp-ится, потому что local array:

```
uint32_t seen[16];
```

не способен хранить больше уникальных chain RVA.

Alignment `UNWIND_INFO`

RVA должен быть 4-byte aligned:

```
if ((unwind_rva & 3u) != 0)
    UA_ALIGNMENT
```

Нарушение не делает header физически нечитаемым, поэтому parser продолжает и сохраняет anomaly.

Это пример distinction:

```
structurally suspicious, but decodable
```

Mapping header

`parse_unwindinfo_header`:

1. преобразует RVA в file offset;
2. проверяет четыре bytes;
3. читает header.

Failure добавляет `UA_TRUNCATED` и возвращает false.

Термин `TRUNCATED` используется расширенно: structure физически недоступна или не помещается в file-backed range.

Header layout

Четыре байта:

```
byte 0: Version:3 | Flags:5
byte 1: SizeOfProlog
```

```
byte 2: CountOfCodes
byte 3: FrameRegister:4 | FrameOffset:4
```

Extraction:

```
ver   = hdr[0] & 0x7;
flags = hdr[0] >> 3;
fr    = hdr[3] & 0x0F;
foff  = hdr[3] >> 4;
```

`FrameOffset` измеряется units of 16 bytes. Actual offset:

```
foff * 16
```

Version policy

Поддерживаются V1 и V2.

V3 для Intel APX имеет другую структуру и не должен интерпретироваться как V1/V2. Проект:

```
UA_VERSION_UNKNOWN
return false
```

Test `test_unsupported_version_is_fatal` закрепляет это поведение.

Отказ лучше ложного успешного decode: неправильная версия сместила бы slots и tail, создав фиктивные handlers.

Frame register checks

Если `frame_reg == 0`, `frame_off` должен быть zero.

Если frame register nonzero, он должен быть допустимым GPR.

После parsing codes проект также требует наличие `UWOP_SET_FPREG`, если frame register задан и record не является chained.

Несогласованность помечается `UA_BAD_OPERAND`, но structure может оставаться valid.

CountOfCodes

`CountOfCodes` - количество двухбайтовых slots, а не логических operations.

Одна operation может занимать:

```
1, 2 или 3 slots
```

Array физически padded до чётного числа slots, чтобы tail был 4-byte aligned:

```
slots_even = (count + 1) & ~1
codes_bytes = slots_even * 2
```

Если `count = 1`, stored slots = 2, bytes = 4.

Второй slot является padding и не входит в semantic operation count.

Structure slot

Первый slot operation:

```
byte 0: CodeOffset
byte 1 low nibble: UnwindOp
byte 1 high nibble: OpInfo
```

`CodeOffset` указывает position в prolog после выполнения соответствующей operation.

Codes хранятся в порядке убывания offsets, потому что unwind применяет действия в обратном направлении.

Если очередной `CodeOffset` больше предыдущего, проект ставит:

```
UA_CODEOFFSET_ORDER
```

Если offset больше `SizeOfProlog`:

```
UA_CODEOFFSET_OUTSIDE
```

V2 epilog descriptors имеют другую семантику, поэтому исключаются из обычной проверки prolog offset.

Определение длины operation

`slots_for_code` получает:

```
version
op
opinfo
v2_seen_epilog
```

и возвращает физическое число slots.

Эта функция должна быть выполнена до чтения operands. Затем проверяется:

```
i + slots <= count
```

Неизвестный opcode получает `UA_BAD_OPCODE`, но parser безопасно продвигается на один slot. Это позволяет собрать остальные anomalies без чтения за границей.

Если известная operation заявляет operands за CountOfCodes, parsing становится fatal.

UWOP_PUSH_NONVOL

Семантика prolog:

```
push register
```

Stack pointer уменьшается на 8, значение register сохраняется.

Summary:

```
fixed_stack += 8
pushed_nonvol_count += 1
register bit added to saved_nonvol_mask
```

Проект допускает некоторые volatile register operands из-за наблюдаемого real-world toolchain behavior, исключая `RSP`. Это implementation compatibility policy, а не простое повторение имени opcode.

UWOP_ALLOC_SMALL

Размер:

```
OpInfo * 8 + 8
```

Диапазон:

```
8..128 bytes
```

Для sample slot:

```
CodeOffset = 4
Op = 2
OpInfo = 4
```

размер:

```
4 * 8 + 8 = 40
```

Поэтому report показывает:

```
prolog_size = 4
fixed_stack_bytes = 40
```

UWOP_ALLOC_LARGE

Имеет два modes.

Mode 0, `OpInfo == 0`:

- следующий slot содержит size / 8;
- actual size = operand * 8;
- total 2 slots.

Mode 1, `OpInfo == 1`:

- следующие два slots содержат полный 32-bit size;
- total 3 slots.

Другой `OpInfo` недопустим.

Project sanity rules отмечают использование large form для размеров, которые выглядят несогласованно с ожидаемым encoding choice. Это anomaly, не parse failure.

UWOP_SET_FPREG

Указывает установку frame pointer register, заданного header.

Operation сама не добавляет fixed stack bytes. Она изменяет способ вычисления frame base.

Повторная operation или отсутствие header frame register создаёт `UA_BAD_OPERAND`.

UWOP_SAVE_NONVOL

Описывает сохранение register в ранее выделенной области stack без `push`.

Следующий slot кодирует scaled offset.

Проект:

- проверяет наличие operand;
- отмечает register в mask;
- не добавляет fixed stack, потому что allocation описывается отдельной operation.

Это различие критично. `SAVE_NONVOL` сообщает расположение сохранённого значения внутри frame, но не означает дополнительное изменение `RSP`.

UWOP_SAVE_NONVOL_FAR

Использует два operand slots для 32-bit offset.

Far form предназначена для больших offsets. Малое или неверно выровненное значение получает `UA_OPERAND_SANITY`.

Parser всё равно сохраняет register mask, если field декодируемо.

XMM saves

`UWOP_SAVE_XMM128` и FAR variant описывают сохранение 128-bit XMM register.

Offsets масштабируются согласно specification, хотя compact summary проекта в основном сохраняет register mask и sanity.

Отдельный `saved_xmm_mask` не смешивает vector state с GPR.

V1 opcodes 6 и 7

Для V1 код поддерживает observed undocumented/common encodings `SAVE_XMM` и `SAVE_XMM_FAR`. Это compatibility behavior, которое должно явно отличаться от официальных V2 meanings.

V2 epilog descriptors

В V2 opcode 6 используется для epilog descriptors, opcode 7 - spare.

Первый descriptor может иметь short или long form; последующие описывают offsets from end.

Parser:

- определяет slot count с учётом first descriptor;
- строит readable `ops_text`;
- не применяет обычную CodeOffset check;
- продолжает декодировать последующие prolog operations.

Unit test содержит short и long records и проверяет `offset_from_end`.

`UWOP_PUSH_MACHFRAME`

Описывает machine frame, сформированный hardware/exception mechanism.

`OpInfo` определяет наличие error code.

Summary добавляет:

```
40 bytes без error code
48 bytes с error code
```

Другой `OpInfo` помечается `UA_BAD_OPERAND`.

Защита суммы fixed stack

`add_fixed_stack` проверяет:

```
UINT32_MAX - total < amount
```

При overflow:

```
total = UINT32_MAX
UA_OPERAND_SANITY
```

Saturation сохраняет upper-bound signal и не позволяет wrap к малому размеру.

Tail после slots

Tail начинается:

```
after_codes = codes_off + codes_bytes;
```

`codes_bytes` уже включает padding.

Содержимое зависит от flags.

Handler tail

При EHANDLER/UHANDLER первые 4 bytes tail - handler RVA.

Проект проверяет physical read и:

```
handler_rva < SizeOfImage
```

Выход за image ставит `UA_HANDLER_OOB`, но read остаётся successful.

Данные после handler RVA являются language-specific. Текущая summary их не декодирует.

Chain tail

При CHAININFO tail содержит ещё один 12-byte `RUNTIME_FUNCTION`.

Проверяются:

- `end > begin`;
- range в `SizeOfImage`;
- nonzero unwind RVA;
- unwind RVA в image.

Нарушение даёт `UA_CHAIN_INVALID`.

Следование по chain

Если enabled, проект начинает с:

```
cur = out->chain.unwind_rva;
```

Массив `seen[16]` хранит уже посещённые RVA. Повтор означает loop.

Внутренний recursive-like вызов:

```
unwind_parse(... follow_chain = false)
```

декодирует один следующий record без собственного продолжения chain. Внешний цикл контролирует depth и loop detection.

Summary parent объединяется:

```
fixed stack sum
pushed count sum
OR register masks
inherited handler
frame consistency check
```

Это compact analytical aggregation. Оно не заменяет точную реализацию `RtlVirtualUnwind` для любого instruction offset.

`valid` и anomalies

В конце:

```
out->valid = true;
return true;
```

Anomalies могут быть nonzero. `valid` означает, что structure была декодирована достаточно для summary, а не что metadata идеальна.

Fatal conditions:

- header unmappable/truncated;

- unsupported version;
- code array за file;
- operation operands за CountOfCodes;
- truncated handler/chain tail.

Nonfatal anomalies:

- alignment;
- bad flags/operands;
- order;
- code offset outside;
- handler OOB;
- chain invalid;
- suspicious operand form.

`parse_error`

При failure `unwind_parse_fail`:

- устанавливает `valid = false`;
- выбирает diagnostic текущего invocation;
- удаляет newline;
- ограничивает длину 159 characters;
- null-terminates field.

Report всегда может показать конкретную причину для function.

Сквозной sample

Runtime table:

```
[0] 1000..1010 -> 4000
[1] 1010..1029 -> 4004
[2] 1029..1040 -> 400C
```

Для record 1:

```
Unwind RVA 0x4004
section .xdata VA 0x4000
raw ptr 0xA00
delta 0x4
file offset 0xA04
```

Header и slot приводят к:

```
Version 1
Flags 0
Prolog 4
Count 1
Frame register 0
ALLOC_SMALL 40
```

JSON function object сохраняет эти значения.

Связь с heuristics

`unwind_parse` не назначает weight.

Он создаёт:

```
us_ok[i]
us[i].anomalies
handler_rva
unwind_rva
```

`heuristics.c` считает:

- parse failures;
- anomaly-bearing records;
- placement unwind RVA;
- placement handler RVA;
- reuse.

Так decode semantics отделена от detection policy.

Tests

`unit_unwind` строит synthetic byte buffers для точной проверки:

- alloc small;
- push nonvol;
- code offset order;
- operand sanity;
- real-world compatible operands;
- V2 epilogs;
- chain inheritance;
- invalid flags/frame;
- preserved parse error;
- unsupported V3;
- truncated handler;
- chain validation/alignment.

Synthetic tests необходимы, потому что позволяют задать отдельные bits без влияния linker.

Regression tests необходимы дополнительно, потому что проверяют полный PE path.

Ограничения decoder

Текущий decoder:

- не выполняет полный `RtlVirtualUnwind`;
- не вычисляет caller context для заданного instruction offset;
- не декодирует language-specific handler data;
- не поддерживает V3;
- агрегирует chain эвристически;
- не сопоставляет operations с disassembly;
- сохраняет masks, но не полные offsets каждого register save в JSON summary.

Это определяет направления развития, но не нарушает заявленный contract summary analyzer.

Расширенная лекция 13. Heuristics evidence engine

Том II. Глава 6. `heuristics.c`: от структурных фактов к оценке свидетельств

Место модуля в системе

`pe.c` и `unwind.c` отвечают на вопросы формата: какие поля присутствуют, какие значения прочитаны, нарушены ли локальные invariants. Они не должны решать, насколько совокупность наблюдений подозрительна.

`heuristics.c` получает уже построенные модели:

```
const PeImage* img
const RuntimeFunction* rf
const UnwindSummary* us
const bool* us_ok
size_t n
```

и формирует `HeuristicReport`.

Это переход от parsing к assessment. Входные данные модуля не являются raw bytes. Они представляют нормализованные факты предыдущих этапов.

Поток данных:

```
PE bytes
-> PeImage + structural counters
-> RuntimeFunction[]
-> UnwindSummary[] + us_ok[]
-> HeuristicReport
```

Если parser неверно извлёк поле, heuristic engine получит неверный факт. Поэтому качество assessment зависит от correctness всех предыдущих contracts.

Что означает слово heuristic

Heuristic, далее эвристика, - правило практического вывода, которое использует наблюдаемые признаки, но не гарантирует единственно правильную классификацию.

Проект не содержит обученной statistical model, malware classifier или probability calibration. Все weights и thresholds заданы константами в source code. Комментарий в начале `heuristics.c` прямо называет модель conservative engineering model, а не mathematically validated classifier.

Следствия:

- `score = 70` не означает 70% вероятности вредоносности;
- `confidence = 0.8` не является частотой правильных предсказаний на независимой выборке;
- threshold `1%` не выведен из стандарта PE;
- weight `25` не является нормативным уровнем Microsoft;
- одинаковый score может быть получен разными наборами evidence.

`score` - ограниченная инженерная сумма весов. `suspicion` - её линейная нормализация `score / 100`. `confidence` - отдельная rule-based оценка полноты и взаимного подтверждения evidence.

Выходная модель

`HeuristicReport` содержит четыре класса данных.

Первый класс - summary assessment:

```
int score;
double suspicion;
double confidence;
bool requires_manual_review;
uint32_t evidence_families_mask;
char verdict[48];
char evidence_level[16];
```

Второй класс - counters наблюдаемых нарушений и ratios.

Третий класс - structured findings:

```
typedef struct HeuristicFinding {
    char code[40];
    char family[32];
    char severity[16];
    char message[192];
    bool has_location;
    uint32_t representative_rva;
    size_t affected_count;
} HeuristicFinding;
```

Четвёртый класс - weakening rules, названные `mitigations` в C model:

```
typedef struct HeuristicMitigation {
    char code[40];
    char message[192];
} HeuristicMitigation;
```

В JSON они выводятся как `weakening_rules_applied`, что точнее отражает назначение: правило не устраняет факт, а снижает confidence вывода.

Валидация arguments и initialization

`heuristics_analyze` требует ненулевые `img`, `out`, `err`. Если `n > 0`, обязаны существовать `rf`, `us` и `us_ok`.

Затем весь output обнуляется:

```
memset(out, 0, sizeof(*out));
```

После этого инициализируется `out->details`. Если allocation fails, функция возвращает false. Это важно для ownership: после успешного init `HeuristicReport` должен быть освобождён через `heuristics_free`.

`out->total = n`, а PE counters копируются из `PeImage`. Таким образом report сохраняет snapshot входных structural facts.

Evidence families

Проект группирует findings в пять семейств.

`table_integrity` включает PE section structure и `RUNTIME_FUNCTION` ranges.

`unwind_integrity` включает parse failures, opcode/operand/chain anomalies и комбинации placement с metadata corruption.

`metadata_placement` включает section context unwind records и handlers.

`unwind_reuse` включает concentration одного `UnwindInfoRVA`.

`handler_dominance` включает concentration одного handler target.

Семейство представлено одним bit в `evidence_families_mask`. `heuristics_evidence_family_count` считает установленные bits.

Family count влияет на confidence, потому что независимые классы evidence считаются более убедительными, чем несколько variants одного pattern. Это engineering assumption. Реальная независимость признаков не доказана статистически.

Frequency map

Для подсчёта повторов используется собственная небольшая hash map с open addressing и linear probing.

`HEntry` хранит:

```
uint32_t key;
uint32_t val;
bool used;
```

Для `unwind_counts` key равен `UnwindInfoRVA`, value - числу records с этим RVA.

Для `handler_counts` key равен `HandlerRVA`, value - числу unwind records с этим handler.

Initial capacity выбирается не меньше 64. При load factor около 70% map растёт:

```
(len + 1) * 10 >= cap * 7
```

`hmap_grow` выделяет новый array и повторно вставляет entries. Это необходимо, потому что index зависит от capacity.

Если map insertion fails после init, loop игнорирует return value:

```
(void)hmap_inc(...)
```

Практически failure возможен главным образом при grow allocation. В таком случае frequency results могут быть неполны, но функция не сигнализирует failure. Это остаточная техническая слабость: для production-grade evidence engine следовало бы propagate OOM и не формировать potentially incomplete reuse ratios. В учебной защите это нужно назвать честно.

Первый проход по `RUNTIME_FUNCTION`

Для каждой записи извлекаются:

```
b = BeginRVA
e = EndRVA
u = UnwindRVA
```

Invalid range

Если:

```
e <= b
```

увеличивается `rf_invalid_ranges`. Первый RVA сохраняется как representative location.

`End == Begin` описывает пустой range. `End < Begin` описывает обратный range. Оба нарушают ожидаемый полуинтервал.

Function range out of image

Если:

```
b >= SizeOfImage || e > SizeOfImage
```

увеличивается `rf_oob`.

Для `begin` используется `>=`, потому что RVA ровно `SizeOfImage` уже находится за последним byte. Для `end` используется `>`, потому что exclusive end может быть равен верхней границе.

Unwind RVA out of image

Если:

```
u >= SizeOfImage || u == 0
```

увеличивается `rf_unwind_oob`.

Zero рассматривается как missing. Даже если header bytes существуют по file offset zero, `UnwindData = 0` не является поддерживаемой valid link на `UNWIND_INFO`.

Unsorted input table

Модуль сравнивает текущий `BeginRVA` с предыдущим:

```
current_begin < previous_begin
```

Каждое такое место считается inversion. Это не полное число инверсий математического массива; это число локальных нарушений nondecreasing order между соседними entries. Поле `rf_unsorted` и сообщение «inversions» поэтому следует понимать в реализации проекта, а не как полный inversion count `O(n²)`.

Reuse counts

Каждый `UnwindRVA`, включая malformed values, добавляется в `unwind_counts`. Это означает, что множество zero или одинаковых OOB values может увеличить reuse ratio. Одновременно они создадут integrity findings. Combination отражает реальную концентрацию указателей, но analyst должен учитывать, что dominant key может быть invalid.

Handler добавляется только если:

```
us_ok[i] && us[i].has_handler
```

Parse-failed record не участвует в handler dominance, потому что handler field не считается надёжно извлечённым.

Placement unwind metadata

Для успешно decoded record проект находит section по `UnwindRVA`.

Если section отсутствует, увеличивается `unwind_unusual_section`.

Если section существует, имя оценивается двумя policy.

Строгий allowlist:

```
.xdata
.rdata
.pdata
.gnu_xdata
.mrdata
xdata
rdata
pdata
```

Дополнительная plausible check принимает имя, содержащее case-insensitive `xdata`, `pdata` или `rdata`.

Это снижает false positives для toolchain-specific names. Но section names не являются гарантией semantics. Специально созданная executable+writable section с именем `.xdata` всё равно будет

обнаружена по permissions.

Независимо от имени проверяются flags:

```
IMAGE_SCN_MEM_EXECUTE
IMAGE_SCN_MEM_WRITE
```

Исполняемая section увеличивает `unwind_exec_section`. Writable section увеличивает `unwind_writable_section`.

Metadata обычно является read-only data. Writable или executable placement создаёт повышенный review interest, но JIT/runtime и нестандартный linker layout остаются возможными benign explanations.

Placement handler

Если record имеет handler, проект находит section по `HandlerRVA`.

Если section отсутствует или не имеет execute flag:

```
handler_nonexec_section++
```

Если section writable:

```
handler_writable_section++
```

Handler - адрес code, поэтому non-executable placement противоречит ожидаемому runtime use. Writable+executable memory возможна, но нарушает распространённую W^X discipline и требует объяснения.

Здесь есть важная модельная граница. Section characteristics в файле не всегда полностью совпадают с runtime page protections после loader или защитного runtime. Поэтому static finding должен коррелироваться с dynamic memory map при серьёзном выводе.

Parse failure и anomaly

Если `us_ok[i] == false`, увеличивается `unwind_parse_fail`.

Если parse succeeded, но `us[i].anomalies != 0`, увеличивается `unwind_anomalies`.

Эти категории различаются.

Parse failure означает, что decoder не смог надёжно завершить поддерживаемую интерпретацию record.

Anomaly означает, что summary построен, но присутствуют нарушения: неизвестная version, bad opcode, code offset order, chain issue, alignment и другие.

Разделение позволяет report сохранять частичные данные для anomalous-but-decodable records.

Overlap и duplicate begin

Input table может быть unsorted, поэтому непосредственное сравнение соседей недостаточно для overlap detection. Проект создаёт `RfIdx[]`, копирует begin/end, сортирует через `qsort`, затем проходит sorted array.

`duplicate_begin`:

```
current.begin == previous.begin
```

`overlap`:

```
current.begin < max_end всех предыдущих records
```

Использование `max_end`, а не только previous end, корректно обнаруживает nested overlap:

```
A = [0x1000, 0x2000)
B = [0x1100, 0x1200)
C = [0x1300, 0x1400)
```

Хотя B закончилась до C, C всё ещё пересекает A. `max_end` остаётся `0x2000`.

Sort array содержит только begin/end и не меняет original arrays. Поэтому parallel relation `rf[i] <-> us[i] <-> us_ok[i]` сохраняется.

Ratios

После counting:

```
unwind_reuse_ratio = max_count_same_unwind / total_records
unwind_unique_ratio = unique_unwind_rvas / total_records
handler_reuse_ratio = max_count_same_handler / handler_count
```

При denominator zero `safe_ratio` возвращает 0.

`unwind_reuse_ratio` отвечает на вопрос о концентрации наиболее частого unwind RVA. Он не равен доле всех reused records. Например, два разных RVA могут встречаться по 20% каждый; max ratio будет 20%.

`unwind_unique_ratio` отражает diversity. Значение 1.0 означает unique RVA для каждого record. Низкое значение показывает reuse, но само по себе не создаёт finding.

`handler_reuse_ratio` считается среди records, которые имеют handler, а не среди всех runtime functions. Это важное различие denominator.

Score saturation

`add_score` вычисляет:

```
score = clamp(score + weight, 0, 100)
```

После достижения 100 дальнейшие findings сохраняются, но числовой score не растёт. Поэтому score 100 не означает одинаковую evidence mass для всех cases. Для анализа насыщенных cases нужно читать findings, affected counts и families.

PE structural findings и веса

Точные implementation weights:

Finding	Weight	Severity
`pe_section_raw_oob`	25	high
`pe_section_virtual_oob`	20	high
`pe_section_raw_overlap`	15	medium
`pe_section_virtual_overlap`	20	high
`pe_section_misaligned`	8	medium
`pe_exception_directory_oob`	25	high

Для каждого finding сохраняется первый representative section RVA и полный affected count.

Representative RVA не означает, что только один объект нарушен. Это адрес первого найденного примера для навигации. Полный count показывает масштаб.

Empty exception table

Если `n == 0` и image имеет непустую executable section, добавляется:

```
code: no_runtime_functions
family: table_integrity
severity: medium
weight: 10
```

Message прямо допускает leaf-only image.

Если executable sections нет, score не начисляется. В details появляется info note о resource-only или forwarding image.

Это context-sensitive rule: одинаковое отсутствие `.pdata` оценивается по-разному в зависимости от наличия executable content.

Процентные integrity thresholds

Для invalid ranges, function OOB, unwind RVA OOB и parse failures threshold high равен строго больше 1%:

```
THRESHOLD_HIGH_PERCENT = 0.01;
```

Сравнение использует `>`, не `>=`.

Веса:

Signal	Low/medium prevalence	High prevalence
invalid range	10	20
function OOB	10	25
unwind RVA OOB	10	25
unwind parse fail	10	25

При высоком варианте severity становится high; иначе medium.

На малом sample один record почти всегда превышает 1%. Например, 1 из 3 равен 33.3%. Поэтому позже confidence получает small-sample penalty, если strong findings отсутствуют. Однако если finding уже high по prevalence, penalty `small_sample_size` не применяется, поскольку condition требует `strong_count == 0`.

Это демонстрирует, что score и confidence решают разные задачи.

Unwind anomaly threshold

Для decoded records с anomaly high threshold равен строго больше 5%:

```
THRESHOLD_UNWIND_ANOM_HIGH = 0.05;
```

Weight:

```
10 при <= 5%
20 при > 5%
```

Severity medium или high соответственно.

Table order findings

Точные weights:

```
rf_unsorted      15, medium
rf_overlaps      25, high
duplicate_begin  10, medium
```

Overlap получает более высокий weight, потому что нарушает однозначность coverage ranges. Unsorted table может быть восстановлена сортировкой для анализа, но runtime contract ожидает sorted entries.

Strong unwind reuse

Strong reuse finding появляется только если:

```
unwind_reuse_ratio > 0.30
n > 200
```

Weight 20, severity high.

Ограничение `n > 200` защищает от малых samples, где несколько одинаковых records дали бы высокий ratio. Strict `>` означает, что ровно 200 records недостаточно, а ровно 30% не проходит.

Representative RVA равен dominant `UnwindInfoRVA`; affected count равен max frequency.

Reuse может быть legitimate из-за shared epilog/prolog patterns, chained info или runtime layout. Поэтому даже strong reuse остаётся heuristic.

Handler dominance с benign-context suppression

Candidate требует:

```
handler_count > 50
handler_reuse_ratio > 0.80
```

Затем проект определяет `has_static_context`: имеются ли integrity issues, placement issues или moderate reuse candidate.

Также проверяется `dominant_runtime_like`:

- dominant handler находится в section;
- section executable;
- section name code-like: `.text`, `text`, содержит `text`, `code` или `init`;
- нет non-executable handler findings.

Если handler выглядит как обычный code runtime helper и других static signals нет, finding подавляется.

Именно это проверяется unit test, где 60 entries законно используют один handler в executable `.text`: score остаётся zero. При добавлении suspicious context dominance появляется.

Если finding создаётся:

```
weight: 15
severity: medium
family: handler_dominance
```

Это важная защита от false positive для central language runtime handlers.

Moderate unwind reuse требует corroboration

Moderate candidate:

```
unwind_reuse_ratio > 0.10
n > 200
```

Finding не создаётся автоматически. Требуется хотя бы один другой static signal: handler dominance, integrity issue, placement issue или anomaly.

Если strong reuse уже создан, moderate finding не добавляется.

Weight 10, severity medium.

Правило реализует принцип corroboration: умеренный repetition без context не считается самостоятельным evidence.

Placement weights

`unwind_in_exec_section`:

```
weight 20
severity high
```

`unwind_unusual_section`:

```
weight 8, severity medium
weight 15, severity high, если count > n/20
```

`n/20` использует integer division. Для `n < 20`, `n/20 == 0`, поэтому любое positive count проходит high branch. Это implementation nuance. Small-sample confidence penalties могут смягчить case только если strong finding отсутствует; здесь high severity создаёт strong count и penalty не применяется. Это потенциально резкое поведение на малых samples и кандидат для будущей calibration.

`unwind_writable_section`:

```
weight 10, severity medium
weight 18, severity high, если count > n/20
```

`handler_nonexec_section`:

```
weight 15
severity high
```

`handler_writable_section`:

```
weight 10
severity medium
```

Combination findings

Модуль начисляет дополнительные weights, когда два разных свойства встречаются вместе.

Executable unwind metadata и non-executable handler

Condition:

```
unwind_exec_section > 0
handler_nonexec_section > 0
```

Finding:

```
placement_exec_nonexec_handler_combo
weight 10
severity high
family metadata_placement
```

Placement и integrity

Condition:

```
(unwind_exec_section || handler_nonexec_section)
&&
(unwind_parse_fail || unwind_anomalies || rf_unwind_oob)
```

Finding:

```
placement_integrity_combo
weight 8
severity medium
family unwind_integrity
```

Reuse и dominance

Condition:

```
unwind_reuse_ratio > 0.10
handler_dominance finding exists
```

Finding:

```
reuse_dominance_combo
weight 8
severity medium
```

Writable metadata и writable handler

Condition:

```
unwind_writable_section > 0
handler_writable_section > 0
```

Finding:

```
writable_metadata_handler_combo
weight 10
severity high
```

Combination weights intentionally double-count related raw observations in aggregate score. Это не статистически независимое суммирование; это explicit policy, что сочетание труднее объяснить benignly.

Finding storage limit

Maximum:

```
HEURISTICS_MAX_FINDINGS = 32
```

Если findings больше, `add_finding` продолжает добавлять human-readable line в `details` и family bit, но structured array больше не растёт.

Последствия:

- score продолжает учитывать rule;
- `details` может содержать больше findings, чем `finding_count`;
- severity counts вычисляются только по сохраненным структурированным findings;
- evidence level и confidence могут недоучесть findings за пределом 32.

Текущий набор rules меньше limit в обычном run, но ограничение модели существует. Промышленная версия могла бы использовать динамический массив или явный флаг усечения.

Severity counts и evidence level

High и critical считаются strong. Medium считается medium. Остальные stored findings относятся к weak.

Evidence level:

```
strong:
  strong_count >= 2 или score >= 70

medium:
  strong_count >= 1
```

```
  или medium_count >= 2
  или score >= 30

weak:
  существует finding или score > 0

none:
  иначе
```

Evidence level не совпадает с verdict thresholds. Например, один high finding при score 15 может дать `evidence_level = medium`, но `verdict = low_suspicion`.

Это не ошибка: поля описывают разные projections. Однако пользователю необходимо объяснять обе шкалы.

Confidence calculation

Initial:

```
confidence = 0.15
```

Additions:

```
+0.11 за evidence family
+0.08 за strong finding
+0.04 за medium finding
+0.08 при hard table issue
+0.08 при parse failure или unwind anomaly
```

Hard table issue:

```
rf_invalid_ranges
rf_oob
rf_unwind_oob
rf_overlaps
```

Если `score == 0`, confidence заменяется на `0.20`. Это означает умеренную уверенность только в ограниченном выводе «сильных static anomalies не обнаружено», а не уверенность в безопасности файла.

Final confidence ограничивается:

```
0.05 <= confidence <= 0.90
```

Static model никогда не выдаёт выше 0.90.

Weakening rules

`single_signal_only`

Применяется, если:

```
finding_count == 1
strong_count == 0
```

Penalty `0.12`.

`small_sample_size`

Применяется, если:

```
total < 32
strong_count == 0
finding_count > 0
```

Penalty `0.06`.

`placement_only_signal`

Применяется, если есть unusual section, но отсутствуют executable/writable placement, handler placement, range issues, parse failures и anomalies.

Penalty `0.10`.

Это правило не применится к `unwind_writable_section`, потому что writable placement рассматривается сильнее простого unusual name.

`single_family_only`

Применяется, если:

```
strong_finding_count == 0
family_count <= 1
finding_count > 0
```

Penalty `0.08`.

Mitigations добавляются в report после findings и не уменьшают score. Следовательно:

```
score описывает weighted anomaly mass
confidence описывает уверенность в статическом verdict
```

Manual review flag

`requires_manual_review` становится true, если:

- score не меньше 20;
- unwind metadata executable;
- unwind metadata writable;
- handler non-executable;
- handler writable;
- есть parse failure;
- применено хотя бы одно weakening rule.

Score 15 с единственным high placement finding также может потребовать review из-за конкретного counter. И наоборот, low score zero не требует review в автоматической модели, хотя общий reverse engineering может быть нужен по внешним причинам.

Verdict

Thresholds:

```
score >= 70 -> high_suspicion
score >= 40 -> moderate_suspicion
score >= 15 -> low_suspicion
иначе       -> no_strong_static_anomalies
```

Название последнего verdict выбрано осторожно. Оно не утверждает `benign`, `clean` или `safe`. Оно утверждает только отсутствие сильных anomalies в реализованном наборе static rules.

Фактический benign sample

Для `benign_richer.exe`:

```
total = 3
all range counters = 0
unwind_parse_fail = 0
unwind_anomalies = 0
placement counters = 0
```

```
reuse conditions не проходят n > 200
score = 0
suspicion = 0.0
confidence = 0.2
verdict = no_strong_static_anomalies
evidence_level = none
requires_manual_review = false
```

Confidence 0.2 не означает, что sample на 20% benign. Это base confidence в узком static conclusion.

Фактический suspicious sample `suspicious_unwind_oob.exe`

Manifest требует:

```
rf_unwind_oob = 1
unwind_parse_fail = 1
score >= 40
verdict = moderate_suspicion
case_strength = strong_unconfirmed_case
```

При малом total один OOB pointer превышает 1%, поэтому:

```
unwind_rva_oob: +25, high
unwind_parse_fail: +25, high
```

Score достигает не менее 50. Два strong findings дают static evidence strong. Dynamic corroboration отсутствует, поэтому report-level case называется strong unconfirmed.

Фактический suspicious sample `suspicious_unwind_writable.exe`

Manifest требует writable unwind metadata и score не менее 15. На малом sample condition `count > n/20` может выбрать high branch weight 18.

Finding говорит, что metadata находится в section с write permission. Он не доказывает tampering. Manual validation должна проверить:

- section characteristics;
- runtime memory protection;
- toolchain;
- наличие JIT/runtime scenario;
- correspondence metadata и code.

Тесты модуля

`unit_heuristics` проверяет базовые score, placement, verdict и confidence.

`unit_heuristics_mitigations` покрывает weakening rules.

`unit_heuristics_context` проверяет benign dominance suppression, suspicious context и writable combinations.

`unit_heuristics_combos` проверяет additional weights при corroborating patterns.

`unit_core_edges` проверяет empty input, resource-only case, nested overlaps и PE structural findings.

Regression manifest связывает synthetic PE samples с ожидаемыми JSON paths.

Тесты подтверждают implementation contract. Они не валидируют weights как научно оптимальные для реального world corpus.

Что следует улучшать в будущей версии

Наиболее значимые направления:

1. Версионировать heuristic policy отдельно от report schema и tool version.
2. Добавить calibration corpus с размеченными compiler/runtime families.
3. Публиковать score contribution каждого finding, а не только total.
4. Добавить truncation flag для fixed finding array.
5. Propagate hash-map allocation failures.
6. Исправить percentage comparison для малых `n`, используя ratio вместо `count > n/20`.
7. Отделить invalid dominant keys от valid reuse statistics.
8. Сохранять all representative locations, а не только первый.
9. Добавить explicit rationale и standard/nonstandard tag каждому finding.
10. Не называть `confidence` вероятностью до empirical calibration.

Итоговая модель главы

`heuristics.c` не обнаруживает «зло» как бинарное свойство. Он:

```
получает проверяемые structural observations
-> агрегирует counts и concentration
-> создаёт named findings
-> группирует findings по evidence families
-> начисляет bounded engineering score
-> оценивает rule-based confidence
-> применяет weakening rules
-> формирует осторожный verdict
-> требует manual validation там, где автоматический вывод недостаточен
```

Именно так модуль должен быть представлен заказчику и на защите: как explainable static evidence engine для EH/unwind anomalies, а не как сертифицированный malware detector.

Расширенная лекция 14. Windows dynamic evidence collector

Том II. Глава 7. `dynamic_win.c`: Windows dynamic evidence collector

Назначение и граница платформы

Статическая часть проекта компилируется и работает на macOS, Linux и Windows, потому что читает PE как массив bytes. Динамическая часть использует Windows Debug API и должна исполнять Windows x64 target. Поэтому она доступна только в Windows build.

Это различие реализовано compile-time branch:

```
### ifdef _WIN32
    /* реальный collector */
### else
    /* unsupported-platform stub */
### endif
```

На non-Windows система не пытается эмулировать process execution. `dynamic_collect_windows` инициализирует report, записывает понятную ошибку и возвращает false.

Следовательно:

```
Linux/macOS host:
  static analysis Windows PE32+ AMD64 = поддерживается
  dynamic Windows exception validation = не поддерживается
```

```
Windows x64 host:
  static analysis = поддерживается
  dynamic validation = поддерживается
```

Wine или virtual machine могут быть отдельной средой исследования, но текущий код напрямую не реализует и не тестирует Wine-specific backend.

Defensive intent

Header `dynamic.h` определяет цель как сбор runtime evidence:

- hits handlers из `UNWIND_INFO`;
- registration и hits VEH/VCH/UEF;
- registration dynamic function tables;
- first-chance и second-chance exception flow.

Collector не заявляет обход защит, извлечение скрытых данных или автоматическую деобфускацию. Он является ограниченным debugger, созданным под конкретные event types.

Модель результата

`DynamicReport` хранит session state, counters, bounded events, bounded trace и human-readable log.

Состояние:

```
bool enabled;
bool ran;
bool ok;
bool timed_out;
bool target_exited;
uint32_t target_exit_code;
```

`enabled` означает, что dynamic mode был запрошен и report инициализирован.

`ran` означает, что `CreateProcessA` succeeded и target действительно был создан.

`ok` означает:

```
CREATE_PROCESS event был получен
&& debug loop не встретил internal failure
&& timeout не наступил
```

`target_exited` отдельно показывает normal observation `EXIT_PROCESS_DEBUG_EVENT`. Session может быть завершена принудительно по timeout, тогда target не считается self-exited.

Events ограничены:

```
DYNAMIC_MAX_EVENTS = 256
```

После заполнения array новые события не перезаписывают старые; увеличивается `dropped_events`.

Trace также ограничен 256 instruction pointers. Input `trace_steps` clamp до 256.

Module map

Breakpoints на system APIs нельзя ставить по address, полученному в process анализатора. Из-за ASLR module bases отличаются.

`ModList` содержит максимум 128 modules:

```
typedef struct ModList {
    ModInfo mods[128];
```

```
    size_t count;
} ModList;
```

`mods_refresh_toolhelp`:

1. сбрасывает count;
2. вызывает `CreateToolhelp32Snapshot`;
3. перечисляет `MODULEENTRY32`;
4. сохраняет basename, base и size;
5. приводит name к lowercase.

Module list fixed-size. Если target загрузил более 128 modules, остаток не сохраняется. Truncation flag отсутствует. Это residual limit: API resolution для позднего module может не сработать без явной ошибки.

Разрешение forwarded API

Windows export может быть forwarded. Например, symbol запрашивается через `kernel32.dll`, но фактический code address принадлежит `kernelbase.dll`.

`local_proc_loc` выполняет:

1. `GetModuleHandleA(preferred_mod)`;
2. `GetProcAddress(symbol)`;
3. `GetModuleHandleExA(...FROM_ADDRESS...)`, чтобы определить real local module;
4. получает basename real module;
5. вычисляет offset:

```
offset = LocalFunctionVA - LocalRealModuleBase
```

После этого `add_api_bps_if_ready` ищет module с таким basename в target:

```
TargetFunctionVA = TargetModuleBase + offset
```

Эта схема корректно переносит symbol location между процессами при одинаковой версии загруженного module. Она предполагает, что local и target используют совместимый module binary с тем же internal offset. Для обычного запуска на одной Windows installation это разумно; cross-architecture или module substitution потребовали бы remote export parsing.

Наблюдаемые API

Collector ставит breakpoints на:

```
AddVectoredExceptionHandler
AddVectoredContinueHandler
SetUnhandledExceptionFilter
RtlAddFunctionTable
RtlInstallFunctionTableCallback
```

Kinds хранятся как bit mask. Один address может соответствовать нескольким logical kinds, поэтому existing breakpoint получает:

```
ex->kinds |= kind;
```

Это предотвращает дублирование `INT3` по одному address.

Breakpoint model

```
typedef struct Breakpoint {
    uint64_t addr;
    uint8_t orig;
    bool armed;
    bool rearm_pending;
    uint32_t kinds;
} Breakpoint;
```

`addr` - target VA.

`orig` - исходный byte, который был заменён.

`armed` - по address ожидается `0xCC`.

`rearm_pending` - исходная instruction временно восстановлена, breakpoint должен быть поставлен снова после single-step.

`kinds` - logical meanings address.

List имеет capacity 512. При переполнении `bp_add_and_arm` возвращает false, но dedicated overflow counter отсутствует.

Запись bytes в remote process

`write_one_byte` сначала вызывает `WriteProcessMemory`. Если write failed, code сохраняет error и пробует временно изменить page protection:

```
VirtualProtectEx(..., PAGE_EXECUTE_READWRITE)
WriteProcessMemory
VirtualProtectEx(..., oldProt)
```

После successful write вызывается `FlushInstructionCache`.

Причина cache flush: processor может иметь ранее полученные instruction bytes. После изменения executable memory debugger должен обеспечить согласованность instruction fetch.

Временный `PAGE_EXECUTE_READWRITE` расширяет permissions целевой страницы. Проект возвращает previous protection, но на короткий период изменяет security state target. Это unavoidable вмешательство выбранной software-breakpoint technique и должно учитываться при anti-debug-sensitive samples.

Установка breakpoint

`bp_set`:

1. читает один byte по `addr`;
2. сохраняет его в `orig`;
3. пишет `0xCC`;
4. устанавливает `armed = true`;
5. очищает `rearm_pending`.

Здесь имеется subtle risk при повторной постановке: `bp_set` каждый раз заново читает current byte и записывает его в `orig`. Correct flow вызывает re-arm после того, как original instruction уже восстановлена, поэтому сохраняется правильный byte. Если функцию ошибочно вызвать на уже armed breakpoint, `orig` мог бы стать `0xCC`; guard в `bp_add_and_arm` предотвращает этот path.

`bp_clear` восстанавливает `orig` только если `armed`.

Self-check

`bp_add_and_arm_selfcheck` после первой попытки установки снова читает byte и добавляет deterministic log line:

```
[SelfCheck] ... OK (byte=0xcc)
```

Если byte не `0xCC`, появляется warning. Self-check также форматирует address как:

```
module.dll+offset (absolute VA)
```

Это улучшает auditability: report показывает не только желание поставить breakpoint, но и результат remote memory verification.

Self-check log не является structured event. Он находится в `DynamicReport.details`, поэтому backend должен сохранять это поле, если требуется полный audit.

Per-thread pending re-arm

После hit breakpoint временно снимается. Original instruction должна выполниться в том же потоке, который hit breakpoint. После single-step этот конкретный breakpoint ставится обратно.

`PendingRearmList` имеет 64 slots:

```
typedef struct PendingRearmEntry {
    DWORD tid;
    Breakpoint* bp;
} PendingRearmEntry;
```

`pending_rearm_set` сначала обновляет existing entry для `tid`, затем ищет free slot. Если 64 threads одновременно находятся между breakpoint hit и single-step, insertion fails и session считается failed.

Один thread может иметь только один pending breakpoint. Это соответствует normal sequence: thread stopped on one breakpoint cannot hit another до выполнения original instruction и single-step.

При `EXIT_THREAD_DEBUG_EVENT` pending breakpoint также re-arm, чтобы завершение потока не оставило shared code без breakpoint.

Командная строка target

Windows `CreateProcessA` принимает mutable command-line string, а не готовый `argv[]`. Поэтому collector строит строку по Microsoft C runtime quoting rules.

`append_cmd_arg`:

- добавляет quotes для empty string, spaces, tabs или internal quotes;
- удваивает backslashes перед quote;
- удваивает trailing backslashes перед closing quote.

Это существенно. Простое соединение arguments через spaces изменило бы boundaries. Например path с пробелом стал бы несколькими arguments.

CLI отделяет target arguments после `--`:

```
eh_unwind_analyzer target.exe --dynamic -- --input sample.dat
```

Collector получает `child_argv` и сохраняет boundaries при построении Windows command line.

Report metadata по умолчанию redacts child arguments, потому что они могут содержать secrets. Это относится к `main.c`; actual CreateProcess command line всё равно включает переданные target arguments.

Создание процесса

Используется:

```
CreateProcessA(
    NULL,
    cmd.s,
    NULL,
    NULL,
    FALSE,
    DEBUG_ONLY_THIS_PROCESS,
    NULL,
    NULL,
    &si,
    &pi
)
```

`DEBUG_ONLY_THIS_PROCESS` означает, что дочерние процессы target автоматически не становятся debuggees этого collector.

Code намеренно не использует `CREATE_SUSPENDED`. Комментарий фиксирует обнаруженную практическую проблему: на некоторых configurations suspended debuggee не выдавал ожидаемый create event, что создавало deadlock. Debug API сам предоставляет ранний `CREATE_PROCESS_DEBUG_EVENT` до обычного user-mode execution.

После success:

```
out->ran = true
hProcess = pi.hProcess
pid = pi.dwProcessId
```

Static handler addresses как input dynamics

До debug loop collector строит unique list `handler_rvas` из успешно parsed records с `has_handler`.

List выделяется размером `rf_count`, минимум один element. Unique check линейный:

```
для каждого handler
  сравнить со всеми уже сохранёнными
```

Worst-case complexity `O(n²)`. Для типичных handler counts это приемлемо, но большое PE с unique handler для каждого record может замедлить preparation. Hash set был бы масштабируемее.

На `CREATE_PROCESS_DEBUG_EVENT`:

```
HandlerVA = base_main + HandlerRVA
```

и ставится `BP_UNWIND_HANDLER`.

Если RVA invalid, write may fail; failure считается best effort и не завершает session.

Debug loop и timeout

Start time:

```
uint64_t start = GetTickCount64();
```

На каждой iteration:

```
elapsed = current - start
if elapsed > timeout_ms -> timeout
```

Сравнение строго `>`. Timeout event добавляется в structured events с `arg0 = timeout_ms`.

`WaitForDebugEvent` вызывается с 100 ms. `ERROR_SEM_TIMEOUT` означает отсутствие event в интервале и приводит к следующей iteration. Другой error завершает loop как failure.

Granularity timeout поэтому примерно 100 ms плюс время обработки события.

Continuation status

Для каждого event initial:

```
DWORD cont = DBG_CONTINUE;
```

Для ordinary exception, не принадлежащего breakpoint machinery:

```
cont = DBG_EXCEPTION_NOT_HANDLED;
```

Это передаёт исключение обычному target dispatch.

Для собственных breakpoint и single-step:

```
cont = DBG_CONTINUE;
```

потому что debugger полностью обработал служебное exception.

Неверный status здесь способен либо поглотить target exception, либо передать приложению `INT3`, вставленный analyzer.

`CREATE_PROCESS_DEBUG_EVENT`

Collector:

1. устанавливает `got_create = true`;
2. сохраняет `base_main = lpBaseOfImage`;
3. refresh modules;
4. пытается поставить API breakpoints;
5. ставит breakpoints на static handler RVA;
6. закрывает `hFile` из event.

Закрытие file handle обязательно по Windows Debug API contract.

`base_main` - actual base. Он используется для статических handler RVA и address-to-section formatting.

`LOAD_DLL_DEBUG_EVENT`

При каждой DLL load:

1. refresh module list;
2. повторно вызвать `add_api_bps_if_ready`;
3. закрыть `hFile`.

Повторная попытка нужна, потому что real module API мог отсутствовать раньше. Existing breakpoints deduplicate по address.

`UNLOAD_DLL_DEBUG_EVENT` только refresh list. Breakpoints внутри выгруженного module не удаляются из `BpList`. Их remote pages уже могут быть unmapped. На final cleanup `bp_clear` best-effort игнорирует failure. Если module выгружен и затем другой module загружен по тому же address, stale breakpoint model может стать проблемой. Это limitation текущего collector.

Ordinary exception event

Для code, отличного от breakpoint и single-step:

```
exceptions_seen++
first_chance_seen++ или second_chance_seen++
```

В event сохраняются:

- type `exception`;
- thread ID;
- exception address;
- exception code;
- first_chance flag;
- module by address.

После этого `DBG_EXCEPTION_NOT_HANDLED`.

Collector не сериализует `NumberParameters` и `ExceptionInformation[]`. Для access violation поэтому не сохраняются attempted address и operation type. Это направление расширения evidence model.

Initial system breakpoint

После создания debuggee Windows генерирует initial breakpoint. Он не принадлежит `BpList`.

Code допускает один неизвестный breakpoint как initial:

```
if !bp && !initial_break_ignored:
    initial_break_ignored = true
    DBG_CONTINUE
```

Следующие unknown breakpoints передаются как not handled.

Это pragmatic rule. Target может выполнить собственный early `INT3` до system breakpoint order, а system behavior может отличаться. Более строгая identification могла бы учитывать module/address и event sequence.

Hit собственного `INT3`

По exception address выполняется `bps_find`.

Если breakpoint найден:

1. `bp_clear` восстанавливает original byte;
2. `rearm_pending = true`;
3. `pending_rearm_set(tid, bp)`;
4. открывается event thread;
5. читается context;
6. `ctx.Rip = addr`;
7. `ctx.EFlags |= 0x100`;
8. анализируются kinds;
9. записывается context;
10. event продолжается.

Комментарий «RIP points to INT3+1; rewind» реализуется присваиванием address exception. Поскольку `addr` равен адресу breakpoint, это возвращает execution на original instruction.

Если clear, pending storage, thread open, context read или context write fails, debug loop помечается failed и завершается после продолжения текущего event.

Capture регистрации VEH

На entry `AddVectoredExceptionHandler` Windows x64 ABI:

```
RCX = First
RDX = Handler
```

Collector:

```
veh_registrations++
event type = veh_registration
arg0 = First
arg1 = Handler
```

Если Handler nonzero, ставится breakpoint `BP_CUSTOM_VEH_HANDLER`.

Collector не наблюдает return value API, поэтому не знает, была ли регистрация successful. Он фиксирует вызов и arguments. Чтобы подтвердить success, нужен return breakpoint или анализ последующего hit.

Capture VCH

Аналогично:

```
RCX = First
RDX = Handler
```

Counters и event types отделены:

```
vch_registrations
vch_registration
BP_CUSTOM_VCH_HANDLER
vch_hits
```

Это сохраняет различие VEH и VCH phases.

Capture UEF

На entry `SetUnhandledExceptionFilter`:

```
RCX = Handler
```

Collector увеличивает `uef_set_calls`, создаёт `uef_registration` и ставит breakpoint на nonzero handler.

Как и для VEH, return value предыдущего filter не записывается.

Microsoft указывает, что top-level filter вызывается, когда exception достигает unhandled filter и process не debugged. Наличие active debugger может менять UEF behavior. Поэтому hit UEF под debugger нельзя считать гарантированным для всех scenarios; bundled smoke target и validation должны учитывать actual Windows semantics.

Capture `RtlAddFunctionTable`

Arguments:

```
RCX = FunctionTable
RDX = EntryCount
R8  = BaseAddress
```

Structured event:

```
type = rtl_add_function_table
arg0 = table pointer
```

```
arg1 = count
arg2 = base
```

Collector не читает remote entries. Для полноценного dynamic table analysis нужно:

1. validate count upper bound;
2. compute byte size safely;
3. read remote array;
4. validate `RUNTIME_FUNCTION`;
5. resolve unwind records against supplied base;
6. snapshot page protections;
7. account for table lifetime and `RtlDeleteFunctionTable`.

Текущая реализация подтверждает API call, но не content.

Capture callback table

`RtlInstallFunctionTableCallback`:

```
RCX = TableIdentifier
RDX = BaseAddress
R8  = Length
R9  = Callback
```

Event stores all four arguments. Callback table is more dynamic: system may call supplied callback to obtain entries on demand. Full analysis потребовал бы breakpoint on callback и decoding return behavior.

Handler hits

Kinds:

```
BP_UNWIND_HANDLER
BP_CUSTOM_VEH_HANDLER
BP_CUSTOM_VCH_HANDLER
BP_CUSTOM_UEF
```

При hit соответствующий counter увеличивается и создаётся event:

```
unwind_handler_hit
veh_handler_hit
vch_handler_hit
uef_handler_hit
```

Для static unwind handler address formatter показывает absolute VA, RVA и section.

Если `trace_steps > 0` и трасса ещё не собиралась, включается single-step trace:

```
tracing = true
trace_tid = current tid
add_trace(current RIP)
trace_left = min(trace_steps, 256) - 1
```

После rewind `current RIP` равен handler entry. Поэтому первым trace element является address original handler instruction до её исполнения.

Трасса запускается только при первом handler hit. Последующие попадания продолжают учитываться отдельными counters и events, но не накапливают новую RIP trace поверх первой. Благодаря этому `trace_count` никогда не превышает заданный `trace_steps`.

EXCEPTION_SINGLE_STEP

Сначала:

```
pr_bp = pending_rearm_take(tid)
bp_set(pr_bp)
```

Это возвращает `0xCC` после выполнения original instruction.

Затем читается context.

Если active tracing принадлежит тому же thread:

```
add_trace(ctx.Rip)
trace_left--
TF remains set
```

Когда `trace_left == 0`, TF снимается.

Если tracing не active для этого thread, TF снимается сразу.

`trace_count` включает initial handler RIP в общий бюджет. При `trace_steps = 32` сохраняется не более 32 addresses: entry point и до 31 последующего single-step addresses.

Collector поддерживает только один active trace session global to report. Повторный handler hit не перезапускает и не перезаписывает трассу, потому что `start_trace` возвращается при `trace_count != 0`. Counters и events повторных hits продолжают записываться. Отдельные одновременные per-thread traces не поддерживаются.

Thread exit

Если thread завершился с pending re-arm, collector повторно ставит breakpoint. Это важно, потому что code memory принадлежит процессу, а не thread.

Если exiting thread был traced, trace state сбрасывается.

Process exit

На `EXIT_PROCESS_DEBUG_EVENT`:

```
process_exited = true
target_exited = true
target_exit_code = event code
```

Добавляется `process_exit` event с exit code в `arg0`. Loop завершится только после `ContinueDebugEvent`, что соответствует Debug API lifecycle.

Exit code zero не означает отсутствие anomalies. Это только status target process.

Session finalization

После loop:

```
timed_out = local timed_out
target_exited = process_exited
ok = got_create && debug_loop_ok && !timed_out
```

Затем collector пытается снять все breakpoints.

Если target сам не завершился, вызывается `TerminateProcess`. После этого collector до одной секунды принимает и продолжает завершающие debug events, включая `EXIT_PROCESS_DEBUG_EVENT`. Такой drain не добавляет фиксированные пять секунд к каждому timeout-case и корректно завершает debug lifecycle.

Закрываются initial thread и process handles, освобождается handler list и command-line buffer.

Если `ok == false`, функция добавляет одну из summary errors:

- timeout;
- failure до create event;
- failure внутри event loop.

Даже при false `DynamicReport` остаётся доступным. `main.c` передаёт его в JSON/HTML, чтобы partial events и failure state не терялись.

Semantics dynamic signal count

Report layer считает не raw events, а категории:

```
1: any unwind handler hits
1: any VEH registration or hit
1: any VCH registration or hit
1: any UEF set or hit
1: any dynamic function table API call
1: any second-chance exception
```

Maximum category count шесть. Десять VEH hits всё равно дают один dynamic signal category.

Это summary diversity, а не event volume.

Runtime evidence level

Strong runtime component:

```
any handler hit
```

Medium components:

```
dynamic table registration
second chance
```

Weak components:

```
handler registrations
first chance
```

Result:

```
strong, если strong>=1 и medium>=1
medium, если strong>=1 или medium>=2
weak, если medium>=1 или weak>=1
none иначе
```

Опять же это engineering taxonomy, не probability.

Structured event limitations

`DynamicEvent` содержит generic `arg0..arg3`. Их meaning зависит от `type`.

Плюс: model compact и позволяет serializing разные APIs.

Минус: schema не выражает types напрямую. Backend должен знать mapping:

```
veh_registration:
  arg0 First
  arg1 Handler

rtl_add_function_table:
  arg0 Table
  arg1 EntryCount
  arg2 BaseAddress
```

Для долгосрочной API лучше использовать typed subobjects или event-specific fields, сохранив backward-compatible base.

Граница безопасности и доверия

Target является untrusted executable. Dynamic mode действительно запускает его. Он может:

- читать и изменять доступные user files;
- обращаться к сети;
- создавать child processes;
- обнаруживать debugger;
- зависать;
- потреблять ресурсы;
- атаковать parser/debugger through event volume.

`DEBUG_ONLY_THIS_PROCESS` не sandbox. Timeout не sandbox. Поэтому production deployment должен запускать dynamic worker в disposable Windows VM или hardened sandbox с resource limits и restricted network.

Backend нельзя запускать collector от имени privileged service account над arbitrary upload.

Bundled dynamic smoke targets

Project содержит Windows test programs:

```
dyn_veh.c
dyn_vch.c
dyn_uef.c
dyn_rtl_ftable.c
```

Они компилируются в:

```
15_dynamic_veh.exe
16_dynamic_vch.exe
17_dynamic_uef.exe
18_dynamic_rtl_function_table.exe
```

Smoke script запускает analyzer с timeout 8000 ms, trace 32 для handlers и JSON validation.

Targets подтверждают ожидаемые counters:

- VEH registration/hit;
- VCH registration/hit;
- UEF set/hit и exception flow;
- dynamic function table calls.

На macOS эти tests не могут проверить реальный Debug API branch. Нужен Windows CI или физический Windows x64 host.

Фактическая Windows validation 8 июня 2026 года

Финальная MinGW-w64 сборка dynamic collector была запущена на Windows x64. Проверенный executable имеет:

```
SHA-256
489e7571827693cd555c1958f9aec7694d1de717f896d33ecfebe8f13e843624
```

Корпус содержал 18 синтетических и 106 реальных PE-файлов:

```
124/124 JSON reports прошли schema и semantic validation
86 dynamic sessions были фактически запущены
54 target завершились самостоятельно
32 target корректно остановлены hard timeout
30 DLL/PYD получили ran=false, потому что CreateProcess не запускает library
8 EXE потребовали elevation
4/4 focused VEH/VCH/UEF/runtime-table scenarios прошли
0 dynamic events потеряно
maximum trace_count = 32 при trace_steps = 32
```

Эта проверка подтверждает не только synthetic smoke, но и устойчивость collector contract на разнообразных реальных EXE: normal exit, nonzero target exit, GUI/long-running timeout и launch failure. DLL/PYD всё равно не были исполнены: для них требуется отдельный host process.

Артефакты сохранены в `reports/dynamic_validation_2026-06-08/`. Скрипт `tests/validate_dynamic_corpus_evidence.py` повторно проверяет 124 индивидуальных reports, aggregate JSON/CSV, trace budget, dropped events, source hashes и четыре expected-signal scenarios без повторного запуска target.

Эмпирическая граница результата: подтверждена MinGW-w64 сборка. MSVC остаётся отдельным compiler gate и должен проходить в Windows CI на финальной revision.

Что сделано правильно

Сильные стороны implementation:

- exact separation Windows/non-Windows;
- forwarded API resolution через real module offset;
- bounded event and trace storage;
- command-line quoting;
- instruction cache flush;
- temporary protection restoration;
- per-thread pending re-arm;
- preservation ordinary exception dispatch;
- repeated API breakpoint setup after DLL load;
- partial report retention on failure;
- timeout and cleanup;
- self-check breakpoint bytes.

Что остаётся улучшить

Наиболее значимые improvements:

1. Remote parse dynamic function tables.
2. Наблюдать removal API и lifetime tables.
3. Typed event schema вместо generic args.
4. Сохранять exception parameters и context snapshots.
5. Поддержать concurrent per-thread traces.
6. Track module unload and invalidate breakpoints.
7. Dynamic capacity или truncation flags для module, breakpoint и pending lists.

8. Return breakpoints для registration API success.
9. Unicode command-line path через `CreateProcessW`.
10. Isolated worker protocol и VM orchestration.
11. Runtime memory protection map.
12. Hardware-breakpoint backend для anti-tamper-sensitive cases.
13. Завершить независимый MSVC gate и расширять Windows version matrix.

Итоговая state machine

Collector можно представить как finite state machine:

```
UNINITIALIZED
-> REPORT_INITIALIZED
-> COMMAND_BUILT
-> PROCESS_CREATED
-> WAITING_EVENT
-> HANDLING_EVENT
-> CONTINUING_EVENT
-> WAITING_EVENT
-> EXITED | TIMED_OUT | FAILED
-> BREAKPOINT_CLEANUP
-> HANDLE_CLEANUP
-> REPORT_READY
```

Внутри breakpoint branch существует вложенная state machine:

```
ARMED
-> HIT
-> ORIGINAL_RESTORED
-> RIP_REWOUND_AND_TF_SET
-> ORIGINAL_INSTRUCTION_EXECUTED
-> SINGLE_STEP
-> REARMED
```

Понимание этих состояний необходимо, чтобы объяснить dynamic part проекта. Она не «просто запускает EXE». Она реализует специализированный user-mode debugger и строит ограниченный, воспроизводимый набор runtime evidence.

Расширенная лекция 15. Reporting, assessment и интеграция

Том II. Глава 8. `report.c`: отчёты, итоговая assessment и интеграционный контракт

Отчёт не является форматированным `printf`

После анализа существуют несколько внутренних моделей:

```
PeImage
RuntimeFunction[]
UnwindSummary[]
us_ok[]
HeuristicReport
DynamicReport
ReportMetadata
```

`report.c` преобразует их в четыре внешних представления:

- brief console output;
- CSV;
- JSON;
- self-contained HTML.

Это serialization boundary. Ошибка здесь может:

- потерять evidence;
- создать invalid JSON;
- внедрить untrusted text в HTML;
- нарушить report schema;
- записать частичный файл как будто он полный;
- исказить числовую semantics.

Поэтому reporting layer проверяет model bounds, metadata, encoding и atomic write.

Brief console report

`report_print_brief` выводит:

```
input path
machine
ImageBase
SizeOfImage
SizeOfHeaders
section count
exception directory
RUNTIME_FUNCTION count
```

Это human-readable summary для terminal. Он не содержит все functions и не предназначен как stable machine contract.

Terminal output может меняться без schema migration. Backend не должен parsing console text.

CSV report

Обычный CSV содержит строку на runtime function:

```
index
begin_rva
end_rva
size
unwind_rva
unwind_ver
unwind_flags
prolog_size
fixed_stack
frame_reg
frame_off
handler_rva
has_chain
anomalies
parse_error
```

При parse failure поля, которые нельзя считать надёжными, получают `NA`, а error сохраняется.

CSV escaping реализован согласно базовым правилам:

- cell с comma, quote или newline оборачивается quotes;
- internal quote удваивается.

CSV подходит для spreadsheet и простого exchange, но не выражает nested structures, typed events или null semantics так полно, как JSON.

Ghidra CSV

При `ghidra_mode` columns:

```
begin_rva,end_rva,name,comment
```

Name строится:

```
sub_%08X
```

Comment содержит summary:

```
version
flags
prolog size
stack bytes
frame register
saved GP/XMM
handler
chain
anomalies
```

Этот report создаёт bridge к Ghidra script. Он не меняет source binary и не является отдельной копией core.

Anomaly text

Bit mask `UnwindSummary.anomalies` преобразуется в compact labels:

```
ver
trunc
op
arg
off
chain_loop
chain_depth
handler_oob
order
operand_sanity
alignment
chain_invalid
```

Если mask zero, записывается `"0"`.

Compact text удобен в table, a numeric mask сохраняется в JSON. Consumer может использовать stable numeric bits, но для долгосрочной schema лучше также публиковать array full anomaly codes.

Report metadata

Полный JSON/HTML требует:

```
typedef struct ReportMetadata {
    const char* tool_version;
    const char* report_schema_version;
    const char* generated_at_utc;
    const char* input_sha256;
    uint64_t input_size;
    const char* command_line;
    const char* build_compiler;
    const char* build_platform;
    const char* build_mode;
} ReportMetadata;
```

Эти поля обеспечивают provenance.

Tool version говорит, какая implementation создала report.

Schema version говорит, как consumer должен интерпретировать JSON structure.

Generated timestamp фиксирует время serialization в UTC.

Input SHA-256 и size идентифицируют bytes.

Command line фиксирует analyzer options, но child arguments могут быть redacted.

Compiler, platform и build mode позволяют различить binaries tool.

Metadata validation

`report_metadata_valid` требует:

- nonempty tool version;
- exact supported версию schema;
- timestamp format длиной 20: `YYYY-MM-DDTHH:MM:SSZ`;
- SHA-256 length 64;
- только lowercase hex;
- nonnull command line;
- nonempty compiler/platform;
- build mode `debug` или `release`.

Проверка timestamp валидацирует syntax positions и digits, но не календарную корректность. Например, month 99 формально прошёл бы. Timestamp генерируется internally, поэтому risk ограничен, но reusable API могла бы выполнять полную date validation.

Schema version comparison strict. Старый или будущий version metadata не записывается этим writer. Это защищает от случайного mislabeled output.

Model validation

`report_model_valid` проверяет:

- если есть sections, pointer не null;
- finding count не выше capacity;
- mitigation count не выше capacity;
- dynamic event count не выше capacity;
- trace count и requested steps не выше array capacity.

Функция не перепроверяет все semantic relationships. Она является boundary sanity guard.

JSON escaping

Input path, error text, section names, command line и dynamic details являются untrusted strings. Они должны быть escaped.

`json_escape` обрабатывает:

```
\  -> \\
"  -> \"
newline -> \n
carriage return -> \r
tab -> \t
control bytes -> \u00xx
```

Для bytes >= 0x80 функция проверяет valid UTF-8 sequence lengths, включая:

- исключение overlong sequences;
- исключение surrogate range;
- ограничение code points до U+10FFFF.

Valid sequence копируется целиком. Invalid byte кодируется как `\u00xx`.

Это гарантирует syntactically valid JSON string, хотя invalid multibyte source преобразуется byte-wise, а не в Unicode replacement character.

JSON numeric representation

Addresses сериализуются strings с `0x`:

```
"begin_rva": "0x00001010"
```

Причины:

- сохраняется hexadecimal representation;
- 64-bit VA не теряется в JavaScript number;
- адрес явно отличается от обычного count.

Counts и ratios сериализуются numbers:

```
"score": 18,
"confidence": 0.43
```

Consumer должен знать type contract. Backend не должен преобразовывать address string в signed 32-bit integer.

Top-level JSON model

Основные sections:

```
input
report_metadata
image_base
size_of_image
size_of_headers
section_alignment
file_alignment
exception_directory
pe_validation
sections
heuristic_analysis          если запрошена и успешна
dynamic_validation          если dynamic был запрошен
assessment                  если есть static или dynamic report
functions
```

Optional blocks отражают выполненные stages. Отсутствие `heuristic_analysis` отличается от score zero.

`pe_validation`

Этот block содержит parser-level structural facts:

```
section_raw_oob
section_virtual_oob
section_raw_overlaps
section_virtual_overlaps
section_misaligned
exception_directory_oob
```

Они доступны независимо от heuristic score. Это полезно для backend, который хочет собственную policy.

`sections`

Для каждой section:

```
name
virtual_address
```

```
virtual_size
raw_offset
raw_size
characteristics
readable
writable
executable
```

Permissions как booleans дублируют bit field для удобства consumer. `characteristics` сохраняет original value.

`heuristic_analysis`

Block включает:

- score, suspicion, confidence;
- manual review flag;
- verdict и evidence level;
- severity counts;
- все raw counters;
- reuse ratios;
- evidence families;
- structured findings;
- weakening rules;
- human-readable details.

Каждый finding содержит:

```
code
family
severity
message
affected_count
representative_rva или null
representative_section
```

Для database нормализованный key - `code`, a message является display text. Business logic не должна зависеть от точной английской строки message.

`dynamic_validation`

Block сохраняет не только successful session:

```
enabled
ran
ok
timed_out
target_exited
target_exit_code
timeout_ms
trace_steps
counters
dynamic_signal_count
dropped_events
details
events
trace
```

Если `ok=false`, partial events всё равно могут быть значимы. Backend должен хранить session status и не отбрасывать block целиком.

Каждый event:

```
type
module
thread_id
exception_code
first_chance
address
arg0..arg3
```

Meaning args определяется event type. Это нужно документировать в API adapter.

Functions

Для каждой table entry JSON содержит original range, section context, parse status, decoded record и comment.

Внутренний `unwind` object:

```
version
flags
prolog_size
count_codes
fixed_stack_bytes
frame_reg
frame_off
has_handler
handler_rva
handler_section
has_chain
anomalies
anomalies_text
chain
```

Даже при `unwind_ok=false` object присутствует с initialized partial/default values, а `unwind_error` объясняет failure.

Это упрощает schema: array shape постоянна.

Итоговая static/dynamic correlation

`has_static_dynamic_correlation` использует две группы rules.

Первая:

```
suspicious static handler context
&&
any unwind/VEH/VCH/UEF handler hit
```

Suspicious handler context:

- handler non-executable;
- handler writable;
- handler dominance family.

Вторая:

```
any placement anomaly
&&
dynamic function table registration
```

Placement anomaly включает unusual/executable/writable unwind metadata или handler placement.

Correlation boolean означает совпадение агрегированных classes evidence. Она не всегда доказывает exact address identity. Поэтому detailed analyst report должен выполнять более строгий address-level join.

Dynamic signal count

Function считает категории, а не события:

```
unwind handler
VEH
VCH
UEF
dynamic function table
second chance
```

Registration и hit одной категории не дают два points.

Это снижает влияние repeated events и отражает diversity runtime mechanisms.

Runtime evidence level

Handler hit относится к strong component, dynamic table call и second chance - medium, registrations и first chance - weak.

Итог `none`, `weak`, `medium`, `strong` строится rule-based. Он не учитывает benign identity handler.

Case strength

`case_strength` комбинирует static и runtime evidence levels:

```
strong_correlated_case
medium_correlated_case
strong_unconfirmed_case
medium_case
weak_case
no_case
```

Если есть correlation и хотя бы одна сторона strong - strong correlated.

Если нет correlation, но одна сторона strong - strong unconfirmed.

Слово case означает аналитический набор evidence, а не юридически доказанный malicious verdict.

Final confidence

Начальная final confidence:

```
hr->confidence, если static report есть
0.10 иначе
```

Добавляется:

```
+0.10 если есть dynamic signals
+0.10 если есть static/dynamic correlation
```

Maximum `0.98`.

Это не Bayesian update. Runtime signal и correlation могут быть зависимы, а increments не калиброваны. Поле следует трактовать как rule-based report confidence.

Final verdict

Если static score:

```
>=70 high_suspicion_correlated/static
>=40 moderate_suspicion_correlated/static
>=15 low_suspicion_correlated/static
```

Если static score ниже 15, но dynamic signals существуют:

```
runtime_signals_only
```

Иначе:

```
no_strong_correlated_evidence
```

Dynamic events не повышают static score. Они влияют на final suffix, confidence, case strength и отдельный evidence level.

Manual validation plan

`collect_manual_validation_steps` строит context-dependent список.

Примеры:

- проверить `.pdata` ranges в PE viewer;
- открыть suspicious `UNWIND_INFO` в hex view;
- сравнить codes с Ghidra prolog;
- проверить section permissions;
- дизассемблировать handler;
- исследовать JIT explanation;
- capture dynamic table arguments;
- single-step registered handlers;
- выполнить dynamic run при наличии только static findings.

Это сильная часть report design: output не заканчивается score, а сообщает следующий проверочный шаг.

Steps являются English display strings, не stable IDs. Для backend localization и workflow automation лучше добавить code каждому step.

HTML security model

HTML self-contained: CSS и JavaScript встроены, external resources отсутствуют.

Content Security Policy:

```
default-src 'none'
style-src 'unsafe-inline'
script-src 'unsafe-inline'
img-src 'none'
base-uri 'none'
form-action 'none'
```

Такой CSP запрещает network loads, images, base URL и forms. Inline style/script разрешены, потому что report содержит собственный dashboard и filter.

Все untrusted strings проходят `html_escape`:

```
& < > " '
```

Invalid UTF-8 bytes превращаются в numeric entities.

Если path содержал `<script>`, он выводится text, а не исполняется.

HTML sections

Dashboard показывает:

- function count;
- parse failures;

- anomalous entries;
- static score/verdict/confidence;
- dynamic status.

Далее:

- provenance;
- PE image fields;
- structural validation;
- sections;
- final assessment;
- manual plan;
- findings;
- weakening rules;
- dynamic events and log;
- filterable runtime functions.

Client-side filter работает только по уже встроенной table и не отправляет данные наружу.

HTML status nuance

Dynamic card class green только если:

```
dr && dr->ran && dr->ok && !dr->timed_out
```

Text различает:

```
not requested
timed out
not run
completed
failed
```

Это предотвращает превращение «dynamic не запрошен» в «dynamic anomalies не найдены».

Atomic file write

CSV, JSON и HTML сначала полностью строятся в `StrBuf`, затем передаются `file_write_atomic`.

Atomic strategy:

1. записать temporary file;
2. flush/close;
3. rename в final path.

Consumer либо видит старый complete file, либо новый complete file, а не partial serialization.

Atomic rename не равен полной crash durability: без directory fsync внезапное отключение питания может иметь platform-dependent outcome. Для обычного report generation гарантия достаточна, но archival evidence storage требует отдельной durability policy.

Memory failure behavior

`StrBuf` имеет sticky `ok` state. Если append failed, дальнейшие operations не превращают truncated buffer в successful report.

Перед write проверяется:

```
sb_ok(all working buffers)
```

Если false, final file не записывается.

Это важнее частичного convenience output: invalid JSON не должен выдаваться как report.

JSON schema

`docs/report.schema.json` формализует expected types and required fields. Tests валидируют generated reports.

Schema version находится и в metadata, и в application constants. Изменение field contract требует:

1. обновить schema;
2. изменить версию schema;
3. добавить migration/compatibility notes;
4. обновить tests;
5. уведомить backend consumer.

Нельзя silently переименовать `rf_unwind_oob`, если коллега уже сохраняет это поле в database.

Как backend должен вызывать core

Текущее ядро предоставляет CLI executable и static library. Для другого разработчика самый безопасный initial integration:

```
backend receives uploaded binary
-> saves immutable job input
-> starts analyzer subprocess with explicit output paths
-> enforces timeout/resource limits
-> checks process exit code
-> parses JSON
-> validates JSON schema
-> stores raw JSON and normalized indexes
```

API между frontend и backend нужен. Между backend и core отдельный network API необязателен: subprocess protocol уже является boundary.

Отдельный core service нужен только если:

- анализ выполняется на другом host;
- нужен Windows-only dynamic worker;
- требуется queue and isolation;
- много concurrent jobs;
- deployment policy запрещает запуск subprocess в backend.

Рекомендуемая database model

Не следует разбивать каждый JSON field в таблицы до потери original report. Надёжная схема:

`analysis_job`:

```
id
status
input_sha256
input_size
original_filename
requested_static
```

```
requested_dynamic
created_at
started_at
finished_at
tool_version
schema_version
exit_code
failure_reason
```

`analysis_report`:

```
job_id
raw_json
json_hash
html_object_key
created_at
```

Нормализованные tables для search:

```
finding
runtime_function
dynamic_event
section
assessment
```

Raw JSON остаётся source of truth для backward compatibility и повторной migration.

Сравнение analyses

Сравнивать можно:

- один hash в разных tool versions;
- разные versions одного binary;
- static-only и static+dynamic runs;
- runs с разными target arguments.

Join keys:

```
input_sha256
tool_version
schema_version
function begin_rva
finding code + representative_rva
dynamic event type + module + offset
```

Absolute runtime VA нельзя напрямую сравнивать между runs из-за ASLR. Его нужно нормализовать к module+offset или RVA.

API adapter не должен менять core

Коллега может написать backend adapter без изменения parser/decoder:

```
HTTP upload
-> job queue
-> subprocess execution
-> JSON schema validation
-> persistence
-> REST response
```

Core следует менять только при необходимости нового analytical field, stable library API или structured streaming. Web concerns, authentication, database transactions и UI не должны входить в C analyzer.

Что сделано правильно

- несколько outputs для разных consumers;

- machine-readable JSON;
- provenance and hash;
- schema validation;
- UTF-8-aware escaping;
- HTML escaping and restrictive CSP;
- atomic writes;
- explicit partial dynamic status;
- structured findings and events;
- manual validation plan;
- static/dynamic assessment без изменения raw evidence.

Что улучшить

1. Добавить explicit policy version heuristics.
2. Typed dynamic event payloads.
3. Stable codes manual steps.
4. Full anomaly code arrays.
5. Score contribution per finding.
6. JSON writer abstraction, уменьшающий риск comma errors.
7. Streaming for extremely large function tables.
8. Optional gzip or JSON Lines for events.
9. Localization layer, не меняющий machine codes.
10. Digital signing of reports для chain of custody.
11. Separate raw observation schema from assessment policy.
12. Compatibility tests against previous версию schemas.

Итог

`report.c` является публичным контрактом продукта. Он соединяет C core с analyst, Ghidra, backend, database и frontend. Полноценность проекта определяется не только тем, может ли parser прочитать `.pdata`, но и тем, может ли downstream system точно понять:

```
что анализировалось
какой версией
какие этапы реально выполнены
какие факты обнаружены
какие выводы построены
какова их ограниченность
что проверять вручную
```

В текущем проекте этот контракт уже достаточно развит для subprocess integration. Сетевую API-прослойку можно строить вне ядра, не изменяя аналитические modules.

Расширенная лекция 16. Build, tests и release readiness

Том II. Глава 9. Сборка, тестирование, воспроизводимость и степень готовности

Что значит «проект работает»

Фраза «работает без ошибок» должна быть разложена на проверяемые утверждения:

- source compiles в заявленных toolchains;
- compiler не выдаёт warnings при strict flags;
- unit tests проходят;
- regression samples дают ожидаемые reports;
- malformed inputs не вызывают memory errors;
- JSON соответствует schema;
- HTML escaping сохраняет security;
- CLI exit codes соответствуют contract;
- release package содержит нужные artifacts;
- Windows dynamic branch проверен на Windows x64;
- реальные benign binaries не создают неприемлемый уровень false positives.

Ни один отдельный successful run не доказывает все пункты.

CMake project model

Top-level project:

```
project(
    eh_unwind_analyzer
    VERSION 1.0.0
    DESCRIPTION "Static and dynamic analyzer for Windows x64 EH/Unwind metadata"
    LANGUAGES C
)
```

Минимальная CMake version 3.23. Language standard:

```
target_compile_features(ehua_core PUBLIC c_std_17)
```

Core собирается как static library:

```
add_library(ehua_core STATIC ...)
```

CLI:

```
add_executable(eh_unwind_analyzer src/main.c)
target_link_libraries(eh_unwind_analyzer PRIVATE ehua_core)
```

Это modular monolith: один deployable CLI, внутри которого modules разделены C interfaces. Static library позволяет будущему native adapter link against core, но public API пока ориентирован на внутренние structs и не обещает ABI stability.

Core sources

Library включает:

```
app_info.c
dynamic_win.c
heuristics.c
pe.c
report.c
unwind.c
```

```
util.c
```

`main.c` не входит в library. Поэтому CLI orchestration отделена от analysis modules.

Platform checks

Если build выполняется на Windows, pointer size должен быть 64 bit:

```
if(WIN32 AND NOT CMAKE_SIZEOF_VOID_P EQUAL 8)
    message(FATAL_ERROR ...)
endif()
```

Это не запрещает 64-bit Linux/macOS static build. Проверка относится только к Windows runtime branch.

При MinGW определяется:

```
__USE_MINGW_ANSI_STDIO=1
```

Это обеспечивает ожидаемое formatting behavior для некоторых specifiers.

Warning policy

MSVC:

```
/W4
/permissive-
/WX при warnings-as-errors
```

GCC/Clang:

```
-Wall
-Wextra
-Wpedantic
-Wconversion
-Wshadow
-Wformat=2
-Werror
```

Warnings as errors включены в base preset. Это предотвращает накопление известных compiler diagnostics.

Warning-free build не доказывает отсутствие bugs, но повышает minimum discipline C code.

Instrumentation

Sanitizer preset:

```
AddressSanitizer
UndefinedBehaviorSanitizer
-fno-omit-frame-pointer
```

ASan обнаруживает многие out-of-bounds, use-after-free и related memory errors. UBSan обнаруживает ряд undefined operations.

Они доступны с GCC/Clang, не с текущим MSVC configuration.

Coverage preset использует:

```
--coverage -O0 -g
```

Sanitizers и coverage запрещены одновременно, чтобы избежать сложной и неподдерживаемой instrumentation combination.

Presets

Configure presets:

```
debug
release
sanitize
coverage
```

Все наследуют Ninja generator, tests enabled, compile commands export и warnings-as-errors.

Типичный release cycle:

```
cmake --preset release
cmake --build --preset release
ctest --preset release
```

Preset фиксирует повторяемый набор options и уменьшает зависимость от ручной command line.

Unit test executables

CTest создаёт девять C unit test binaries.

`unit_unwind`

Проверяет decoder:

- operation decoding;
- code order;
- flags;
- operand length;
- chain errors;
- boundary conditions.

`unit_heuristics`

Проверяет базовые static score, placement, verdict и confidence.

`unit_heuristics_mitigations`

Проверяет weakening rules:

- single signal;
- small sample;
- placement only;
- single family.

`unit_heuristics_context`

Проверяет context-sensitive rules:

- benign handler dominance suppression;
- dominance при corroborating context;
- writable metadata and handler combination.

`unit_heuristics_combos`

Проверяет additional combination findings и weights.

`unit_core_edges`

Проверяет:

- empty function table;
- resource-only image;
- overflow/bounds;
- RVA mapping;
- nested overlaps;
- PE structure findings.

`unit_report_json`

Проверяет:

- complete static/dynamic serialization;
- metadata;
- findings/events;
- UTF-8 safety;
- expected JSON fragments.

`unit_report_html`

Проверяет:

- HTML escaping;
- dashboard sections;
- filter script;
- отсутствие unescaped hostile strings.

`unit_matrix_report`

Проверяет combined assessment:

- strong static without dynamic;
- weak static plus weak runtime;
- strong correlated case;
- comma placement around optional blocks;
- final verdict and case strength.

Assertions в release unit tests

С `assert` обычно отключается macro `NDEBUG` в release builds. CMake принудительно undefines:

```
/UNDEBUG для MSVC
-UNDEBUG для GCC/Clang
```

Иначе release CTest мог бы «пройти», не выполнив проверки assertions.

Python integration tests

При наличии Python добавляются ещё семь tests.

`regression_samples`

Запускает CLI на synthetic PE corpus и проверяет `tests/samples/manifest.json`.

Corpus включает:

```
benign_minimal.exe
benign_richer.exe
suspicious_unwind_oob.exe
suspicious_unwind_writable.exe
suspicious_section_raw_oob.exe
suspicious_eh_unwind_tamper.exe
```

Manifest проверяет paths внутри generated JSON, minimum score, verdict и case strength.

`dynamic_validator`

Проверяет validator logic для Windows dynamic reports. На non-Windows это не заменяет реальный Debug API smoke.

`report_semantics`

Проверяет semantic relationships report, а не только JSON syntax.

`release_metadata`

Проверяет consistency version, schema и release files.

`release_hygiene`

Проверяет, что source tree и install rules не пропускают AppleDouble `._*`, `.DS_Store`, Python cache, `.plist` и неоднозначные старые Windows executables. Дополнительно закрепляет фильтрацию real-corpus runner и CPack.

`cli_contract`

Проверяет arguments, exit codes, expected outputs и errors.

`mutation_fuzz`

Создаёт 120 deterministic mutated inputs и запускает parser через CLI. Цель - обнаружить crashes, hangs и unsafe malformed-input paths.

Это mutation testing, а не exhaustive fuzzing.

Общее число CTest cases

Девять unit tests плюс семь Python tests дают:

```
16 CTest tests
```

Именно это число относится к финальной release-candidate configuration.

Regression corpus semantics

Synthetic samples полезны, потому что каждое изменение контролируется и ожидаемый signal известен.

`benign_minimal` и `benign_richer` ожидают:

```
score 0
no_strong_static_anomalies
evidence none
manual review false
case no_case
```

`suspicious_unwind_oob` проверяет broken pointer и parse failure.

`suspicious_unwind_writable` проверяет placement.

`suspicious_section_raw_oob` проверяет file layout and overlap.

`suspicious_eh_unwind_tamper` проверяет writable metadata, non-executable/writable handler и combined case.

Synthetic corpus не измеряет false-positive rate на diverse real software.

Реальный benign corpus

В проектной validation использовался corpus 106 реальных PE files. Его роль:

- проверить parser на разнообразных compiler/linker layouts;
- обнаружить unexpected failures;
- оценить benign false positives;
- проверить performance.

Для научной воспроизводимости нужно сохранять:

- список SHA-256;
- происхождение файлов;
- license/redistribution constraints;
- tool version;
- exact command;
- machine-readable aggregate results.

Если corpus не включён в repository из-за licensing, manifest hashes и acquisition procedure должны быть documented.

Фраза «проверено на 106 файлах» без списка и результатов слабее воспроизводимого benchmark.

На финальной revision повторный прогон дал:

```
106/106 valid reports
174 795 RUNTIME_FUNCTION records
0 parse failures
0 structural unwind anomalies
0 nonzero heuristic scores
maximum analysis time 765 ms
```

Исходные PE остаются в `real_samples/inbox` для локальной инженерной проверки, но не включаются в публичные source/customer ZIP из-за размера и необходимости отдельно учитывать redistribution terms. Их происхождение и download hashes записаны в `real_samples/SOURCES.md`.

Fuzzing levels

Проект имеет два уровня.

Deterministic mutation runner изменяет bytes existing samples и запускает full CLI.

Optional libFuzzer target:

```
ehua_parser_fuzzer
```

Он доступен при `EHUA_BUILD_FUZZER=ON` и Clang с libFuzzer runtime.

LibFuzzer выполняет coverage-guided input generation и может значительно глубже исследовать parser state space. Для release readiness полезно сохранять seed corpus и regression inputs для найденных crashes.

Fuzzing не доказывает отсутствие vulnerabilities. Оно увеличивает explored input space.

На финальной revision выполнены 1000 deterministic mutations:

```
accepted valid reports: 634
controlled rejected inputs: 366
crashes, hangs, invalid reports: 0
```

LibFuzzer target успешно собирается Homebrew LLVM 21. Полная bounded campaign остаётся Linux CI gate; локальная macOS campaign с крупными PE seeds оказалась медленнее установленного wall-clock budget.

Sanitizer limitations

ASan/UBSan на macOS/Linux проверяют static code paths и non-Windows dynamic stub. Windows-specific branch под `#ifdef _WIN32` при таком run не компилируется и не исполняется.

Для dynamic branch нужны:

- Windows x64 build;
- MSVC warning build;
- MinGW build;
- smoke targets;
- желательно Application Verifier или Windows-compatible sanitizer where supported.

Cross-compilation MinGW подтверждает compile compatibility, но не runtime correctness Debug API.

Dynamic smoke validation

`tests/run_dynamic_smoke_windows.bat` запускает четыре purpose-built targets.

Parameters:

```
--timeout-ms 8000
--trace-steps 32 для handler targets
--json report path
```

После runs `validate_dynamic_reports.py` сверяет reports со schema и expected counters.

Это минимальная обязательная проверка перед заявлением, что dynamic mode работает.

На Windows x64 8 июня 2026 года получены следующие результаты:

```
124/124 schema/semantic-valid reports
86 started dynamic sessions
54 normal completions
32 controlled timeouts
4/4 focused smoke scenarios
0 dropped events
trace budget violations: 0
```

Полный набор можно проверить без выполнения targets:

```
python3 tests/validate_dynamic_corpus_evidence.py
```

При наличии исходных corpus roots валидатор дополнительно пересчитывает размеры и SHA-256 каждого входного PE.

Для более сильной validation нужны:

- Windows 10/11;
- разные patch levels;

- MSVC и MinGW analyzer builds;
- single-thread и multi-thread target;
- module unload/reload;
- high event volume;
- timeout target;
- invalid handler RVA;
- JIT/dynamic table from real runtime;
- anti-debug-aware sample.

CLI exit codes

Main orchestration различает:

- argument/usage error;
- parse/read failure;
- requested stage failure;
- report write failure.

Даже если heuristic или dynamic stage failed, code может попытаться записать partial report. Process exit code остаётся nonzero, чтобы automation не приняла partial result за complete success.

Backend должен проверять и exit code, и JSON status. Только один из них недостаточен.

Report schema validation

Generated JSON должен проверяться против `docs/report.schema.json`.

Schema validation отвечает на вопрос:

```
соответствует ли structure declared contract
```

Она не отвечает:

```
верны ли вычисленные addresses и score
```

Для второго нужны semantic tests и golden cases.

Release metadata

Project version CMake `1.0.0`, package version `1.0.0`. Stable final означает, что текущая ревизия объявлена презентационной и проходит финальные release gates.

Report schema `1.0.0`.

Version dimensions:

```
tool implementation version
package/release version
report версию schema
heuristic policy version, пока отсутствует отдельно
```

Их не следует смешивать.

Installation

CMake installs:

- CLI в `bin`;

- docs;
- Ghidra integration;
- examples;
- README, LICENSE, CHANGELOG, SECURITY, CONTRIBUTING.

Core static library не включена в install targets. Если коллега хочет link directly, потребуется добавить library/header installation и stable API policy. Для subprocess integration это не нужно.

CPack

CPack generator создаёт ZIP:

```
eh-unwind-analyzer-1.0.0-System-Processor.zip
```

Archive package является distribution artifact, а не отдельным изменённым проектом. Source workspace остаётся основным местом edits. Архив содержит installed artifacts согласно CMake rules, поэтому он может намеренно не включать source и tests.

Именно это объясняет прежнее недоразумение: Windows project folder и release archive выполняют разные роли.

CPack package является native installed package. Дополнительно `scripts/create_release_bundles.py` формирует delivery set для разных получателей. Это четыре архива, но не четыре разные реализации программы.

Windows x64 package

```
eh-unwind-analyzer-1.0.0-windows-x64.zip
```

Содержит проверенный MinGW-w64 executable, пользовательскую документацию, JSON Schema, examples, Ghidra importer и release passport. Это основной пакет для Windows-заказчика. В нём нет source tree, test corpus и промежуточных build artifacts.

macOS arm64 package

```
eh-unwind-analyzer-1.0.0-Darwin-arm64.zip
```

Это CPack build для Apple Silicon. Он выполняет статический анализ Windows PE-файлов, формирует reports и поддерживает Ghidra workflow. Windows dynamic collector в macOS binary недоступен и возвращает controlled unsupported result.

Source package

```
eh-unwind-analyzer-1.0.0-source.zip
```

Содержит C source, headers, CMake, tests, CI, документацию, исходники книги, release scripts и manifests. Он предназначен разработчику или заказчику, которому передаётся исходный код. Реальные сторонние PE-файлы и локальные build directories не включаются.

Evidence package

```
eh-unwind-analyzer-1.0.0-evidence.zip
```

Содержит не программу для обычного запуска, а проверяемую доказательную базу: 124 individual JSON reports, logs, aggregate CSV/JSON/Markdown, итоговый DOCX, schema и validators. Его назначение - аудит утверждений о Windows dynamic validation.

Все package hashes записываются в `release-output/SHA256SUMS.txt`. Повторная сборка release bundles дала те же SHA-256, то есть их content reproducible при неизменном source tree и native CPack input.

Convenience scripts больше не создают неоднозначный executable в корне:

```
build_mingw.bat -> build\mingw-local\eh_unwind_analyzer.exe
build_msvc.bat  -> build\msvc-local\eh_unwind_analyzer.exe
```

Readiness dimensions

Готовность нельзя выразить одним абсолютным процентом без шкалы.

Static core

Сильные стороны:

- bounded parser;
- PE structure validation;
- detailed unwind decoder;
- explainable heuristics;
- JSON/HTML/CSV;
- unit/regression/fuzz tests;
- cross-platform file analysis.

Остаточные задачи:

- broader real corpus calibration;
- full code/metadata correspondence analysis;
- policy versioning;
- expanded formats/architectures.

Dynamic core

Сильные стороны:

- functional Windows debugger loop;
- handler/API breakpoints;
- per-thread re-arm;
- events and trace;
- smoke targets.
- фактическая MinGW-w64 Windows validation на 124 reports;
- воспроизводимый evidence validator.

Остаточные задачи:

- production sandbox;
- remote dynamic table decoding;
- richer context;
- финальный независимый MSVC gate;
- concurrency/module-lifetime hardening.

Product platform

CLI core готов к ручному и backend subprocess use.

Для multi-user product отсутствуют в самом core:

- authentication;
- upload API;
- queue;
- database;
- worker isolation;
- frontend;
- observability;
- retention policy.

Это не обязано входить в analytical core; другой человек может построить платформу вокруг JSON contract.

Можно ли защищать как ВКР

Да, если формулировка соответствует фактическому scope:

```
разработан инструмент статического анализа PE32+ AMD64 EH/unwind metadata
с explainable heuristic assessment,
Windows-only dynamic evidence collection,
machine-readable and interactive reports,
Ghidra integration,
automated tests and reproducible build
```

Нельзя заявлять:

- универсальный reverse engineering suite;
- гарантированный malware detector;
- полную динамическую песочницу;
- поддержку всех PE architectures;
- доказанную statistical accuracy без benchmark.

Сильная защита показывает не отсутствие границ, а осознанное управление ими.

Release gate перед передачей

Перед каждым release:

1. Clean configure release.
2. Build with warnings-as-errors.
3. Run 16 CTest tests.
4. Run sanitizer suite.
5. Run mutation fuzz.
6. Validate synthetic corpus.
7. Validate representative real corpus.
8. Build Windows MSVC and MinGW.
9. Run Windows dynamic smoke.
10. Validate JSON schema.

11. Generate HTML and inspect hostile escaping sample.
12. Verify version consistency.
13. Build native CPack package.
14. Build Windows, source and evidence delivery packages.
15. Revalidate extracted source/evidence packages.
16. Compute and verify all package SHA-256.
17. Publish known limitations and compiler gates.

Фактический паспорт качества final revision

На 8 июня 2026 года зафиксированы:

Gate	Result
Release CTest	16/16
ASan/UBSan CTest	16/16
Coverage CTest	16/16
Line coverage без Windows-only branch	86,3%
Function coverage	96,2%
Branch coverage	62,8%
Deterministic mutations	1000 PASS
Real benign corpus	106/106
Windows report validation	124/124
Focused dynamic smoke	4/4
Clang Static Analyzer	0 findings
Extracted source package	16/16
Extracted evidence package	124/124

Эти цифры относятся к конкретной revision и не заменяют future regression testing. Проверенный Windows binary - MinGW-w64. Поэтому версия остаётся `1.0.0`, когда independent MSVC job и Windows dynamic corpus пройдены на той же release revision.

Итог

Проект уже вышел за уровень одноразового prototype: у него есть modular core, strict build, report contract, tests, fuzzing, release metadata и dynamic smoke infrastructure.

Однако «полностью законченный top product» требует operational layer и дальнейшей empirical validation. Наиболее точная характеристика текущего состояния:

```
зрелое ядро исследовательского инструмента release-candidate уровня,
готовое к ВКР, ручному использованию и backend integration,
но не завершённая multi-user sandbox platform
```

Такое определение не уменьшает ценность работы. Оно правильно отделяет завершённость аналитического ядра от завершённости всей будущей информационной системы.

Расширенная лекция 17. Разбор нормального benign-файла

Том II. Глава 10. Полный разбор `benign_richer.exe`: от bytes до assessment

Цель сквозного разбора

Предыдущие главы объясняли modules отдельно. Здесь один реальный test binary проходит весь pipeline. Ни одно значение не предполагается: все числа взяты из фактического файла и JSON, созданного текущим release build 8 июня 2026 года.

Input:

```
tests/samples/bin/benign_richer.exe
```

Размер:

```
5641 bytes
```

SHA-256:

```
8b79fe76e06a44637caa1238460b9a83c50c49d80f2f355a614cef021bacc1ec
```

Command:

```
build/release/eh_unwind_analyzer \
  tests/samples/bin/benign_richer.exe \
  --heuristics \
  --json /tmp/teacher_benign.json \
  --html /tmp/teacher_benign.html
```

Build platform `macos-arm64`, compiler Clang 21.0.0, mode release. Это допустимо, потому что выполняется static parsing Windows PE, а не Windows dynamic mode.

Первые bytes

Hex dump:

```
00000000: 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00
00000010: b8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000030: 00 00 00 00 00 00 00 00 00 00 00 00 80 00 00 00
```

Bytes `4d 5a` дают little-endian word `0x5A4D`, signature `MZ`.

По offset `0x3C`:

```
80 00 00 00
```

Little-endian conversion:

```
e_lfanew = 0x00000080
```

Следовательно, PE signature ожидается по file offset `0x80`. Parser проверяет, что этот offset и следующие headers лежат внутри 5641-byte buffer.

PE identity

Из COFF и optional header report получает:

```
Machine      = 0x8664
ImageBase    = 0x140000000
SizeOfImage  = 0x7000
SizeOfHeaders = 0x400
Sections     = 6
```

`0x8664` соответствует AMD64. Optional header имеет PE32+ magic, иначе parser завершил бы analysis unsupported-format error.

`SizeOfImage = 0x7000` задаёт RVA range:

```
[0x0000, 0x7000)
```

Alignment

```
SectionAlignment = 0x1000
FileAlignment    = 0x0200
```

Section RVAs:

```
0x1000
0x2000
0x3000
0x4000
0x5000
0x6000
```

Все кратны `0x1000`.

Raw offsets ненулевых raw sections:

```
0x400
0x600
0x800
0xA00
0xC00
```

Все кратны `0x200`.

`section_misaligned = 0`.

Sections

.text

```
RVA          0x1000
VirtualSize  0x40
RawOffset    0x400
RawSize      0x200
Flags        0x60000020
Permissions  R-X
```

Code functions находятся в `[0x1000, 0x1040)`. RawSize больше VirtualSize из-за file alignment padding.

.rdata

```
RVA          0x2000
VirtualSize  0x50
RawOffset    0x600
RawSize      0x200
Permissions  R--
```

.pdata

```
RVA          0x3000
VirtualSize  0x24
RawOffset    0x800
RawSize      0x200
Permissions  R--
```

Значимые 36 bytes содержат три `RUNTIME_FUNCTION`.

.xdata

```
RVA          0x4000
VirtualSize  0x14
RawOffset    0xA00
RawSize      0x200
Permissions  R--
```

Значимые 20 bytes содержат три unwind records.

.bss

```
RVA          0x5000
VirtualSize  0x10
RawOffset    0
RawSize      0
Permissions  RW-
```

`.bss` virtual-only. Отсутствие raw bytes корректно.

.idata

```
RVA          0x6000
VirtualSize  0x18
RawOffset    0xC00
RawSize      0x200
Permissions  R--
```

Structural checks

Для каждой raw section:

```
RawOffset + RawSize <= 5641
```

Для каждой virtual section:

```
VirtualAddress + max(VirtualSize, RawSize) <= 0x7000
```

Pairwise raw и virtual intervals не пересекаются в запрещённой форме. Result:

```
section_raw_oob = 0
section_virtual_oob = 0
section_raw_overlaps = 0
section_virtual_overlaps = 0
section_misaligned = 0
```

Exception directory

```
RVA  = 0x3000
Size = 0x24
```

RVA находится в `.pdata`.

Conversion:

```
delta = 0x3000 - 0x3000 = 0
file offset = 0x800 + 0 = 0x800
```

Size `0x24 = 36`. Entry size 12:

```
36 / 12 = 3
```

Incomplete tail отсутствует.

Raw .pdata bytes

```
00000800:
00 10 00 00  10 10 00 00  00 40 00 00
10 10 00 00  29 10 00 00  04 40 00 00
29 10 00 00  40 10 00 00  0c 40 00 00
```

Разбивка по 12 bytes.

Entry 0

```
00 10 00 00 -> BeginRVA  0x1000
10 10 00 00 -> EndRVA    0x1010
00 40 00 00 -> UnwindRVA 0x4000
```

Function size:

```
0x1010 - 0x1000 = 0x10 = 16
```

Entry 1

```
10 10 00 00 -> 0x1010
29 10 00 00 -> 0x1029
04 40 00 00 -> 0x4004
```

Size:

```
0x1029 - 0x1010 = 0x19 = 25
```

Entry 2

```
29 10 00 00 -> 0x1029
40 10 00 00 -> 0x1040
0c 40 00 00 -> 0x400C
```

Size:

```
0x1040 - 0x1029 = 0x17 = 23
```

Table invariants

Ranges:

```
[0x1000,0x1010)
[0x1010,0x1029)
[0x1029,0x1040)
```

Каждый end больше begin. Все bounds внутри `SizeOfImage`. Entries sorted. Соседние ranges соприкасаются только exclusive/end boundary, overlap отсутствует. Begin values unique.

Unwind RVA:

```
0x4000
0x4004
0x400C
```

Все nonzero и внутри image.

Conversion unwind RVA

`.xdata`:

```
VirtualAddress = 0x4000
RawOffset      = 0xA00
```

Conversions:

```
0x4000 -> 0xA00 + 0x0 = 0xA00
0x4004 -> 0xA00 + 0x4 = 0xA04
0x400C -> 0xA00 + 0xC = 0xA0C
```

Raw `.xdata` bytes

```
00000a00:
01 00 00 00
01 04 01 00 04 42 00 00
01 04 01 00 04 42 00 00
```

Record 0 at `0xA00`

Header:

```
byte0 = 0x01
Version = 0x01 & 0x07 = 1
Flags   = 0x01 >> 3 = 0

SizeOfProlog = 0
CountOfCodes = 0
Frame byte   = 0
```

No code array, no handler, no chain.

Summary:

```
v1
prolog 0
stack 0
frame none
anomalies 0
```

Record 1 at `0xA04`

Header:

```
01 04 01 00
```

Interpretation:

```
Version = 1
Flags = 0
SizeOfProlog = 4
CountOfCodes = 1
FrameRegister = 0
FrameOffset = 0
```

Code slot:

```
04 42
```

`CodeOffset = 0x04`.

Second byte nibbles:

```
UnwindOp = low nibble 0x2 = UWOP_ALLOC_SMALL
OpInfo   = high nibble 0x4
```

Allocation:

```
4 * 8 + 8 = 40 bytes
```

CountOfCodes odd, поэтому aligned tail находится после padding slot `00 00`. Flags zero, tail semantics нет.

Record 2 at `0xA0C`

Bytes идентичны record 1:

```
01 04 01 00 04 42 00 00
```

Summary также stack 40 bytes.

Parallel arrays

Main allocates:

```
rf[3]
us[3]
us_ok[3]
```

After decoding:

```
us_ok = [true, true, true]
```

`us[0]` относится к `rf[0]`, и так далее. Sorting heuristic копии не нарушает эту relation.

Heuristic aggregation

Raw counters:

```
invalid_ranges = 0
oob = 0
unwind_oob = 0
unsorted = 0
```

```
overlaps = 0
duplicate_begins = 0
parse_fail = 0
anomalies = 0
placement = 0
handler placement = 0
```

Unwind frequency map содержит три distinct keys, frequency каждого 1:

```
max_count = 1
unique = 3
total = 3
```

Ratios:

```
unwind_reuse_ratio = 1/3 = 0.333333
unwind_unique_ratio = 3/3 = 1.0
```

Хотя max ratio больше 0.30, strong reuse finding требует `n > 200`. Здесь `n = 3`, поэтому finding не создаётся.

Handler count zero.

Static assessment

Ни одно scoring rule не сработало:

```
score = 0
suspicion = 0.0
confidence = 0.2
evidence_level = none
verdict = no_strong_static_anomalies
requires_manual_review = false
```

`details` содержит:

```
- [info] No strong anomalies detected by heuristics.
```

Final assessment

Dynamic mode не запрошен:

```
runtime_evidence_level = none
dynamic_signal_count = 0
correlation = false
```

Final:

```
final_verdict = no_strong_correlated_evidence
final_confidence = 0.2
case_strength = no_case
```

Что именно доказал этот run

Доказано для SHA-256 указанного input и текущей implementation:

- file соответствует поддерживаемой минимальной PE32+ AMD64 model;
- шесть section headers прочитаны;
- поддерживаемые structural checks не нашли нарушений;
- exception directory содержит три полные entries;
- ranges sorted, non-overlapping и bounded;
- три unwind records decoded;
- два records описывают fixed stack allocation 40 bytes;

- parser anomalies отсутствуют;
- реализованные static heuristics не создали findings.

Не доказано:

- что binary безопасен;
- что machine code полностью соответствует metadata;
- что все PE directories корректны;
- что file будет вести себя benign при запуске;
- что в нём нет иных методов obfuscation;
- что dynamic exception behavior отсутствует.

Как выполнить ручную correspondence check

Для functions:

```
preferred VA 0x140001010
preferred VA 0x140001029
```

в Ghidra нужно проверить первые четыре bytes prolog. Metadata ожидает stack allocation 40 bytes. Типичная instruction:

```
sub rsp, 28h
```

Если фактическая instruction подтверждает это, достигается третий уровень evidence: correspondence metadata и code.

Почему этот sample важен

Он показывает, что clean baseline не означает отсутствие metadata. Напротив, report подробно восстанавливает function ranges и stack effects, но не начисляет suspicion.

Это distinction между:

```
наличием сложной системной структуры
и
нарушением её invariants
```

Проект ценен не только findings. Он формирует normalized map EH/unwind metadata даже для normal binary.

Расширенная лекция 18. Разбор подозрительных samples

Том II. Глава 11. Расследование четырёх подозрительных regression samples

Назначение samples

Regression samples созданы для контролируемой проверки rules. Они не являются коллекцией реального malware и не используются для оценки detection accuracy.

Каждый sample изменяет известное свойство `benign_richer.exe`, поэтому можно связать:

```
изменённый факт
-> parser observation
-> finding
-> score contribution
-> manual verification
```

`suspicious_unwind_oob.exe`

Фактический report

```
score = 50
confidence = 0.69
verdict = moderate_suspicion
evidence_level = strong
case_strength = strong_unconfirmed_case
```

Findings:

```
unwind_rva_oob
severity high
affected 1
representative RVA 0x9000

unwind_parse_fail
severity high
affected 1
representative RVA 0x9000
```

Почему `0x9000` invalid

Image size sample сохраняет `0x7000`. Допустимые RVA:

```
[0x0000, 0x7000)
```

`0x9000 >= 0x7000`, поэтому pointer выходит за image.

Score derivation

Один invalid pointer из трёх records:

```
1 / 3 = 33.33%
```

Это больше threshold 1%.

Weights:

```
unwind_rva_oob high = 25
unwind_parse_fail high = 25
total = 50
```

Confidence derivation

Base:

```
0.15
```

Families:

```
table_integrity
unwind_integrity
2 * 0.11 = 0.22
```

Strong findings:

```
2 * 0.08 = 0.16
```

Hard table bonus:

```
+0.08
```

Unwind hard bonus:

```
+0.08
```

Total:

```
0.15 + 0.22 + 0.16 + 0.08 + 0.08 = 0.69
```

Weakening rules отсутствуют, потому что findings strong.

Что проверять вручную

1. Открыть `.pdata` по raw offset.
2. Найти entry с `UnwindData = 0x9000`.
3. Проверить `SizeOfImage`.
4. Убедиться, что RVA не принадлежит section.
5. Определить, является ли это corruption, deliberate tamper или ошибка generator.
6. Исследовать function prolog без опоры на broken metadata.

Ограниченный вывод

Один runtime function ссылается на unwind RVA вне declared image, поэтому Windows x64 unwind contract для этой записи нарушен. Намерение изменения автоматически не установлено.

`suspicious_unwind_writable.exe`

Фактический report

```
score = 18
confidence = 0.34
verdict = low_suspicion
evidence_level = medium
case_strength = medium_case
```

Finding:

```
unwind_writable_section
severity high
affected 3
representative RVA 0x4000
```

Structural fact

Все три `UnwindRVA` остаются valid и decode succeeds. Изменено section characteristic `.xdata`: section объявлена writable.

Score

Affected count 3, total 3. Condition:

```
3 > 3/20
```

Integer `3/20 = 0`, поэтому high branch:

```
weight = 18
```

Confidence

Base `0.15`.

One family:

```
+0.11
```

One strong finding:

```
+0.08
```

Total `0.34`.

Weakening rules не применяются, потому что `strong_count = 1`.

Почему score low, evidence medium

Verdict threshold:

```
15 <= 18 < 40 -> low_suspicion
```

Evidence level:

```
strong_count >= 1 -> medium
```

Fields не противоречат друг другу: score band low, но finding severity high enough for medium evidence level.

Возможные benign explanations

- runtime or JIT modifies unwind records;
- custom loader changes protections;
- nonstandard toolchain;
- sample deliberately built for test.

Manual verification

1. Проверить original `Characteristics`.
2. Запустить на Windows и запросить actual page protection.
3. Наблюдать writes к `.xdata`.
4. Проверить signatures compiler/runtime.
5. Сравнить metadata до и после execution.

Ограниченный вывод

Все records structurally valid, но metadata размещена в writable section. Это расширяет возможность runtime modification и требует проверки, однако не доказывает tampering.

`suspicious_section_raw_oob.exe`

Фактический report

```
score = 60
confidence = 0.46
verdict = moderate_suspicion
evidence_level = strong
case_strength = strong_unconfirmed_case
```

Findings:

```
pe_section_raw_oob
high
affected 1
RVA 0x5000

pe_section_raw_overlap
medium
affected 5
RVA 0x5000

pe_section_virtual_overlap
high
affected 1
RVA 0x6000
```

Score

```
raw OOB          25
raw overlap      15
virtual overlap  20
total            60
```

Confidence

All findings belong to one family `table_integrity`:

```
base               0.15
one family        +0.11
two strong        +0.16
one medium        +0.04
total              0.46
```

Hard table bonus in implementation applies only to runtime-function hard issues, not PE section counters. Поэтому additional `+0.08` нет.

Почему пять raw overlaps

Изменённая section raw range может стать настолько большой, что пересекает несколько других sections. Counter считает каждую pairwise collision.

Representative location показывает первый violating section, a affected count отражает число overlaps.

Manual verification

1. Выписать raw interval каждой section.
2. Использовать формулу пересечения полуинтервалов.
3. Проверить file size.
4. Сравнить behavior Windows loader и parser.
5. Определить, какие bytes фактически shared между section descriptions.

Ограниченный вывод

Section table внутренне противоречива: по крайней мере один raw range выходит за file и ranges пересекаются. Любая дальнейшая interpretation bytes через section mapping должна считаться потенциально неоднозначной.

`suspicious_eh_unwind_tamper.exe`

Фактический report

```
score = 53
confidence = 0.54
verdict = moderate_suspicion
evidence_level = strong
case_strength = strong_unconfirmed_case
```

Findings:

```
unwind_writable_section
high
affected 1
RVA 0x2010

handler_nonexec_section
high
affected 1
RVA 0x2000

handler_writable_section
```

```
medium
affected 1
RVA 0x2000

writable_metadata_handler_combo
high
affected 0
no representative RVA
```

Interpretation

Одна unwind record перенаправлена в writable section near `.rdata`-like area. Handler RVA указывает на `0x2000`, который расположен в non-executable, writable section.

Score

```
unwind writable high       18
handler non-executable     15
handler writable           10
combined writable pattern  10
total                      53
```

Confidence

Families:

```
metadata_placement
```

All findings use same family in this case.

Severity:

```
3 strong
1 medium
```

Calculation:

```
base             0.15
one family      +0.11
three strong    +0.24
one medium      +0.04
total            0.54
```

No single-family penalty because strong findings exist.

Why combination affected count is zero

Combination finding создаётся без `annotate_finding`. Он описывает relation между aggregate counters, а не отдельный object. Поэтому:

```
has_location = false
affected_count = 0
```

Это data-model nuance, не отсутствие combination.

Manual investigation

1. Найти exact `RUNTIME_FUNCTION`, указывающую `0x2010`.
2. Decode record вручную.
3. Проверить handler flags.
4. Прочитать handler RVA `0x2000`.
5. Проверить section permissions.
6. Дизассемблировать bytes по `0x2000`.
7. Если bytes не code, определить, загрузил бы Windows record и когда возник failure.

8. На Windows поставить breakpoint на actual handler VA.
9. Наблюдать, достигается ли он.
10. Проверить runtime page protection и modifications.

Ограниченный вывод

Metadata и handler placement одновременно противоречат обычному read-only-data/executable-code layout. Сочетание является сильным static anomaly, но sample intent и runtime reachability требуют отдельного подтверждения.

Почему все cases `unconfirmed`

Dynamic mode не запускался. Поэтому:

```
runtime evidence = none
correlation = false
```

Strong static evidence даёт:

```
strong_unconfirmed_case
```

Это корректное название. Статика подтверждает structural anomalies, но не runtime activation.

Как превратить case в correlated

Для OOB record dynamic correlation может быть не достижима, если Windows loader или exception path не использует function.

Для writable handler case:

1. Запустить target в isolated Windows VM.
2. Trigger representative exception path.
3. Получить handler hit или exception flow.
4. Нормализовать address к module+RVA.
5. Связать с static representative RVA.

Если static suspicious handler context совпадает с handler hit, project summary устанавливает correlation.

Regression test и реальное расследование

Regression test отвечает:

```
сработало ли правило на специально созданном нарушении
```

Real investigation отвечает:

```
почему нарушение существует в неизвестном binary и что оно делает
```

Первое автоматизировано. Второе требует reverse engineer. Проект уменьшает объём ручной работы, но не отменяет её.

Расширенная лекция 19. Типовые проблемы и решения

Том II. Глава 12. Задачи с полными решениями

Как работать с задачами

Решение следует сначала получить самостоятельно, затем сравнить не только final number, но и последовательность проверок. В reverse engineering правильный ответ без проверяемого пути недостаточен.

Задача 1. Little-endian field

По file offset `0x3C` находятся bytes:

```
80 00 00 00
```

Определить 32-bit unsigned value.

Решение

Little-endian означает, что byte с наименьшим весом хранится первым:

```
value =
0x80 * 256^0
+ 0x00 * 256^1
+ 0x00 * 256^2
+ 0x00 * 256^3
= 0x00000080
```

Это `e_lfanew`; PE header начинается по file offset 128 decimal.

Ошибка была бы прочитать bytes как big-endian `0x80000000`, что почти наверняка вывело бы parser за file.

Задача 2. RVA в file offset

Section:

```
VirtualAddress   = 0x4000
VirtualSize      = 0x180
PointerToRawData = 0xA00
SizeOfRawData    = 0x200
```

Найти file offset для RVA `0x40B4`.

Решение

Проверка нижней границы:

```
0x40B4 >= 0x4000
```

Delta:

```
0x40B4 - 0x4000 = 0xB4
```

Virtual span:

```
max(0x180,0x200) = 0x200
0xB4 < 0x200
```

Raw availability:

```
0xB4 < 0x200
```

Offset:

```
0xA00 + 0xB4 = 0xAB4
```

Дополнительно требуется проверить:

```
0xAB4 < file_size
```

Без file size окончательная безопасность чтения не доказана.

Задача 3. Virtual-only address

Section:

```
VirtualAddress = 0x5000
VirtualSize = 0x300
RawSize = 0x100
RawOffset = 0xC00
```

RVA `0x5180`. Принадлежит ли section? Есть ли byte в file?

Решение

Delta:

```
0x5180 - 0x5000 = 0x180
```

Virtual span:

```
max(0x300,0x100) = 0x300
0x180 < 0x300
```

Адрес принадлежит virtual section range.

Но:

```
0x180 >= RawSize 0x100
```

Соответствующего raw byte нет. Загрузчик может сформировать zero-filled memory, но static parser не должен читать file offset `0xD80` как часть этой section.

Задача 4. Число runtime functions

Exception directory size `0x54`. Сколько полных AMD64 `RUNTIME_FUNCTION`?

Решение

Entry size 12 decimal, то есть `0xC`.

```
0x54 = 84 decimal
84 / 12 = 7
```

Remainder:

```
84 % 12 = 0
```

Семь полных entries, неполного хвоста отсутствует.

Если size был бы `0x55`, осталось бы один лишний byte, и строгая policy должна сообщить malformed directory.

Задача 5. Range invariants

Даны entries:

```
A [0x1000,0x1100)
B [0x1100,0x1200)
C [0x1180,0x1300)
D [0x1300,0x1300)
```

Какие нарушения?

Решение

A и B соприкасаются:

```
B.begin == A.end
```

Overlap отсутствует.

C:

```
C.begin 0x1180 < B.end 0x1200
```

Overlap B/C.

D:

```
End == Begin
```

Invalid empty range.

Если sorted overlap algorithm использует `max_end`, после C он станет `0x1300`. D begin равен `0x1300`, поэтому overlap с предыдущими отсутствует, хотя range сам invalid.

Задача 6. Decode header

Header bytes:

```
09 06 02 35
```

Разобрать fields.

Решение

Byte0 `0x09 = 00001001b`.

Version - low 3 bits:

```
001b = 1
```

Flags - high 5 bits:

```
00001b = 1
```

То есть `UNW_FLAG_EHANDLER`.

`SizeOfProlog = 0x06 = 6`.

`CountOfCodes = 0x02 = 2 slots`.

Frame byte `0x35`:

```
FrameRegister = low nibble = 5
FrameOffset = high nibble = 3
FrameOffsetBytes = 3 * 16 = 48
```

Register encoding 5 соответствует RBP.

После двух slots code array уже DWORD-aligned. Поскольку EHandler flag установлен, далее ожидается 32-bit HandlerRVA.

Задача 7. `UWOP_ALLOC_SMALL`

Code slot:

```
04 72
```

Решение

`CodeOffset = 4`.

Second byte:

```
low nibble 2 -> UWOP_ALLOC_SMALL
high nibble 7 -> OpInfo 7
```

Allocation:

```
7 * 8 + 8 = 64 bytes
```

Требуется `SizeOfProlog >= 4`, иначе code-offset anomaly.

Задача 8. `UWOP_ALLOC_LARGE`

Slots:

```
08 01
20 00
```

Решение

Operation byte `0x01`:

```
UnwindOp = 1 -> UWOP_ALLOC_LARGE
OpInfo = 0
```

Следующий slot как little-endian `uint16`:

```
20 00 -> 0x0020 = 32
```

Scale 8:

```
32 * 8 = 256 bytes
```

Логическая operation потребила два slots: основной и operand.

Задача 9. Code ordering

`SizeOfProlog = 8`, code offsets:

```
8, 5, 6
```

Решение

Каждый offset не превышает prolog size.

Но descending order нарушен:

```
5 -> 6 увеличивается
```

Устанавливается `UA_CODEOFFSET_ORDER`.

Decoder может продолжить summary, если structure readable, но report должен отметить anomaly.

Задача 10. Alignment tail

`CountOfCodes = 3`. Где начинается tail относительно начала `UNWIND_INFO`?

Решение

Header 4 bytes.

Три meaningful slots:

```
3 * 2 = 6 bytes
```

После header offset был бы:

```
4 + 6 = 10
```

Tail должен быть DWORD-aligned. Slots округляются до четырёх:

```
align slots = 4
4 + 4*2 = 12
```

Tail начинается по offset 12 (`0xC`) от начала record. Один padding slot занимает bytes 10-11.

Задача 11. Score calculation

Report counters:

```
rf_oob: 2 из 100
unwind_parse_fail: 1 из 100
unwind_exec_section: 3
handler_nonexec_section: 1
```

Рассчитать minimum score, учитывая combinations.

Решение

`rf_oob` prevalence:

```
2/100 = 2% > 1%
```

Weight high:

```
+25
```

`unwind_parse_fail`:

```
1/100 = 1%
```

Threshold strict >1%, поэтому low:

```
+10
```

`unwind_exec_section`:

```
+20
```

`handler_nonexec_section`:

```
+15
```

Combination executable metadata + non-executable handler:

```
+10
```

Placement + integrity, потому что executable placement и parse failure:

```
+8
```

Total:

```
25 + 10 + 20 + 15 + 10 + 8 = 88
```

Score clamps at 100, но здесь 88.

Verdict high suspicion, evidence strong.

Задача 12. Confidence calculation

Для предыдущего case предположим six stored findings:

```
4 high
2 medium
3 evidence families
hard table issue present
hard unwind issue present
```

Решение

```
base                  0.15
families 3*0.11      +0.33
strong 4*0.08        +0.32
medium 2*0.04        +0.08
table bonus          +0.08
unwind bonus         +0.08
raw total             1.04
```

Weakening rules не применяются из-за strong findings и multiple families.

Static confidence clamps:

```
0.90
```

Если dynamic signals и correlation добавятся в report-level summary:

```
0.90 + 0.10 + 0.10 = 1.10
```

Final confidence clamps at `0.98`.

Это демонстрирует engineered ceiling.

Задача 13. Breakpoint state

Breakpoint по `0x7FF600001000` заменил original byte `0x55` на `0xCC`. Возник `EXCEPTION_BREAKPOINT`, context RIP равен `0x7FF600001001`. Какие изменения нужны?

Решение

1. Записать `0x55` обратно по `0x...1000`.
2. Flush instruction cache.
3. Установить:

```
RIP = 0x7FF600001000
```

4. Установить TF:

```
EFlags |= 0x100
```

5. Сохранить pending re-arm для current thread ID.
6. `ContinueDebugEvent(..., DBG_CONTINUE)`.
7. На `EXCEPTION_SINGLE_STEP` снова записать `0xCC`.
8. Снять TF, если trace не продолжается.

Если не rewind RIP, original instruction будет пропущена. Если не re-arm, последующие hits потеряются.

Задача 14. First-chance continuation

Target создал access violation, которое должно быть обработано его VEH. Что должен вернуть collector?

Решение

Для ordinary exception collector должен:

```
DBG_EXCEPTION_NOT_HANDLED
```

Это не означает, что exception останется необработанным окончательно. Status сообщает debugger layer, что он сам exception не обработал; Windows продолжит application dispatch, включая handlers.

`DBG_CONTINUE` поглотил бы exception на уровне debugger и мог не позволить VEH выполнить expected path.

Задача 15. Dynamic function table event

На breakpoint `RtlAddFunctionTable`:

```
RCX = 0x000001F000100000
RDX = 12
R8  = 0x000001F000000000
```

Что доказано?

Решение

Доказано:

- execution достиг entry соответствующего API;
- на момент entry argument registers содержали указанные values;
- target пытался вызвать function с pointer, count 12 и base.

Не доказано:

- API вернул TRUE;
- pointer readable;
- table содержит 12 valid entries;
- entries sorted;
- unwind data valid;
- table использовалась exception dispatcher;
- code malicious.

Для усиления evidence нужны return observation, remote memory snapshot и последующий runtime use.

Задача 16. Correlation

Static report:

```
handler_writable_section = 1
representative RVA = 0x5200
```

Dynamic event:

```
type = veh_handler_hit
module = target.exe
address = ActualBase + 0x7000
```

Можно ли утверждать exact correlation с handler `0x5200`?

Решение

Нет. Aggregate project rule может считать suspicious handler context плюс handler hit correlated at class level, но exact addresses различаются:

```
0x5200 != 0x7000
```

Нужно определить, был ли static finding frame handler, a dynamic event - VEH. Это разные mechanisms.

Корректный вывод:

```
в run наблюдался handler activity при наличии suspicious static handler context,
но exact address-level identity не установлена
```

Задача 17. Backend result

Analyzer exit code nonzero, но JSON существует и содержит:

```
dynamic_validation.ran = true
dynamic_validation.ok = false
dynamic_validation.timed_out = true
heuristic_analysis.score = 50
```

Как сохранять job?

Решение

Job не должен получать общий status `success`.

Рекомендуемая модель:

```
job status = partial_failed или failed_with_report
static stage = completed
dynamic stage = timed_out
process exit code = nonzero
raw JSON = stored
findings = indexed
failure reason = dynamic timeout
```

Отбрасывать JSON нельзя: static evidence valid и dynamic partial events могут быть полезны.

Показывать пользователю complete success также нельзя.

Задача 18. Сравнение двух runs

Run A handler address `0x7FF612345200`. Run B address `0x7FF712345200`. Module base A `0x7FF612340000`, base B `0x7FF712340000`.

Решение

Offsets:

```
A: 0x7FF612345200 - 0x7FF612340000 = 0x5200
B: 0x7FF712345200 - 0x7FF712340000 = 0x5200
```

Absolute VAs различаются из-за ASLR, module-relative offset совпадает. События относятся к одной позиции image, если module binary identity совпадает.

Задача 19. Критика verdict

Report:

```
score = 0
verdict = no_strong_static_anomalies
```

Можно ли написать заказчику «файл безопасен»?

Решение

Нет.

Score zero означает только, что реализованные rules не нашли strong static EH/unwind anomalies. Analyzer:

- не выполняет full malware analysis;
- не проверяет все PE directories;
- не доказывает code semantics;

- может не видеть поведение, созданное во время выполнения;
- не покрывает недостижимые пути.

Корректная формулировка:

```
в поддерживаемой PE32+ AMD64 EH/unwind model структурных и placement anomalies,
покрываемых текущей heuristic policy, не обнаружено
```

Задача 20. Проектирование нового finding

Нужно обнаруживать несоответствие между stack allocation в metadata и машинным prolog и машинный prolog. Где реализовать?

Решение

Одного изменения `heuristics.c` недостаточно. Нужен новый этап анализа:

1. Декодер инструкций для x86-64.
2. Сопоставление begin RVA функции с байтами файла.
3. Декодировать instructions в пределах `SizeOfProlog`.
4. Вычислить фактические effects на `RSP` и nonvolatile saves.
5. Сравнить с `UnwindSummary`.
6. Создать соответствие модели наблюдения.
7. Передать counters/findings в heuristic policy.
8. Добавить поля JSON и версию schema.
9. Добавить синтетические positive/negative tests.
10. Проверить варианты compiler-a.

Декодер инструкций должен быть структурированная library, а не случайные шаблоны байтов без полноценного декодера. Иначе prefixes, альтернативные encodings и варианты optimization создадут ложные результаты.

Итог контроля

Если задачи решаются с пониманием, студент умеет:

- читать little-endian fields;
- переводить RVA;
- проверять section and range invariants;
- декодировать unwind header и operations;
- вручную воспроизводить score/confidence;
- объяснять breakpoint lifecycle;
- различать registration, hit и success;
- проектировать backend status;
- формулировать ограниченные выводы;
- видеть, где новый feature требует архитектурного stage, а не одной эвристики.

Расширенная лекция 20. Термины, источники и карта защиты

Приложение A. Связный терминологический указатель

Как пользоваться указателем

Определения ниже не заменяют основные лекции. Они предназначены для повторения. Термин следует понимать вместе с указанным отношением к соседним понятиям и проекту.

Программа, файл и исполнение

Source code - текст программы на языке программирования. Для C он проходит preprocessing и compilation. В проекте source code находится в `src/`, `include/` и tests.

Preprocessor - стадия C translation, обрабатывающая `#include`, macros и conditional compilation. Благодаря `#ifdef _WIN32` один source file содержит Windows implementation и non-Windows stub.

Compiler - программа, переводящая translation units в object code и выполняющая diagnostics/optimization. Clang, GCC и MSVC поддерживаются build system в разных средах.

Object file - результат отдельной compilation unit до окончательного linking. Содержит machine code, data, symbols и relocations, но ещё не является готовым executable image.

Linker - компоновщик, объединяющий object files и libraries, разрешающий symbols и создающий PE image.

Executable image - файл, предназначенный для отображения loader в memory; в Windows обычно EXE или DLL в формате PE.

Loader - часть runtime system, отображающая image sections, применяющая relocations, разрешающая imports и подготавливающая process execution.

Process - выполняющийся экземпляр программы с virtual address space, handles, modules и threads.

Thread - единица исполнения внутри process, имеющая собственные registers, stack и instruction pointer.

Module - загруженный PE image внутри process: main EXE или DLL.

Host - операционная система и machine, на которых работает analyzer.

Target или **debuggee** - программа, которую analyzer исследует или запускает под debugger.

Данные и адреса

Bit - минимальная binary unit со значением 0 или 1.

Byte - в рассматриваемых платформах восемь bits.

Unsigned integer - целое без знака; все bit combinations выражают nonnegative value.

Signed integer - целое со знаком; на современных systems обычно two's complement.

Overflow - результат arithmetic вне диапазона типа. Для unsigned C он вычисляется modulo power of two; для signed overflow поведение undefined.

Endianness - порядок bytes многобайтового значения. PE AMD64 использует little-endian.

Little-endian - least significant byte хранится по меньшему адресу.

Pointer - typed C value, обозначающее address object/function в адресном пространстве текущего процесса.

File offset - позиция byte в файле на диске.

Virtual Address, VA - адрес в virtual address space process.

Relative Virtual Address, RVA - VA относительно module base: `RVA = VA - base`.

ImageBase - preferred base PE image из optional header.

Actual module base - фактический base загруженного module; может отличаться из-за ASLR.

ASLR - Address Space Layout Randomization, выбор изменяемых bases для усложнения предсказания addresses.

Address range - interval addresses. В проекте функции представлены полуинтервалом `[begin,end)`.

Out of bounds, OOB - address или range выходит за declared boundary, например file size или SizeOfImage.

Alignment - требование кратности address/offset определённому числу bytes.

Saturation - ограничение результата maximum/minimum вместо wraparound; используется при aggregate stack size.

CPU и ABI

Instruction Set Architecture, ISA - модель instructions, registers, address sizes и архитектурных effects. Проект ориентирован на AMD64/x86-64 target.

Machine code - bytes instructions конкретной ISA.

Assembly language - textual representation machine instructions.

Register - быстрое архитектурное состояние CPU.

Instruction pointer - register address следующей/текущей instruction; на x86-64 `RIP`.

Stack pointer - register вершины stack; на x86-64 `RSP`.

Flags register - register condition/control bits; в context используется `EFlags`.

Application Binary Interface, ABI - правила двоичного взаимодействия: argument registers, preserved registers, stack alignment, object formats и unwind conventions.

Calling convention - часть ABI, описывающая передачу arguments, return value и register preservation.

Volatile register - register, который callee может изменить без восстановления.

Nonvolatile register - register, который callee обязан сохранить и восстановить, если использует.

Shadow space - 32-byte область, резервируемая caller в Windows x64 для первых четырёх register arguments.

Stack alignment - требование к кратности `RSP` в определённых execution points.

Function - region code с agreed entry/return semantics.

Caller - function, выполняющая call.

Callee - вызываемая function.

Prolog - ограниченная начальная sequence, создающая stack frame и сохраняющая nonvolatile state.

Epilog - sequence, отменяющая frame effects и возвращающая control caller.

Stack frame - часть thread stack и register relation, принадлежащая одному active function invocation.

Frame pointer/register - nonvolatile register, используемый как стабильная reference point frame.

Leaf function - функция, для которой Windows может выполнить простую unwind без table entry при соблюдении ABI constraints.

Nonleaf function - функция, обычно требующая unwind metadata из-за calls, stack allocation или nonvolatile state.

Context - snapshot выбранного архитектурного состояния thread.

PE/COFF

PE - Portable Executable, Windows image format.

COFF - Common Object File Format, структура object/image headers и related records.

DOS header - начальный compatibility header PE с signature `MZ`.

`e_lfanew` - field по offset `0x3C`, указывающее file offset PE signature.

PE signature - bytes `PE\0\0`.

COFF file header - header с Machine, NumberOfSections и SizeOfOptionalHeader.

Optional header - обязательный для image header с ImageBase, sizes, alignments и data directories.

PE32+ - 64-bit variant optional header, magic `0x20B`.

Machine type - architecture identifier; AMD64 имеет value `0x8664`.

Section - basic unit code/data mapping в PE.

Section header - record с name, virtual/raw ranges и characteristics.

VirtualSize - значимый virtual size section.

SizeOfRawData - размер file-backed section bytes.

PointerToRawData - file offset начала raw section data.

Characteristics - bit flags назначения/permissions section.

Data directory - RVA+size системной table в optional header.

Exception directory - data directory, указывающая AMD64 runtime function table.

`.text` - распространённое имя executable code section; имя не является жёстким нормативным условием.

`.pdata` - распространённое имя section runtime function entries.

`.xdata` - распространённое имя section unwind records.

`.bss` - распространённое имя zero-initialized virtual data без обязательных raw bytes.

`.idata` - распространённое имя import data.

SizeOfImage - declared size загруженного image в RVA space.

SizeOfHeaders - file/memory extent PE headers.

SectionAlignment - alignment virtual sections.

FileAlignment - alignment raw section data.

Исключения и unwind

Exception - событие, прерывающее normal instruction flow и инициирующее Windows dispatch.

Exception code - numeric identifier класса exception.

Structured Exception Handling, SEH - Windows mechanism поиска и обработки exceptions.

Dispatch - поиск подходящего handler.

Unwind - восстановление caller states и сворачивание frames.

Virtual unwind - вычисление caller context без исполнения original epilog.

Search phase - проход frames для выбора handler.

Unwind phase - фактическое сворачивание до выбранной точки.

`RUNTIME_FUNCTION` - AMD64 record `{BeginRVA, EndRVA, UnwindRVA}`.

Function table - sorted array `RUNTIME_FUNCTION`.

`UNWIND_INFO` - variable-length record effects prolog, handler или chain.

Version - format version в low three bits header.

Flags - high five bits header, включая EHANDLER, UHANDLER и CHAININFO.

`SizeOfProlog` - длина declared prolog в bytes.

`CountOfCodes` - число two-byte unwind slots, не число logical operations.

`UNWIND_CODE` - two-byte slot с CodeOffset и opcode/OpInfo.

CodeOffset - offset в prolog, связанный с operation.

Opcode - тип unwind operation.

Operand - дополнительное encoded value operation.

`UWOP_PUSH_NONVOL` - описание push nonvolatile register.

`UWOP_ALLOC_SMALL` - compact small stack allocation.

`UWOP_ALLOC_LARGE` - stack allocation с additional operands.

`UWOP_SET_FPREG` - установка frame register.

`UWOP_SAVE_NONVOL` - сохранение GP nonvolatile register в stack slot.

`UWOP_SAVE_XMM128` - сохранение nonvolatile XMM register.

`UWOP_PUSH_MACHFRAME` - описание system machine frame.

Handler RVA - image-relative адрес frame handler.

Chained unwind info - record, ссылающаяся на primary runtime function/unwind description.

Frame-based handler - handler, связанный с stack frame через unwind metadata.

VEH - Vectored Exception Handler, process-level registration до frame search.

VCH - Vectored Continue Handler, vectored continuation mechanism.

UEF - Unhandled Exception Filter, top-level process filter.

Dynamic function table - runtime-registered table для generated code.

JIT compiler - Just-In-Time compiler, создающий machine code во время process execution.

Fragment в Unwind V3 - непрерывный диапазон machine code, описанный одной парой `RUNTIME_FUNCTION` и `UNWIND_INFO_V3`.

WOD - Winding Operation Descriptor, variable-length descriptor одной операции unwind в V3.

WOD pool - packed byte array всех WOD prolog и epilog одного V3 fragment.

PayloadWords - число 16-bit words payload V3 после четырёхбайтового header; handler или chained record в это число не входят.

Debugging

Debugger - process/tool, управляющий target через Debug API.

Debug event - Windows notification о create/exit, DLL load или exception.

`WaitForDebugEvent` - ожидание следующего event.

`ContinueDebugEvent` - продолжение target с указанным status.

First chance - первое уведомление debugger до окончательной application handling.

Second chance - уведомление после неуспешного normal handling.

`DBG_CONTINUE` - debugger объявляет event обработанным.

`DBG_EXCEPTION_NOT_HANDLED` - debugger передаёт ordinary exception дальнейшей системе dispatch.

Breakpoint - контролируемая остановка execution по адресу/условию.

Software breakpoint - замена instruction byte на trap instruction.

`INT3` - однобайтовая x86 breakpoint instruction `0xCC`.

Original byte - byte target code, сохранённый до вставки `INT3`.

Re-arm - повторная установка breakpoint после исполнения original instruction.

Trap Flag, TF - bit `EFlags`, вызывающий single-step exception после одной instruction.

Single-step - режим пошагового исполнения.

Pending re-arm - association thread с breakpoint, который нужно восстановить на следующем single-step.

Remote process memory - память target, читаемая/изменяемая analyzer через Debug API.

Instruction cache - CPU cache fetched instructions; после code modification требуется flush.

Timeout - верхняя граница времени dynamic session, но не security sandbox.

Trace - в проекте bounded sequence RIP после handler hit, не полный execution log.

Анализ и доказательства

Parser - component, преобразующий bytes в structured fields с bounds checks.

Decoder - component, интерпретирующий encoded domain structure, например unwind operations.

Invariant - условие, которое должно сохраняться для корректной model.

Наблюдение (observation) - непосредственно полученный факт конкретного file/run, то есть файл или конкретный запуск, то есть файл или конкретный запуск.

Finding - named analytical record, созданный rule из observations.

Severity - qualitative importance finding: medium/high и т. п.

Evidence family - группа related findings одного analytical слоя.

Score - bounded engineering sum weights 0-100.

Suspicion - в проекте `score / 100`, не probability.

Confidence - rule-based оценка силы/разнообразия evidence, не эмпирически калиброванная probability.

Verdict - categorical label по thresholds.

Weakening rule - правило, снижающее confidence при benign alternatives или недостатке context.

Mitigation - internal C name weakening rule; не исправляет input.

Correlation - relation static и dynamic evidence. Aggregate correlation слабее exact address-level identity.

False positive - benign object, ошибочно classified suspicious.

False negative - suspicious behavior, не обнаруженное analyzer.

Ground truth - эталонная разметка, относительно которой проверяется правильность результата.

True positive, TP - размеченное нарушение, которое инструмент обнаружил.

True negative, TN - корректный объект, который инструмент не отметил положительным сигналом.

Sensitivity / Recall - доля обнаруженных положительных примеров: `TP / (TP + FN)`.

Specificity - доля правильно отклонённых отрицательных примеров: `TN / (TN + FP)`.

Precision - доля подтверждённых сигналов среди всех положительных сигналов: `TP / (TP + FP)`.

F1 - гармоническое среднее precision и recall.

Confidence interval - диапазон статистически правдоподобных значений параметра при заданной модели выборки; не устраняет смещение corpus.

Calibration - подбор/проверка thresholds на размеченном corpus.

Corpus - набор samples для testing/measurement.

Regression test - test, сохраняющий ранее установленное expected behavior.

Unit test - проверка отдельного module/contract.

Integration test - проверка взаимодействия components и external outputs.

Fuzzing - генерация множества inputs для поиска crashes и unsafe paths.

Mutation fuzzing - изменение seed inputs.

Coverage-guided fuzzing - генерация inputs с использованием code coverage feedback.

ASan - AddressSanitizer.

UBSan - UndefinedBehaviorSanitizer.

Reproducibility - возможность повторить analysis с тем же input, versions и commands.

Отчёты и интеграция

Serialization - преобразование in-memory model во внешний format.

CSV - flat tabular text format.

JSON - structured text format objects/arrays/scalars.

HTML - document markup; report является self-contained dashboard.

Schema - formal description allowed JSON structure and types.

Schema version - version public data contract.

Provenance - сведения о происхождении result: input hash, tool/build, time, command.

SHA-256 - 256-bit cryptographic hash, используемый для identity bytes.

Atomic write - запись temporary file и rename, предотвращающие наблюдение partial result.

UTF-8 - variable-length Unicode encoding, валидируемая report writer.

Escaping - преобразование special characters для безопасного JSON/HTML representation.

Content Security Policy, CSP - browser policy, ограничивающая sources content в HTML report.

Backend - server-side application, управляющее upload, jobs, core execution и persistence.

Frontend - user interface, взаимодействующее с backend API.

API - формальный интерфейс между components. CLI+JSON также является API, хотя не network API.

Subprocess - отдельный OS process, запускаемый backend для analysis.

Job queue - механизм асинхронного планирования analyses.

Database - persistent structured storage jobs/reports.

Raw JSON - неизменённый full report; рекомендуется сохранять как source of truth.

Normalization - выделение indexed fields JSON в relational/search structures.

Sandbox - изолированная execution environment. Debugger и timeout сами по себе sandbox не образуют.

Release candidate, RC - предварительный release, близкий к stable, но ещё допускающий blocking changes.

Приложение B. Нормативные и технические источники

Microsoft

1. PE Format - PE/COFF headers, sections, RVA, directories и `.pdata`.
2. x64 exception handling - `RUNTIME_FUNCTION`, `UNWIND_INFO`, operations и unwind procedure.
3. x64 Unwind Information V3 - preview layout V3, explicit epilogs, WOD pool и Intel APX registers.
4. x64 calling convention - registers, arguments, alignment и unwindability.
5. x64 prolog and epilog - ограничения prolog/epilog.
6. Structured Exception Handling - общая модель SEH.
7. Debugging Events - lifecycle debug events.
8. Writing the Debugger's Main Loop - `WaitForDebugEvent`/`ContinueDebugEvent`.
9. AddVectoredExceptionHandler.
10. SetUnhandledExceptionFilter.
11. RtlAddFunctionTable.
12. RtlAddGrowableFunctionTable.
13. WaitForDebugEventEx.

Стандарты и документация tools

1. ISO/IEC 9899:2018 - язык C17. Текст стандарта может распространяться на условиях ISO; для практического reference используется документация compiler и committee drafts.
2. AMD64 Architecture Programmer's Manual - architecture registers, instructions и exception model.
3. Intel 64 and IA-32 Architectures Software Developer's Manual - x86-64 instruction and system semantics.
4. CMake Documentation - targets, presets, CTest и CPack.
5. Ghidra Documentation - import, disassembly, decompiler и scripting.
6. JSON Schema - формальная валидация JSON documents.
7. NIST Static Analysis Tool Study Methodology - различие accuracy, precision и recall при оценке инструменты analysis.
8. NIST/SEMATECH e-Handbook of Statistical Methods - основы статистической оценки и доверительные интервалы.

Приложение C. Карта ответов на защите

Что делает проект

Проект анализирует Windows PE32+ AMD64 files. Он безопасно читает PE layout, извлекает exception directory, декодирует `RUNTIME_FUNCTION` и `UNWIND_INFO`, выявляет structural/placement/reuse anomalies, а на Windows может запустить target под Debug API и наблюдать exception-related registrations, handler hits и dynamic function tables. Результаты выводятся в terminal, CSV, JSON и HTML.

Почему это нужно

EH/unwind metadata требуется Windows для exception dispatch, stack unwinding и debugging. Она также содержит function boundaries, stack effects и handler relations, полезные reverse engineer даже без symbols. Повреждённая или необычно организованная metadata может указывать на corruption, custom runtime, JIT, obfuscation или exception-driven control flow.

В чём научно-техническая задача

Задача состоит не в чтении одного поля PE, а в построении безопасного and explainable pipeline:

```
untrusted bytes
-> validated structural model
-> decoded ABI metadata
-> evidence aggregation
-> optional runtime observations
-> reproducible reports
```

Почему архитектура не microservices

Core - modular monolith: static C library плюс CLI. Для одного analytical tool это уменьшает deployment complexity и сохраняет чёткие module boundaries. Backend, database, queue и Windows dynamic workers могут быть отдельными services вокруг stable JSON/CLI contract.

Почему static работает на macOS/Linux

Static parser не исполняет input и не вызывает Windows loader. Он интерпретирует bytes по documented PE format. Dynamic mode требует actual Windows Debug API и поэтому Windows-only.

Почему score не probability

Weights и thresholds заданы engineering rules. Нет обученной model и empirical probability calibration. Score ранжирует anomaly evidence; confidence оценивает разнообразие и strength rules.

Что означает successful analysis

Input прочитан; минимальная PE32+ AMD64 model извлечена; requested stages завершились; outputs записаны. Это не означает, что file benign или что все возможные behaviors исследованы.

Что является главным ограничением

Static stage не сравнивает полностью metadata с машинный prolog и не анализирует весь binary semantics. Dynamic stage покрывает только достигнутый path и должен запускаться в isolated Windows environment.

Можно ли подключить frontend и database без изменения core

Да. Backend запускает CLI как subprocess, валидирует JSON schema и сохраняет raw report. Core changes нужны только для новых analytical fields или stable native API, но не для обычной web integration.

Почему проект готов к ВКР

Есть работающий core, объяснимая архитектура, documented standards, static and dynamic modes, structured reports, Ghidra integration, strict build, 15 automated CTest cases, fuzzing, synthetic and real-corpus validation и явно сформулированные limitations.

Что развивать после ВКР

Приоритеты:

1. Windows sandbox worker.
2. Remote decoding dynamic function tables.
3. Code-to-unwind correspondence.
4. Broader labeled corpus and calibration.
5. Backend/database/frontend.
6. Versioned heuristic policy.
7. Richer typed event schema.

Расширенная лекция 21. Evidence, coverage и проверяемость

Том III. Техническая теория обнаружения аномалий и границ доказательства

Зачем нужна отдельная теория над parser и эвристиками

Предыдущие главы объяснили устройство PE32+ AMD64, таблицы `RUNTIME_FUNCTION`, записей `UNWIND_INFO`, Windows Debug API и конкретных модулей проекта. Однако знания отдельных структур недостаточно для ответа на главный технический вопрос:

```
каким образом наблюдаемое свойство бинарного файла
превращается в обоснованный вывод об аномалии
```

Между чтением байтов и выводом существует несколько логических переходов. Если их смешать, возникает типичная ошибка: любое отличие от ожидаемого расположения объявляется вредоносным вмешательством. Проект устроен консервативнее. Он разделяет:

1. непосредственно прочитанный факт;
2. проверяемый инвариант формата или ABI;
3. зарегистрированное нарушение;
4. эвристический finding;
5. совокупную оценку;
6. решение аналитика после ручной проверки.

Эта последовательность является технической основой объяснимости проекта.

Наблюдение

Наблюдение, или observation, - значение, непосредственно полученное из входного файла или динамического события.

Примеры статических наблюдений:

- `SizeOfImage = 0x9000`;
- section имеет `VirtualAddress = 0x4000`;

- `RUNTIME_FUNCTION.BeginAddress = 0x1050`;
- `RUNTIME_FUNCTION.EndAddress = 0x1030`;
- `UnwindInfoRVA = 0x7000`;
- section characteristics содержат `IMAGE_SCN_MEM_WRITE`;
- один `HandlerRVA` встречается в 120 записях.

Примеры динамических наблюдений:

- target process успешно создан;
- получено `EXCEPTION_DEBUG_EVENT`;
- выполнена регистрация VEH;
- вызван `RtlAddFunctionTable`;
- breakpoint по статически найденному handler сработал;
- процесс завершился сам или был остановлен по timeout.

Наблюдение ещё не является оценкой. Значение `UnwindInfoRVA = 0x7000` само по себе ни корректно, ни ошибочно. Для интерпретации необходимо сопоставить его с `SizeOfImage`, section table и доступным диапазоном файла.

Инвариант

Инвариант, или invariant, - условие, которое должно выполняться для корректной модели на определённом уровне.

Инварианты проекта делятся на четыре класса.

Инварианты безопасного чтения

Они не описывают Windows ABI. Они защищают процесс анализатора.

Пример:

```
offset <= file_size
size <= file_size - offset
```

Эта форма проверки предпочтительнее выражения:

```
offset + size <= file_size
```

потому что сложение может переполниться до сравнения. Нарушение означает, что запрошенный диапазон нельзя безопасно прочитать. Оно не доказывает намеренное вмешательство: файл может быть повреждён случайно.

Инварианты PE layout

Они связывают section table, файл и виртуальный образ.

Примеры:

```
PointerToRawData + SizeOfRawData <= file_size
VirtualAddress + virtual_span <= SizeOfImage
exception directory полностью лежит внутри image
```

Такие условия определяют, можно ли построить однозначное отображение между RVA и file offset.

Инварианты `RUNTIME_FUNCTION`

Для half-open диапазона функции:

```
[BeginAddress, EndAddress)
```

базовое условие:

```
BeginAddress < EndAddress
```

Оба конца должны принадлежать допустимому image space, а таблица по нормативному контракту должна быть отсортирована по начальному адресу.

Инварианты `UNWIND_INFO`

Они зависят от версии записи. Для V1/V2 проверяются:

- допустимая версия;
- длина header и массива slots;
- существование дополнительных operands;
- допустимость opcode и `OpInfo`;
- согласованность frame register;
- порядок `CodeOffset`;
- положение handler или chained record;
- отсутствие loop и превышения chain depth.

Инвариант должен быть сформулирован для конкретной версии формата. Именно поэтому V3 нельзя безопасно проверять правилами V1/V2.

Нарушение и аномалия

Нарушение, или violation, - невыполнение конкретного инварианта.

Аномалия, или anomaly, - наблюдаемое отклонение от нормативной, компиляторной или статистически ожидаемой структуры, которое требует объяснения.

Эти понятия пересекаются, но не тождественны.

Жёсткое нарушение:

```
EndAddress <= BeginAddress
```

Невозможно согласовать с нормальным half-open диапазоном функции.

Контекстная аномалия:

```
UNWIND_INFO находится в writable section
```

Само наличие права записи не делает адрес недопустимым. Но обычная сформированная linker metadata обычно не нуждается в записи во время исполнения, поэтому свойство является значимым сигналом.

Статистическая аномалия:

```
более 30% RUNTIME_FUNCTION используют один UnwindInfoRVA
при числе записей больше 200
```

Такой reuse может быть законным, поэтому требует контекста.

Finding как публичная единица результата

Внутренние counters и anomaly bits слишком низкоуровневые для внешнего потребителя. Проект преобразует их в **finding** - именованную, машиночитаемую единицу свидетельства.

Finding содержит:

```
code
family
severity
message
affected_count
representative_rva
section
```

`code` является стабильным идентификатором правила. Человек может читать `message`, а backend - группировать данные по `code`.

`affected_count` показывает масштаб нарушения. `representative_rva` предоставляет точку входа для Ghidra или debugger, но не заменяет полный список затронутых объектов.

Почему finding не равен verdict

Один finding отвечает на узкий вопрос:

```
обнаружено ли конкретное свойство
```

Verdict отвечает на другой вопрос:

```
насколько сильна совокупность статических свидетельств
```

Например, `unwind_unusual_section` может быть единственным сигналом нестандартного linker layout. Проект регистрирует finding, но weakening rules снижают confidence. Если одновременно имеются parse failure и handler в non-executable section, совокупность сильнее любого одиночного наблюдения.

Evidence family

Evidence family - семейство признаков, относящихся к одному техническому слою.

Проект использует пять семейств:

1. `table_integrity`;
2. `unwind_integrity`;
3. `metadata_placement`;
4. `unwind_reuse`;
5. `handler_dominance`.

Количество семейств важно для confidence. Несколько findings из одного семейства могут иметь общую первопричину. Сигналы из независимых слоёв труднее объяснить одной случайной особенностью.

Например:

```
unwind_rva_oob
+ unwind_parse_fail
```

могут быть двумя проявлениями одного повреждённого указателя.

Но сочетание:

```
rf_overlaps
+ writable UNWIND_INFO
+ runtime handler hit
```

охватывает table layout, permissions и фактическое исполнение.

Severity

Severity описывает техническую серьёзность finding, а не вероятность вредоносности.

`high` применяется, когда свойство:

- разрушает однозначность структуры;
- выводит адрес за image;
- делает handler неисполняемым;
- затрагивает широкий процент записей;
- образует сильную комбинацию.

`medium` применяется, когда:

- нарушение может иметь benign explanation;
- оно локально;
- требуется дополнительный context;
- это статистический pattern, а не жёсткое повреждение.

Severity не вычисляется из имени malware family и не является оценкой ущерба от запуска файла.

Score

Score является ограниченной суммой инженерных весов:

```
score = clamp(sum(weights), 0, 100)
```

Он нужен для triage и сортировки. Важные свойства:

- score детерминирован;
- каждый вклад можно объяснить;
- score насыщается на 100;
- одинаковый score может быть получен разными наборами findings;
- score не является вероятностью.

Насыщение означает, что значения выше 100 теряют различия. Поэтому полный список findings всегда информативнее одного числа.

Suspicion

Поле:

```
suspicion = score / 100
```

является нормализованным представлением score для JSON и интерфейсов.

Если `suspicion = 0.73`, это означает:

```
score = 73 из 100
```

Это не означает 73-процентную вероятность атаки или модификации.

Confidence

Confidence в проекте - инженерная оценка того, насколько разнообразна и устойчива интерпретация статического verdict.

Она увеличивается при:

- наличии нескольких evidence families;
- high findings;
- нескольких medium findings;
- жёстких нарушениях table/unwind integrity.

Она уменьшается при:

- единственном нестрогом сигнале;
- малом числе функций;
- только необычном placement;
- единственном evidence family.

Confidence ограничивается диапазоном:

```
0.05 .. 0.90
```

Верхняя граница 0.90 предотвращает представление эвристической оценки как абсолютной уверенности.

Weakening rule

Weakening rule, или mitigation в именах C-структур, - правило, фиксирующее benign alternative или недостаток контекста.

В текущей реализации четыре правила:

- `single_signal_only`;
- `small_sample_size`;
- `placement_only_signal`;
- `single_family_only`.

Weakening rule не удаляет наблюдение и не уменьшает score. Оно уменьшает confidence. Это принципиально: факт сохраняется, но интерпретация становится осторожнее.

Manual review

`requires_manual_review` означает, что автоматический результат требует проверки аналитиком.

Ручная проверка включает:

1. переход к `representative_rva`;
2. проверку section characteristics;
3. сопоставление `RUNTIME_FUNCTION` с дизассемблированным диапазоном;
4. проверку prolog против unwind operations;
5. анализ handler target;
6. проверку динамического достижения;
7. поиск benign explanation: compiler runtime, JIT, protector, custom linker.

Флаг manual review не означает ошибку анализатора. Он является частью правильного рабочего процесса.

Модель классов вмешательств

Чтобы обсуждать покрытие, сначала нужно определить пространство возможных изменений.

Класс A. Повреждение файлового layout

Изменяются section headers, raw ranges, virtual ranges, alignments или exception directory.

Проект проверяет:

- выход raw data за файл;
- выход virtual span за image;
- overlaps;
- alignment;
- доступность exception directory.

Класс B. Повреждение function table

Изменяются `BeginAddress`, `EndAddress`, `UnwindInfoRVA` или порядок entries.

Проект проверяет:

- пустые/обратные диапазоны;
- выход за image;
- недоступный unwind RVA;
- отсутствие сортировки;
- overlaps;
- duplicate begins.

Класс C. Повреждение `UNWIND_INFO`

Изменяются header, slots, operands, flags, tail или chain.

Проект проверяет parser failure и двенадцать внутренних anomaly reasons.

Класс D. Перемещение metadata

Корректные по синтаксису записи помещаются в sections с необычными permissions или именами.

Проект проверяет executable, writable и unusual placement.

Класс E. Перенаправление handler

Handler RVA изменяется на non-executable, unknown или writable location.

Проект проверяет свойства целевой section и dominance pattern.

Класс F. Централизация

Большое число функций ссылается на один unwind record или handler.

Проект проверяет reuse и dominance только при достаточном размере таблицы и контексте.

Класс G. Runtime-only изменение

Metadata создаётся после загрузки через dynamic function table или регистрируются process-level handlers.

Статический анализ этого не видит. Windows collector наблюдает выбранные API регистрации и handler hits во время фактически достигнутого execution path.

Класс H. Семантически согласованная подмена

Атакующий изменяет машинный prolog и `UNWIND_INFO` согласованно, сохраняет валидные ranges, permissions и статистику.

Текущие структурные эвристики могут не обнаружить такую подмену. Для неё нужны:

- disassembly;
- восстановление фактических stack effects;
- code-to-unwind correspondence;
- сравнение с доверенным baseline;
- подпись или provenance исходного артефакта.

Эта граница должна называться прямо.

Покрытие не является полнотой

Coverage, или покрытие, отвечает на вопрос:

```
какие заранее определённые классы нарушений способен заметить инструмент
```

Completeness, или полнота, означала бы:

```
инструмент обнаруживает любое возможное вмешательство
```

Для структурного анализатора второе утверждение недостижимо без ограничения модели противника. Валидная структура может содержать нежелательную семантику. Невыполненный runtime path может скрывать динамическое действие. Неизвестная будущая версия формата может быть корректной, но неподдержанной.

Правильное утверждение проекта:

```
инструмент системно покрывает определённые структурные, placement,
reuse и наблюдаемые runtime-классы аномалий
```

Почему число признаков не доказывает покрытие

Количество rules не является прямой мерой качества.

Сто поверхностных проверок section names могут покрывать меньше, чем одна строгая bounds check. Поэтому 27 findings оцениваются через:

- источник инварианта;
- независимость от других findings;
- класс вмешательства;
- возможность локализовать нарушение;
- тестируемость;
- benign alternatives;
- стоимость false positive.

Число 27 является результатом текущей декомпозиции публичных результатов, а не нормативной константой PE/COFF.

Двенадцать внутренних anomaly reasons

Finding `unwind_anomalies` агрегирует двенадцать низкоуровневых причин:

1. `UA_VERSION_UNKNOWN`;
2. `UA_TRUNCATED`;
3. `UA_BAD_OPCODE`;
4. `UA_BAD_OPERAND`;
5. `UA_CODEOFFSET_OUTSIDE`;
6. `UA_CHAIN_LOOP`;
7. `UA_CHAIN_DEPTH_EXCEEDED`;
8. `UA_HANDLER_OOB`;
9. `UA_CODEOFFSET_ORDER`;
10. `UA_OPERAND_SANITY`;
11. `UA_ALIGNMENT`;
12. `UA_CHAIN_INVALID`.

Это не дополнительные публичные findings. Они являются диагностическими подтипами одного слоя `UNWIND_INFO`. Такое разделение предотвращает искусственное раздувание score, когда одна повреждённая запись нарушает сразу несколько зависимых условий.

Traceability

Traceability, или прослеживаемость, означает возможность пройти цепочку:

```
источник правила
-> условие в коде
-> unit test
-> finding в JSON
-> ручная проверка по RVA
```

Для проекта эта цепочка выглядит так:

```
Microsoft PE/x64 contract
-> pe.c / unwind.c / heuristics.c
-> unit_core_edges / unit_unwind / unit_heuristics*
-> report.c
-> JSON findings и manual_validation_steps
```

Если правило нельзя связать с инвариантом, кодом и проверкой, его нельзя считать достаточно обоснованным.

False positive и false negative

False positive - корректный или допустимый объект получает положительный сигнал.

Причины:

- custom linker;
- protector;
- JIT/runtime metadata;
- редкий compiler layout;
- неподдержанная новая версия формата;

- слишком резкий threshold;
- неправильная benign policy.

False negative - интересующее нарушение присутствует, но сигнал не возникает.

Причины:

- изменение не затрагивает проверяемый инвариант;
- структура и код изменены согласованно;
- runtime path не достигнут;
- handler зарегистрирован через ненаблюдаемый механизм;
- threshold подавил слабый pattern;
- формат не распознан.

Оба типа ошибок являются свойствами пары:

```
инструмент + определённая задача + определённый corpus
```

Нельзя говорить о false positive или false negative без ground truth.

Технически корректная формулировка результата

После анализа следует говорить:

```
обнаружено нарушение X в N объектах;
оно относится к семейству Y;
первый пример расположен по RVA Z;
автоматическая оценка равна S;
существуют такие benign alternatives;
для подтверждения нужны такие ручные действия
```

Не следует заменять эту формулировку утверждением:

```
файл вредоносный
```

Проект не решает задачу универсальной классификации вредоносности.

Контрольные вопросы

1. Чем observation отличается от finding?
2. Почему жёсткий structural violation не обязательно означает атаку?
3. Почему placement является контекстной аномалией?
4. Почему score и confidence изменяются независимо?
5. Почему weakening rule не должен удалять finding?
6. Что требуется для заявления о coverage?
7. Какой класс согласованной подмены остаётся вне текущей модели?
8. Почему runtime collector не гарантирует наблюдение всех paths?
9. Зачем внутренние `UA_*` причины агрегируются?
10. Как построить traceability chain для нового правила?

Расширенная лекция 22. Подробная логика findings

Том III. Полная система 27 признаков: основания, связи и ручная проверка

Что именно означает число 27

В текущей версии `heuristics.c` существует 27 возможных публичных finding codes. Это не 27 полей PE и не 27 типов атак. Это 27 именованных результатов, которые система может включить в массив:

```
"findings": []
```

Один finding может агрегировать множество однотипных нарушений. Например, `rf_oob` создаётся один раз, даже если за границы image выходят десять `RUNTIME_FUNCTION`.

Число 27 получено декомпозицией технических слоёв:

```
6  findings PE layout
7  findings RUNTIME_FUNCTION
2  findings UNWIND_INFO integrity
3  findings reuse и handler statistics
5  findings placement и permissions
4  combined findings
---
27 публичных finding codes
```

Внутри `unwind_anomalies` дополнительно различаются 12 причин `UA_*`. Поэтому глубина проверки больше, чем простое число публичных кодов, но выдавать их за 39 независимых признаков было бы неправильно.

Критерии включения признака

Finding включён в текущую систему, если выполняются следующие условия.

1. Он проверяет отдельный технический инвариант или отдельный класс

контекстной аномалии.

2. Его можно вычислить детерминированно.
3. Его можно локализовать или количественно измерить.
4. Для него существует понятный manual validation workflow.
5. Его можно закрепить unit или regression test.
6. Он не дублирует полностью другой finding.
7. Для него можно назвать границы и benign alternatives.

Эти критерии важнее самого количества.

Группа I. Шесть признаков PE layout

1. `pe_section_raw_oob`

Проверяемый объект. Поля `PointerToRawData` и `SizeOfRawData` section header.

Инвариант.

```
PointerToRawData <= file_size
SizeOfRawData <= file_size - PointerToRawData
```

Что означает нарушение. Section объявляет байты, которых нет во входном файле. Любая попытка читать полный raw range без bounds check вышла бы за buffer.

Почему признак важен для EH/unwind. `.pdata` или `.xdata` может находиться в такой section. Тогда RVA формально принадлежит image, но фактические байты metadata отсутствуют.

Вес и severity.

```
weight 25
severity high
family table_integrity
```

Высокий вес связан с прямым нарушением физической доступности данных.

Benign alternatives. Обрезанная передача, повреждение архива, неверный dump memory image, нестандартный инструмент создания файла. Они объясняют происхождение, но не делают диапазон безопасным.

Ручная проверка. Сравнить сумму полей с фактическим размером файла, проверить overlay и определить, является ли объект обычным disk PE или неполным memory dump.

Что не обнаруживается. Raw range может быть доступным, но содержать семантически неверные данные.

2. `pe_section_virtual_oob`

Проверяемый объект. `VirtualAddress`, virtual span section и `SizeOfImage`.

Инвариант.

```
VirtualAddress + virtual_span <= SizeOfImage
```

Проект вычисляет virtual span с учётом `VirtualSize` и `SizeOfRawData`, поскольку реальный mapping policy должен учитывать как объявленный размер в памяти, так и доступный raw content.

Что означает нарушение. Section выходит за объявленную границу image. RVA внутри её хвоста нельзя уверенно считать принадлежащим загружаемому образу.

Связь с EH/unwind. `BeginAddress`, `EndAddress`, `UnwindInfoRVA` и `HandlerRVA` являются image-relative. Ошибка `SizeOfImage` или section span разрушает основу их проверки.

Вес и severity.

```
weight 20
severity high
```

Вес немного ниже raw OOB, потому что фактическое loader behavior может зависеть от дополнительных правил выравнивания, но модель image уже несогласованна.

Ручная проверка. Сопоставить `SizeOfImage`, `SectionAlignment`, последний section end и результат загрузки Windows loader в изолированной среде.

Граница. Признак не доказывает, что Windows обязательно загрузит файл или откажет: analyzer проверяет собственный строгий parser contract.

3. `pe_section_raw_overlap`

Проверяемый объект. Пары raw ranges sections.

Для sections A и B:

```
A = [A.raw_begin, A.raw_end)
B = [B.raw_begin, B.raw_end)
```

Overlap существует, если:

```
A.raw_begin < B.raw_end
and
B.raw_begin < A.raw_end
```

Что означает нарушение. Одни байты файла одновременно объявлены частью двух sections.

Почему это важно. Разные parser, loader и inspection tools могут интерпретировать такие bytes по-разному. Атакующий может использовать неоднозначность для parser differential.

Вес и severity.

```
weight 15
severity medium
```

Overlap значим, но некоторые упаковщики и специально сформированные images могут использовать нестандартные layouts.

Ручная проверка. Выписать обе пары raw boundaries, определить реальную область пересечения и сравнить отображение в нескольких независимых tools.

Граница. Пересечение не доказывает, что именно EH metadata была модифицирована.

4. `pe_section_virtual_overlap`

Проверяемый объект. Virtual ranges sections.

Что означает нарушение. Один RVA попадает более чем в одну section по объявленным spans.

Почему severity выше raw overlap. EH/unwind адреса выражены в RVA. Если RVA имеет две возможные section identities, неоднозначными становятся permissions, имя section и mapping в file offset.

Вес и severity.

```
weight 20
severity high
```

Возможное вмешательство. Handler может быть представлен одним parser как исполняемый, а другим как неисполняемый; metadata может скрываться в перекрывающихся mappings.

Ручная проверка. Построить отсортированный список virtual intervals, определить все sections для спорного RVA и проверить фактическое memory map после загрузки.

Граница. Проект сообщает структурную неоднозначность, но не моделирует все частные loader decisions.

5. `pe_section_misaligned`

Проверяемый объект. Соответствие section fields объявленным `FileAlignment` и `SectionAlignment`.

Типовые условия:

```
PointerToRawData mod FileAlignment = 0
VirtualAddress mod SectionAlignment = 0
```

с учётом допустимых специальных случаев формата.

Что означает нарушение. Layout не соответствует собственной alignment policy optional header.

Почему признак medium. Alignment errors могут появляться в custom, handcrafted или very-small-alignment images. Не каждое отклонение приводит к неправильному чтению EH metadata.

Вес и severity.

```
weight 8
severity medium
```

Ручная проверка. Проверить значения alignments, кратность offsets, специальные ограничения PE и фактическую загрузку.

Граница. Это общий PE signal. Он не является специфичным для `.pdata`.

6. `pe_exception_directory_oob`

Проверяемый объект. Data directory с индексом exception table.

Инвариант. Полный диапазон:

```
[ExceptionRVA, ExceptionRVA + ExceptionSize)
```

должен принадлежать image и быть отображаемым в доступные file bytes для статического разбора.

Что означает нарушение. Analyzer не может безопасно получить полный массив `RUNTIME_FUNCTION`.

Почему это прямой EH/unwind признак. Это корневая directory, из которой начинается весь статический pipeline.

Вес и severity.

```
weight 25
severity high
```

Ручная проверка. Проверить directory entry, section mapping, кратность размера 12 байтам и возможное усечение файла.

Граница. Пустая directory и directory OOB различаются. Пустота может быть допустима для leaf-only image; OOB означает недоступный объявленный диапазон.

Группа II. Семь признаков `RUNTIME_FUNCTION`

7. `no_runtime_functions`

Условие.

```
runtime_function_count == 0
and image has executable content
```

Если executable sections отсутствуют, finding не создаётся.

Почему отсутствие не всегда ошибка. Leaf function, не изменяющая nonvolatile state и не выделяющая stack frame, может раскручиваться без function table entry. Теоретически небольшой image может состоять только из таких функций.

Почему сигнал всё же нужен. Для крупного x64 image с вызовами, stack allocations и nonleaf functions полное отсутствие `.pdata` требует объяснения.

Вес и severity.

```
weight 10
severity medium
```

Ручная проверка. Дизассемблировать executable sections, найти `call`, изменения `RSP`, сохранения nonvolatile registers и определить, действительно ли весь код leaf-only.

Граница. Проект пока не выполняет эту disassembly-проверку автоматически.

8. `invalid_ranges`

Инвариант.

```
EndAddress > BeginAddress
```

Что означает нарушение. Диапазон пустой или направлен назад. Он не может однозначно описывать непустой фрагмент кода.

Threshold.

```
<= 1% affected: weight 10, medium
>  1% affected: weight 20, high
```

Почему используется доля. Одна повреждённая entry в таблице из десятков тысяч отличается по масштабу от массового нарушения. При этом даже единичная entry сохраняется как finding.

Ручная проверка. Найти entry по representative RVA, проверить endian decode, соседние records и исходные 12 bytes.

Возможное вмешательство. Изменение `EndAddress` может скрывать часть function coverage или ломать поиск текущего frame.

Граница. Корректный диапазон ещё не означает, что он совпадает с фактической функцией.

9. `rf_oob`

Инвариант.

```
BeginAddress < SizeOfImage
EndAddress <= SizeOfImage
```

Дополнительно диапазон должен соответствовать доступной executable области при ручной интерпретации.

Что означает нарушение. Function entry описывает код вне объявленного image.

Threshold и веса.

```
<= 1%: +10, medium
>  1%: +25, high
```

Почему высокий вариант имеет вес 25. Массовые out-of-image ranges разрушают function table как индекс текущего instruction pointer.

Ручная проверка. Проверить `SizeOfImage`, section ends, ASLR-independent RVA и отсутствие ошибочного смешения RVA с VA.

Граница. Внутри image range может всё равно указывать на данные, а не на код; это отдельный возможный будущий признак.

10. `unwind_rva_oob`

Проверяемый объект. Третье поле `RUNTIME_FUNCTION`.

Инвариант. `UnwindInfoRVA` должен указывать на доступную и достаточно длинную запись внутри image/file mapping.

Что означает нарушение. Function range существует, но описание обратного восстановления контекста отсутствует по указанному адресу.

Threshold и веса.

```
<= 1%: +10, medium
>  1%: +25, high
```

Почему это один из основных признаков вмешательства. Изменение одного 32-bit RVA способно перенаправить decoder на чужие данные, вызвать неправильный unwind или скрыть handler relation.

Ручная проверка. Выполнить `pe_rva_to_fileoff`, проверить section, alignment и первые четыре header bytes.

Граница. Доступный RVA может вести к синтаксически корректной, но чужой записи. Такой случай частично покрывается reuse/statistics, но требует code-to-metadata correspondence.

11. `rf_unsorted`

Нормативное ожидание. Function table entries должны быть отсортированы по `BeginAddress`.

Как считается. Во время исходного прохода проект фиксирует inversions: следующий `BeginAddress` меньше предыдущего.

Почему порядок важен. Runtime lookup использует упорядоченную таблицу. Нарушение может изменить результат поиска или сделать его implementation-dependent.

Вес и severity.

```
weight 15
severity medium
```

Почему не high автоматически. Analyzer может скопировать entries, отсортировать их и продолжить проверку overlaps. Наличие inversion не всегда означает повреждение самих ranges.

Ручная проверка. Сравнить исходный порядок `.pdata` с сортировкой, проверить, является ли directory цельной таблицей или смешением данных.

Граница. Сортировка для анализа не исправляет input и не гарантирует runtime acceptance.

12. `rf_overlaps`

Проверяемый объект. Отсортированные function intervals.

Для предыдущего максимального конца `max_end`:

```
current.BeginAddress < max_end
```

означает overlap.

Что означает нарушение. Один instruction RVA покрывается несколькими `RUNTIME_FUNCTION`.

Почему это сильный сигнал. Function table должна давать однозначную запись для virtual unwind. Overlap нарушает однозначность выбора metadata.

Вес и severity.

```
weight 25
severity high
```

Ручная проверка. Определить обе entries, проверить chained fragments и убедиться, что overlap не является ошибкой собственного определения interval.

Граница. Раздельные noncontiguous fragments допустимы через chained info, но они не должны произвольно перекрывать один instruction range.

13. duplicate_begin

Условие.

```
entry[i].BeginAddress == entry[i-1].BeginAddress
```

после сортировки.

Что означает нарушение. Две entries претендуют на один начальный адрес.

Отличие от overlap. Duplicate begin является более конкретной формой неоднозначности. Он может сопровождаться overlap, но сохраняется отдельно для диагностики причины.

Вес и severity.

```
weight 10
severity medium
```

Почему вес не суммируется как два независимых доказательства. Findings могут иметь общую первопричину. Поэтому analyst должен смотреть на family и raw entries, а не считать каждый code независимым экспериментом.

Ручная проверка. Сравнить `EndAddress` и `UnwindInfoRVA` обеих entries.

Граница. Finding не определяет, какая из дубликатных entries является правильной.

Группа III. Два признака UNWIND_INFO integrity

14. unwind_parse_fail

Источник. `unwind_parse` вернул `false` для одной или нескольких entries.

Parse failure означает, что decoder не смог безопасно завершить структурное чтение. Причинами могут быть:

- неизвестная version;
- усечённый header;
- недостающий operand;
- невозможный tail;
- chain read failure;
- выход mapping за доступный buffer.

Threshold и веса.

```
<= 1%: +10, medium
>  1%: +25, high
```

Почему failure отделён от anomaly. Некоторые anomalies можно отметить и продолжить parsing. Failure означает, что итоговая `UnwindSummary` не может считаться полностью полученной.

Ручная проверка. Использовать `parse_error`, исходный `UnwindInfoRVA`, hex dump и официальное описание конкретной version.

Критическая оговорка. Корректная новая version, которую parser ещё не поддерживает, также создаст parse failure. Это compatibility gap, а не повреждение файла.

15. unwind_anomalies

Источник. `UnwindSummary.anomalies != 0`.

Что агрегируется. Двенадцать `UA_*` flags.

Threshold.

```
<= 5% affected records: +10, medium
>  5% affected records: +20, high
```

Почему threshold выше, чем для OOB. Некоторые operand sanity и ordering отклонения могут зависеть от редких compiler patterns. Для массового нарушения вероятность системной проблемы выше.

Почему один finding, а не двенадцать. Несколько flags могут возникнуть из одного повреждённого header. Отдельное начисление веса за каждый bit искусственно увеличивало бы score.

Ручная проверка. Расшифровать short names из report:

```
ver, trunc, op, arg, off, chain_loop, chain_depth,
handler_oob, order, operand_sanity, alignment, chain_invalid
```

и проверить конкретные bytes.

Граница. Finding сообщает структурную проблему metadata, но не устанавливает её происхождение.

Группа IV. Три статистических признака

16. `unwind_reuse_moderate`

Метрика.

```
unwind_reuse_ratio =
max_count_for_one_UnwindInfoRVA / total_runtime_functions
```

Candidate condition.

```
ratio > 0.10
total > 200
```

Дополнительное условие. Должен существовать другой static signal. Умеренный reuse без corroboration подавляется.

Вес и severity.

```
weight 10
severity medium
```

Почему reuse может быть нормальным. Одинаковые prologs, shared metadata, compiler folding и runtime helpers могут законно уменьшать размер таблиц.

Почему признак всё же полезен. В сочетании с placement, integrity или handler dominance reuse может указывать на централизованную схему перенаправления.

Ручная проверка. Найти dominant `UnwindInfoRVA`, перечислить функции, проверить одинаковы ли их реальные prologs и не используется ли chained info.

17. `unwind_reuse_strong`

Условие.

```
ratio > 0.30
total > 200
```

Вес и severity.

```
weight 20
severity high
```

Почему нужен минимальный размер. В таблице из трёх entries две одинаковые дают 66.7%, но статистически это слабое основание. Условие `total > 200` исключает такие малые samples.

Почему threshold не является универсальной нормой. Он является консервативным engineering default и должен проверяться на расширенном corpus разных toolchains.

Ручная проверка. Сравнить dominant record с распределением function sizes, compiler/runtime identity и фактическими prologs.

Граница. Сильный reuse не доказывает обфускацию.

18. `handler_dominance`

Метрика.

```
handler_reuse_ratio =
max_count_for_one_HandlerRVA / handler_count
```

Candidate condition.

```
handler_count > 50
ratio > 0.80
```

Контекстное подавление. Если dominant handler находится в executable code-like section и других static anomalies нет, finding не создаётся.

Почему это необходимо. `__C_specific_handler`, `__CxxFrameHandler3/4` и другие runtime handlers могут законно обслуживать множество функций.

Вес и severity.

```
weight 15
severity medium
```

Когда finding становится значимым. При сочетании с writable/non-executable placement, integrity errors или необычным reuse.

Ручная проверка. Идентифицировать handler, проверить imports/symbols, section permissions, xrefs, language-specific data и runtime hits.

Граница. Статический dominant handler может никогда не исполняться на исследованном path.

Группа V. Пять placement-признаков

19. `unwind_in_exec_section`

Условие. Section, содержащая `UnwindInfoRVA`, имеет `IMAGE_SCN_MEM_EXECUTE`.

Ожидаемая модель. Unwind metadata является данными и обычно размещается в read-only `.xdata`, `.rdata` или related section.

Почему executable placement подозрителен. Bytes могут одновременно интерпретироваться как code и metadata; это повышает риск parser differential, сокрытия и self-referential layouts.

Вес и severity.

```
weight 20
severity high
```

Ручная проверка. Проверить section flags, дизассемблирует ли инструмент эту область как code, есть ли control-flow edges внутрь metadata.

Граница. Нестандартный linker может объединить code и read-only data. Finding требует объяснения, но не является доказательством атаки.

20. `unwind_unusual_section`

Условие. Section name не относится к разрешённым xdata/rdata/pdata-like именам и не распознаётся как plausibly unwind-named.

Threshold.

```
count <= total / 20: +8, medium
count >  total / 20: +15, high
```

Почему имя является слабым источником. PE section name не определяет семантику и может быть произвольным. Поэтому permissions сильнее имени.

Зачем правило существует. Необычное размещение помогает analyst быстро найти custom linker/protector regions и сопоставить их с другими signals.

Weakening. Placement-only и single-family penalties могут снизить confidence.

Ручная проверка. Проверить characteristics, entropy, соседние data, compiler identity и mapping всех affected records.

Implementation nuance. При `total < 20` integer division даёт `total / 20 == 0`, поэтому любое положительное count попадает в high branch. Это кандидат для будущей calibration.

21. `unwind_writable_section`

Условие. Section с `UNWIND_INFO` имеет `IMAGE_SCN_MEM_WRITE`.

Почему это сильнее необычного имени. Writable permission является функциональным свойством memory mapping. После загрузки metadata потенциально может изменяться без изменения disk file.

Threshold и веса.

```
до 5%: +10, medium
более 5%: +18, high
```

Возможные benign причины. JIT infrastructure, custom runtime, post-load fixups, объединённая data section.

Ручная проверка. Определить, изменяются ли bytes во время исполнения; сравнить disk и memory copies; проверить calls к APIs управления memory protection.

Граница. Сам permission не доказывает фактическую модификацию.

22. `handler_nonexec_section`

Условие. `HandlerRVA` не принадлежит известной executable section.

Почему это сильный structural signal. Handler должен быть передан управлению как код. Target в non-executable data противоречит обычной модели исполнения.

Вес и severity.

```
weight 15
severity high
```

Возможные объяснения. Неверный RVA, повреждённая metadata, section table anomaly, runtime protection transition, custom loader.

Ручная проверка. Проверить mapping после загрузки, DEP permissions, возможное изменение protection и фактическое значение handler в memory.

Граница. Static section flags не всегда равны текущим page protections после `VirtualProtect`.

23. `handler_writable_section`

Условие. Handler target находится в section с write permission.

Почему это важно. Writable executable code облегчает runtime modification и затрудняет утверждение, что disk bytes равны исполнявшимся bytes.

Вес и severity.

```
weight 10
severity medium
```

Почему не high автоматически. Некоторые runtime-generated или unpacked regions законно проходят состояние writable+executable.

Ручная проверка. Проверить W^X policy, фактические page protections, изменения bytes и момент установки handler.

Граница. Section-level permission грубее page-level runtime protection.

Группа VI. Четыре combined findings

24. `placement_exec_nonexec_handler_combo`

Условие.

```
unwind_in_exec_section exists
and handler_nonexec_section exists
```

Логика. Metadata размещена там, где ожидается code, а handler указывает туда, где code не ожидается. Два направления нарушения усиливают друг друга.

Вес и severity.

```
weight 10
severity high
```

Почему это отдельный finding. Он выражает не новую raw observation, а policy о значимости совместного появления.

Ручная проверка. Проверить обе стороны по конкретным RVA и исключить ошибку section mapping.

25. `placement_integrity_combo`

Условие.

```
(unwind in executable section or handler non-executable)
and
(parse failure or unwind anomaly or unwind RVA OOB)
```

Логика. Необычное размещение подкреплено повреждением содержимого или указателя.

Вес и severity.

```
weight 8
severity medium
family unwind_integrity
```

Почему вес умеренный. Исходные findings уже внесли основной вклад. Combo добавляет ограниченный corroboration bonus.

Граница. Сигналы могут иметь одну первопричину, поэтому их нельзя интерпретировать как независимые вероятности.

26. `reuse_dominance_combo`

Условие.

```
unwind_reuse_ratio > 0.10
and handler_dominance finding exists
```

Логика. Централизация происходит одновременно на уровне unwind records и handler targets.

Вес и severity.

```
weight 8
severity medium
```

Возможная benign причина. Общий language runtime handler плюс compiler-generated shared unwind descriptions.

Ручная проверка. Проверить, соответствует ли dominant handler известному runtime и согласованы ли реальные prologs с общей metadata.

27. `writable_metadata_handler_combo`

Условие.

```
unwind_writable_section exists
and handler_writable_section exists
```

Логика. И описание unwind, и код handler находятся в изменяемых regions. Это создаёт возможность согласованного runtime изменения обеих сторон.

Вес и severity.

```
weight 10
severity high
```

Ручная проверка. Снять memory snapshots до и после инициализации, сравнить bytes, page protections и события регистрации.

Граница. Permission создаёт возможность изменения, но не доказывает, что изменение состоялось.

Почему отдельных findings не больше

Проект не создаёт отдельные публичные codes для каждого:

- `UWOP`;
- register number;
- `UA_*` bit;
- section name;
- dynamic exception code.

Причина - контроль гранулярности. Слишком мелкие findings:

- многократно оценивают одну первопричину;
- переполняют report;

• затрудняют сравнение файлов;

• делают score зависимым от случайного количества вторичных симптомов.

Низкоуровневые детали сохраняются в `UnwindSummary`, counters, operations text и event timeline.

Почему отдельных findings не меньше

Объединение всех ошибок в один code `malformed_unwind` потеряло бы:

• слой возникновения;

• severity;

• локализацию;

• manual workflow;

• возможность threshold calibration;

• проверяемость regression tests.

Разделение на 27 codes сохраняет техническую причину результата.

Матрица покрытия классов вмешательств

Класс изменения	Основные findings	Что подтверждается
Усечение или ложные file ranges	`pe_section_raw_oob`, `pe_exception_directory_oob`	Объявленные bytes недоступны
Ложный virtual layout	`pe_section_virtual_oob`, overlaps, misalignment	Image model неоднозначна или несогласованна
Подмена function boundaries	`invalid_ranges`, `rf_oob`, `rf_unsorted`, `rf_overlaps`, `duplicate_begin`	Function table нарушает structural contract
Подмена unwind pointer	`unwind_rva_oob`, `unwind_parse_fail`	Запись недоступна или не разбирается
Повреждение unwind encoding	`unwind_anomalies`	Нарушены version-specific rules
Перемещение metadata	три unwind placement findings	Section context необычен
Перенаправление handler	два handler placement findings	Target section не соответствует обычному коду
Централизация	reuse и dominance	Распределение ссылок необычно
Сочетание независимых слоёв	четыре combo findings	Несколько свойств взаимно подкрепляются

Какие вмешательства 27 признаков не гарантируют обнаружить

1. Изменение машинного кода при полностью согласованной metadata.
2. Замена handler на другой допустимый executable RVA.
3. Подмена semantics внутри существующего handler без изменения placement.
4. Runtime modification после окончания observation window.
5. Динамическая регистрация через ненаблюдаемый механизм.
6. Неисполненный path.
7. Корректная новая версия metadata без поддержки parser.
8. Изменение language-specific handler data, которое не нарушает текущие

structural checks.

9. Подмена файла на другой валидный файл без trusted baseline.

Как добавить новый признак правильно

Новый finding должен пройти последовательность.

1. Сформулировать технический инвариант или статистическую гипотезу.
2. Определить raw observations.
3. Назвать benign alternatives.
4. Выбрать family.
5. Обосновать severity и вес относительно существующих rules.
6. Добавить representative location и affected count.
7. Добавить unit test positive.
8. Добавить unit test benign suppression.
9. Проверить влияние на corpus и thresholds.
10. Обновить schema semantics, evidence matrix и учебный курс.

Итоговое обоснование числа 27

Технически корректный ответ:

> *В проекте 27 публичных finding codes, потому что текущая модель декомпозирована на 6 проверок PE layout, 7 проверок function table, 2 агрегированных проверки* `UNWIND_INFO`*, 3 статистических pattern, 5 placement-проверок и 4 combined rules. Число не объявляется полным числом возможных атак. Полнота оценивается по покрываемым классам вмешательств, traceability и тестам каждого правила. Внутри unwind decoder дополнительно различаются 12 диагностических причин* `UA_*`*.*

Контрольные вопросы

1. Почему `duplicate_begin` и `rf_overlaps` сохраняются отдельно?
2. Почему moderate reuse требует corroboration?
3. Почему handler dominance подавляется для обычного runtime handler?
4. Почему section name слабее section permission?
5. Почему `unwind_parse_fail` и `unwind_anomalies` разделены?
6. Почему combo finding может добавлять вес, хотя raw observations уже

учтены?

7. Какие признаки являются жёсткими structural violations?
8. Какие признаки являются контекстными?
9. Какие признаки являются статистическими?
10. Как доказать, что новый finding не дублирует существующий?

Расширенная лекция 23. Accuracy validation и дизайн эксперимента

Том III. Точность, валидация и проектирование строгого эксперимента

Почему вопрос «какова точность» неполон

Точность нельзя вычислить, пока не определены:

1. единица классификации;

2. положительный класс;
3. отрицательный класс;
4. ground truth;
5. threshold решения;
6. состав corpus;
7. правила учёта неоднозначных cases.

Для EH/Unwind Analyzer возможны разные задачи:

- определить, разбирается ли PE без structural failure;
- определить, существует ли конкретное нарушение;
- определить, должен ли finding быть создан;
- определить, требует ли файл ручной проверки;
- определить, присутствовало ли намеренное вмешательство;
- определить, является ли файл вредоносным.

Проект непосредственно решает первые четыре задачи. Последние две требуют дополнительного ground truth и анализа semantics.

Единица оценки

Unit of evaluation, или единица оценки, определяет, что считается одним примером.

File-level

Один PE-файл является одним примером.

Вопрос:

```
должен ли файл получить хотя бы один finding
```

Преимущество - простая итоговая таблица. Недостаток - файл с одной повреждённой entry и файл с тысячами повреждений учитываются одинаково.

Record-level

Одна `RUNTIME_FUNCTION` или одна `UNWIND_INFO` является примером.

Вопрос:

```
правильно ли отмечена конкретная запись
```

Это точнее для parser и per-record anomalies, но требует детальной разметки каждой записи.

Finding-level

Один потенциальный finding code для одного файла является примером.

Вопрос:

```
должен ли code X присутствовать в отчёте файла Y
```

Так можно измерять precision и recall отдельно для каждого из 27 rules.

Event-level

Одно динамическое событие является примером.

Вопрос:

```
зарегистрирован ли фактический вызов или handler hit
```

Требуется внешний trace или специально контролируемый target.

Ground truth

Ground truth - заранее установленная эталонная разметка, с которой сравнивается результат инструмента.

Ground truth должен отвечать:

- какое нарушение присутствует;
- где оно находится;
- почему оно считается нарушением;
- каким способом это подтверждено;
- кто выполнил разметку;
- допускается ли неоднозначный verdict.

Источники ground truth:

1. специально сконструированный sample;
2. differential comparison с независимыми parsers;
3. compiler/linker provenance;
4. ручная экспертная проверка;
5. сравнение disk/memory/baseline versions;
6. instrumented dynamic target;
7. consensus нескольких экспертов.

Synthetic sample даёт точный ground truth по внесённому изменению, но плохо представляет разнообразие реальных toolchains. Реальный corpus представляет разнообразие, но требует дорогой разметки.

Confusion matrix

Для бинарного решения используются четыре результата.

True Positive, TP

Нарушение присутствует, и инструмент его обнаружил.

False Positive, FP

Нарушение отсутствует, но инструмент создал положительный сигнал.

True Negative, TN

Нарушение отсутствует, и инструмент не создал сигнал.

False Negative, FN

Нарушение присутствует, но инструмент его не обнаружил.

Матрица:

Ground truth	Сигнал есть	Сигнала нет
Нарушение есть	TP	FN
Нарушения нет	FP	TN

Без размеченных positive examples нельзя вычислить TP и FN. Без размеченных negative examples нельзя вычислить TN и FP.

Accuracy, или доля правильных решений:

```
accuracy = (TP + TN) / (TP + TN + FP + FN)
```

Accuracy сильно зависит от баланса classes.

Если в corpus 990 benign и 10 anomalous files, инструмент, всегда отвечающий «аномалии нет», получит:

```
accuracy = 990 / 1000 = 99%
```

но:

```
recall = 0 / 10 = 0%
```

Поэтому overall accuracy не должна быть единственной метрикой triage tool.

Sensitivity и recall

Sensitivity, также **recall** или true positive rate:

```
recall = TP / (TP + FN)
```

Она отвечает:

```
какую долю реально существующих нарушений обнаружил инструмент
```

Для проекта recall нужно измерять per finding class. Например, отдельный recall для:

- `rf_overlaps`;
- `unwind_rva_oob`;
- `UA_BAD_OPCODE`;
- writable handler placement;
- runtime `RtlAddFunctionTable` registration.

Объединение всех нарушений в одну категорию скрывает слабые классы.

Specificity

Specificity, или true negative rate:

```
specificity = TN / (TN + FP)
```

Она отвечает:

```
какую долю корректных объектов инструмент не пометил ошибочно
```

Для analyst-facing triage specificity важна, потому что false positives создают ручную нагрузку.

Precision

Precision, или positive predictive value:

```
precision = TP / (TP + FP)
```

Она отвечает:

```
какая доля созданных положительных сигналов подтверждается ground truth
```

Precision зависит от prevalence нарушений в corpus. Даже неизменный detector может иметь разную precision в исследовательском наборе с 50% anomalies и в реальном потоке с 0.1% anomalies.

Negative Predictive Value

```
NPV = TN / (TN + FN)
```

Показатель отвечает, насколько надёжен отрицательный результат в выбранном corpus. Для проекта его нельзя интерпретировать как гарантию отсутствия любого вмешательства, потому что проверяется ограниченное пространство классов.

F1

F1 является гармоническим средним precision и recall:

```
F1 = 2 * precision * recall / (precision + recall)
```

F1 не использует TN и полезна при дисбалансе classes, но также скрывает различие между precision и recall. Поэтому обе исходные метрики должны публиковаться отдельно.

False Positive Rate

```
FPR = FP / (FP + TN) = 1 - specificity
```

Если specificity равна 99%, FPR равна 1%. Для потока из миллиона benign файлов это теоретически до десяти тысяч сигналов при аналогичном распределении. Абсолютная analyst workload зависит от объёма данных.

False Negative Rate

```
FNR = FN / (FN + TP) = 1 - recall
```

Этот показатель особенно важен для жёстких structural violations, где ожидается почти детерминированное обнаружение при условии поддерживаемой версии формата.

Threshold

Проект имеет несколько thresholds:

- prevalence выше 1% для ряда integrity findings;
- anomaly prevalence выше 5%;
- reuse выше 10% или 30%;
- dominance выше 80%;
- score verdict thresholds 15, 40 и 70.

Изменение threshold меняет trade-off.

Снижение threshold:

- обычно увеличивает recall;
- обычно увеличивает false positives.

Повышение threshold:

- обычно увеличивает specificity;
- может увеличить false negatives.

Threshold нельзя выбирать только по synthetic samples, специально созданным под существующие правила.

ROC и PR curves

Если система выдаёт непрерывный score, можно изменять decision threshold и строить curve.

ROC curve показывает:

```
true positive rate против false positive rate
```

PR curve показывает:

```
precision против recall
```

Для редких positive classes PR curve часто информативнее ROC, потому что сильнее отражает влияние false positives.

Текущий score является суммой policy weights. Построение curve возможно технически, но имеет смысл только на независимом размеченном corpus.

Calibration

Calibration отвечает на вопрос, соответствует ли числовая оценка эмпирической частоте события.

Если система заявляет вероятность 0.8 для ста cases, приблизительно 80 из них должны подтверждаться при хорошей calibration.

Поле `suspicion = score / 100` не calibrated probability. Поэтому для него нельзя применять вероятностную интерпретацию.

Confidence interval

Результат sample является оценкой неизвестного свойства более широкой совокупности.

Если в 106 benign files нет false positives:

```
observed specificity = 106 / 106 = 100%
```

Это не доказывает universal specificity 100%. При простой биномиальной модели двусторонний 95% Wilson interval имеет нижнюю границу приблизительно 96.5%.

Wilson interval для доли:

```
center =
(p_hat + z^2/(2n)) / (1 + z^2/n)

half_width =
z / (1 + z^2/n)
* sqrt(p_hat(1-p_hat)/n + z^2/(4n^2))
```

где:

```
p_hat = x / n
z = 1.96 для приблизительного 95% interval
```

Даже interval не исправляет corpus bias. Если все files получены от ограниченного набора vendors и toolchains, математическая формула не делает выборку репрезентативной автоматически.

Independence assumption

Простая биномиальная модель предполагает независимые испытания.

106 files могут содержать:

- десятки DLL одной CPython distribution;
- несколько binaries одного compiler version;
- общие runtime libraries;
- похожие linker options.

Такие samples коррелированы. Эффективное количество независимых layouts меньше числа файлов. Поэтому полезно группировать результаты по:

- vendor;
- product;
- compiler;
- linker;
- architecture feature;
- presence of handlers;
- number of runtime functions.

Что доказывает corpus 106/106

Зафиксированный результат:

```
106/106 valid reports
174 795 RUNTIME_FUNCTION
0 parse failures
0 structural unwind anomalies
0 nonzero heuristic scores
```

Он подтверждает:

1. совместимость parser с выбранным benign corpus;
2. отсутствие наблюдаемых false positives по правилу

«ненулевой score/finding является положительным сигналом»;

3. способность обработать большое число реальных entries;
4. стабильность outputs на этих layouts.

Он не подтверждает:

- recall на реальных вмешательствах;
- поддержку всех Windows toolchains;
- поддержку V3;
- отсутствие false positives в мировой совокупности;
- общую accuracy.

Что доказывают 6 regression samples

Два benign и четыре suspicious fixtures проверяют:

- ожидаемые paths parser;
- срабатывание конкретных известных rules;
- score/verdict contract;

- стабильность JSON.

Это **functional regression coverage**.

Оно не является оценкой external sensitivity, потому что suspicious samples сконструированы под известные conditions. Инструмент фактически проверяется на задачах, которые разработчик заранее вложил в fixtures.

Что доказывают 1000 mutations

Результат:

```
634 accepted valid reports
366 контролируемый отказ
0 crashes
0 hangs
0 invalid reports
```

подтверждает robustness относительно выполненных mutations:

- malformed inputs обрабатываются контролируемо;
- parser не упал;
- report contract сохранился.

Это не classification accuracy. Для большинства mutations неизвестно, какой finding должен был быть истинным.

Что доказывают 124/124 Windows reports

Результат означает, что все собранные reports прошли JSON Schema и cross-field semantic validation.

Проверяется:

- структура report;
- согласованность counters;
- status fields;
- event bounds;
- metadata;
- отсутствие нарушений schema.

Это не означает, что все возможные dynamic paths были достигнуты.

Что доказывают 4/4 focused smoke scenarios

Четыре purpose-built targets подтверждают работу selected mechanisms:

- VEH;
- VCH;
- UEF;
- runtime function-table registration.

Это проверка известных scenarios. Она не измеряет coverage реальных программ, anti-debug behavior, multi-process logic и всех способов регистрации.

Parser accuracy и detector accuracy

Эти понятия необходимо разделять.

Parser structural accuracy

Вопрос:

```
правильно ли bytes преобразованы в поля и summaries
```

Методы:

- byte-level golden fixtures;
- differential comparison;
- round-trip там, где применимо;
- compiler-generated samples;
- boundary unit tests.

Heuristic detection accuracy

Вопрос:

```
правильно ли findings соответствуют размеченным аномалиям
```

Методы:

- per-class labeled corpus;
- precision/recall;
- threshold sweep;
- error analysis.

Runtime observation accuracy

Вопрос:

```
зарегистрировал ли collector фактически произошедшие события
```

Методы:

- instrumented target ground truth;
- independent debugger/ETW trace;
- timestamps and event identities;
- repeated runs.

Нельзя объединять эти три слоя в один процент.

Строгий benchmark plan

Шаг 1. Зафиксировать taxonomy

Для каждого из 27 findings определить:

- positive condition;
- negative condition;
- ambiguous/excluded condition;
- expected location;

- expected affected count.

Шаг 2. Собрать corpus

Минимальные strata:

- MSVC разных versions;
- MinGW-w64;
- Clang/LLVM for Windows;
- system binaries;
- user applications;
- DLL, EXE и PYD;
- binaries с C, C++ EH и без handlers;
- packed/protected samples;
- handcrafted malformed samples;
- V1, V2 и V3 после реализации поддержки.

Шаг 3. Разделить development и test

```
development set -> выбор rules и thresholds
test set        -> однократная итоговая оценка
```

Использование test set для настройки превращает его в development set.

Шаг 4. Выполнить независимую разметку

Желательно два эксперта. Расхождения фиксируются и разрешаются отдельно. Нужно хранить rationale, а не только label.

Шаг 5. Измерить per-class metrics

Для каждого finding:

- TP;
- FP;
- TN;
- FN;
- precision;
- recall;
- specificity;
- F1;
- confidence interval.

Шаг 6. Проверить thresholds

Запустить score sweep и prevalence threshold variants. Выбирать policy по заранее определённой цели, например:

```
минимизировать false-positive burden
при recall жёстких structural violations не ниже заданного уровня
```

Шаг 7. Провести error analysis

Каждый FP и FN должен получить:

- root cause;
- affected toolchain;
- решение: исправить, документировать или исключить;
- regression fixture, если исправление принято.

Шаг 8. Опубликовать limitations

Benchmark должен указывать:

- даты;
- hashes;
- версии;
- состав corpus;
- exclusions;
- доверительные интервалы;
- known bias.

Какая метрика главная для этого проекта

Одной главной метрики нет.

Для hard structural rules приоритет:

```
высокий per-class recall
```

Для контекстных placement/reuse rules:

```
высокая precision
и низкая analyst false-positive burden
```

Для benign compatibility:

```
specificity
```

Для parser robustness:

```
crash-free execution, контролируемый отказ, sanitizer/fuzz results
```

Для dynamic collector:

```
event recall на instrumented targets
и корректность event semantics
```

Технически корректный ответ о текущей точности

> *Общая accuracy проекта пока не установлена, потому что нет достаточно большого независимого размеченного positive corpus для всех классов вмешательств. На выбранном benign corpus наблюдаемая file-level specificity составила 106/106, то есть 100%; нижняя граница простого 95% Wilson interval приблизительно 96.5%, но выборка ограничена. Шесть regression samples подтверждают ожидаемую функциональность известных rules, 1000 mutations - robustness, 124 Windows reports - корректность schema/semantics, а 4 smoke scenarios - работу выбранных dynamic mechanisms. Эти результаты нельзя складывать в один процент.*

Технические источники

1. NIST Static Analysis Tool Study Methodology - определения precision,

accuracy и recall при сравнении инструменты analysis: https://www.nist.gov/system/files/documents/2021/03/24/CAS%202012%20Static%20Analysis%20Tool%20Study%20Methodology.pdf

2. NIST/SEMATECH e-Handbook of Statistical Methods - доверительные интервалы и

основы статистической оценки: https://www.itl.nist.gov/div898/handbook/

3. Документация проекта:

docs/evidence_matrix.md, docs/threshold_calibration.md, docs/PRODUCT_READINESS.md.

Контрольные вопросы

1. Почему 106/106 не является общей accuracy?
2. Чем specificity отличается от precision?
3. Почему synthetic positive samples не дают внешнюю sensitivity?
4. Что измеряют mutations?
5. Зачем разделять file-level и finding-level metrics?
6. Почему один accuracy-процент опасен при class imbalance?
7. Что означает confidence interval?
8. Почему correlated binaries уменьшают effective sample diversity?
9. Как избежать leakage между development и test set?
10. Какие метрики нужны отдельно для parser, heuristics и dynamics?

Расширенная лекция 24. UNWIND_INFO v3 compatibility

Том III. UNWIND_INFO V3: новая модель, совместимость и план поддержки

Почему V3 требует отдельной главы

В ранних главах курс описывает UNWIND_INFO V1/V2. Текущий decoder проекта поддерживает именно эти версии. В мае 2026 года Microsoft опубликовала предварительную спецификацию x64 Unwind Information V3.

V3 нельзя считать небольшим добавлением одного opcode. Она сохраняет RUNTIME_FUNCTION и общие .pdata/.xdata conventions, но существенно меняет внутреннюю организацию unwind record:

- вместо V1/V2 slots появляется пул WOD переменной длины;
- отдельно хранятся смещения IP;
- epilogs получают явные descriptors, то есть явные описатели эпилогов, то есть явные описатели эпилогов;
- кодирование integer-регистров расширяется до 5 bits для APX registers;
- появляется UNW_FLAG_LARGE;
- payload, то есть полезная часть записи, имеет собственный размер в 16-bit words.

Попытка читать V3 decoder-ом, то есть декодером,, то есть декодером, V1/V2 создаст неверные boundaries, operations и tail offset. Поэтому контролируемый отказ текущей версии проекта безопаснее ложного успешного разбора.

Статус спецификации

Microsoft прямо обозначает V3 как предварительную спецификацию. Это означает:

- структура уже документирована;
- compiler/assembler support может появляться экспериментально;
- возможны несовместимые изменения;
- production parser должен versionировать реализацию и тестовые fixtures;
- нельзя считать текущий текст неизменяемым навсегда.

На дату подготовки главы официальный источник указывает обновление 21 мая 2026 года.

Назначение V3

V3 добавляет поддержку Intel APX, включая расширенный набор general-purpose registers R16-R31. Она также предоставляет compiler больше свободы в prolog и epilog и позволяет точнее описывать операции обоих направлений.

Ключевое различие:

```
V1/V2 в основном кодируют действия prolog,
а unwind procedure восстанавливает контекст в обратном порядке.

V3 хранит общий pool operation descriptors,
на который ссылаются prolog и явные epilog descriptors.
```

Что осталось неизменным

`.pdata` по-прежнему является отсортированным массивом 12-byte `RUNTIME_FUNCTION`:

```
offset 0: BeginAddress
offset 4: EndAddress
offset 8: UnwindInfoAddress
```

Все три поля остаются 32-bit RVA.

Следовательно, существующие findings function table остаются концептуально применимыми:

- `invalid_ranges`;
- `rf_oob`;
- `unwind_rva_oob`;
- `rf_unsorted`;
- `rf_overlaps`;
- `duplicate_begin`.

Но decoder объекта по `UnwindInfoAddress` должен выбирать implementation по version.

Header V3

Header занимает четыре bytes.

Byte	Bits	Поле
0	2:0	`Version = 3`
0	7:3	`Flags`
1	7:0	`SizeOfProlog` low byte
2	7:0	`PayloadWords`

Byte	Bits	Поле
3	4:0	NumberOfOps
3	7:5	NumberOfEpilogs

Сходство размера header с V1/V2 не означает совместимость полей.

В V1/V2 byte 2 - CountOfCodes, а byte 3 содержит frame register и frame offset. В V3:

```
byte 2 = PayloadWords
byte 3 = NumberOfOps + NumberOfEpilogs
```

Старый decoder истолковал бы эти значения как slots и frame declaration.

PayloadWords

PayloadWords задаёт число 16-bit words после header:

```
payload_size = PayloadWords * 2
```

Payload содержит:

1. optional large-prolog extension;
2. prolog смещения IP;
3. epilog descriptors;
4. epilog смещения IP;
5. WOD pool;
6. возможный padding до полного word.

Handler RVA или chained RUNTIME_FUNCTION в PayloadWords не входят.

Tail offset

Положение handler/chain вычисляется:

```
tail =
align_up(sizeof(UNWIND_INFO_V3) + PayloadWords * 2, 4)
```

После tail:

• при UNW_FLAG_EHANDLER или UNW_FLAG_UHANDLER находится 4-byte

handler RVA и language-specific data;

• при UNW_FLAG_CHAININFO находится 12-byte RUNTIME_FUNCTION.

Общий принцип tail сохранился, но его исходная длина определяется payload, а не количеством V1/V2 slots.

Flags V3

V3 сохраняет:

```
UNW_FLAG_EHANDLER  = 0x01
UNW_FLAG_UHANDLER  = 0x02
UNW_FLAG_CHAININFO = 0x04
```

Добавляется:

```
UNW_FLAG_LARGE = 0x08
```

Bit 4 зарезервирован и должен быть zero.

Decoder должен проверять:

- несовместимые combinations;
- reserved bit;
- наличие required extension;
- достаточную длину payload;
- tail bounds.

Large prolog

Без `UNW_FLAG_LARGE` размер prolog ограничен 8-bit полем.

При `UNW_FLAG_LARGE` первый byte payload является `UNWIND_INFO_LARGE_V3.SizeOfPrologHighByte`.

Полный размер:

```
SizeOfProlog16 =
(SizeOfPrologHighByte << 8) | SizeOfProlog
```

Кроме того, prolog смещения IP становятся 16-bit вместо 8-bit.

Это означает, что parser должен вычислять element width из flags до чтения массива offsets.

Prolog смещения IP

Количество элементов равно `NumberOfOps`.

Обычная форма:

```
NumberOfOps bytes
```

Large form:

```
NumberOfOps * 2 bytes
```

Первый offset относится к операции, ближайшей к function body, то есть к последней prolog instruction с unwind effect. Последний offset относится к операции, ближайшей к function entry.

Таким образом сохраняется принцип reverse execution order.

Epilog descriptors

`NumberOfEpilogs` кодируется тремя bits и имеет диапазон 0-7 для одного fragment. Если epilogs больше, specification предлагает subfragments.

Каждый `EPILOG_INFO_V3` содержит:

```
Flags          3 bits
NumberOfOps    5 bits
EpilogOffset   signed 16 bits
```

При `NumberOfOps > 0` следует extended descriptor:

- `FirstOp`;
- `IpOffsetOfLastInstruction`;
- массив смещения IP.

При `NumberOfOps == 0` descriptor наследует operation description от предыдущего epilog. Parser должен запретить наследование без предыдущего полного descriptor.

Epilog offset

`EpilogOffset` является signed displacement.

Для первого descriptor:

- положительное значение считается от fragment start;
- отрицательное - от fragment tail.

Для последующих descriptors значение является delta от предыдущего epilog.

Все epilogs должны использовать одинаковое направление:

```
либо ascending from start,
либо descending from tail
```

Нарушение sign/order policy должно стать отдельной V3 anomaly reason или подтипом общего descriptor anomaly.

Parent fragment transfer

Epilog flag `EPILOG_INFO_PARENT_FRAGMENT_TRANSFER` сообщает, что epilog передаёт управление parent fragment, например через `JMP`, а не возвращается непосредственно caller.

Для reverse engineer это важно: control transfer нельзя автоматически интерпретировать как обычный `RET`.

Large epilog

Flag `EPILOG_INFO_LARGE` выбирает extended descriptor с 16-bit `IpOffsetOfLastInstruction` и 16-bit elements массива offsets.

Parser должен независимо учитывать:

- large prolog policy;
- large policy конкретного epilog.

Нельзя использовать один global element width для всего payload.

WOD

Winding Operation Descriptor, WOD - variable-length packed descriptor одной unwind operation.

WOD pool является последовательностью bytes без выравнивания между descriptors. Размер одного WOD определяется opcode и составляет от одного до пяти bytes.

Prolog WOD list начинается с offset zero WOD pool. Epilog использует `FirstOp`, который является byte offset, а не порядковым номером operation.

Почему variable-length decoding сложнее

В V1/V2 основная единица массива - двухбайтовый slot. Длина operation выражается числом slots.

В V3:

```
operation length = 1..5 bytes
opcode width = 3, 4, 6 или 8 bits
```

Decoder должен проверять opcode в определённом порядке. Если сначала сравнить полный byte, можно неправильно распознать компактные opcodes, которые используют только младшие bits.

Порядок dispatch WOD

Безопасная логика:

1. проверить 3-bit compact opcodes;
2. проверить 4-bit opcodes;
3. проверить 6-bit `WOD_PUSH2`;
4. проверить full-byte opcodes;
5. иначе отметить invalid opcode.

Этот порядок является частью формата, а не только implementation detail.

Основные WOD

WOD	Размер	Смысл
`WOD_SET_FPREG`	2	Установить frame register
`WOD_ALLOC_HUGE`	5	Большое unscaled выделение stack
`WOD_ALLOC_LARGE`	3	Выделение stack с scale 8
`WOD_PUSH_CANONICAL_FRAME`	2	Canonical OS/hardware frame
`WOD_PUSH`	1	Push одного integer register
`WOD_SAVE_NONVOL_FAR`	5	Far save integer register
`WOD_SAVE_NONVOL`	3	Scaled save integer register
`WOD_PUSH_CONSECUTIVE_2`	1	Push двух последовательных registers
`WOD_ALLOC_SMALL`	1	Compact stack allocation
`WOD_SAVE_XMM128_FAR`	5	Far save XMM
`WOD_SAVE_XMM128`	3	Scaled save XMM
`WOD_PUSH2`	2	APX push двух arbitrary registers

Integer register encoding

V3 compact integer operations используют 5-bit register number:

```
0..15  -> RAX..R15
16..31 -> R16..R31
```

Это одно из принципиальных APX-изменений. Текущий `UnwindSummary` имеет:

```
uint16_t saved_nonvol_mask;
```

Такой mask способен представить только 16 register numbers. Для V3 он недостаточен.

Поддержка V3 требует как минимум:

```
uint32_t saved_nonvol_mask;
```

или отдельной extensible register-set structure.

Изменение структуры влияет на:

- `unwind.h`;
- `report.c`;
- JSON schema;
- CSV;
- HTML;

- tests;
- backward compatibility.

WOD_PUSH

Первый byte:

```
bits 2:0 = 100b
bits 7:3 = 5-bit register
```

Operation уменьшает RSP на 8 при исполнении prolog. При unwind register восстанавливается из stack, а simulated RSP увеличивается на 8.

Parser должен проверять, допустим ли register для данной ABI role, а не только помещается ли номер в пять bits.

WOD_PUSH2

WOD_PUSH2 занимает два bytes и кодирует два arbitrary 5-bit registers.

Stack effect:

```
16 bytes
```

Для summary одного pushed_nonvol_count достаточно для количества, но masks должны включать оба registers.

WOD_PUSH_CONSECUTIVE_2

Кодирует register N и N+1.

Обязательная sanity check:

```
N <= 30
```

Иначе второй register имел бы номер 32 вне 5-bit APX range.

Allocation WOD

WOD_ALLOC_SMALL:

```
allocation = (Size + 1) * 8
range 8..128
```

WOD_ALLOC_LARGE:

```
allocation = uint16_size * 8
```

WOD_ALLOC_HUGE:

```
allocation = raw uint32 byte count
```

При суммировании fixed stack bytes нужно проверять integer overflow. Даже если individual field допустим, сумма нескольких operations может выйти за uint32_t.

Save WOD

WOD_SAVE_NONVOL использует 16-bit displacement co scale 8.

WOD_SAVE_NONVOL_FAR использует 32-bit unscaled displacement.

WOD_SAVE_XMM128 использует scale 16.

WOD_SAVE_XMM128_FAR использует 32-bit unscaled displacement.

Sanity checks должны сопоставлять:

- displacement;
- stack allocation;
- alignment;
- register class;
- operation direction в prolog/epilog.

WOD_SET_FPREG

Encoding включает 4-bit register и 4-bit offset.

Смысл:

```
frame_register = RSP + offset * 16
```

В отличие от compact APX integer operations, field register здесь документирован как 4-bit. Parser не должен автоматически расширять его до R16-R31.

WOD_PUSH_CANONICAL_FRAME

Operation описывает canonical frame, созданный hardware или OS mechanism. Type различает варианты frame.

До появления стабильных SDK constants parser должен:

- сохранять raw type;
- распознавать документированные значения;
- неизвестное значение отмечать как unsupported/unknown;
- не вычислять stack effect по догадке.

Fragment и chaining

V3 вводит термин **fragment** как непрерывную область machine code, описываемую парой `RUNTIME_FUNCTION/UNWIND_INFO_V3`.

Одна source-level функция может иметь:

- main fragment;
- subfragments;
- chained descriptions.

Это усиливает важность различия:

```
source function != обязательно один непрерывный runtime range
```

Findings overlap и duplicate begin должны учитывать только нормативно допустимую fragment model, а не предполагать простое соответствие один к одному.

Почему текущий проект правильно отказывает V3

Текущий код:

```
version != 1 && version != 2
-> UA_VERSION_UNKNOWN
-> parse failure
```

Это предотвращает:

- чтение `PayloadWords` как `CountOfCodes`;

- чтение `NumberOfOps/NumberOfEpilogs` как frame fields;
- неправильный tail offset;
- фиктивный handler RVA;
- выход за buffer при ложной slot length;
- ложные register masks.

С точки зрения memory safety и semantic honesty такой отказ корректен.

Почему отказ всё равно является продуктовым ограничением

После появления корректных V3 binaries текущая версия:

- не сможет декодировать их metadata;
- создаст `unwind_parse_fail`;
- может увеличить score;
- может потребовать manual review для benign file;
- ухудшит specificity на corpus с V3.

Следовательно, V3 support является compatibility requirement, а не только дополнительной функцией.

Как report должен различать malformed и unsupported

Текущий aggregate finding `unwind_parse_fail` объединяет причины.

Для V3 полезно ввести явное разделение:

```
unsupported_version
malformed_supported_version
```

На уровне per-record report:

```
{
  "version": 3,
  "parse_status": "unsupported",
  "parse_error": "UNWIND_INFO v3 is not supported by this build"
}
```

Unsupported корректный record не должен автоматически получать тот же severity, что truncated V1.

План безопасной реализации V3

Этап 1. Version dispatch

```
switch (version) {
case 1:
case 2:
    parse_v1_v2(...);
    break;
case 3:
    parse_v3(...);
    break;
default:
    unsupported(...);
}
```

Общие tail/handler helpers можно переиспользовать только после получения правильного version-specific payload end.

Этап 2. Новая модель данных

Нужны:

- 32-bit integer register mask или extensible set;
- epilog summaries;
- count WOD;
- payload size;
- support status;
- version-specific anomaly flags.

Не следует перегружать поля `count_codes`, `frame_reg`, `frame_off` несовместимой семантикой.

Этап 3. Bounded cursor

V3 parser удобнее строить через cursor:

```
payload_begin
payload_end
current
```

Перед каждым чтением:

```
needed <= payload_end - current
```

WOD pool boundaries должны быть вычислены после полного прохода descriptors, но до decoding `FirstOp`.

Этап 4. Descriptor validation

Проверить:

- reserved flags;
- large extension;
- `NumberOfOps`;
- `NumberOfEpilogs`;
- inheritance;
- sign consistency offsets;
- range epilogs внутри fragment;
- `FirstOp` внутри WOD pool;
- WOD count и byte lengths;
- IP offset ordering;
- last instruction boundary.

Этап 5. WOD decoding

Использовать documented dispatch order и отдельную функцию:

```
bool v3_decode_wod(
    const uint8_t* pool,
    size_t pool_size,
    size_t offset,
    V3Wod* out,
    size_t* consumed);
```

Function не должна изменять cursor при failure.

Этап 6. Summary

Вычислять:

- prolog stack effect;
- saved integer registers;
- saved XMM registers;
- frame register;
- epilog operation references;
- handler/chain.

При переполнении summary counter сохранять raw operation, но ставить anomaly.

Этап 7. Reports и schema

Добавить version-aware fields без удаления V1/V2:

```
encoding_family: "slots_v1_v2" | "wod_v3"
payload_words
prolog_operation_count
epilog_count
```

Schema version должна измениться при изменении публичного contract.

Этап 8. Tests

Минимальный набор:

1. V3 empty payload.
2. One `WOD_ALLOC_SMALL`.
3. APX `WOD_PUSH`.
4. `WOD_PUSH2`.
5. Large prolog.
6. One full epilog.
7. Inherited epilog.
8. Large epilog.
9. Shared WOD region.
10. Handler tail.
11. Chained fragment.
12. Invalid reserved flag.
13. Truncated WOD.
14. `FirstOp` OOB.
15. Invalid inheritance.
16. Mixed epilog offset signs.
17. Register 31 consecutive-pair error.
18. Differential sample from `ml64 /unwindv3`.

Этап 9. Corpus gate

До объявления поддержки нужны:

- compiler-generated V3 objects;
- linked PE images;
- comparison с Microsoft/LLVM tools, когда они поддерживают вывод;
- benign zero-false-positive corpus;
- mutation/fuzz seeds V3;
- backward regression V1/V2.

Как V3 влияет на 27 findings

Без изменений остаются PE layout и function table findings.

Требуют version-aware logic:

- `unwind_parse_fail`;
- `unwind_anomalies`;
- placement findings, если parser должен получить handler;
- handler dominance;
- combo findings.

Reuse по `UnwindInfoRVA` можно считать до полного decode, но интерпретация должна учитывать fragment sharing.

Технически корректный ответ о V3

> *Текущая версия безопасно поддерживает* `UNWIND_INFO` *V1/V2 и намеренно отказывает неизвестную V3, чтобы не интерпретировать её по старому layout. Microsoft опубликовала V3 как предварительную спецификацию для Intel APX в мае 2026 года. V3 сохраняет* `.pdata` *и 12-byte* `RUNTIME_FUNCTION`*, но вводит* `PayloadWords`*, явные epilog descriptors и пул WOD переменной длины. Поэтому поддержка требует отдельного decoder, расширенной модели registers, новых tests и обновления report schema. До этого корректные V3 files являются известной границей совместимости и могут создавать parse-failure signals.*

Нормативный источник

Microsoft Learn, `Exception Handling Unwind Information V3`:

`https://learn.microsoft.com/en-us/cpp/build/x64-unwind-information-v3?view=msvc-170`

Microsoft Learn, conventional x64 exception handling V1:

`https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64?view=msvc-170`

Контрольные вопросы

1. Почему одинаковый размер header не делает V3 совместимой с V1/V2?
2. Что содержит `PayloadWords` и чего оно не содержит?
3. Почему WOD decoder должен проверять opcodes в определённом порядке?
4. Для чего нужны explicit epilog descriptors?
5. Почему текущего 16-bit register mask недостаточно?

6. Чем large prolog отличается от large epilog?
7. Почему unsupported V3 нельзя считать malformed автоматически?
8. Какие существующие findings остаются применимыми без изменений?
9. Какие поля JSON должны стать version-aware?
10. Какой test corpus нужен перед объявлением поддержки V3?

Приложение A. Таблица основных UWOP

Код	Имя	Что описывает	Как влияет на unwind
0	`UWOP_PUSH_NONVOL`	`push` nonvolatile-регистра	прочитать регистр со стека, поднять `RSP` на 8
1	`UWOP_ALLOC_LARGE`	большое выделение стека	прибавить к `RSP` размер allocation
2	`UWOP_ALLOC_SMALL`	малое выделение стека	прибавить `OpInfo * 8 + 8`
3	`UWOP_SET_FPREG`	установка frame register	использовать frame register model
4	`UWOP_SAVE_NONVOL`	store nonvolatile-регистра в stack slot	восстановить регистр из `[frame + offset*8]`
5	`UWOP_SAVE_NONVOL_FAR`	store nonvolatile с большим offset	восстановить регистр из far offset
8	`UWOP_SAVE_XMM128`	сохранение nonvolatile XMM	восстановить XMM из offset*16
9	`UWOP_SAVE_XMM128_FAR`	сохранение XMM с большим offset	восстановить XMM из far offset
10	`UWOP_PUSH_MACHFRAME`	machine frame от системного события	восстановить frame с учетом optional error code

Приложение B. Словарь терминов без тумана

`RVA` - смещение от `ImageBase` внутри загруженного образа. Не file offset.

`VA` - виртуальный адрес в адресном пространстве процесса. Для PE-объекта обычно `VA = ImageBase + RVA`.

`File offset` - позиция байта в файле на диске. Для перехода от RVA к file offset нужна section table.

`ImageBase` - базовый адрес образа в памяти, предпочтительный или фактический после загрузки.

`Prolog` - начало функции, где создается stack frame и сохраняются nonvolatile- регистры.

`Epilog` - конец или выходной путь функции, где stack frame разбирается обратно.

`Volatile register` - регистр, который callee может испортить; caller не должен ожидать сохранения.

`Nonvolatile register` - регистр, который callee обязан сохранить, если меняет.

`Shadow space` - 32 байта, которые caller резервирует на стеке для первых четырех регистровых аргументов по Microsoft x64 calling convention.

`Unwind` - восстановление контекста caller-а из текущего кадра.

`Virtual unwind` - вычисление предыдущего контекста без буквального исполнения машинных инструкций возврата.

`Handler RVA` - RVA функции-обработчика, указанный после unwind codes при соответствующих flags.

`Language-specific data` - данные, которые понимает конкретный handler/runtime, например C SEH scope table.

`Finding` - структурированный сигнал анализатора: код, family, severity, адрес и количество affected entries.

`Mitigation` - правило, которое ослабляет уверенность, если есть нормальное объяснение или слишком мало данных.

Приложение C. Нормативные источники и ориентиры

Основные официальные источники:

- Microsoft PE Format: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format
- Microsoft x64 exception handling: https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64
- Microsoft x64 calling convention: https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention
- Microsoft RtlLookupFunctionEntry: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtllookupfunctionentry
- Microsoft RtlVirtualUnwind: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlvirtualunwind
- Microsoft RtlAddFunctionTable: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdfunctiontable
- Microsoft RtlInstallFunctionTableCallback: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlinstallfunctiontablecallback
- Microsoft RaiseException: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-raiseexception
- Microsoft AddVectoredExceptionHandler: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredexceptionhandler
- Microsoft SetUnhandledExceptionFilter: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-setunhandledexceptionfilter

Книжные ориентиры для параллельного развития:

- Eldad Eilam, `Reversing: Secrets of Reverse Engineering` - стиль мышления по слоям бинаря и системному анализу.
- Chris Eagle, `The IDA Pro Book` - дисциплина работы с дизассемблером и доказательным анализом.
- Bruce Dang, Alexandre Gazet, Elias Bachaalany, `Practical Reverse Engineering` - связь assembly, ОС, debugging и практики.
- Randal Bryant, David O'Hallaron, `Computer Systems: A Programmer's Perspective` - фундамент "программа как данные, память, процессор, linking".
- Andrew Tanenbaum, `Structured Computer Organization` - слои компьютерной системы.

Эти книги не копируются и не заменяют документацию Microsoft по конкретным структурам Windows x64. Их ценность здесь - в методе: проверять выводы через реальные байты, адреса, ABI и runtime behavior.

Приложение D. Чек-лист ручной проверки одного finding

1. Записать код finding и evidence family.
2. Найти representative RVA и преобразовать его в VA/file offset.
3. Открыть соответствующую запись `.pdata`.
4. Проверить Begin/End/UnwindData как little-endian RVA.
5. Проверить секцию для begin, unwind и handler RVA.
6. Разобрать `UNWIND_INFO` по bytes, count и slots.
7. Сравнить unwind operations с prolog disassembly.
8. Проверить, есть ли normal compiler/runtime explanation.
9. Если включен dynamic mode, сопоставить runtime event address с module base.
10. Сформулировать вывод в виде "факт -> правило -> следствие -> ограничение".

Приложение E. Как говорить о проекте на защите

Нельзя говорить: "Программа определяет защищенные файлы". Это слишком широко и нестрого.

Лучше говорить: "Программа анализирует EH/Unwind metadata PE32+ AMD64 и выявляет структурные несоответствия, необычное размещение metadata, переиспользование unwind information, dominance handlers и runtime-события, которые полезны для reverse engineering и ручной проверки protected binaries".

Нельзя говорить: "Score - вероятность вредоносности". Лучше: "Score - инженерная шкала suspiciousness по явной evidence matrix; confidence показывает уверенность в интерпретации, а mitigations защищают от чрезмерного вывода".

Нельзя говорить: "Если metadata writable, файл плохой". Лучше: "Writable placement не доказывает вредоносность, но нарушает обычное ожидание read-only unwind metadata и требует проверки в сочетании с другими признаками".